

TRƯỜNG ĐẠI HỌC GIAO THÔNG VẬN TẢI
KHOA CÔNG NGHỆ THÔNG TIN
BỘ MÔN KHOA HỌC MÁY TÍNH
TRẦN VĂN DŨNG – PHẠM XUÂN TÍCH



BÀI GIẢNG

CHƯƠNG TRÌNH DỊCH

SỐ TÍN CHỈ: 3

HÀ NỘI - 2015

LỜI NÓI ĐẦU

Môn học “Chương trình dịch” đã được đưa vào giảng dạy tại Khoa Công nghệ Thông tin của Trường đại học Giao thông Vận tải từ năm học 2003-2004. Do các ngôn ngữ lập trình và môi trường thực thi của các phần mềm trên mạng Internet ngày càng phát triển và mở rộng, nên Chương trình dịch đã trở thành một thành phần bắt buộc và quan trọng cho mọi hệ thống ứng dụng. Để đáp ứng yêu cầu học tập và tự tìm hiểu của sinh viên các chuyên ngành Công nghệ Thông tin, Bộ môn Khoa học máy tính, Khoa Công nghệ Thông tin, trường đại học Giao thông Vận tải đã tổ chức biên soạn bài giảng này. Nội dung của nó được dựa trên một số tài liệu, nhưng chủ yếu là hai cuốn sách: Andrew W. Appel và Jens Palsberg: *Modern Compiler Implementation in Java*; Alfred V. Aho, Ravi Sethi, and Jeffrey D. Ullman, *Compilers: Principles, Techniques and Tools*. Các cuốn sách trên đã được dùng làm tài liệu giảng dạy tại nhiều trường đại học. Với mục đích trang bị các kiến thức cơ sở vừa đủ và giúp cho sinh viên hiểu được bản chất của các bước thực thi của Chương trình dịch, trong giáo trình các tác giả đã cố gắng trình bày tóm tắt các phần lý thuyết cơ bản và đưa ra các ứng dụng thực tế. Đồng thời cũng trình bày một số xu hướng phát triển của ngôn ngữ lập trình như đặc trưng của các ngôn ngữ lập trình hướng đối tượng, các ngôn ngữ lập trình hàm và lập trình tổng quan.

Bài giảng gồm 8 chương. Chương mở đầu nêu tổng quan về Chương trình dịch; Chương 2 giới thiệu một số nguyên lý thiết kế ngôn ngữ lập trình như lập trình thủ tục, lập trình hướng đối tượng, lập trình hàm và tổng quan; Chương 3 tóm tắt về phân tích từ vựng, chuyển một chương trình thành dãy các từ tố; Chương 4 trình bày những khái niệm cơ bản về Phân tích cú pháp, Chương 5 xây dựng cây cú pháp cho các dòng lệnh và cả chương trình dựa trên việc lập bảng; Chương 6 giới thiệu về phân tích ngữ nghĩa: cú pháp trừu tượng, rút gọn cây cú pháp mà giữ nguyên ngữ nghĩa, đưa ra các phương pháp tính toán, ước lượng các hành động ngữ nghĩa; Chương 7 trình bày kiểm tra kiểu, đa hình và đa kiểu; Chương 8 nói về môi trường thực thi: Bản ghi kích hoạt của các đoạn chương trình và các lời gọi hàm, xem xét việc dịch sang mã trung gian là biểu diễn khái quát độc lập với cấu trúc máy.

T.V.Dũng và P.X.Tích

Do lần đầu biên soạn và chưa có nhiều kinh nghiệm thực tế, nên không tránh khỏi những sai sót và lỗi in ấn nhất định. Các tác giả xin vui lòng tiếp nhận mọi sự đóng góp giúp cho bài giảng “Chương trình dịch” ngày càng tốt hơn. Mọi ý kiến xây dựng xin gửi về theo địa chỉ sau: Trần Văn Dũng hoặc Phạm Xuân Tích, Khoa Công nghệ Thông tin, Đại học Giao thông Vận tải, Láng Thượng, Đống Đa, Hà nội.

Các tác giả

CHƯƠNG 1:

MỞ ĐẦU VỀ CHƯƠNG TRÌNH DỊCH

MỞ ĐẦU

Bài giảng này dựa trên cuốn sách “Cài đặt chương trình dịch hiện đại trong Java” của các tác giả Andrew W. Appel và Jens Palsberg, nhà xuất bản Cambridge University Presss, 2004.

Cuốn sách này mô tả kỹ thuật, cấu trúc dữ liệu, và thuật toán để dịch một chương trình từ một ngôn ngữ lập trình sang mã thực hiện được. Chương trình dịch hiện đại thường tổ chức chia thành nhiều pha, mỗi pha thao tác trên ‘ngôn ngữ’ trừu tượng khác nhau.

Để mô tả các vấn đề trong khi dịch một ngôn ngữ lập trình thực tế, chúng ta sẽ chỉ rõ dịch MiniJava như thế nào, đây là một tập con các lệnh không tầm thường của Java. Các bài tập lập trình trong mỗi chương để cài đặt các pha tương ứng, sinh viên mà cài đặt tất cả các pha trong cuốn sách sẽ có một chương trình dịch làm việc được. MinJava dễ dàng bổ sung để hỗ trợ việc mở rộng các lớp, các hàm bậc cao. Phụ lục A sẽ mô tả ngôn ngữ MiniJava.

Giao diện giữa các module của chương trình dịch cũng rất quan trọng như các thuật toán bên trong các module. Để mô tả giao diện đúng, sẽ rất có ích nếu ta viết nó trong ngôn ngữ lập trình thực tế. Cuốn sách này sử dụng Java - một ngôn ngữ lập trình hướng đối tượng. Java là an toàn, trong chương trình Java không có tình huống hệ thống kiểu vi phạm các khái quát hoá; và nó có thể tự giải phóng bộ nhớ, việc này làm đơn giản quản lý cấp bộ nhớ động. Cả hai tính chất này rất có ích khi viết chương trình dịch, cũng như mọi phần mềm khác.

1.1 TỔNG QUAN VỀ CHƯƠNG TRÌNH DỊCH

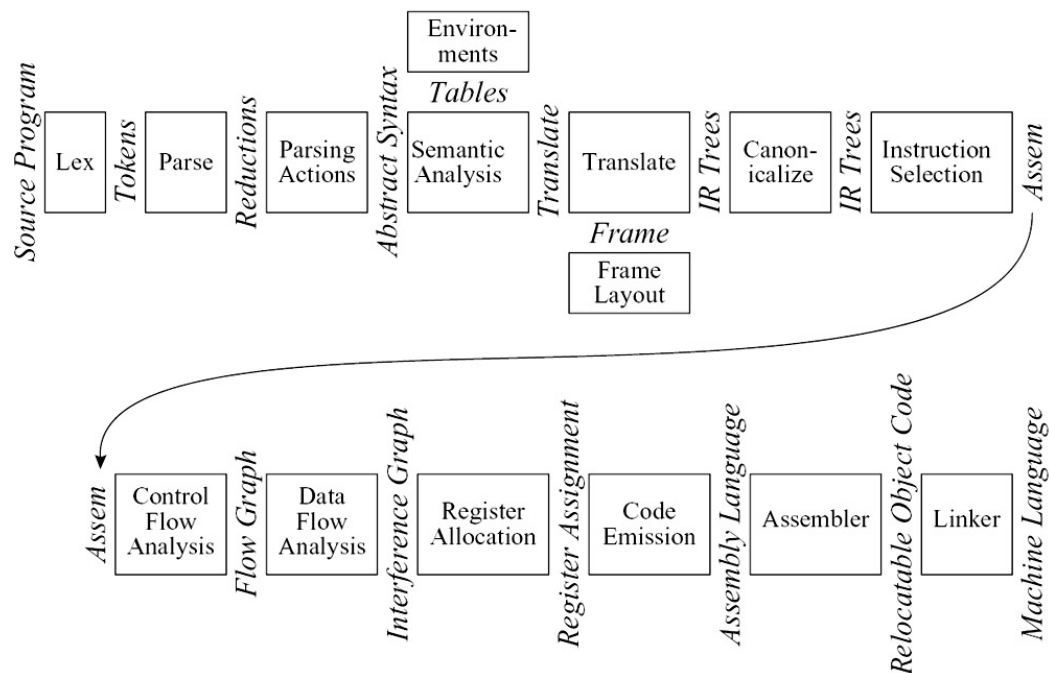
Bất cứ hệ thống phần mềm lớn sẽ dễ hiểu và dễ cài đặt, nếu người thiết kế quan tâm đến việc khái quát hoá và giao diện một cách cơ bản. Hình 1.1 nêu các pha của một chương trình dịch tiêu biểu. Mỗi pha được cài đặt bởi một hay một số module phần mềm. Chia chương trình dịch ra thành nhiều phần nhỏ cho phép tái sử dụng các thành phần. Chẳng hạn, để thay đổi máy đích mà chương trình dịch tạo ra ngôn ngữ máy, ta chỉ cần thay các module Frame Layout (Hiển thị khung) và Instruction Selection (Chọn chỉ lệnh). Để thay đổi ngôn ngữ nguồn được dịch, chỉ cần đổi module Translate (Dịch). Chương trình dịch có thể đi kèm với bộ soạn thảo hướng ngôn ngữ trong giao diện Cú pháp trừu tượng (Abstract Syntax interface).

Kinh nghiệm học để đi đến khái quát đúng thông qua một số bước lặp Suy nghĩ – Cài đặt - Thiết kế lại (think – implement – redesign) là không thể bỏ qua được. Do đó, chúng tôi trình bày dự án thực hành mà ở đó việc khái quát và giao diện đã được nghĩ kỹ, làm cho gọn và tổng quát nhất có thể được.

Một số giao diện như Cú pháp trừu tượng (Abstract Syntax), cây thể hiện trung gian IR (Intermediate Representation) và Assem, có dạng cấu trúc dữ liệu: chẳng hạn như, pha Phân tích cú pháp (Parsing Action phase) xây dựng cấu trúc dữ liệu Cú pháp Trừu tượng (Abstract Syntax) và truyền nó cho pha Phân tích ngữ nghĩa (Semantic Analysis phase). Các giao diện khác cũng là kiểu dữ liệu trừu tượng; giao diện Dịch (Translate) là tập các hàm mà Phân tích ngữ nghĩa có thể gọi, và các giao diện Từ tổ là các hàm mà Phân tích từ vựng (Parser) có thể gọi để lấy từ tổ tiếp theo trong chương trình đầu vào.

Mỗi chương của cuốn sách này mô tả một pha của chương trình dịch, như đã chỉ ra trong Bảng 1.2. Việc module hoá này là tiêu biểu cho nhiều chương trình dịch thực tế. Nhưng một số chương trình dịch gộp các pha Phân tích từ vựng, cú pháp, ngữ nghĩa, dịch và tối ưu hoá vào một pha; một cách khác đưa phần Chọn chỉ lệnh để về sau như chúng ta đã làm. Một số chương trình dịch đơn giản bỏ qua Phân tích luồng điều khiển, phân tích luồng dữ liệu, và pha đăng ký bộ nhớ. Chúng ta sẽ thiết kế chương trình dịch trong cuốn sách này đơn giản nhất có thể.

1.2 CÁC BƯỚC CHÍNH CỦA CHƯƠNG TRÌNH DỊCH



Bảng 1.2: Mô tả các pha của chương trình dịch.

Phần	Pha	Mô tả
2	Phân tích từ vựng Lex	– Tách file chương trình nguồn thành các từ riêng biệt gọi là từ tố.
3	Phân tích cú pháp Parse	- Phân tích cấu trúc của chương trình cần dịch.
4	Hành động ngữ nghĩa Semantic Actions	- Xây dựng phần cây cú pháp trừu tượng tương ứng với các pha.
5	Phân tích ngữ nghĩa Semantic Analysis	- Xác định mỗi pha có ý nghĩa gì, liên kết các biến với định nghĩa của chúng, kiểm tra kiểu của các biểu thức, yêu cầu dịch từng pha.
6	Hiển thị khung Frame Layout	– Đặt các biến, các tham số hàm số, ... vào các bản ghi kích hoạt (khung ngăn xếp) tùy theo từng loại máy.
7	Dịch – Translate	Tạo cây thể hiện trung gian (IR trees), mà không gắn chặt với một ngôn ngữ nguồn hay kiến trúc đích nào.
8	Chuẩn hoá Canonicalize	- Loại bỏ các tác động phụ của biểu thức, và xoá bỏ các nhánh điều kiện phục vụ cho các pha tiếp theo.
9	Chọn lệnh - Instruction Selection	- Nhóm các đỉnh cây IR thành thao tác mà tương ứng với hành động của lệnh máy đích.
10	Phân tích luồng điều khiển - Control Flow Analysis	- Phân tích dãy các lệnh thành đồ thị luồng điều khiển mà thể hiện mọi luồng điều khiển có thể buộc chương trình tuân theo khi thực hiện.
10	Phân tích luồng dữ liệu Dataflow Analysis	- Tập hợp mọi thông tin về các biến của chương trình; chẳng hạn, phân tích tính sống, tìm các chỗ mà ở đó chương trình lưu giá trị còn cần đến của các biến.
11	Đăng ký bộ nhớ Register Allocation	- Chọn thanh ghi lưu giá trị của mỗi biến và các giá trị trung gian sử dụng trong chương trình; các biến

Bảng 1.2: Mô tả các pha của chương trình dịch.

Phần	Pha	Mô tả
		không sống tại cùng thời điểm cần chia sẻ cùng thanh ghi.
12	Chuyển dịch code Code Emission	- Thay các tên trung gian trong mỗi lệnh máy bằng thanh ghi của máy.

Trong giáo trình này chúng ta chỉ trình bày những phần cơ bản nhất của chương trình dịch, nội dung chủ yếu đến phần 7 trong bảng nêu trên.

CÔNG CỤ VÀ PHẦN MỀM

Hai trong số các trừu tượng được sử dụng rộng rãi nhất trong chương trình dịch hiện đại là văn phạm phi ngữ cảnh để phân tích cú pháp và biểu thức chính qui để phân tích từ vựng. Để sử dụng các trừu tượng này một cách tốt nhất ta có công cụ chuyên dụng như Yacc (mà chuyển văn phạm thành chương trình phân tích cú pháp) và Lex (mà chuyển đặc tả mô tả thành chương trình phân tích cú pháp). May mắn là chương trình này luôn có đối với Java.

Các bộ sinh Phân tích cú pháp JavaCC và SableCC là miễn phí trên Internet. Có thể tham khảo các trang Web sau:

<http://uk.cambridge.org/resources/052182060X> (bên ngoài Bắc Mỹ);

<http://us.cambridge.org/titles/052182060X.html> (bên trong Bắc Mỹ).

1.3 THÔNG DỊCH VÀ BIÊN DỊCH

Trình thông dịch – Interpreter

Trình thông dịch là một trình để thông dịch ngôn ngữ, tức là dịch từng câu lệnh một. Trong thông dịch mã nguồn của cả chương trình không được dịch trước thành ngôn ngữ máy, mà khi chạy chương trình mã nguồn mới được dịch và thực thi từng dòng lệnh một.

Tất cả các ngôn ngữ không biên dịch ra mã máy một lần duy nhất đều phải sử dụng trình thông dịch (PHP, WScripts, Perl, Linux Shell, Python....). Các ngôn ngữ theo trình thông

dịch thường được gọi là kịch bản (script).

Như vậy chương trình viết bằng ngôn ngữ kịch bản phải có một trình thông dịch kèm theo khi chạy chương trình.

Ưu điểm:

- Phát triển nhanh chóng
- Có thể chỉnh sửa mã nguồn bất kỳ khi nào
- Mạnh xử lý cú pháp
- Uyển chuyển mềm dẻo.
- Có thể chạy trên mọi nền tảng (Platform, Hệ điều hành - HĐH) nếu có trình thông dịch tương ứng, tại vì không phải là ngôn ngữ máy (chỉ là file văn bản) nên không bị phụ thuộc vào HĐH. tiêu biểu là Perl, PHP, Python.

Nhược điểm:

- Vì là ngôn ngữ thông dịch chạy từng dòng, nên ngôn ngữ thông dịch không hỗ trợ đa luồng (multi thread) và giao dịch (transaction).
- Cũng do chạy từng dòng, nên tốc độ thực thi không nhanh bằng các chương trình viết bằng ngôn ngữ biên dịch (C, C++, VB...) đã chuyển trực tiếp ra ngôn ngữ máy.

Trình biên dịch - Compiler

Trình biên dịch sẽ dịch cả mã nguồn thành mã máy trên một HĐH xác định và chỉ chạy trên HĐH đó, do đó các chương trình được biên dịch sẽ phụ thuộc nhiều vào nền tảng và HĐH.

Ưu điểm:

- Ràng buộc chặt chẽ về kiểu trong ngôn ngữ.
- Hỗ trợ các tính năng đa luồng, transtion ...
- Do đã biên dịch phụ thuộc vào hệ điều hành nên chương trình có thể tận dụng toàn bộ các tính năng đặc trưng của HĐH
- Tốc độ thực thi tốt
- Bảo mật tốt (không thể xâm phạm mã nguồn làm thay đổi chức năng của chương trình)

Nhược điểm:

- Sau khi biên dịch ra ngôn ngữ máy thì chỉ có thể chạy trên một HĐH xác định.

Bảng so sánh

Biên dịch – Compiler	Thông dịch - Interpreter
- Chương trình được biên dịch ra thành ngôn ngữ máy trên một HĐH xác định trước và chỉ chạy trên HĐH	- Mã nguồn không được dịch trước thành ngôn ngữ máy mà mỗi lần cần chạy chương trình thì mã nguồn mới được dịch để thực thi từng dòng một.

đó (C++ -> .exe chỉ chạy trên Win, C++ -> .o chạy trên Unix/Linux)	Tất cả các ngôn ngữ không biên dịch ra mã máy điều phải sử dụng trình thông dịch, thường được gọi là ngôn ngữ kịch bản.
- Ưu điểm của trình biên dịch là chương trình được tối ưu tốt cho HĐH và kiến trúc phần cứng ngay lúc dịch sang mã máy.	- Trình thông dịch thì có ưu điểm là có thể chạy trên nhiều HĐH và kiến trúc máy tính khác nhau, miễn là có bộ thông dịch tương ứng trên HĐH.
- Tuy quá trình này tốn thời gian, nhưng chỉ thực hiện có 1 lần mà thôi.	- Trình thông dịch thì mỗi lần chạy sẽ chuyển chương trình sang mã máy, mỗi lần dịch thì thời gian tốn ít thôi, nhưng bù lại có thể lần nào chạy cũng phải dịch (trừ khi bộ thông dịch cache lại kết quả của lần dịch trước đó).
- Trình biên dịch tạo ra file executable lúc này đã là mã máy, nên trên đĩa nó bao nhiêu memory thì sẽ load lên bấy nhiêu.	- Trình thông dịch thì trên memory còn có bộ thông dịch, và bộ thông dịch phải load chương trình nguồn lên rồi dịch thành mã máy...cho nên thường quá trình chạy một chương trình thông dịch sẽ tốn memory hơn.
Java cũng là ngôn ngữ biên dịch mà có thể chạy mọi HĐH bởi vì Java không biên dịch ra ngôn ngữ máy mà biên dịch ra bytecode, và bytecode đó phải chạy trên JVM (Java Virtual Machine) là một máy ảo.	

VÍ DỤ: CẤU TRÚC DỮ LIỆU CHO NGÔN NGỮ CÂY

Nhiều cấu trúc dữ liệu được dùng trong chương trình dịch làm thể hiện trung gian cho chương trình được dịch. Thông thường các thể hiện này có dạng cây, với một số kiểu đỉnh, mỗi đỉnh có các thuộc tính khác nhau. Các cây như vậy xuất hiện trong nhiều giao diện của các pha trong Hình 1.1.

Cây thể hiện có thể được mô tả bằng văn phạm giống như ngôn ngữ lập trình. Để giới thiệu khái niệm này, chúng ta sẽ đưa ra một ngôn ngữ lập trình đơn giản với các lệnh và biểu thức nhưng không có vòng lặp và lệnh điều kiện if (được gọi là chương trình dòng trực tiếp).

Cú pháp cho ngôn ngữ này cho trong Văn phạm 1.3.

VĂN PHẠM (GRAMMAR) 1.3: Ngôn ngữ chương trình dòng trực tiếp.

$Stm \rightarrow Stm; Stm$ (CompoundStm)

$Stm \rightarrow id := Exp$ (AssignSt)

$Stm \rightarrow \text{print } (ExpList)$	(PrintStm)
$Exp \rightarrow \text{id}$	(IdExp)
$Exp \rightarrow \text{num}$	(NumExp)
$Exp \rightarrow Exp \text{ Binop } Exp$	(OpExp)
$Exp \rightarrow (Stm, Exp)$	(EseqExp)
$ExpList \rightarrow Exp, ExpList$	(PairExpLi)
$ExpList \rightarrow Exp$	(LastExpLi)
$Binop \rightarrow +$	(Plus)
$Binop \rightarrow -$	(Minus)
$Binop \rightarrow \times$	(Times)
$Binop \rightarrow /$	(Div)

Ngữ nghĩa không hình thức của ngôn ngữ trên như sau. Mỗi Stm là một lệnh, mỗi Exp là một biểu thức, $s_1; s_2$ là thực hiện lệnh s_1 , sau đó đến lệnh s_2 . Lệnh gán $i := e$ là tính toán giá trị biểu thức e , sau đó lưu kết quả vào biến i . Và lệnh $\text{print}(e_1, e_2, \dots, e_n)$ hiện giá trị của tất cả biểu thức trong ngoặc từ trái sang phải, cách nhau khoảng trống và kết thúc bằng việc xuống dòng mới.

Biểu thức định danh, như i , chứa nội dung hiện thời của biến i . Một số ước lượng giá trị nguyên của biến. Biểu thức toán tử $e_1 \text{ op } e_2$ tính giá trị e_1 , sau đó e_2 , rồi áp dụng toán tử hai ngôi. Dãy biểu thức (s, e) giống như dấu phẩy trong ngôn ngữ C, tính toán giá trị của s cho tác động phụ trước khi tính và trả về kết quả của biểu thức e .

Chẳng hạn thực hiện chương trình sau:

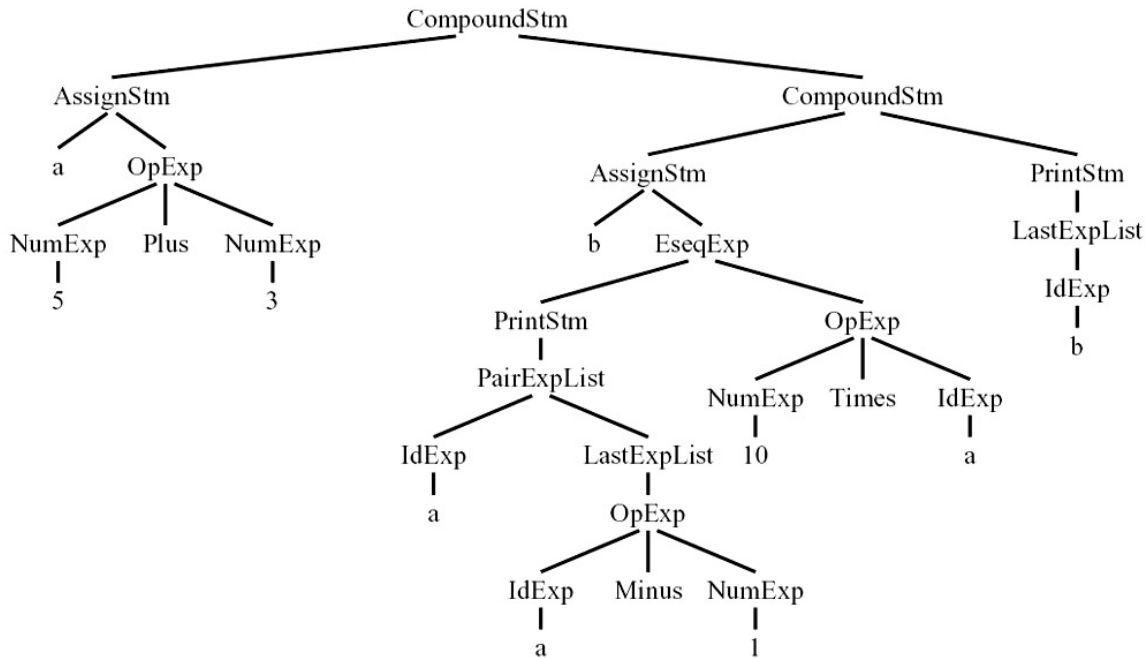
```
a := 5+3; b := (print(a, a-1), 10*a); print(b)
```

sẽ in ra kết quả

```
8 7
```

```
8 0
```

Chương trình này được thể hiện như thế nào bên trong chương trình dịch. Thể hiện trên là chương trình nguồn, với các ký tự mà lập trình viên viết. Nhưng nó không dễ dàng được thực hiện. Một thể hiện tốt hơn là cấu trúc dữ liệu cây, với mỗi đỉnh cho một lệnh (Stm) hoặc một biểu thức (Exp). Hình 1.4 thể hiện cây biểu diễn của chương trình đó, mỗi đỉnh được sinh ra bởi luật trong Văn phạm 1.3 và mỗi đỉnh có nhiều con ứng với các ký hiệu ở vế phải của luật sinh tương ứng với đỉnh đó.



`a := 5 + 3 ; b := ( print ( a , a - 1 ) , 10 * a ) ; print ( b )`

Hình 1.4: Biểu diễn cây của chương trình dòng lệnh trực tiếp

Chúng ta có thể dịch trực tiếp văn phạm thành các định nghĩa cấu trúc dữ liệu như trong Chương trình 1.5. Mỗi ký hiệu của văn phạm tương ứng với một lớp trừu tượng (abstract class) trong cấu trúc dữ liệu:

Lớp Văn phạm - Grammar Class

<i>Stm</i>	Stm
<i>Exp</i>	Exp
<i>ExpList</i>	ExpList
<i>id</i>	String
<i>num</i>	Int

CHƯƠNG TRÌNH 1.5: Thể hiện của chương trình dòng trực tiếp.

```
public abstract class Stm {}

public class CompoundStm extends Stm {
    public Stm stm1, stm2;
    public CompoundStm(Stm s1, Stm s2) {stm1=s1; stm2=s2;}}
```

```
public class AssignStm extends Stm {
    public String id; public Exp exp;
    public AssignStm(String i, Exp e) {id=i; exp=e;}}

public class PrintStm extends Stm {
    public ExpList exps;
    public PrintStm(ExpList e) {exps=e;}}

public abstract class Exp {}

public class IdExp extends Exp {
    public String id;
    public IdExp(String i) {id=i;}}

public class NumExp extends Exp {
    public int num;
    public NumExp(int n) {num=n;}}

public class OpExp extends Exp {
    public Exp left, right; public int oper;
    final public static int Plus=1, Minus=2, Times=3, Div=4;
    public OpExp(Exp l, int o, Exp r) {left=l; oper=o; right=r;}}

public class EseqExp extends Exp {
    public Stm stm; public Exp exp;
    public EseqExp(Stm s, Exp e) {stm=s; exp=e;}}

public abstract class ExpList {}

public class PairExpList extends ExpList {
    public Exp head; public ExpList tail;
    public PairExpList(Exp h, ExpList t) {head=h; tail=t;}}

public class LastExpList extends ExpList {
    public Exp head;
    public LastExpList(Exp h) {head=h;}}
```

Đối với mỗi luật văn phạm có một hàm tạo (constructor) thuộc vào lớp cho ký hiệu ở vế trái. Chúng ta đơn giản mở rộng lớp trừu tượng với lớp cụ thể cho mỗi luật của văn phạm. Tên của hàm tạo (lớp) được chỉ ra ở vế phải của Văn phạm 1.3. Mỗi luật văn phạm có các thành phần ở vế phải, mà cần được thể hiện trong cấu trúc dữ liệu. Lệnh hợp *CompoundStm* có hai lệnh đơn *Stm* ở vế phải. Lệnh gán *AssignStm* có tên và biểu thức, ... Những cái này trở thành các trường của các lớp con trong cấu trúc dữ liệu Java. Như vậy *CompoundStm* có hai trường (còn được gọi là các biến thể hiện) *Stm1* và *Stm2*; Lệnh gán *AssignStm* có trường *Id* và *exp*. Đối với phép toán hai ngôi *Binop* ta có thể làm điều gì đó đơn giản hơn. Mặc dù ta có thể tạo thành class *Binop* với các subclass *Plus*, *Minus*, *Times*, *Div* - điều này làm số lượng tăng quá nhiều, vì không lớp con nào cần trường nào cả. Thay vào đó chúng ta sẽ tạo ra kiểu số (enumeration type – trong Java là *Integer*) của các hằng số (là các biến *final int*) cục bộ đối với lớp *OpExp*.

Phong cách lập trình. Chúng ta sẽ tuân theo một số qui định để thể hiện cây cấu trúc trong Java:

- Cây được mô tả bằng văn phạm
- Cây được mô tả bằng một hay nhiều lớp mà mỗi lớp tương ứng với một ký hiệu trong văn phạm.
- Mỗi lớp trừu tượng được mở rộng bởi một hay một số lớp con, mỗi lớp cho một qui tắc văn phạm.
- Đối với mỗi ký hiệu không tầm thường ở về phải một luật sinh, có một trường trong lớp tương ứng. (Ký hiệu tầm thường là các ký hiệu ngăn cách như dấu chấm phẩy trong lệnh *CompoundStmt*).
- Mỗi lớp có một hàm tạo gán giá trị ban đầu cho mọi trường của nó.
- Các cấu trúc dữ liệu được gán giá trị ban đầu khi chúng khởi tạo (bằng các hàm tạo) và không bao giờ bị thay đổi sau đó (trừ khi chúng đột nhiên biến mất).

Các nguyên tắc lắp ráp được (*modularity*) của các chương trình Java:

- Mỗi pha hay module của chương trình dịch thuộc vào gói của nó.
- Khai báo nhập theo yêu cầu “*Import on demand*” với dấu * không được sử dụng, mà phải khai báo cụ thể. Nếu file Java được bắt đầu như sau:
`import A.F.*; import A.G.*; import B.*; import C.*;`
thì người đọc buộc phải nhìn ra bên ngoài file này xem các gói nào định nghĩa *X* mà được sử dụng trong biểu thức *X.put()*;
- Các khai báo kiểu đơn “*Single-type import*” là giải pháp tốt hơn. Nếu module bắt đầu:
`import A.F.W; import A.G.X; import B.Y; import C.Z;`
thì bạn không cần nhìn ra ngoài file này có thể nói *X* đến từ gói *A.G*.
- Java bản chất là hệ thống đa luồng (*Multi-thread*). Chúng ta mong muốn hỗ trợ chương trình dịch đồng thời đa luồng và dịch hai chương trình đồng thời, mỗi chương trình trong một luồng. Như vậy, các biến tĩnh sẽ tránh dùng trừ khi chúng là các hằng không thay đổi (*final*). Chúng ta không muốn hai luồng của chương trình dịch cùng cập nhật giá trị của cùng một biến (*static*).

THÔNG DỊCH CHƯƠNG TRÌNH DÒNG TRỰC TIẾP

Cài đặt chương trình phân tích và chương trình dịch cho chương trình dòng trực tiếp. Bài tập này như một phần để giới thiệu môi trường (bảng ký hiệu ánh xạ tên biến vào thông tin về các biến), cú pháp trừu tượng (cấu trúc dữ liệu biểu diễn cấu trúc câu của chương trình), cấu trúc đệ quy trên cây, sử dụng trong nhiều phần của chương trình dịch và phong cách lập trình hàm mà không có lệnh gán.

Nó được coi như bài tập ôn lại lập trình Java. Các lập trình viên có kinh nghiệm làm trên các ngôn ngữ khác cũng có thể làm được các bài tập này, nhưng cần tài liệu bổ sung (như giáo trình Java) để hỗ trợ.

Chương trình để biên dịch đã sẵn sàng chuyển sang cú pháp trừu tượng như mô tả bằng kiểu dữ liệu trong Chương trình 1.5. Tuy nhiên chúng ta chưa muốn quan tâm đến phân tích ngôn ngữ, nên chúng ta viết chương trình này bằng cách dùng các hàm tạo dữ liệu.

```

Stm prog =
  new CompoundStm(new AssignStm("a",
                                new OpExp(new NumExp(5),
                                             OpExp.Plus, new NumExp(3))),
                  new CompoundStm(new AssignStm("b",
                                                  new EseqExp(new PrintStm(new PairExpList(new IdExp("a"),
                                                                 new LastExpList(new OpExp(new IdExp("a"),
                                                                 OpExp.Minus, new NumExp(1))))),
                                                  new OpExp(new NumExp(10), OpExp.Times,
                                                            new IdExp("a")))),
                                  new PrintStm(new LastExpList(new IdExp("b")))));

```

Viết chương trình dịch không có các tác động phụ (như lệnh gán cập nhật biến và cấu trúc dữ liệu) là phần giới thiệu tốt cho văn phạm ngữ nghĩa và văn phạm thuộc tính là phương pháp mô tả chương trình làm gì. Nó cũng là kỹ thuật có ích dùng để viết chương trình dịch, chương trình dịch cũng cho biết ngôn ngữ lập trình làm gì. Như vậy trong cài đặt các chương trình này, ta không gán giá trị mới cho biến hoặc trường của đối tượng nào trừ khi khởi tạo chúng. Đối với các biến cục bộ, ta sử dụng dạng khởi tạo khi khai báo (chẳng hạn `int i = j+3;`), còn đối với mỗi lớp, dùng hàm tạo (giống như hàm tạo của *CompoundStm* trong Chương trình 1.5).

1. Viết hàm Java `int maxags(Stm s)` mà cho biết số các đối số là lớn nhất trong lệnh in bất kỳ bên trong biểu thức con nào đó của lệnh đã cho. Chẳng hạn `maxags(prog)` bằng 2.
2. Viết hàm Java `void interp(Stm s)` mà thông dịch một chương trình trong ngôn ngữ này. Để viết trong phong cách lập trình hàm - bạn không khi nào dùng lệnh gán, khởi tạo mỗi biến cục bộ khi bạn khai báo nó.

Các hàm của bạn mà kiểm tra một biểu thức *Exp* sẽ sử dụng *instanceof* để xác định lớp con nào mà biểu thức thuộc và sau đó chuyển đến lớp con đó. Hoặc bạn muốn bổ sung các phương thức cho các lớp *Exp* và *Stm* để tránh dùng *instanceof*.

Đối với **phần 1**, các lệnh in có thể chứa các biểu thức có các lệnh in khác.

Đối với **phần 2**, tạo các hàm đệ qui hai biến *interpStm* và *interpExp*. Thể hiện bằng table như danh sách các cặp `id × int`, ánh xạ các định danh vào các giá trị nguyên gán cho chúng.

```

class Table {
    String id; int value; Table tail;
    Table(String i, int v, Table t) {id=i; value=v; tail=t;}
}

```

Khi đó *interpStm* được khai báo như

```
Table interpStm(Stm s, Table t)
```

lấy bảng t_1 như biến và tạo ra bảng mới t_2 mà giống như t_1 ngoại trừ một số định danh ánh xạ đến các số nguyên khác nhau như kết quả của lệnh đó.

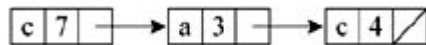
Chẳng hạn, bảng t_1 ánh xạ a vào 3 và c vào 4, mà chúng ta viết ký hiệu toán dạng $\{a \rightarrow 3, c \rightarrow 4\}$, có thể thể hiện như danh sách kết nối 

Bây giờ, giả sử bảng t_2 giống bảng t_1 , ngoại trừ ánh xạ c vào 7 thay vì 4. Về mặt toán học ta có thể viết

$$t_2 = \text{update}(t_1, c, 7)$$

Ở đây hàm cập nhật *update* trả về bảng mới $\{a \rightarrow 3, c \rightarrow 7\}$.

Trong máy tính chúng ta coi như cài đặt t_2 bằng cách đặt ô mới ở đầu danh sách kết nối



Và giả thiết sự xuất hiện đầu tiên của c trong danh sách này sẽ ưu tiên hơn mọi xuất hiện khác của c trong danh sách đó.

Như vậy hàm cập nhật dễ cài đặt; và hàm tìm kiếm *lookup* tương ứng

```
int lookup(Table t, String key)
```

tìm ngay trong danh sách kết nối. Tất nhiên trong phong cách lập trình hướng đối tượng `int lookup(Table t, String key)` là một phương thức của lớp bảng (*Table class*).

Dịch biểu thức là phức tạp hơn dịch lệnh, vì biểu thức trả về giá trị nguyên và có các tác động phụ. Chúng ta mong muốn mô phỏng lệnh gán của ngôn ngữ lập trình dòng trực tiếp mà không tạo nên tác động phụ nào trong chương trình dịch. (Tuy nhiên lệnh `in print` vẫn kèm theo các tác động phụ của chương trình dịch). Lời giải là khai báo *interpExp* như sau:

```
class IntAndTable {int i; Table t;
    IntAndTable(int ii, Table tt) {i=ii; t=tt;}
}
IntAndTable interpExp(Exp e, Table t) ...
```

Kết quả dịch của biểu thức e_1 với bảng t_1 là số nguyên i và bảng mới t_2 . Khi dịch biểu thức với hai biểu thức con (như trong *OpExp*) bảng t_2 nhận kết quả từ biểu thức con thứ nhất có thể được sử dụng để xử lý biểu thức con thứ hai.

BÀI TẬP

1.1 Tối ưu hóa đoạn chương trình sau:

```
int sumcalc(int a, int b, int N)
{
    int i;
    int x, y;
    x = 0;
    y = 0;
```

```
    for (i=0; i <= N, i++){
        x = x + (4*a/b)*i + (i+1)*(i+1);
        x = x + b*y;
    }
    return x;
}
```

1.2 Chương trình đơn giản sau cài đặt cây tìm kiếm nhị phân dạng hàm thường trực, như vậy, nếu `tree2=insert(x,tree1)`, thì cây `tree1` vẫn sẵn sàng cho phép được tìm kiếm, ngay cả khi cây `tree2` có thể đang được sử dụng.

```
class Tree {Tree left; String key; Tree right;
    Tree(Tree l, String k, Tree r) {left=l; key=k; right=r;}

Tree insert(String key, Tree t) {
    if (t==null) return new Tree(null, key, null)
    else if (key.compareTo(t.key) < 0)
        return new Tree(insert(key,t.left),t.key,t.right);
    else if (key.compareTo(t.key) > 0)
        return new Tree(t.left,t.key,insert(key,t.right));
    else return new Tree(t.left,key,t.right);
}
```

a. Cài đặt hàm thành viên `member` mà trả về `true`, nếu *item* được tìm thấy, ngược lại `false`.

b. Mở rộng chương trình trên để không chỉ bao gồm thành viên mà còn ánh xạ khoá vào các trói buộc:

```
Tree insert(String key, Object binding, Tree t);
Object lookup(String key, Tree t);
```

c. Các cây này không cân bằng (*not balanced*); mô tả hành vi của hai dãy chèn sau:

- a. t s p i p f b s t
- b. a b c d e f g h i

*d. Tìm hiểu cây tìm kiếm cân bằng trong Sedgewick [1997] và đề xuất cấu trúc dữ liệu cây cân bằng cho bảng ký hiệu dạng hàm. Gợi ý: để giữ phong cách lập trình hàm, thuật toán cần phải cân bằng lại *khi* chèn, không phải khi tìm kiếm, vì vậy cấu trúc dữ liệu như cây trái rộng không còn thích hợp nữa.

e. Viết theo phong cách hướng đối tượng (không phải phong cách lập trình hàm), sao cho lệnh chèn bây giờ là `t.insert(key)` thay vì `insert(key,t)`. Gợi ý: bạn cần lớp con cây rỗng `EmptyTree`.

CHƯƠNG 2:

THIẾT KẾ NGÔN NGỮ LẬP TRÌNH

TỔNG QUAN

Ngôn ngữ lập trình là một hệ thống được ký hiệu hóa để miêu tả những tính toán (qua máy tính) trong một dạng mà cả con người và máy đều có thể đọc và hiểu được.

Theo định nghĩa ở trên thì một ngôn ngữ lập trình phải thỏa mãn được hai điều kiện cơ bản sau:

- Dễ hiểu và dễ sử dụng đối với người lập trình, để có thể dùng để giải quyết nhiều bài toán khác nhau.
- Miêu tả một cách đầy đủ và rõ ràng các tiến trình, để chạy được trên các hệ máy tính khác nhau.

Một tập hợp các chỉ thị được biểu thị qua ngôn ngữ lập trình nhằm mục đích thực hiện các thao tác máy tính nào đó được gọi là một chương trình. Khái niệm này còn có những tên khác như chương trình máy tính.

2.1 LỊCH SỬ NGÔN NGỮ LẬP TRÌNH

ASSEMBLY

Ngôn ngữ Assembly là một bước tiến vượt ra khỏi với ngôn ngữ máy khó hiểu. Ngôn ngữ này xuất hiện vào những năm 1950, nó được thiết kế để máy tính trở nên thân thiện hơn với người sử dụng. Assembly đưa ra khái niệm biến (variable), nhờ đó mà ta có thể gán một ký hiệu cho một vị trí nào đó trong bộ nhớ mà không phải viết lại địa chỉ này dưới dạng nhị phân mỗi lần sử dụng. Assembly cũng chứa vài "phép toán giả", tức là ta có thể biểu diễn mã phép toán dưới dạng lệnh (statement) thay vì dưới dạng nhị phân. Sau khi viết một chương trình Assembly, lập trình viên phải chạy nó thông qua Assembler. Assembler là một chương trình biên dịch (compiler), nó dịch chương trình Assembly sang ngôn ngữ máy.

FORTRAN

Fortran là ngôn ngữ lập trình cấp cao cổ nhất, nó được phát triển vào những năm 1950. Fortran là tên viết tắt của Formula Translator. Nó có rất nhiều phiên bản khác nhau. Fortran IV được lấy làm chuẩn vào năm 1966. Fortran 77 sau đó có thêm nhiều đặc điểm mới hơn. Gần đây Fortran 90 đã được phát triển. Ngoài các chuẩn Fortran kể trên còn có một số "dị bản" khác, ví dụ như F là pha trộn của Fortran với ngôn ngữ hướng module,

hay High Performance Fortran (HPF) dùng cho các cluster. Fortran chẳng có gì đặc sắc lắm. Nó không được sử dụng nhiều ngoại trừ trong giới khoa học, nơi cần thực hiện nhiều tính toán toán học. Fortran có logic lập trình hay nên nó cũng được sử dụng để dạy về lập trình trong trường học.

COBOL

Cobol là tên viết tắt của Common Business Oriented Language. Nó được phát triển cuối những năm 1950 và ngôn ngữ cấp cao cổ thứ nhì (sau Fortran). Cobol hướng vào giới thương mại. Nó là một ngôn ngữ rất dài dòng. Chương trình Cobol thường dài hơn bất kỳ chương trình viết bằng ngôn ngữ nào khác. Dù vậy, nó là một ngôn ngữ tốt và dễ đọc. Dù cổ nhưng Cobol vẫn được sử dụng phổ biến trong thế giới thương mại.

PASCAL

Đây là ngôn ngữ do giáo sư Niklaus Wirth thiết kế vào năm 1970 với mục đích giảng dạy ý niệm lập trình có cấu trúc. Nhưng sau một thời gian do tính ưu việt của nó nên Pascal đã được sử dụng rộng rãi.

Pascal là ngôn ngữ lập trình bậc cao. Trước khi Pascal được phát triển thì việc lập trình được thực hiện trên các ngôn ngữ cấp thấp, các lập trình viên rất khó khăn trong việc xây dựng các chương trình lớn. Pascal dùng ngôn ngữ sát với ngôn ngữ tự nhiên hơn do đó nó thân thiện với người lập trình hơn. Do vậy nó giảm bớt các công việc nặng nhọc cho người lập trình.

Pascal kết hợp giữa đặc tính gọn, dễ nhớ, khả năng truy cập cấp thấp, và các cấu trúc giữ liệu đa dạng. Pascal còn hỗ trợ khả năng đưa các chương trình viết bằng ASSEMBLER vào chương trình của bạn, khả năng đồ hoạ và hướng đối tượng.

Pascal là ngôn ngữ lập trình có cấu trúc. Tính cấu trúc của Pascal được thể hiện qua 3 yếu tố: cấu trúc trong dữ liệu, cấu trúc trong các toán tử và cấu trúc trong công cụ thủ tục.

- Tính cấu trúc của dữ liệu được thể hiện qua phần mô tả. Cũng như các ngôn ngữ lập trình khác, Pascal có một số kiểu dữ liệu được định nghĩa sẵn và các phép toán trên các kiểu dữ liệu này. Từ các kiểu dữ liệu đó, người lập trình có thể xây dựng các kiểu dữ liệu phức tạp hơn. Sau đó để khai báo đối tượng thuộc kiểu dữ liệu phức tạp đó ta không cần trình bày lại cấu trúc thiết lập, mà chỉ cần tham chiếu đến kiểu đó.

- Tính cấu trúc của các toán tử được thể hiện ở chỗ bên trong các toán tử thực hiện một động tác, còn có các toán tử thực hiện nhiều động tác, song sự quan trọng nhất của Pascal là toán tử hợp thành. Toán tử hợp thành được xây dựng bắt đầu bằng từ khoá BEGIN, sau đó đến đây các toán tử thành phần và kết thúc bằng từ khoá END.

- Tính cấu trúc trong công cụ thủ tục thể hiện thông qua khả năng phân tích chương trình thành các modul độc lập và lời gọi đệ quy thủ tục.

Pascal không phải là ngôn ngữ khó học hơn ngôn ngữ dành cho những người mới bắt đầu làm quen với lập trình (BASIC) nhưng nó lại tỏ ra có những đặc tính cấu trúc hoá tốt hơn và không có những cú pháp mang lỗi.

Pascal không phân biệt chữ hoa và chữ thường, do vậy người lập trình có thể thoải mái hơn trong việc viết các câu lệnh và đặt tên cho các đối tượng của mình.

Việc dịch một chương trình viết bằng Pascal được thực hiện bằng một trình biên dịch. Khi chương trình chứa một lỗi cú pháp nào đó thì mã máy sẽ không được sinh ra. Còn nếu không có lỗi thì sau khi dịch xong sẽ nhận được một bản mã đối tượng.

Với những chương trình lớn thì dùng ngôn ngữ lập trình có cấu trúc để quản lý sẽ là rất khó khăn, vì vậy để đáp ứng nhu cầu của người lập trình những phiên bản của Pascal về sau này đã có hỗ trợ lập trình hướng đối tượng.

Do mục đích ban đầu của Pascal và các đặc điểm của nó, Pascal rất thích hợp dùng để giảng dạy trong các nhà trường và cho những người mới bắt đầu học lập trình. Còn đối với những bài toán ứng dụng trong thực tế thì Pascal ít được sử dụng.

NGÔN NGỮ C

Ngôn ngữ C được phát triển từ ngôn ngữ B trên máy UNIX. Đến nay ANSI ban hành chuẩn về C.

Cũng giống như PASCAL, C là ngôn ngữ lập trình có cấu trúc. Nhưng nói chặt chẽ về mặt kỹ thuật thì C không phải là ngôn ngữ lập trình có cấu trúc chính cống vì trong C không cho phép các khối giống nhau (chẳng hạn bạn không thể khai báo hàm này trong hàm khác).

C là ngôn ngữ cấp trung vì nó cho phép thao tác trên các bit, byte, và địa chỉ. C kết hợp các yếu tố mềm dẻo của ngôn ngữ bậc cao và khả năng điều khiển mạnh của ASSEMBLER. Do vậy, C tỏ ra thích hợp với lập trình hệ thống.

Chương trình viết bằng C là tập hợp các hàm riêng biệt, giúp cho việc che giấu mã và dữ liệu trở nên dễ dàng. Hàm được viết bởi những người lập trình khác nhau không ảnh hưởng đến nhau và có thể được biên dịch riêng biệt trước khi ráp nối thành chương trình. So với PASCAL thì C thoáng hơn, chẳng hạn C không kiểm tra kiểu khi chạy, điều này do người lập trình đảm nhiệm.

C tỏ ra ít gán bó hơn so với các ngôn ngữ bậc cao, nhưng C lại thực tế hơn so với các ngôn ngữ khác.

Một đặc điểm nổi bật của C là C có tính tương thích cao. Chương trình viết bằng C cho một loại máy hoặc hệ điều hành này có thể chuyển dễ dàng sang loại máy hoặc hệ điều hành khác. Hiện nay hầu hết các loại máy tính đều có trình biên dịch C.

Một chương trình được viết bằng C sẽ dễ tối ưu, chạy với tốc độ cao và tiết kiệm bộ nhớ. Tuy vậy, C chỉ thích hợp với những chương trình hệ thống hoặc những chương trình đòi hỏi tốc độ. Còn với những bài toán lớn và phức tạp thì cũng như PASCAL, C rất khó kiểm soát chương trình.

NGÔN NGỮ C++

Ngôn ngữ C++ được phát triển từ ngôn ngữ C. Có thể coi C++ là một cải tiến của C. Nó mang đầy đủ các đặc tính của C. Một chương trình viết bằng C có thể biên dịch được bằng trình biên dịch của C++. Ở những phiên bản đầu tiên, thực chất mã nguồn C++ đầu tiên được dịch sang mã nguồn C, rồi từ mã nguồn C tiếp tục được biên dịch tiếp.

C++ là ngôn ngữ lập trình hướng đối tượng, do vậy nó có đầy đủ các tính chất của một ngôn ngữ lập trình hướng đối tượng: tính kế thừa, tính đóng kín và tính đa dạng.

Nhưng C++ không phải là ngôn ngữ hướng đối tượng hoàn toàn mà là ngôn ngữ "đa hướng". Vì C++ hỗ trợ cả lập trình hướng mệnh lệnh và lập trình hướng đối tượng. C++ kết hợp những ý tưởng hay nhất của lập trình có cấu trúc với việc phân chia một bài toán thành các nhóm nhỏ có quan hệ với nhau, mỗi nhóm con đó là một đối tượng chứa các lệnh và dữ liệu riêng của nó.

C++ đưa vào các khái niệm hàm ảo, tải đè hàm, tải đè toán tử cho phép tạo ra các kiểu dữ liệu trừu tượng, hỗ trợ thừa kế bội.

Mục tiêu của C++ là tiếp cận những ý tưởng của phương pháp luận hướng đối tượng và trừu tượng dữ liệu.

Các đặc tính của C++ cho phép người lập trình xây dựng những thư viện phần mềm có chất lượng cao phục vụ những đề án lớn.

C++ là ngôn ngữ thích hợp cho việc xây dựng những chương trình lớn như các hệ soạn thảo, chương trình dịch, các hệ quản trị cơ sở dữ liệu, các hệ truyền thông, ...

JAVA

JAVA được tạo ra trước năm 1990 bởi nhóm các nhà phát triển của Sun Microsystem có nhiệm vụ phải viết phần mềm hệ thống để nhúng vào các sản phẩm điện tử của khách hàng. Họ đã khắc phục một số hạn chế của C++ để tạo ra ngôn ngữ lập trình JAVA.

Do được phát triển từ C++ nên JAVA rất giống C++. Nhưng JAVA là ngôn ngữ hướng đối tượng hoàn toàn, còn C++ là ngôn ngữ đa hướng.

JAVA là ngôn ngữ lập trình mạnh vì nó hội tụ được các yếu tố sau:

- JAVA là ngôn ngữ hướng đối tượng (object oriented programming): Các ngôn ngữ lập trình hướng đối tượng có các modul có thể thay đổi và được xác định trước mà người lập trình có thể gọi ra để thực hiện những nhiệm vụ cụ thể. Trong JAVA các modul này gọi là các lớp (class) và chúng được lưu trữ trong thư viện lớp tạo nên cơ sở của bộ công cụ phát triển JAVA (Java Development Kit). Trong JAVA tất cả các hàm và biến đều phải là thành phần của một lớp.

- Đơn giản (simple): Mặc dù dựa trên cơ sở của C++ nhưng JAVA đã được lược bỏ các tính năng khó nhất của C++ làm cho ngôn ngữ này dễ dùng hơn. Do vậy việc đào tạo một lập trình viên JAVA ngắn hơn và JAVA trở nên thân thiện với người sử dụng hơn. Trong JAVA không có các con trỏ, không hỗ trợ toán tử Overloading, không có tiền xử lý. Tất cả mọi đối tượng trong một chương trình JAVA đều được tạo trên heap bằng toán tử new - chúng không bao giờ được tạo trên stack. JAVA cũng là ngôn ngữ gom rác (garbage - collected language), vì vậy nó không cần đếm từng new với delete - một nguồn bộ nhớ chung để thoát trong các chương trình của C++. Trong thực tế không có toán tử delete trong JAVA.

- Đa luồng (multithread): Có nghĩa là JAVA cho phép xây dựng các trình ứng dụng, trong đó, nhiều tiến trình có thể xảy ra đồng thời. Tính đa luồng cho phép các nhà lập trình có thể biên soạn các phần mềm đáp ứng tốt hơn, tương tác hơn và thực hiện theo thời gian thực.

- JAVA độc lập với cấu trúc máy: Đây là thuộc tính đặc sắc nhất của JAVA. Có nghĩa là JAVA không phụ thuộc vào hệ máy, các ứng dụng bằng JAVA có thể dùng được trên hầu

như mọi máy tính.

Có thể nói JAVA là ngôn ngữ lập trình cho Web:

- **Hiệu mạng:** JAVA được viết ra để hoạt động trên mạng và có các thủ tục để có thể quản lý các giao thức TCP/IP, FTP, HTTP. Nói cách khác JAVA được xây dựng để hoàn toàn tương thích trên Internet.
- **JAVA** cho phép tạo ra các trang Web động, các ứng dụng nhúng.
- **An toàn:** Đặc tính an toàn của ngôn ngữ lập trình này bắt nguồn từ việc nó có những phân hạn chế được cài đặt sẵn nhằm đề phòng các chương trình JAVA thực hiện các chức năng như ghi vào ổ cứng hoặc cho phép vi rút xâm nhập vào từ mạng.

VISUAL BASIC

Visual Basic là một môi trường lập trình được phát triển bởi Microsoft nhằm cung cấp cho những người lập trình một phương pháp phát triển các ứng dụng trên Windows nhanh và dễ nhất.

Visual Basic cung cấp cho người lập trình một môi trường tích hợp, nơi mà người lập trình có thể sử dụng các công cụ để tạo ra giao diện người sử dụng một cách nhanh chóng và dễ dàng tạo ra mã để trả lời lại các tác động từ phía người sử dụng. Visual Basic cung cấp cho người lập trình cả kỹ thuật lập trình hướng sự kiện và kỹ thuật lập trình hướng đối tượng.

Môi trường phát triển của Visual Basic có những công cụ soạn thảo và gỡ rối tinh vi, nó cho phép người lập trình gắn mã với giao diện một cách nhanh chóng cho mỗi sự kiện. Visual Basic cung cấp cho bạn một giao diện nhanh nhất nhưng bù lại bạn phải cũng phải trả giá về tốc độ. Do vậy nếu yêu cầu bài toán cần đến tốc độ thì bạn nên dùng C++.

Còn có rất nhiều các ngôn ngữ khác nữa với các đặc điểm khác nhau, được thiết kế cho một vài mục đích đặc biệt nào đó, ví dụ: Perl, SQL, HTML ... là các ngôn ngữ khá thông dụng.

NGÔN NGỮ KỊCH BẢN

Một **ngôn ngữ kịch bản** hay **ngôn ngữ lập trình kịch bản** (tiếng Anh: *scripting language* hay *script language*) là một ngôn ngữ lập trình hỗ trợ viết kịch bản (*script*). Các chương trình được viết cho ngôn ngữ kịch bản có thể được thực thi thông qua lời gọi từ các tác vụ thay vì chạy thủ công bởi người dùng.

Kịch bản có thể được viết và thực hiện "on-the-fly", tức là không cần biên dịch (compile) và liên kết (linked). Khác với các ngôn ngữ thông thường, chương trình viết ra cần biên dịch thành mã máy trước khi đến tay người dùng, chương trình viết bằng ngôn ngữ kịch bản thường được chạy trực tiếp từ mã nguồn hoặc bytecode, do đó người sử dụng hoàn toàn xem và đổi nó với mã nguồn của chương trình.

2.2 THIẾT KẾ NGÔN NGỮ LẬP TRÌNH

NGÔN NGỮ LẬP TRÌNH THỦ TỤC

Mặc dù Pascal là một ngôn ngữ giảng dạy và hàn lâm rất thành công, nhưng C đã làm lu mờ Pascal như một sản phẩm ngôn ngữ lập trình. Một lý do mà không liên quan đến bản thân thiết kế ngôn ngữ, là sự phổ cập của hệ điều hành Unix, mà được viết trên C. Khi mà các chương trình được viết chạy trên Unix, mọi lời gọi hệ thống cơ bản sẽ ngay lập tức được hỗ trợ bởi C. Do đó, dễ dàng viết nhiều ứng dụng Unix trên C hơn là các ngôn ngữ lập trình khác. Một lý do khác cho sự phổ cập của C là nó có mô hình bộ nhớ đặc biệt mà rất gần với phần cứng bên dưới. Mặc dù C có nhiều khái niệm giống với Pascal, C ít cứng nhắc hơn trong việc tuân thủ các nguyên lý cơ bản và các hạn chế.

C được thiết kế nguyên bản và cài đặt từ 1969 đến 1973, như một phần của dự án Hệ điều hành Unix tại Bell Laboratories. C được thiết kế bởi Dennis Ritchie, một trong những người ban đầu thiết kế ra Unix, vì vậy ông ta và Ken Thompson có thể xây dựng Unix trên ngôn ngữ nào mà họ muốn. Thiết kế được bắt nguồn từ ngôn ngữ B (do đó sau này có tên là C, chữ cái tiếp theo trong bảng chữ cái), mà bản thân B dựa trên ngôn ngữ gọi là BCPL. Những thay đổi đáng kể của C được xảy ra từ 1977 đến 1979, như phần bắt buộc để đạt được tính chuyển mang của hệ thống Unix, và vào giữa những năm 1980 khi mà hội đồng chuẩn quốc gia Hoa kỳ chuẩn hóa ngôn ngữ. BCPL là ngôn ngữ lập trình hệ thống được thiết kế vào những năm 1960 và được sử dụng bởi các thành viên của dự án Hệ điều hành Multics trong Bell Laboratories. B là phương án cắt gọt bớt của BCPL, được thiết kế chạy trên máy tính nhỏ sử dụng bởi dự án Unix. Sự khác biệt chính giữa B và C ở chỗ B là ngôn ngữ không kiểu còn ngôn ngữ C có kiểu và có các qui tắc kiểm tra kiểu.

Một đặc tính để phân biệt C với các ngôn ngữ phổ cập khác là cách nó xử lý các vị trí nhớ, các mảng và con trỏ. Đây là một phần của C được kế thừa từ BCPL và B. Kiểu dữ liệu duy nhất trong B là 'word' hoặc 'cell', mẫu bit có độ dài cố định. Bộ nhớ trong BCPL và B được thể hiện cho lập trình viên dưới dạng mảng tuyến tính các từ 'word'. Con trỏ được xử lý như các chỉ số nguyên đến mảng này, và các mảng được khai báo bởi lập trình viên được xem như các từ kề nhau lấy từ mảng lớn của tất cả các từ bộ nhớ. Cách nhìn như vậy có một số hệ quả sau. Một là mảng và con trỏ khá tương đương nhau như mô tả dưới đây. Hai là, vì con trỏ là chỉ số dương trong mảng bộ nhớ, phép toán trên con trỏ cũng được xem xét là có ý nghĩa: nếu p là địa chỉ của một vị trí nhớ, thì p+1 là địa chỉ của vị trí nhớ tiếp theo.

CON TRỎ VÀ MẢNG TRONG C

Trong C, các con trỏ và mảng được khai báo khác nhau, vì con trỏ và mảng là các kiểu của các giá trị khác nhau. Chẳng hạn, đoạn code sau khai báo con trỏ p đến số nguyên và mảng A các số nguyên:

```
int*p;
```

```
int A[5];
```

Trong hầu hết các ngôn ngữ lập trình, đều có một thao tác nào đó xác định vị trí (dereferencing) con trỏ. Dereferencing là thao tác trả về vị trí được trỏ bởi con trỏ. Trong hầu hết các ngôn ngữ có dùng mảng, có một thao tác đánh chỉ số mà có thể được dùng để tìm một vị trí trong mảng. Có sự giống nhau giữa hai thao tác này, nhưng các ngôn ngữ có kiểu đều coi dereferencing đến tên mảng và đánh chỉ số con trỏ đều không hợp lệ. Tuy nhiên, trong C các mảng được xử lý rất hiệu quả xem như con trỏ. Để trích dẫn xem C Reference Manual, Dennis Ritchie, 1975:

Mỗi khi định danh kiểu mảng xuất hiện trong biểu thức, nó chuyển về con trỏ đến phần tử đầu tiên của mảng. Theo định nghĩa, thao tác chỉ số [] được dịch theo cách sau 'E1[E2]' được đồng nhất với '((E1)+(E2))'. Vì theo qui tắc chuyển kiểu mà áp dụng cho +, nếu E1 là mảng và E2 là số nguyên, thì E1[E2] tham chiếu đến phần tử thứ E2 của E1.*

NGÔN NGỮ LẬP TRÌNH HƯỚNG ĐỐI TƯỢNG

Đặc trưng quan trọng của các ngôn ngữ hướng đối tượng là khái niệm *mở rộng extention hoặc kế thừa inheritance*. Nếu một ngữ cảnh chương trình nào đó (như tham số hình thức của hàm hoặc phương thức) mà có một đối tượng hỗ trợ các phương thức m_1, m_2, m_3 , thì nó cũng sẽ chấp nhận đối tượng mà hỗ trợ m_1, m_2, m_3, m_4 .

MỞ RỘNG LỚP (CLASS EXTENTION)

Chương trình 2.1 mô tả việc sử dụng mở rộng lớp trong Java. Mỗi `Vehicle` là một `Object`; mỗi `Car` là một `Vehicle`; như vậy mỗi `Car` cũng là một `Object`. Mỗi `Vehicle` (và như vậy mỗi `Car` và `Truck`) có một trường nguyên `position` và một phương thức `move`.

PROGRAM 2.1: An object-oriented program.

```
class Vehicle {
    int position;
    void move (int x) { position = position + x; }
}
class Car extends Vehicle{
    int passengers;
    void await(Vehicle v) {
        if (v.position < position)
            v.move(position - v.position);
        else
            this.move(10);
    }
}
class Truck extends Vehicle{
    void move(int x) {
        if (x <= 55) { position = position + x; }
```



```
    }  
}  
class Main{  
    public static void main(String args[]) {  
        Truck t = new Truck();  
        Car c = new Car();  
        Vehicle v = c;  
        c.passengers = 2;  
        c.move(60);  
        v.move(70);  
        c.await(t);  
    }  
}
```

Ngoài ra Car có trường nguyên `passengers` và phương thức `await`. Các biến trong phạm vi đầu vào của `await` là

`passengers` vì nó là một trường của Car,
`position` vì nó là (không tường minh) một trường của Car,
`v` nó là tham số hình thức của `await`,
`this` vì nó là (không tường minh) tham số hình thức của `await`.

Tại lời gọi đến `c.await(t)`, xe tải truck `t` được gắn với tham số hình thức `v` của phương thức `await`. Đó đó khi `v.move` được gọi, nó kích hoạt thân phương thức `Truck_move`, chứ không phải `Vehicle_move`.

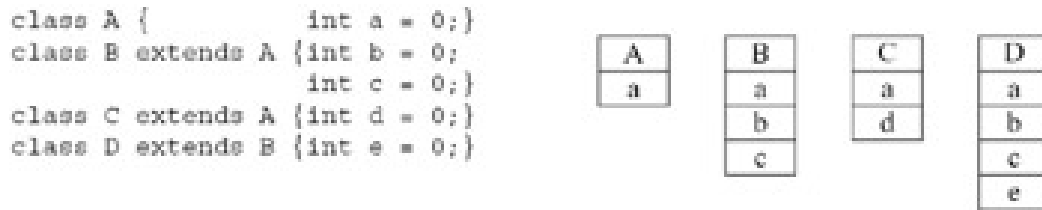
Chúng ta sử dụng ký hiệu `A_m` để chỉ một bản phương thức (*method instance*) `m` được khai báo bên trong lớp `A`. Nó không phải là một phần cú pháp của Java, nó chỉ để dùng khi phân biệt ngữ nghĩa của chương trình Java. Mỗi khai báo khác nhau của một phương thức là một bản phương thức khác nhau. Hai bản phương thức khác nhau có thể có cùng tên phương thức nếu, chẳng hạn, cái này đè cái kia.

ĐƠN KẾ THỪA CỦA CÁC TRƯỜNG DỮ LIỆU

Để lấy giá trị biểu thức `v.position`, trong đó `v` thuộc vào lớp `Vehicle`, chương trình dịch cần phải sinh ra code để đính kèm với trường `position` từ đối tượng (bản ghi) mà `v` trỏ đến.

Điều này trông có vẻ khá đơn giản: bản nhập môi trường cho biến `v` chứa (trong số nhiều thứ khác) con trỏ đến mô tả kiểu (lớp) của `Vehicle`; Nó có danh sách các trường và các lề (offsets) của chúng. Nhưng tại thời điểm chạy biến `v` có thể chứa con trỏ đến `Car` hoặc `Truck`; ở đó sẽ là trường `position` trong đối tượng `Car` hoặc `Truck`?

Đơn kế thừa. Đối với các ngôn ngữ đơn kế thừa, ở đó mỗi lớp mở rộng chỉ từ một lớp cha, kỹ thuật tiên tổ *prefixing* đơn giản làm việc tương đối tốt. Khi `B` extends `A`, các trường của `B` mà được kế thừa từ `A` sẽ được đặt phía trên trong bản ghi `B` *tại điểm bắt đầu, theo thứ tự mà chúng xuất hiện trong bản ghi của A*. Các trường trong `B` mà không kế thừa từ `A` sẽ được đặt sau đó, như chỉ ra trên Hình 2.2.



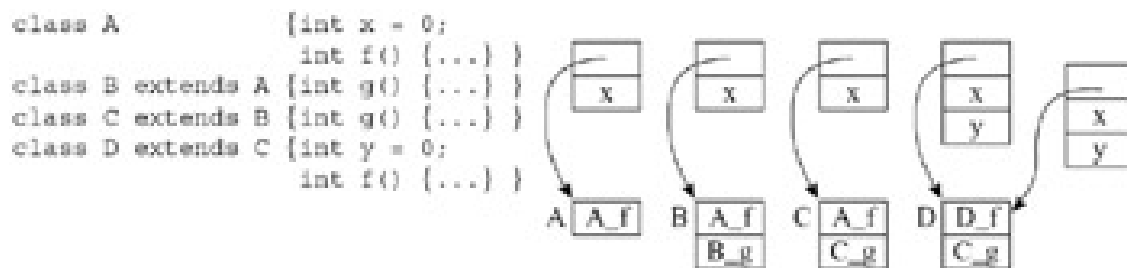
Hình 2.2: Đơn kế thừa các trường dữ liệu

CÁC PHƯƠNG THỨC

Kế thừa phương thức được dịch giống như hàm: Nó trả về mã máy mà nằm tại địa chỉ cụ thể của không gian các chỉ lệnh. Chẳng hạn, phương thức kế thừa `Truck_move` có bản nhập tại nhãn của mã máy `Truck_move`. Trong pha phân tích ngữ nghĩa của chương trình dịch, bản nhập môi trường của mỗi biến chứa con trỏ đến mô tả lớp; mỗi mô tả lớp chứa con trỏ đến lớp cha của nó, và cũng như là danh sách các bản phương thức; và mỗi bản phương thức có nhãn mã máy.

Các phương thức tĩnh (Static methods). Một số ngôn ngữ hướng đối tượng cho phép một số phương thức được khai báo là tĩnh: *static*. Mã máy mà thực thi khi `c.f()` được gọi phụ thuộc vào kiểu của biến `c`, không phải kiểu của đối tượng *object* mà `c` đang giữ. Để dịch lời gọi phương thức từ `c.f()`, chương trình dịch sẽ tìm lớp của `c`; giả sử đó là lớp `C`. Khi đó nó tìm trong lớp `C` phương thức `f`; giả sử không tìm thấy. Khi đó nó tìm tiếp trong lớp cha của `C` là lớp `B`, xem có phương thức `f` không; nếu không thì lớp cha của `B`; và tiếp tục như vậy. Giả sử tại lớp tổ tiên nào đó là lớp `A` nó tìm thấy phương thức tĩnh `f`; khi đó nó dịch lời gọi hàm đến nhãn `A_f`.

Phương thức động (Dynamic methods). Kỹ thuật trên sẽ không làm việc đối với phương thức động. Nếu phương thức `f` trong `A` là phương thức động, thì nó sẽ được đề trong một lớp `D` nào đó mà là lớp con của `C` (xem Hình 2.3). Nhưng không có cách nào nói cho chương trình dịch biết là biến `c` được trỏ đến đối tượng của lớp `D` (trong trường hợp mà `D_f` cần được gọi) hoặc lớp `C` (trong trường hợp `A_f` cần được gọi).



Hình 2.3: Mô tả lớp để tìm kiếm phương thức động.

Để giải quyết vấn đề này, mô tả lớp cần chứa vector với kế thừa phương thức đối với mỗi tên phương thức (không tĩnh). Khi lớp `B` kế thừa từ `A`, bảng phương thức *bắt đầu với các bản nhập* cho mọi tên phương thức đã biết đối với `A`, và sau đó tiếp tục với các phương

thức *mới* được khai báo bởi B. Điều này rất giống với việc sắp xếp các trường trong các đối tượng kế thừa.

Hình 2.3 chỉ ra điều gì xảy ra khi lớp D ghi đè phương thức f . Mặc dù bản nhập cho f ở vị trí đầu trong bảng phương thức của D, nó cũng ở vị trí đầu trong bảng phương thức của lớp tổ tiên A, nó trở nên bản phương thức khác vì nó đã được ghi đè.

Để thực thi $c.f()$, trong đó f là phương thức động, code được dịch cần thực thi các chỉ lệnh sau:

1. Tìm mô tả lớp d tại lề (offset) 0 từ đối tượng c .
2. Tìm con trỏ bản nhập phương thức p từ lề (hằng số) f của d .
3. Nhảy đến địa chỉ p , lưu địa chỉ trả về (nơi mà gọi p).

LẬP TRÌNH HÀM

Hàm - func-tion: là ánh xạ toán học mà gán chính xác một phần tử của một tập hợp cho mỗi phần tử của tập hợp đó hoặc tập hợp khác.

Khái niệm toán học của hàm số là nếu $f(x) = a$ "lúc này", thì $f(x) = a$ "thời điểm tiếp theo"; ở đây không có giá trị nào khác bằng $f(x)$. Điều này cho phép sử dụng suy luận đẳng thức tương tự như đại số: Nếu $a = f(x)$, thì $g(f(x), f(x))$ là tương đương với $g(a, a)$. Các ngôn ngữ lập trình hàm thuần túy khuyến khích kiểu lập trình mà ở đó có thể suy luận đẳng thức được, như nó vẫn làm trong toán học.

Các ngôn ngữ lập trình mệnh lệnh *Imperative* có cú pháp tương tự như: $a \leftarrow f(x)$. Nhưng nếu ta cho phép điều này $b \leftarrow f(x)$, thì không có gì đảm bảo rằng $a = b$; hàm f có thể có *tác động phụ* (*side effects*) trên các biến tổng thể mà làm nó trả về các giá trị khác nhau ở mỗi thời điểm. Ngoài ra, một chương trình có thể gán cho biến x giữa các lời gọi đến $f(x)$, do đó $f(x)$ trên thực tế mỗi thời điểm cho giá trị khác nhau.

Hàm bậc cao (Higher-order functions). Các ngôn ngữ lập trình hàm cũng cho phép các hàm được truyền như tham số cho các hàm khác hoặc được trả về như kết quả. Các hàm mà nhận các đối số hàm được gọi là các hàm bậc cao.

Các hàm bậc cao được quan tâm đặc biệt, nếu ngôn ngữ cho phép các hàm lồng nhau với phạm vi từ vựng (hay còn được gọi là *cấu trúc khối*). Phạm vi từ vựng có nghĩa là mỗi hàm có thể tham chiếu đến các biến hoặc các tham số của một hàm bất kỳ mà nó lồng bên trong hàm đó. Một *ngôn ngữ hàm bậc cao* là ngôn ngữ với các hàm bậc cao và phạm vi lồng nhau.

Cái gì là bản chất ngôn ngữ lập trình hàm? Có phải là suy luận đẳng thức hay là các hàm bậc cao? Ở đây không có một sự nhất trí rõ ràng về câu trả lời cho câu hỏi này. Trong chương này chúng ta sẽ bàn về ba đặc tính hay của ngôn ngữ lập trình hàm:

- **FunJava.** Ngôn ngữ MiniJava với các hàm bậc cao. Vì các tác động phụ vẫn còn cho phép (và như vậy, suy luận đẳng thức không làm việc), Đây gọi là *ngôn ngữ hàm bậc cao không thuần nhất*; Các ngôn ngữ dạng này như Scheme, ML, và Smalltalk.
- **PureFunJava.** Một ngôn ngữ với các hàm bậc cao và không có tác động phụ, giữ được bản chất của các ngôn ngữ hàm thuần nhất, chặt (như một tập con hàm thuần nhất của ML).
- **LazyFunJava.** Một *ngôn ngữ hàm thuần nhất không chặt* mà sử dụng ước lượng “lười” như ngôn ngữ Haskell. Các ngôn ngữ hàm không chặt hỗ trợ suy luận đẳng thức rất tốt.

Một *ngôn ngữ hàm thuần nhất bậc nhất* như SISAL hỗ trợ suy luận đẳng thức nhưng không hỗ trợ hàm bậc cao.

MỘT NGÔN NGỮ HÀM ĐƠN GIẢN

Để tạo ngôn ngữ mới Fun-MiniJava, chúng ta bổ sung kiểu hàm (*function types*) cho MiniJava:

```
ClassDecl → type id = TypeExp ;
TypeExp  → TypeExp -> TypeExp
          → ( TypeList ) -> TypeExp
          → ( TypeExp )
          → Type TypeList
          → TypeExp TypeRest*
          →
TypeRest →, TypeExp
```

Kiểu `int->String` là kiểu của các hàm mà nhận đối số nguyên đơn và trả về kết quả là xâu (giả thiết rằng lớp `String` đã được khai báo). Kiểu `(int, String)->int[]` mô tả các hàm mà nhận hai đối số (một số nguyên, một xâu) và trả về kết quả là mảng số nguyên.

Biến bất kỳ có thể có kiểu hàm; các hàm có thể truyền như đối số và trả về như kết quả. Như vậy, kiểu `(int->int)->(int->int)` là hoàn toàn hợp lệ; Thao tác `->` là kết hợp phải, vì vậy đây là kiểu các hàm mà nhận đối số là hàm `int->int` và trả về kết quả là hàm `int->int`.

Chúng ta cũng điều chỉnh định dạng của biểu thức *CALL*, sao cho hàm được gọi là biểu thức bất kỳ, không có thành phần `.methodname`, và sao cho bản thân phương thức có thể là kết quả của biểu thức:

```
Exp → Exp ( ExpList )
Exp → Exp . id
```

Nếu `v` là đối tượng của lớp với phương thức `int m(int[])`, thì biểu thức `v.m` ước lượng cho giá trị hàm kiểu `(int[])->int`. Việc ước lượng `v.m` không gọi phương thức đó.

Chúng ta cho phép các khai báo biến và các khai báo hàm (phương thức) ngay ở đầu của lệnh phức (tức là, các hàm lồng nhau). Chúng ta loại bỏ lệnh `if` và bổ sung biểu thức `if`: như vậy, `(if (E) B else C)` tính toán `E`, và sau đó tính toán `B` nếu `E` là true, ngược lại tính toán `C`. Giá trị của toàn bộ biểu thức `if` là giá trị của `B` hoặc `C`.

```
MethodDecl → public Type id ( FormalList ) Compound
Compound  → { VarDecl* MethodDecl* Statement* return Exp ; }
Exp        → Compound
           → if ( Exp ) Exp else Exp
```

Cuối cùng, chúng ta dịch ngữ nghĩa của `return` theo cách khác: Thay vì tạo kết quả cho toàn bộ thân hàm, nó tạo kết quả cho lệnh thành phần của chúng. Chẳng hạn, biểu thức `{return 3;}+{return 4;}` ước lượng thành 7.

Chương trình 2.4 minh họa cách sử dụng các kiểu hàm. Hàm `add` nhận đối số nguyên và trả về hàm `h`. Như vậy, `addFive` là phiên bản của `h` mà biến `n` là 5, nhưng `addSeven` là hàm `h(x) = 7 + x`. Sự cần thiết đối với mỗi bản tạo khác nhau của `h` phải “nhớ” giá trị thích hợp cho biến không *cục bộ* `n` thúc đẩy kỹ thuật cài đặt của *bao đóng* - *closures*, mà được mô tả sau này.

CHƯƠNG TRÌNH 2.4: Chương trình FunJava.

```
type intfun = int -> int;

class C {
  public intfun add(n: int) {
    public int h(int m) { return n+m; }
    return h;
  }
  public intfun twice(f: intfun) {
    public int g(int x) {return f(f(x));}
    return g;
  }
  public int test() {
    intfun addFive = add(5);
    intfun addSeven = add(7);
    int twenty = addFive(15);
    int twentyTwo = addSeven(15);
    intfun addTen = twice(addFive);
    int seventeen = twice(add(5))(7);
    intfun addTwentyFour = twice(twice(add(6)));
    return addTwentyFour(seventeen);
  }
}
```

Hàm `twice` nhận đối số `f` mà là hàm từ `int` vào `int`, và kết quả của `twice(f)` là hàm `g` mà áp dụng hai lần hàm `f`. Như vậy, `addTen` là hàm `g(x) = addFive(addFive(x))`. Mỗi bản tạo của `g(x)` cần phải nhớ giá trị `f` đúng, cũng như mỗi bản tạo `h` cần phải nhớ `n`.

LẬP TRÌNH ĐA HÌNH

Một số các hàm thực hiện theo cách mà nó độc lập với kiểu dữ liệu trên đó chúng thao tác. Một số cấu trúc dữ liệu được cấu trúc theo cách không phụ thuộc vào kiểu các phần tử của chúng.

Chẳng hạn, xét một hàm ghép danh sách kết nối trong Java. Chúng ta định nghĩa lớp `List`, các lớp con cho danh sách rỗng và khác rỗng, và một phương thức (không phá hủy) `append`:

```
abstract class IntList {
    IntList append(IntList more);
}
class IntCons extends IntList {
    Integer head; IntList tail;
    IntCons (Integer head, IntList tail) {
        this.head=head; this.tail=tail;
    }
    IntList append(IntList more) {
        return new IntCons(head, tail.append(more));
    }
}
class IntNull extends IntList {
    IntNull () {}
    IntList append(IntList more) {
        return more;
    }
}
```

Ở đây không có mã riêng cho kiểu dữ liệu `IntList` hoặc phương thức `append`, mà có thể là bất cứ cái gì khác chẳng hạn kiểu phần tử có thể là `String` hoặc `Tree` thay vì `Integer`. Chúng ta thấy `append` có thể làm việc trên mọi kiểu danh sách. Tất nhiên chúng ta có thể sử dụng từ `Object` thay cho `Integer` để khai báo `head`, nhưng sau đó nếu chúng ta vô tình lẫn lộn số nguyên `Integers` và xâu `Strings` như các phần tử của cùng một danh sách, bộ kiểm tra kiểu của chương trình dịch không thể có khả năng phân hồi hữu ích: chúng ta nhận được lỗi thời gian chạy tại nơi hạ kiểu ở đâu đó.

Một hàm là *đa hình* nếu nó có thể thao tác trên đối số của nhiều kiểu. Ở đây có hai kiểu đa hình chính

Đa hình tham số. Một hàm là đa hình tham số nếu nó tuân theo cùng một thuật toán không phụ thuộc vào kiểu của đối số của nó. Cơ chế tổng quan của ngôn ngữ Ada hoặc Modula-3, C++ *templates*, hoặc lược đồ kiểu ML (*type schemes*) là các ví dụ về đa hình tham số.

Tải chồng (overloading). Một định danh hàm được tải chồng (*overloaded*) nếu nó đứng tên cho các thuật toán khác nhau tùy thuộc vào kiểu của đối số của nó. Chẳng hạn, trong hầu hết các ngôn ngữ phép toán `+` được tải chồng, nghĩa là cộng số nguyên trên các đối nguyên và cộng số phẩy động (mà là thuật toán khác hoàn toàn) trên các đối dấu phẩy

động. Trong nhiều ngôn ngữ, bao gồm cả Ada, C++, và Java, các lập trình viên có thể tạo các hàm được tải chồng theo cách riêng của họ.

Hai kiểu đa hình này là hoàn toàn khác nhau – hầu như không liên quan đến nhau – và đòi hỏi các kỹ thuật cài đặt khác nhau.

ĐA HÌNH THAM SỐ

Hàm đa hình $f(t\ x)$ nhận tham số nào đó x có kiểu t , trong đó t có thể được khởi tạo trên một kiểu thực tế nào đó. Trong phong cách *tường minh* của đa hình tham số, chúng ta truyền kiểu như một đối số cho hàm, như chúng ta viết một cái gì đó như $f<t>(t\ x)$, và một lời gọi hàm có thể trông như sau $f<\text{int}>(3)$ hoặc $f<\text{string}>(\text{"three"})$. Trong một ngôn ngữ với đa hình *ẩn*, chúng ta đơn giản viết định nghĩa như $f(x)$, và lời gọi như $f(3)$ hoặc $f(\text{"three"})$ – tham số kiểu t không được nói rõ. Các ngôn ngữ lập trình suy luận được có thể được thiết kế cả hai cách.

Trong chương này chúng ta sẽ trình bày *Generic Java* (viết tắt là GJ), một mở rộng đa hình của Java (các hàm đa hình cũng được gọi là các hàm tổng quan trong tài liệu của các ngôn ngữ lập trình). Trong GJ, các lớp và các phương thức là đa hình: Mỗi lớp và phương thức có thể nhận các tham số kiểu trong ngoặc nhọn:

```

ClassDecl → class id TyParams Ext { VarDecl* MethodDecl* }
      Ext → extends Type
      →
MethodDecl → public TyParams Type id ( FormalList )
      { VarDecl* Statement* return Exp ; }
      TyParams → < id Ext TyParRest* >
      →
      TyParRest → , id Ext

```

Bổ sung vào các kiểu `int` và `boolean` (và các kiểu khác), một *Type* được sử dụng trong khai báo các biến có thể nhận các đối số mà bản thân cũng là kiểu:

```

Type → id < Type TypeRest* >
TypeRest → , Type

```

Cuối cùng, các hàm tạo của lớp cũng có thể nhận các đối số kiểu:

```

Exp → new id < Type TypeRest* >()

```

GJ sử dụng kết hợp đa hình tường minh và ẩn: Một lập trình viên luôn cần phải viết các tham số kiểu hình thức (tại các khai báo lớp), nhưng các tham số kiểu thực tế (khi gọi hàm tạo lớp) có thể thường xuyên bị bỏ qua. Trong chương này chúng ta sẽ trình bày chỉ GJ hoàn toàn tường minh.

Sử dụng đa hình, chúng ta có thể tạo lớp `List` tổng quan, với nó chúng ta có thể tạo danh sách các số nguyên, hoặc danh sách các xâu, nhưng mà ngăn cản các lỗi không mong muốn của cái này đối với cái kia:

```
abstract class List<X> {
    List<X> append(List<X> more);
}
class Cons<X> extends List<X> {
    X head; List<X> tail;
    Cons (X head, List<X> tail) {this.head=head; this.tail=tail;}
    List<X> append(List<X> more) {
        return new Cons<X>(head, tail.append(more));
    }
}

class Null<X> extends List<X> {
    Null () {}
    List<X> append(List<X> more) {
        return more;
    }
}
```

Sử dụng khai báo này, chúng ta có thể tạo danh sách các số nguyên (3,4) với biểu thức,

```
List<Integer> list34 =
    new Cons<Integer>(new Integer(3),
        new Cons<Integer>(new Integer(4),
            new Null<Integer>));
```

Chúng ta có thể tạo danh sách của danh sách các số nguyên:

```
List<List<Integer>> lislis =
    new Cons<List<Integer>>(list34,
        new Null<List<Integer>>());
```

Trong GJ chúng ta cũng có thể gắn kết một tham số biến hình thức bằng cách chỉ ra rằng nó cần phải là một lớp con của một lớp cơ sở nào đó. Giả sử chẳng hạn chúng ta có lớp `Printable`:

```
abstract class Printable { void print_me(); }
```

với các lớp con, một số trong chúng đã được khai báo ở đây và một số còn cần được khai báo:

```
class PrintableInt extends Printable {
    int x;
    void print_me() {... print x ...}
}
class PrintableBool extends Printable {
    boolean b;
    void print_me() {... print b ...}
}
```

Trong Java thông thường chúng ta tạo một cặp in được như sau:

```
class Pair {
    Printable a;
    Printable b;
    void print_me() { a.print_me(); b.print_me(); }
}
```

và điều này sẽ làm việc tốt, cho đến khi chúng ta không nghĩ rằng trong một cặp Pair nào đó, các thành phần `a` và `b` có thể thuộc các lớp con khác nhau của `Printable`. Nhưng nếu chúng ta muốn tạo một cặp in “gắn kết”, ở đó cả hai thành phần cần phải thuộc vào cùng một lớp con, chúng ta sẽ sử dụng đa hình gắn kết trong GJ như sau:

```
class GPair<X extends Printable> {
    X a;
    X b;
    void print_me() { a.print_me(); b.print_me(); }
}
```

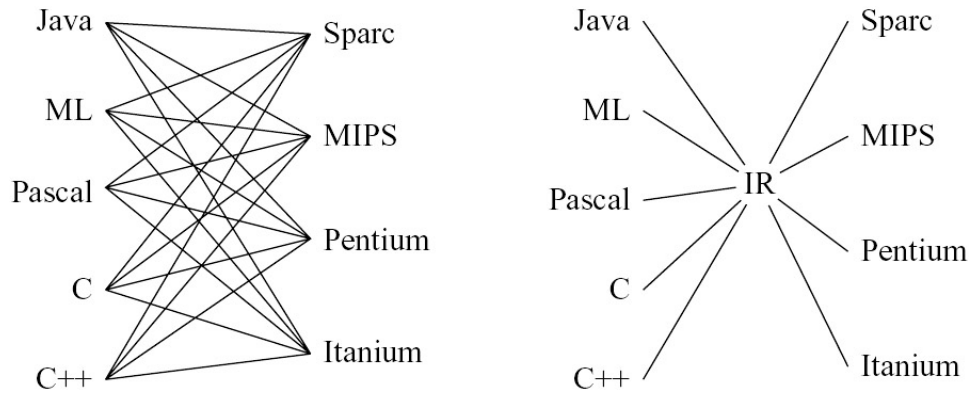
Bây giờ mỗi đối tượng của kiểu `GPair<PrintableInt>` có các thành phần `a` và `b` mà cả hai đều khởi tạo từ `PrintableInt`, và tương ứng đối với `GPair<PrintableBool>`, và đối với các lớp con khác của `Printable` mà có thể được khai báo trong tương lai. Chúng ta nói rằng `Printable` được gắn với tham số kiểu `X`.

Kiểu con trong GJ. Trong Java, nếu chúng ta tạo lớp `class Triple extends Pair`, thì `Triple` là kiểu con của `Pair`, và bất kỳ đối tượng `Triple` nào cũng có thể được truyền như tham số cho phương thức bất kỳ mà đón nhận `Pair`. Trong GJ, chúng ta có thể tạo lớp `class GTriple<X extends Printable> extends GPair<X>`, và khi đó `GTriple<PrintableInt>` là kiểu con của `GPair<PrintableInt>`. Nhưng nếu `class MyInt extends PrintableInt`, thì nó không phải là trường hợp mà `GTriple<MyInt>` là kiểu con của `GPair<PrintableInt>`. Và đặc biệt nó không phải trường hợp mà `GTriple` là kiểu con của `GPair`, vì chúng không phải là kiểu, chúng là *cấu trúc* (*constructors*) tạo kiểu, mà trở thành kiểu chỉ khi áp dụng cho các đối số.

2.3 DỊCH SANG MÃ TRUNG GIAN

Pha Phân tích ngữ nghĩa của chương trình dịch cần phải dịch cú pháp trừu tượng sang mã máy trừu tượng. Nó có thể thực hiện điều đó sau khi kiểm tra kiểu, hoặc cùng một thời điểm.

Mặc dù nó có thể được dịch trực tiếp sang mã máy thực tế, nhưng nó cản trở tính dịch chuyển và module hóa. Giả sử chúng ta muốn chương trình dịch cho N ngôn ngữ nguồn khác nhau sang M kiểu máy đích khác nhau. Về mặt nguyên tắc cần $N \cdot M$ chương trình dịch (xem hình 2.5a), đây là nhiệm vụ cài đặt lớn.



Hình 2.5: Chương trình dịch cho năm ngôn ngữ và bốn máy đích:
(a) không dùng IR, (b) sử dụng IR.

Biểu diễn trung gian (*intermediate representation* - IR) là ngôn ngữ máy trừu tượng mà có thể thể hiện các thao tác máy đích mà không phải thỏa thuận quá nhiều về chi tiết đặc thù của máy. Nhưng nó cũng độc lập với chi tiết của ngôn ngữ gốc. Phần trước của chương trình dịch gồm phân tích từ vựng, phân tích cú pháp, phân tích ngữ nghĩa và dịch ra biểu diễn trung gian. Phần sau thực hiện tối ưu biểu diễn trung gian và dịch sang ngôn ngữ máy.

Chương trình dịch chuyển mang được sẽ dịch ngôn ngữ gốc sang IR và sau đó dịch từ IR sang ngôn ngữ máy, như thể hiện trên hình 2.5b. Bây giờ chỉ cần N phần đầu và M phần sau. Nhiệm vụ cài đặt như vậy là chấp nhận được.

Ngay cả khi một phần đầu và một phần sau được xây dựng riêng biệt, biểu diễn trung gian IR tốt có thể module hóa nhiệm vụ này, như vậy phần đầu không bị phức tạp với các chi tiết đặc thù của máy, và phần sau không cần quan tâm đến các thông tin đặc thù của ngôn ngữ gốc. Nhiều kiểu IR khác nhau được sử dụng trong chương trình dịch; Đối với chương trình dịch này chúng ta chọn các cây biểu diễn đơn giản.

CÂY BIỂU DIỄN TRUNG GIAN

Ngôn ngữ cây biểu diễn trung gian được định nghĩa trong gói `Tree`, bao gồm các lớp trừu tượng `Stm` và `Exp` và các lớp con của nó, như được nêu trong Hình 2.6.

```
package Tree;

abstract class Exp
CONST(int value)
NAME(Label label)
TEMP(Temp.Temp temp)
BINOP(int binop, Exp left, Exp right)
```

```
MEM(Exp exp)
CALL(Exp func, ExpList args)
ESEQ(Stm stm, Exp exp)

abstract class Stm
MOVE(Exp dst, Exp src)
EXP(Exp exp)
JUMP(Exp exp, Temp.LabelList targets)
CJUMP(int relop, Exp left, Exp right, Label iftrue, Label iffalse)
SEQ(Stm left, Stm right)
LABEL(Label label)

other classes:
ExpList(Exp head, ExpList tail)
StmList(Stm head, StmList tail)

other constants:
final static int BINOP.PLUS, BINOP.MINUS, BINOP.MUL, BINOP.DIV,
BINOP.AND,
    BINOP.OR, BINOP.LSHIFT, BINOP.RSHIFT, BINOP.ARSHIFT, BINOP.XOR;

final static int CJUMP.EQ, CJUMP.NE, CJUMP.LT, CJUMP.GT, CJUMP.LE,
    CJUMP.GE, CJUMP.ULT, CJUMP.ULE, CJUMP.UGT, CJUMP.UGE;
```

Hình 2.6: Các cây biểu diễn trung gian.

Biểu diễn trung gian tốt có một số đặc tính sau:

- Nó cần phải thuận lợi cho pha phân tích ngữ nghĩa để tạo ra ϵ . Nó cần phải thuận lợi để dịch ra ngôn ngữ máy thực tế cho mọi máy đích mong muốn.
- Mỗi cấu trúc cần phải có ý nghĩa rõ ràng và đơn giản, sao cho việc tối ưu chuyển đổi mà viết lại các biểu diễn trung gian được dễ dàng đặc tả và cài đặt.

Các đoạn riêng của cú pháp trừu tượng có thể phức tạp, như chỉ số mảng, lời gọi thủ tục... Các chỉ lệnh riêng của máy thực tế cũng có thể có tác động phức tạp (mặc dù điều này không còn đúng đối với các máy so với các kiến trúc cũ). Không may, không phải lúc nào cũng là trường hợp các đoạn phức tạp của cú pháp trừu tượng cũng tương ứng chính xác với các chỉ lệnh phức tạp mà máy có thể thực thi.

Do đó, biểu diễn trung gian cần phải có các thành phần riêng lẻ mà mô tả chỉ một việc rất đơn giản: việc đính, lưu, bổ sung, chuyển hay nhảy đơn. Khi đó mọi đoạn nhỏ của cú pháp trừu tượng có thể được dịch sang tập các chỉ lệnh của máy trừu tượng; và các nhóm chỉ lệnh máy trừu tượng có thể bó lại với nhau tạo thành các chỉ lệnh máy thực tế.

Sau đây là mô tả ý nghĩa của từng thao tác cây. Trước hết, biểu thức (Exp), mà dành cho tính toán giá trị nào đó (có thể với các tác động phụ):

1. $\text{CONST}(i)$ Hằng số nguyên i .
2. $\text{NAME}(n)$ Hằng số ký hiệu n (tương ứng với nhãn của ngôn ngữ assembly).

3. TEMP(t) Tạm thời t . Tạm thời trong máy trừu tượng tương tự như thanh ghi trong máy thực tế. Tuy nhiên máy trừu tượng có vô hạn các tạm thời.
4. BINOP(o, e_1, e_2) Áp dụng toán tử hai ngôi o cho hai thành phần e_1, e_2 . Biểu thức e_1 được tính trước e_2 . Các toán tử số học số nguyên là PLUS, MINUS, MUL, DIV; các toán tử logic theo từng bit nguyên là AND, OR, XOR; các toán tử dịch logic số nguyên là LSHIFT, RSHIFT; dịch phải số học số nguyên là ARSHIFT. Ngôn ngữ MiniJava chỉ có một toán tử logic, nhưng ngôn ngữ trung gian là độc lập với ngôn ngữ nguồn, vì vậy các toán tử logic có thể được sử dụng để cài đặt các tính chất khác của MiniJava.
5. MEM(e) Nội dung gồm số bytes cỡ từ ($wordSize$ bytes) của bộ nhớ bắt đầu từ địa chỉ e (trong đó $wordSize$ được định nghĩa trong module `Frame`). Lưu ý rằng khi MEM được sử dụng như con trái của MOVE, nó có nghĩa là lưu "store", nhưng ở các chỗ khác nó có nghĩa là tìm về "fetch."
6. CALL(f, l) Lời gọi thủ tục: áp dụng hàm f cho danh sách biến l . Biểu thức con f được tính toán trước các biến mà được tính toán từ trái qua phải.
7. ESEQ(s, e) Lệnh s được tính toán các tác động phụ, sau đó e được tính toán cho kết quả.

Các lệnh (`stm`) của ngôn ngữ cây thực hiện các tác động phụ và luồng kiểm soát sau:

- MOVE(TEMP t, e) Tính e và chuyển nó vào tạm thời t .
- MOVE(MEM(e_1) e_2) Tính e_1 , sinh ra địa chỉ a . Sau đó tính e_2 , và lưu kết quả vào số bytes cỡ $wordSize$ của bộ nhớ bắt đầu từ a .
- EXP(e) Tính e và bỏ qua kết quả.
- JUMP($e, labs$) Truyền điều khiển (jump) đến địa chỉ e . Đích e có thể là nhãn ký tự, như trong NAME(lab), hoặc nó có thể là địa chỉ được tính từ bất cứ kiểu biểu thức nào khác. Chẳng hạn, lệnh `switch(i)` của ngôn ngữ C có thể được cài đặt bằng việc thực hiện số học trên i . Danh sách nhãn $labs$ chỉ rõ mọi vị trí mà biểu thức e có thể tính đến; điều này cần thiết cho phân tích dòng dữ liệu về sau. Trường hợp chung của việc nhảy đến nhãn đã biết l được viết JUMP(NAME l , `new LabelList(l, null)`), nhưng lớp JUMP có thêm hàm tạo sao cho nó có thể viết tắt JUMP(l).
- CJUMP(o, e_1, e_2, t, f) Tính e_1, e_2 theo thứ tự, cho ra các giá trị a, b . Sau đó so sánh a, b sử dụng toán tử quan hệ o . Nếu kết quả là `true`, nhảy đến t ; ngược lại nhảy đến f . Các toán tử quan hệ là EQ và NE đối với so sánh bằng nhau hay không bằng nhau của số nguyên (có dấu hoặc không có dấu); bất đẳng thức nguyên có dấu LT, GT, LE, GE; và bất đẳng thức nguyên không dấu ULT, ULE, UGT, UGE.
- SEQ(s_1, s_2) Lệnh s_1 được nối tiếp sau bởi lệnh s_2 .
- LABEL(n) Xác định giá trị hằng tên n là địa chỉ mã máy hiện thời. Điều này giống như định nghĩa nhãn trong ngôn ngữ assembly. Giá trị NAME(n) có thể là đích của các bước nhảy hoặc các lời gọi, ...

Hầu như có thể gán ngữ nghĩa hình thức cho ngôn ngữ `Tree`. Tuy nhiên, ở đây không có chỗ trong ngôn ngữ này dành cho các định nghĩa hàm và thủ tục – chúng ta có thể đặc tả

chỉ thân của mỗi hàm. Dãy đăng nhập và thoát ra khỏi mỗi thủ tục sẽ được bổ sung sau như việc gắn đặc điểm riêng cho mỗi ngôn ngữ đích.

2.4 MÁY ẢO JAVA

Tất cả các chương trình muốn thực thi được thì phải được biên dịch ra mã máy. Mã máy của từng kiến trúc CPU của mỗi máy tính là khác nhau (tập lệnh mã máy của CPU Intel, CPU Solarix, CPU Macintosh ... là khác nhau), vì vậy trước đây một chương trình sau khi được biên dịch xong chỉ có thể chạy được trên một kiến trúc CPU cụ thể nào đó. Đối với CPU Intel chúng ta có thể chạy các hệ điều hành như Microsoft Windows, Unix, Linux, OS/2,...

Chương trình thực thi được trên Windows được biên dịch dưới dạng file có đuôi .EXE còn trên Linux thì được biên dịch dưới dạng file có đuôi .ELF, vì vậy trước đây một chương trình chạy được trên Windows muốn chạy được trên hệ điều hành khác như Linux chẳng hạn thì phải chỉnh sửa và biên dịch lại.

Ngôn ngữ lập trình Java ra đời, nhờ vào máy ảo Java mà khó khăn nêu trên đã được khắc phục. Một chương trình viết bằng ngôn ngữ lập trình Java sẽ được biên dịch ra mã của máy ảo java (mã java bytecode). Sau đó máy ảo Java chịu trách nhiệm chuyển mã java bytecode thành mã máy tương ứng. Sun Microsystem chịu trách nhiệm phát triển các máy ảo Java chạy trên các hệ điều hành trên các kiến trúc CPU khác nhau.

JVM độc lập trên các hệ điều hành. Một Java Virtual Machine có 3 thành phần chính là:

1. Class-Loader Subsystem : chuyên tìm kiếm và load các file .class vào vùng nhớ của Java.
2. Runtime Data Area : vùng nhớ hệ thống cấp phát cho Java Virtual Machine.
3. Execution Engine: chuyển các lệnh của JVM trong file .class thành các lệnh của máy, hệ điều hành tương ứng và thực thi chúng.

BÀI TẬP

1. Cài đặt chương trình Java 2.1 và 2.4.
2. Tiến trình linking cho C++ là tương đối phức tạp. Tuy nhiên, cần phải nhấn mạnh rằng, nếu < là phép toán ghi đè, thì phiên bản đúng của < cần phải được định danh khi đoạn code đã dịch cho sort được link với chương trình gọi. Chẳng hạn, xét hàm tổng quát sort sau:

```
template <typename T>
void sort( int count, T * A[count] ) {
    for (int i=0; i <count-1; i++)
        for (int j=i+1; j<count-1; j++)
```

```
        if (A[j] < A[i]) swap(A[i],A[j]);  
    }
```

Nếu A là mảng kiểu T, thì sort(n, A) sẽ làm việc chỉ khi phép toán < được định nghĩa cho kiểu T. Yêu cầu này của sort không được khai báo ở đâu trong đoạn code C++. Tuy nhiên khi template hàm được khởi tạo, kiểu thực tế T cần phải có phép toán < được định nghĩa hoặc lỗi trong thời gian link sẽ thông báo và không đoạn code đối tượng thực thi nào được tạo ra. Kiểm nghiệm các đánh giá trên qua việc cài đặt trên thực tế.

3. Tải và cài chương trình SML (Standard ML) và chạy chương trình sau. Giải thích kiểu ML cho mỗi khai báo sau đây:

- a. `fun a(x,y) = x+2*y;`
- b. `fun b(x,y) = x+y/2.0;`
- c. `fun c(f) = fn y => f(y);`
- d. `fun d(f,x) = f(f(x));`
- e. `fun e(x,y,b) = if b(y) then x else y;`

4. Chạy chương trình tính độ dài của danh sách sau trong ML:

```
-- fun length(nil) = 0  
| length( x :: xs ) = 1 + length(xs);  
val length = fn : 'a list -> int
```

5. Sắp xếp đa hình

Hàm này, sẽ sắp xếp chèn trên danh sách, có các đối số là hàm so sánh less và một danh sách l gồm các phần tử cần sắp xếp. Đoạn code được dịch và chạy đúng:

```
fun sort(less, nil) = nil |
    sort(less, a: :l) =
        let
            fun insert(a, nil) = a: :nil |
                insert(a, b: :l) = if less(a,b) then a: :(b: :l)
                                   else b:: insert(a, l)
        in
            insert(a, sort(less, l))
        end;
```

6. Kiểu của hàm đó là gì? Giải thích ngắn gọn, bao gồm cả kiểu của hàm phụ insert. Bạn không cần chạy thuật toán trên đoạn code này, chỉ cần giải thích vì sao người lập trình ML thông thường mong muốn đoạn code kiểu đó.
7. Điểm bất động đa hình

Điểm bất động của hàm f là giá trị sao cho $x = f(x)$. Ở đây có mối liên hệ giữa đệ qui và điểm bất động mà mô tả bởi định nghĩa ML của hàm giai thừa factorial: $\text{int} \rightarrow \text{int}$:

```
fun Y f x = f(Yf)x;

fun F f x = if x = 0 then 1 else x*f(x-1);

val factorial = Y F;
```

Hàm đầu Y là phép toán điểm bất động. Hàm thứ hai F là hàm trên các hàm mà điểm bất động của nó của nó là factorial. Cả hai đều là các hàm: sử dụng cú pháp ML $\text{fn } x \rightarrow \dots$ for $\lambda x \dots$, chúng ta sẽ viết hàm F như sau:

```
fun F(f) = fn x =>
    if x=0 then 1 else x*f(x-1)
```

F là hàm mà khi áp dụng vào thuật toán f , trả về hàm mà khi áp dụng vào x , có giá trị được cho bởi biểu thức nếu $x = 0$ thì bằng 1; nếu không thì bằng $x*f(x-1)$.

- a. Chương trình dịch ML sẽ kết luận kiểu gì cho F?
- b. Chương trình dịch ML sẽ kết luận kiểu gì cho Y?

CHƯƠNG 3: PHÂN TÍCH TỪ VỰNG

TỔNG QUAN

Để dịch chương trình từ ngôn ngữ này sang ngôn ngữ khác, chương trình cần phải tách chúng ra, hiểu cấu trúc và ý nghĩa, rồi đặt chúng lại theo cách khác. Kỳ đầu của chương trình dịch sẽ tiến hành phân tích và kỳ sau thực hiện việc tổng hợp.

Việc phân tích sẽ tách thành:

Phân tích từ vựng (Lexical analysis): tách đầu vào thành các từ riêng biệt gọi là từ tố ("tokens");

Phân tích cú pháp (Syntax analysis): phân tích cấu trúc câu của chương trình và

Phân tích ngữ nghĩa (Semantic analysis): tìm ý nghĩa của chương trình.

Phân tích từ vựng nhận dòng các ký tự và tạo ra dòng tên, từ khoá và các dấu cách. Nó bỏ qua các khoảng trắng và ghi chú giữa các từ tố. Tránh làm cho phân tích từ vựng phức tạp, lúc nào cũng phải tính đến các khoảng trắng, ta cần phải tách phân tích từ vựng ra khỏi phân tích ngữ nghĩa. Phân tích từ vựng không phức tạp, nhưng chúng ta sẽ giải quyết nó bằng các công cụ và việc hình thức hoá rất mạnh, vì việc hình thức hoá như vậy rất có ích trong việc giải quyết phân tích từ vựng và các công cụ tương tự có nhiều ứng dụng trong các lĩnh vực khác ngoài chương trình dịch.

3.1 BIỂU THỨC CHÍNH QUI

TỪ TỐ (TOKENS)

Từ tố là một dãy các ký tự mà gắn kết với nhau như một thành phần đơn vị trong văn phạm của ngôn ngữ lập trình. Ngôn ngữ lập trình phân loại các từ tố thành một tập hữu hạn các kiểu từ tố. Chẳng hạn một số kiểu từ tố của một ngôn ngữ lập trình tiêu biểu là:

Type	Examples
ID	foo n14 last
NUM	73 0 00 515 082
REAL	66.1 .5 10. 1e67 5.5e-10
IF	if
COMMA	,
NOTEQ	!=
LPAREN	(

RPAREN)

Từ tổ là một phần câu lệnh như IF, VOID, RETURN được xây dựng từ bảng chữ cái được gọi là các từ dành riêng (*reserved words*) và trong hầu hết các ngôn ngữ nó không được dùng đặt tên biến.

Các ví dụ không phải là từ tổ

<i>Chú giải</i>	<code>/* thử lại */</code>
<i>Hướng tiền xử lý – preprocessor directive</i>	<code>#include<stdio.h></code>
<i>preprocessor directive</i>	<code>#define NUMS 5, 6</code>
<i>Gói lệnh - macro</i>	<code>NUMS</code>
<i>Dấu cách, cách khoảng (tab), và xuống dòng</i>	

Trong các ngôn ngữ yêu cầu đòi hỏi các gói lệnh tiền xử lý, tiền xử lý thao tác trên dòng ký tự nguồn, tạo ra dòng ký tự nguồn khác sau đó truyền sang cho phân tích từ vựng. Có thể tích hợp gói lệnh tiền xử lý với phân tích từ vựng.

Cho chương trình như sau

```
float match0(char *s) /* tìm số không */
{if (!strcmp(s, "0.0", 3))
    return 0.;
}
```

Phân tích từ vựng sẽ trả về dòng

FLOAT	ID(match0)	LPAREN	CHAR	STAR	ID(s)	RPAREN
LBRACE	IF	LPAREN	BANG	ID(strcmp)	LPAREN	ID(s)
COMMA	STRING(0.0)	COMMA	NUM(3)	RPAREN	RPAREN	
RETURN	REAL(0.0)	SEMI	RBRACE	EOF		

ở đây kiểu từ tổ của mỗi từ tổ được thay thế; một số từ tổ như tên biến hoặc hằng xâu, có giá trị ngữ nghĩa gắn với chúng, cho thêm một số thông tin phụ về kiểu từ tổ.

Làm sao có thể mô tả các qui tắc từ vựng của ngôn ngữ lập trình? Trong ngôn ngữ nào có thể viết phân tích từ vựng? Chúng ta có thể mô tả các từ tổ của ngôn ngữ lập trình bằng tiếng Việt; ví dụ sau đây là mô tả tên biến trong C hoặc Java:

Tên biến là dãy các chữ và các số; ký tự đầu tiên bắt buộc là chữ. Dấu gạch dưới cũng coi như chữ. Chữ hoa và chữ thường là khác nhau. Nếu dòng đầu vào đã tách thành các từ tổ cho đến ký tự đã cho, thì từ tổ tiếp theo sẽ được lấy bao gồm xâu dài nhất các ký tự mà có thể tạo nên từ tổ. Dấu cách, dấu Tab, xuống dòng và chú giải được bỏ qua trừ khi chúng được dùng để tách các từ tổ. Một khoảng trắng được đòi hỏi để tách các biến, từ khoá và hằng kề nhau.

Và nói chung, bất kỳ ngôn ngữ lập trình suy luận được nào cũng cài đặt bộ phân tích từ vựng đơn giản. Nhưng chúng ta sẽ đặc tả các từ tổ dùng ngôn ngữ hình thức của các biểu thức chính qui, cài đặt các từ tổ sử dụng otomat hữu hạn tất định, và sử dụng toán học để kết nối hai khái niệm đó. Điều đó dẫn đến bộ phân tích từ vựng đơn giản và dễ đọc hơn.

BIỂU THỨC CHÍNH QUI

Chúng ta có thể nói ngôn ngữ là tập hợp các xâu; mỗi xâu là dãy hữu hạn các ký hiệu. Bản thân các ký hiệu được lấy từ một bảng chữ hữu hạn.

Ngôn ngữ Pascal là tập hợp các xâu mà tạo thành các chương trình Pascal hợp lệ. Ngôn ngữ của các số nguyên tố là tập các xâu các chữ số thập phân, mà thể hiện các số nguyên tố. Ngôn ngữ của các từ dành riêng trong C là tập tất cả các xâu trên bảng chữ mà không thể dùng để đặt tên cho các biến trong ngôn ngữ lập trình C. Hai ngôn ngữ đầu là ngôn ngữ vô hạn, ngôn ngữ sau là hữu hạn. Trong tất cả các trường hợp trên, bảng chữ là tập các ký tự ASCII.

Khi chúng ta nói về ngôn ngữ như trên, chúng ta không gán ý nghĩa nào cho các xâu; chúng ta mới chỉ phân loại từng xâu có thuộc ngôn ngữ hay không.

Để đặc tả một số ngôn ngữ có thể là vô hạn với mô tả hữu hạn, chúng ta sẽ sử dụng khái niệm biểu thức chính qui. Mỗi biểu thức chính qui đại diện cho một tập các xâu.

Ký hiệu - Symbol: Đối với mỗi ký hiệu a trong bảng chữ của ngôn ngữ, biểu thức chính qui a ký hiệu ngôn ngữ chứa đúng một xâu là chữ a .

Tùy chọn - Alternation: Cho hai biểu thức chính qui M và N , phép toán tùy chọn được viết như gạch dọc $|$ tạo thành biểu thức chính qui mới $M | N$. Một xâu thuộc ngôn ngữ $M | N$, nếu nó thuộc ngôn ngữ M hoặc thuộc ngôn ngữ N . Như vậy ngôn ngữ của $a | b$ chứa hai xâu a và b .

Phép ghép - Concatenation: Cho hai biểu thức chính qui M và N , phép toán ghép $\cdot$ tạo thành biểu thức chính qui mới $M \cdot N$. Một xâu thuộc ngôn ngữ $M \cdot N$ nếu nó được ghép từ hai xâu α và β sao cho α thuộc ngôn ngữ M và β thuộc ngôn ngữ N . Như vậy, biểu thức chính qui $(a | b) \cdot a$ xác định ngôn ngữ chứa hai xâu aa và ba .

Epsilon: Biểu thức chính qui ϵ biểu diễn ngôn ngữ mà xâu duy nhất của nó là xâu rỗng. Như vậy, $(a \cdot b) | \epsilon$ biểu diễn ngôn ngữ $\{\epsilon, "ab"\}$.

Phép lặp - Repetition: Cho biểu thức chính qui M , bao đóng Kleene của nó là M^* . Một xâu thuộc M^* nếu nó là ghép của không hay nhiều xâu, mà mọi xâu đó thuộc M . Như vậy, $((a | b) \cdot a)^*$ biểu diễn tập vô hạn $\{\epsilon, "aa", "ba", "aaaa", "baaa", "aaba", "baba", "aaaaaa", \dots\}$.

Sử dụng các ký hiệu, phép tùy chọn, phép ghép, epsilon, và phép lặp Kleene chúng ta có thể đặc tả tập các ký hiệu ASCII tương ứng với các từ tổ của ngôn ngữ lập trình. Trước hết, xét các ví dụ sau:

$(0 | 1)^* \cdot 0$ Số chẵn biểu diễn dạng bít.

$b^*(abb^*)^*(a| \epsilon)$ Các xâu gồm a và b mà không có hai a liên tiếp.

$(a|b)^*aa(a|b)^*$ Các xâu gồm a và b mà chứa hai a liên tiếp.

Khi viết các ngôn ngữ lập trình, chúng ta đôi khi bỏ qua ký hiệu phép ghép hoặc epsilon, và chúng ta giả thiết rằng phép đóng Kleene có độ ưu tiên cao hơn phép ghép, và phép ghép cao hơn tùy chọn; như vậy $ab | c$ có nghĩa là $(a \cdot b) | c$, và $(a |)$ có nghĩa là $(a | \epsilon)$.

Bây giờ chúng ta đưa ra một số ký hiệu viết tắt: $[abcd]$ có nghĩa là $(a | b | c | d)$, $[b-g]$ có nghĩa là $[bcdefg]$, $[b-gM-Qkr]$ có nghĩa là $[bcdefgMNOPQkr]$, $M?$ có nghĩa là $(M | \epsilon)$, và M^+ có nghĩa là $(M \cdot M^*)$. Những mở rộng này tạo ra sự thuận tiện, nhưng không làm tăng khả năng mô tả của các biểu thức chính qui: Tập tất cả các xâu mà có thể mô tả bởi các ký hiệu viết tắt này cũng có thể được mô tả bởi các phép toán cơ bản. Mọi phép toán này được tổng hợp trong Hình 3.1.

a	Ký tự bình thường biểu diễn chính nó.
ϵ	Xâu rỗng. Cách khác viết xâu rỗng.
$M N$	Tùy chọn, chọn M hoặc N .
$M \cdot N$	Phép ghép, M được nối tiếp bởi N .
MN	Cách khác viết phép ghép.
M^*	Phép lặp (không hoặc một số lần).
M^+	Phép lặp, một hoặc một số lần.
$M?$	Không hoặc một lần xuất hiện của M .
$[a - zA - Z]$	Tập ký tự tùy chọn.
$\cdot$	Dấu chấm ký hiệu ký tự đơn bất kỳ trừ dấu dòng mới.
"a . + *"	Ngoặc kép, xâu trong ngoặc kép ký hiệu hằng xâu cho chính nó.

Hình 3.1: Ký hiệu các biểu thức chính qui.

Sử dụng ngôn ngữ này, chúng ta có thể đặc tả các từ tổ của một ngôn ngữ lập trình (Hình 3.2).

Dòng thứ năm mô tả đoán nhận các chú giải hoặc khoảng trắng, nhưng không báo lại Bộ phân tích. Chú giải cho bộ phân tích này được bắt đầu bởi hai dấu gạch ngang, chỉ chứa các ký tự từ điển và kết thúc với ký hiệu xuống dòng.

if	IF
[a-z][a-z0-9]*	ID
[0-9]+	NUM
([0-9]+ "." [0-9]*) ([0-9]* "." [0-9]+)	REAL - số thực
("--"[a-z]*"\n") (" " "\n" "\t")+	không phải từ tổ, chỉ là khoảng trắng
.	lỗi

Hình 3.2: Biểu thức chính qui đối với một số từ tổ.

Cuối cùng, đặc tả từ vựng cần phải là *hoàn chỉnh*, luôn sánh xâu con khởi đầu của một đầu vào; chúng ta luôn đạt được điều này bằng cách có qui tắc mà sánh ký tự đơn bất kỳ (và trong trường hợp này, in ra thông báo lỗi "ký tự không hợp lệ" và tiếp tục).

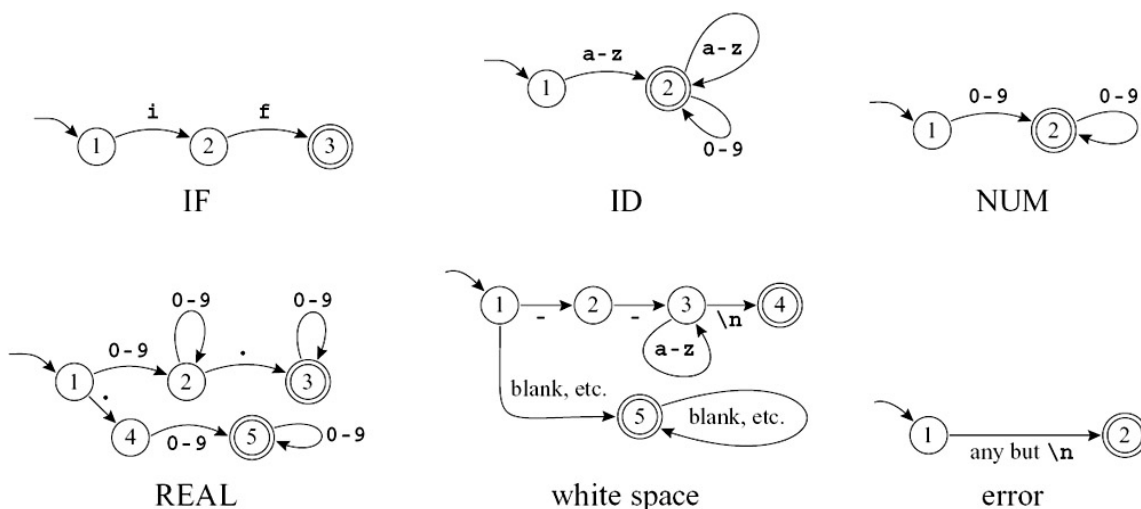
Các qui tắc này hơi nhập nhằng một chút. Chẳng hạn, *if8* sánh với tên biến hay là hai từ tổ *if* và *8*? Xâu *if 89* bắt đầu bởi tên biến hay là từ dành riêng. Có hai qui tắc tránh nhập nhằng quan trọng được sử dụng bởi Lex, JavaCC, SableCC và các bộ sinh phân tích từ vựng khác:

- **Sánh dài nhất:** Xâu con bắt đầu dài nhất của đầu vào mà có thể sánh với biểu thức chính qui bất kỳ sẽ được lấy như từ tổ tiếp theo.
- **Ưu tiên Qui tắc:** Đối với xâu con bắt đầu dài nhất nào đó, biểu thức chính qui đầu tiên mà sánh được sẽ xác định kiểu từ tổ. Như vậy thứ tự thoả mãn các biểu thức chính qui cũng có ý nghĩa quan trọng.

Chẳng hạn *if8* sánh với tên biến theo qui tắc sánh dài nhất, và *if* sánh với từ dành riêng theo qui tắc ưu tiên.

3.2 OTOMAT HỮU HẠN

Biểu thức chính qui thuận tiện cho đặc tả các từ tổ, nhưng chúng ta cần hình thức để có thể được cài đặt trong chương trình máy tính. Để làm điều đó chúng ta sử dụng otomat hữu hạn. Otomat hữu hạn có một tập hữu hạn các trạng thái; các cạnh đi từ trạng thái này đến trạng thái kia, và cạnh được gán nhãn bởi ký hiệu. Một trạng thái là trạng thái ban đầu và một số trạng thái được đánh dấu là các trạng thái kết thúc. Hình 3.3 nêu ra một số trạng thái hữu hạn. Chúng ta đánh số trạng thái cho thuận tiện khi thảo luận. Trạng thái bắt đầu nói chung đánh số 1. Cạnh có thể gán nhãn một số ký tự cho gọn trong trường hợp có nhiều cạnh giữa hai trạng thái; chẳng hạn trong máy ID có 26 cạnh từ đỉnh 1 đến đỉnh 2, mỗi cạnh gán một chữ cái khác nhau.

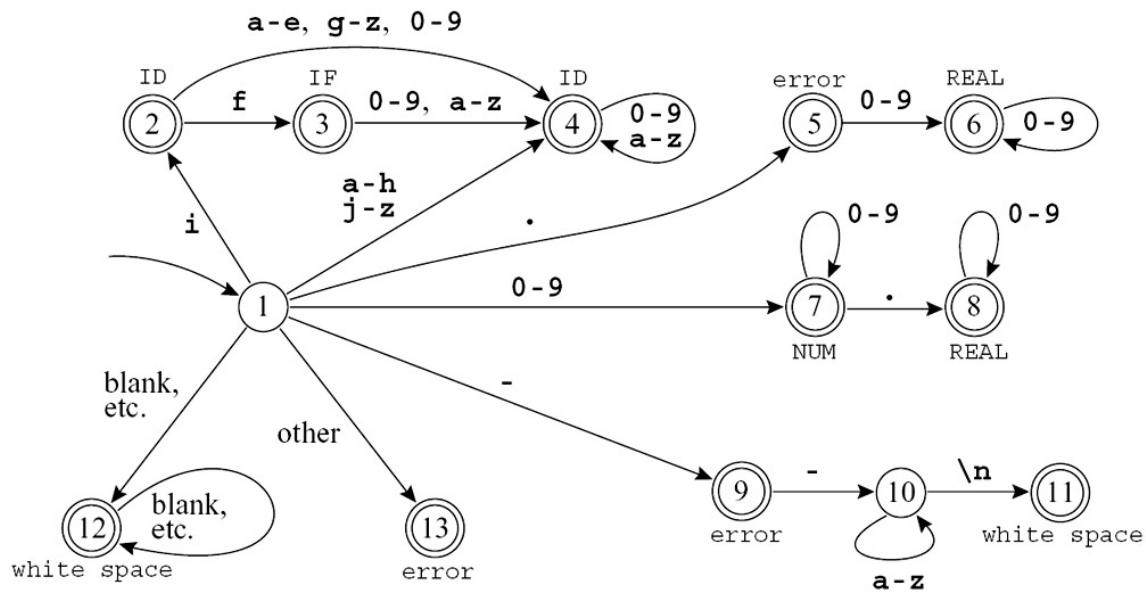


Hình 3.3: Otomat hữu hạn cho các từ tổ. Các trạng thái được chỉ ra bằng các khoanh tròn; các trạng thái kết được chỉ rõ bằng khoanh tròn kép. Trạng thái bắt đầu có mũi tên đến không xuất phát từ đâu. Cạnh được gán nhãn bằng một số ký tự được viết tắt thay cho các cạnh song song

Trong otomat tất định hữu hạn (DFA), không có hai cạnh từ một trạng thái mà có cùng một nhãn như nhau. Otomat tất định hữu hạn (DFA) chấp nhận hoặc từ chối các chuỗi như sau. Bắt đầu từ trạng thái ban đầu, đối với mỗi ký tự của chuỗi đầu vào, otomat chuyển theo một cạnh đến một trạng thái tiếp theo. Cạnh cần phải gán nhãn với các ký tự đầu vào. Sau khi thực hiện n phép chuyển dịch ứng với chuỗi đầu vào n ký tự, nếu otomat ở trạng thái kết, thì nó chấp nhận chuỗi đầu vào đó. Nếu nó không ở trạng thái kết, hoặc ở một thời điểm nào đó, không có cạnh có nhãn tương ứng, thì otomat từ chối đầu vào đó. Ngôn ngữ đoán nhận được bởi otomat chính là tập tất cả các chuỗi mà otomat đó chấp nhận.

Chẳng hạn, rõ ràng rằng, một chuỗi bất kỳ trong ngôn ngữ đoán nhận bởi otomat ID cần phải bắt đầu bởi một chữ nào đó. Một chữ đơn bất kỳ cũng dẫn đến trạng thái 2, mà là trạng thái kết; như vậy chuỗi chữ đơn bất kỳ cũng được chấp nhận. Từ trạng thái 2, một chữ hay một số dẫn ngược trở lại trạng thái 2, như vậy chuỗi bắt đầu bởi chữ và tiếp sau là các số hoặc các chữ cũng được chấp nhận. Trên thực tế, máy được nêu trong Hình 3.3 chấp nhận cùng ngôn ngữ như các biểu thức chính quy trong Hình 3.2.

Đây là sáu otomat rời nhau, làm sao có thể kết hợp chúng lại thành otomat duy nhất để làm bộ phân tích từ vựng. Chúng ta sẽ nghiên cứu kỹ cách làm một cách hình thức trong phần sau, nhưng ở đây chúng ta làm theo trực quan: Hình 3.4 chỉ ra máy như vậy. Mỗi trạng thái kết cần gán nhãn kiểu từ tổ mà nó chấp nhận. Trạng thái 2 của máy này phản ánh khía cạnh trạng thái 2 của máy IF và trạng thái 2 của máy ID; vì trạng thái sau là kết thúc, nên trạng thái kết hợp này cũng là kết thúc. Trạng thái 3 giống trạng thái 3 của máy IF và trạng thái 2 của máy ID; vì cả hai đều là trạng thái kết, nên ta theo qui tắc ưu tiên để giải quyết nhập nhằng – chúng ta gán nhãn 3 bởi IF vì muốn từ tổ này được đoán nhận như từ dành riêng, chứ không phải tên biến.



Hình 3.4. Otomat hữu hạn kết hợp

Chúng ta sẽ ký hiệu máy này như ma trận chuyển: mảng hai chiều (véctor của các véctor), được đánh chỉ số dưới bởi số trạng thái và ký tự đầu vào. Ở đây có thể có trạng thái “chết” 0 mà quay vòng đến chính nó với mọi ký tự; chúng ta sử dụng nó để ký hiệu không có cạnh đó.

```
int edges[][] = { /* ...012...-...e f g h i j... */
/* state 0 */    {0,0,...0,0,0...0...0,0,0,0,0,0...},
/* state 1 */    {0,0,...7,7,7...9...4,4,4,4,2,4...},
/* state 2 */    {0,0,...4,4,4...0...4,3,4,4,4,4...},
/* state 3 */    {0,0,...4,4,4...0...4,4,4,4,4,4...},
/* state 4 */    {0,0,...4,4,4...0...4,4,4,4,4,4...},
/* state 5 */    {0,0,...6,6,6...0...0,0,0,0,0,0...},
/* state 6 */    {0,0,...6,6,6...0...0,0,0,0,0,0...},
/* state 7 */    {0,0,...7,7,7...0...0,0,0,0,0,0...},
/* state 8 */    {0,0,...8,8,8...0...0,0,0,0,0,0...},
    et cetera
}
```

Cần có mảng “kết thúc” ánh xạ các số của trạng thái vào hành động ứng với kiểu từ tổ tương ứng - trạng thái kết thúc 2 ánh xạ vào hành động ID.

ĐOÁN NHẬN SÁNH DÀI NHẤT

Dễ nhận thấy bảng trên được sử dụng như thế nào để đoán nhận là chấp nhận hay từ chối các xâu, nhưng công việc của Phân tích từ vựng là tìm sánh dài nhất, khúc đầu dài nhất của đầu vào được xác định đúng. Khi dịch chuyển theo các cung, bộ phân tích sẽ giữ sánh dài nhất có thể và vị trí của chỗ sánh đó. Giữ sánh dài nhất có nghĩa là nhớ lại thời điểm cuối mà otomat ở trạng thái kết với hai biến, trạng thái sau cùng *Last –Final* (trạng thái

kết sau cùng ghi nhận được) và vị trí đầu vào tại trạng thái kết sau cùng *Input-Position-at-Last-Final*. Mỗi khi đạt trạng thái kết, bộ phân tích cập nhật các biến này; khi đạt được trạng thái “chết”, các biến cho biết từ tổ vừa sánh và kết thúc của nó ở đâu.

Hình 3.5 chỉ ra các thao tác của bộ phân tích từ vựng mà đoán nhận sánh dài nhất; lưu ý rằng vị trí đầu vào hiện tại có thể vừa mới ở chỗ mà bộ đoán nhận gặp trạng thái kết. (*Last Final* – Trạng thái kết cuối đi qua; *Current state* – trạng thái hiện tại của Otomat)

Last Final	Current State	Current Input	Accept Action
0	1	$\overline{i}f \text{ --not-a-com}$	
2	2	$i\overline{f} \text{ --not-a-com}$	
3	3	$if\overline{\text{ --not-a-com}}$	
3	0	$if\overline{\text{ --not-a-com}}$	<i>return IF</i>
0	1	$i\overline{f} \text{ --not-a-com}$	
12	12	$if \text{ --not-a-com}$	
12	0	$if \text{ --not-a-com}$	<i>found white space; resume</i>
0	1	$if\overline{\text{ --not-a-com}}$	
9	9	$if \overline{\text{ --not-a-com}}$	
9	10	$if \overline{\text{ --not-a-com}}$	
9	10	$if \overline{\text{ --not-a-com}}$	
9	10	$if \overline{\text{ --not-a-com}}$	
9	10	$if \overline{\text{ --not-a-com}}$	
9	0	$if \overline{\text{ --not-a-com}}$	<i>error, illegal token '-' ; resume</i>
0	1	$if\overline{\text{ --not-a-com}}$	
9	9	$if- \overline{\text{ --not-a-com}}$	
9	0	$if- \overline{\text{ --not-a-com}}$	<i>error, illegal token '-' ; resume</i>

Hình 3.5. Otomat của hình 3.4 đoán nhận một số từ tổ. Ký hiệu | chỉ ra vị trí đầu vào ở lần gọi tiếp theo của bộ phân tích từ vựng, ký hiệu $\perp$ chỉ ra vị trí hiện tại của otomat, và T chỉ ra vị trí mà otomat vừa ở trạng thái kết.

3.3 VĂN PHẠM CHÍNH QUI

Một văn phạm chính qui là một bộ bốn (N, Σ, P, S) , với N là tập hữu hạn các ký hiệu chưa tận, Σ là bảng chữ tận cùng hữu hạn, P là tập hữu hạn các luật sinh và S là chưa tận xuất phát, sao cho mọi luật sinh P là một trong các dạng sau:

1. $B \rightarrow a$ trong đó B là chưa tận trong N và a là một tận cùng trong Σ
2. $B \rightarrow aC$ trong đó B và C ở trong N và a trong Σ

3. $B \rightarrow \epsilon$ trong đó B ở trong N và ϵ ký hiệu chuỗi rỗng.

Ví dụ. Cho văn phạm sau:

```
digit      → 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9
REAL       → digit REAL1
REAL1      → digit REAL1
REAL1      → . INTEGER
INTEGER    → digit INTEGER
INTEGER    → digit
```

Trong đó $\{REAL, REAL1, INTEGER\}$ là tập các chưa tận xuất, $REAL$ là chưa tận xuất phát; $\{0, 1, 2, 3, 4, 5, 6, 7, 8, 9, .\}$ là tập các tận cùng. Và ở đây ta sử dụng $|$ để ghép nhiều luật sinh có cùng vế trái: viết $B \rightarrow 0 | 1$ thay cho cặp $B \rightarrow 0$ và $B \rightarrow 1$.

Lấy một số nguyên và một số thập phân bất kỳ, chẳng hạn 2015 và 45.23, ta có dãy suy diễn sau bắt đầu từ $REAL$, mỗi lần áp dụng một luật sinh và dừng ở một trong 2 số trên:

$REAL \rightarrow INTEGER \rightarrow \text{digit } INTEGER \rightarrow 2 \text{ } INTEGER \rightarrow 2\text{digit } INTEGER \rightarrow$
 $20 \text{ } INTEGER \rightarrow 20 \text{ digit } INTEGER \rightarrow 201 \text{ } INTEGER \rightarrow 201 \text{ digit} \rightarrow 2015$

$REAL \rightarrow \text{digit } REAL1 \rightarrow 4 \text{ } REAL1 \rightarrow 4 \text{ digit } REAL1 \rightarrow 45 \text{ } REAL1 \rightarrow 45. \text{ } INTEGER$
 $\rightarrow 45.\text{digit } INTEGER \rightarrow 45.2 \text{ } INTEGER \rightarrow 45.2 \text{ } INTEGER \rightarrow 45.2\text{digit} \rightarrow 45.23$

Ta nói 2015 và 45.23 được suy diễn từ Văn phạm trên. Hay còn nói ngôn ngữ sinh ra bởi Văn phạm trên là tập các số nguyên và các số thập phân.

Như ta đã biết, tập các số nguyên và các số thập phân được biểu diễn dạng biểu thức chính qui sau:

$[0-9]^+ \text{ NUM}$
 $([0-9]^+ \text{ " . " } [0-9]^+) \mid ([0-9]^+ \text{ " . " } [0-9]^+) \text{ REAL - số thực}$

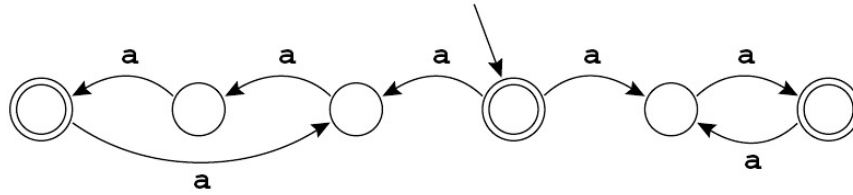
Người ta chứng minh được rằng, một ngôn ngữ được sinh bởi một biểu thức chính qui, thì cũng được sinh bởi một văn phạm chính qui nào đó; và ngược lại: một ngôn ngữ được sinh bởi một văn phạm biểu thức chính qui, thì cũng được sinh bởi một biểu thức chính qui nào đó. Các ngôn ngữ đó được gọi là ngôn ngữ chính qui.

Dễ dàng thực hiện bước chuyển từ văn phạm chính qui sang otomat không tắt định, mà đoán nhận cùng một ngôn ngữ chính qui.

OTOMAT KHÔNG TẮT ĐỊNH HỮU HẠN

Otomat không tắt định hữu hạn (NFA) là máy mà ở một trạng thái nào đó có ít nhất hai cạnh cùng nhãn đi ra hoặc nó có thể có cạnh đặc biệt gán nhãn ϵ , mà có thể biến đổi trạng thái theo đó mà không mất ký tự nào ở đầu vào.

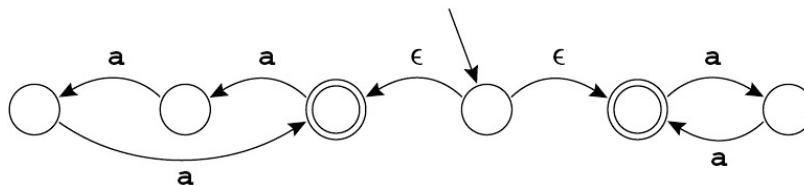
Sau đây là ví dụ về NFA:



Ở trạng thái xuất phát với ký tự đầu vào a , otomat có thể dịch sang phải hoặc sang trái. Nếu sang trái được chọn, thì các xâu gồm toàn chữ a có độ dài là bội của 3 sẽ được chấp nhận. Nếu sang phải được chọn, các xâu có độ dài chẵn được chấp nhận. Như vậy ngôn ngữ đoán nhận bởi otomat không tắt định đó là tập các xâu gồm chữ a và có độ dài chẵn hoặc là bội của 3.

Trong dịch chuyển đầu tiên, máy này sẽ chọn đi theo đường nào. Có thể đòi hỏi chấp nhận sâu là nếu có lựa chọn đường đi dẫn đến việc chấp nhận. Khi đó, cần phải đoán và cần phải luôn đoán đúng.

Các cạnh gán nhãn $\in$ có thể được tính đến mà không cần sử dụng ký hiệu nào ở đầu vào. Sau đây là NFA khác mà cũng đoán nhận cùng một ngôn ngữ với NFA nêu trên:

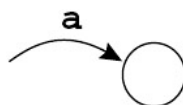


Và ở đây máy trên có thể chọn các cạnh $\in$ để đi. Nếu có trạng thái với một số $\in$ cạnh và một số cạnh gán nhãn bởi các ký hiệu khác, thì máy có thể chọn cách nuốt các ký hiệu đầu vào (và theo các cạnh gán nhãn ký hiệu tương ứng) hoặc thay vào đó đi theo các cạnh $\in$.

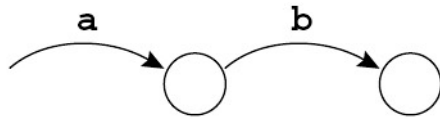
BIẾN ĐỔI BIỂU THỨC CHÍNH QUI THÀNH OTOMAT KHÔNG TẮT ĐỊNH - NFA

Otomat không tắt định là khái niệm hữu ích vì rất dễ dàng chuyển biểu thức chính qui (tĩnh, khai báo) thành NFA (mô phỏng và gần thực hiện).

Thuật toán chuyển đổi biến mỗi biểu thức chính qui thành NFA với đuôi (trạng thái xuất phát) và đầu (trạng thái kết). Chẳng hạn biểu thức chính qui ký hiệu đơn a biến đổi thành NFA trong hình sau.



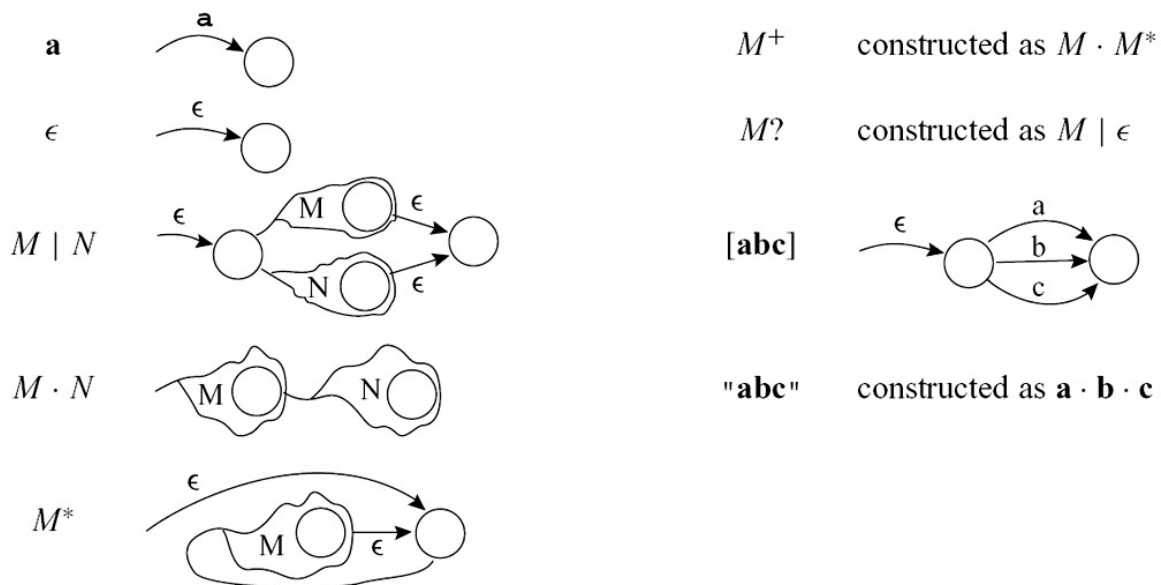
Biểu thức chính qui **ab** , mà kết nối **a** với **b** bằng phép ghép được thực hiện bởi việc ghép nối hai otomat NFA, lắp đầu của **a** với đuôi của **b** . Máy kết quả có đuôi nhãn **a** và đầu là nơi mà cạnh **b** đi đến đó.



Nói một cách tổng quát, mỗi biểu thức chính qui M bất kỳ có một NFA tương ứng nào đó có một đuôi và một đầu.



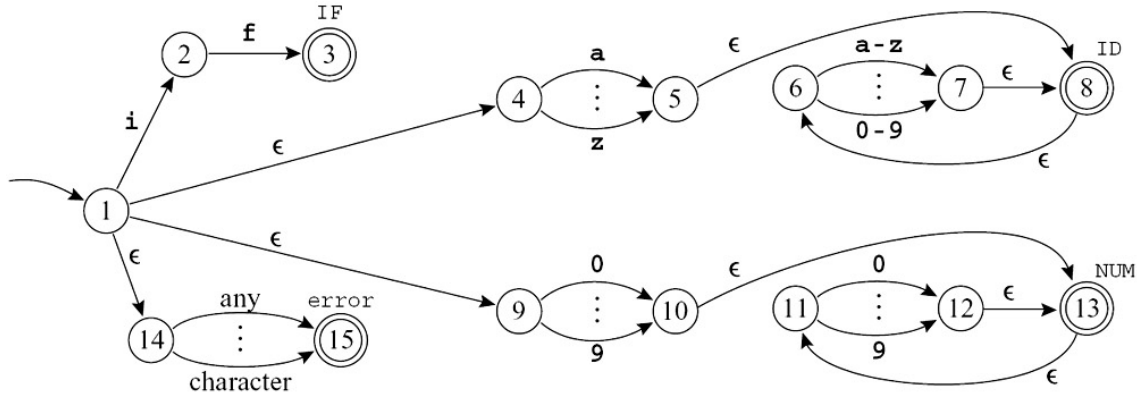
Chúng ta có thể định nghĩa việc chuyển đổi biểu thức chính qui thành NFA bằng đệ qui như sau. Biểu thức chính qui hoặc là nguyên thủy (tức là ký hiệu duy nhất hoặc là ϵ) hoặc được xây dựng từ các biểu thức chính qui nhỏ hơn. Tương tự, NFA hoặc là nguyên thủy hoặc được xây dựng từ các NFA nhỏ hơn.



Hình 3.6: Chuyển biểu thức chính qui thành NFA

Hình 3.6 chỉ ra các qui tắc chuyển biểu thức chính qui thành otomat không tắt định. Chúng ta mô tả thuật toán dựa trên một số biểu thức trên Hình 3.2 - đối với các từ tổ IF, ID, NUM và error. Mỗi biểu thức được chuyển thành NFA, trạng thái “đầu” của mỗi

NFA được đánh dấu kết với kiểu từ tổ khác nhau; và đuôi của mọi biểu thức được nối thành đỉnh xuất phát mới. Kết quả sau khi trộn các trạng thái NFA tương đương được chỉ ra trên Hình 3.7.



Hình 3.7: Bốn biểu thức chính qui được chuyển thành NFA

BIẾN ĐỔI OTOMAT KHÔNG TẮT ĐỊNH (NFA) THÀNH OTOMAT TẮT ĐỊNH (DFA)

Như chúng ta đã thấy việc cài đặt DFA như một chương trình máy tính là rất dễ dàng. Nhưng cài đặt NFA thì chưa khả thi, vì không có phần cứng có thể đoán được đường đi. Chúng ta sẽ loại bỏ việc đoán đường đi bằng cách mỗi lần thử một khả năng có thể.

Giả sử ta mô phỏng NFA trên Hình 3.7 với xâu đầu vào là *in*. Chúng ta bắt đầu từ trạng thái 1. Bây giờ thay vì phải đoán xem sử dụng cạnh ϵ nào, chúng ta sẽ nói là tại thời điểm này NFA có thể lấy bất cứ ϵ cạnh nào trong số chúng, như vậy sẽ là một trong số các trạng thái $\{1, 4, 9, 14\}$. Đây chính là bao đóng ϵ của $\{1\}$, rõ ràng vì không có trạng thái nào thêm có thể đạt được mà không dùng ký tự đầu vào.

Bây giờ chúng ta thực hiện phép chuyển trên ký tự *i*. Từ trạng thái 1 có thể đạt 2, từ trạng thái 4 có thể đạt 5, từ 9 không đi đâu cả, và từ 14 đến 15. Như vậy ta có tập hợp $\{2, 5, 15\}$. Chúng ta cần tính tiếp bao đóng ϵ . Từ 5 có cạnh ϵ đến 8, và từ 8 đến 6. Như vậy NFA phải ở một trong số các trạng thái $\{2, 5, 6, 8, 15\}$.

Với ký tự đầu vào *n*, ta đi từ trạng thái 6 đến trạng thái 7. từ 2 không đi đâu, từ 5 không đi đâu, từ 8 không đi đâu, và từ 15 không đi đâu. Vậy ta có tập hợp $\{7\}$ và bao đóng ϵ của nó là $\{6, 7, 8\}$.

Bây giờ chúng ta đang ở cuối xâu *in*, khi đó NFA có ở trạng thái kết không. Một trong số các trạng thái của tập trạng thái có thể đạt được là 8, đây là trạng thái kết. Do đó *in* là từ tổ ID.

Chúng ta định nghĩa một cách hình thức bao đóng ϵ như sau. Giả sử **edge**(s, c) là tập tất cả các trạng của NFA đạt được từ trạng thái s theo cạnh nhãn c .

Đối với tập các trạng thái S nào đó, **closure**(S) là tập các trạng thái mà đạt được từ trạng thái nào đó trong S mà không cần đọc thêm ký hiệu đầu vào nào, chỉ cần đi theo các cạnh ϵ . Về mặt toán học, chúng ta có thể biểu diễn ý tưởng đi theo các cạnh ϵ bằng cách nói rằng **closure**(S) là tập T nhỏ nhất sao cho:

$$T = S \cup \left(\bigcup_{s \in T} \text{edge}(s, \epsilon) \right).$$

Chúng ta có thể tính T bởi phép lặp:

```

 $T \leftarrow S$ 
repeat  $T' \leftarrow T$ 
         $T \leftarrow T' \cup \left( \bigcup_{s \in T'} \text{edge}(s, \epsilon) \right)$ 
until  $T = T'$ 
    
```

Tại sao thuật toán này cho kết quả mong muốn? T chỉ có thể tăng sau mỗi bước lặp, như vậy T cuối cùng sẽ vẫn chứa S . Nếu $T = T'$ sau một bước lặp nào đó, thì T cũng sẽ chứa cả $\bigcup_{s \in T'} \text{edge}(s, \epsilon)$. Cuối cùng, thuật toán cần phải dừng, vì chỉ có hữu hạn các trạng thái khác nhau trong NFA.

Bây giờ, khi đang mô phỏng NFA như mô tả ở trên, giả sử chúng ta đang ở trong tập $d = \{s_i; s_k; s_l\}$ của các trạng thái NFA là $s_i; s_k; s_l$. Bằng việc xuất phát trong d và nhận ký hiệu vào c , chúng ta đạt được tập mới các trạng thái NFA; gọi tập này là **DFAedge**($d; c$):

$$\text{DFAedge}(d, c) = \text{closure}\left(\bigcup_{s \in d} \text{edge}(s, c)\right)$$

Sử dụng **DFAedge**, chúng ta có thể viết thuật toán mô phỏng NFA một cách hình thức hơn như sau. Nếu trạng thái bắt đầu của NFA là s_1 và xâu đầu vào là $c_1, \dots, c_k$, thì thuật toán là

```

 $d \leftarrow \text{closure}(\{s_1\})$ 
for  $i \leftarrow 1$  to  $k$ 
     $d \leftarrow \text{DFAedge}(d, c_i)$ 
    
```

Thao tác trên tập các trạng thái là rất tốn kém khi chúng ta muốn phân tích cú pháp từng ký hiệu đầu vào trong chương trình nguồn. Nhưng có thể tính toán trước mọi tập trạng thái. Chúng ta xây dựng DFA từ NFA, sao cho mỗi tập của NFA tương ứng với một trạng thái của DFA. Vì NFA có n trạng thái, nên DFA cũng có hữu hạn trạng thái và có nhiều nhất là 2^n trạng thái.

Việc xây dựng DFA là dễ dàng khi mà chúng ta có các thuật toán **closure** và **DFAedge**. Trạng thái bắt đầu d_1 của DFA chính là **closure**(s_1), giống như thuật toán mô phỏng NFA. Một cách khái quát, ở đây có cạnh từ d_i đến d_j gán nhãn là c nếu $d_j = \mathbf{DFAedge}(d_i, c)$. Giả sử Σ là bảng chữ.

```

states[0]  $\leftarrow$  {};   states[1]  $\leftarrow$  closure( $\{s_1\}$ )
p  $\leftarrow$  1;   j  $\leftarrow$  0
while j  $\leq$  p
  foreach c  $\in$   $\Sigma$ 
    e  $\leftarrow$  DFAedge(states[j], c)
    if e = states[i] for some i  $\leq$  p
      then trans[j, c]  $\leftarrow$  i
    else p  $\leftarrow$  p + 1
          states[p]  $\leftarrow$  e
          trans[j, c]  $\leftarrow$  p
  j  $\leftarrow$  j + 1

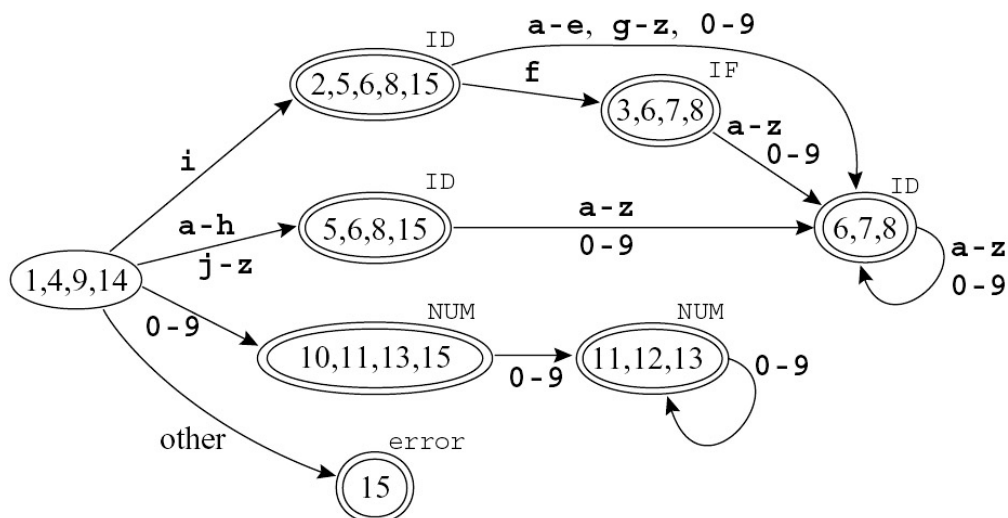
```

Thuật toán không viếng thăm các đỉnh không đạt được của DFA. Điều này rất quan trọng, vì về mặt nguyên tắc DFA có 2^n trạng thái, nhưng trên thực tế chúng ta chỉ tìm được khoảng n trạng thái là đạt được từ trạng thái đầu. Rất quan trọng vì tránh bùng nổ kích thước của bảng chuyển trạng thái DFA của chương trình dịch.

Trạng thái d là kết thúc trong DFA, nếu có một trạng thái nào đó trong $states[d]$ là kết thúc trong NFA. Chỉ gán nhãn trạng thái kết là chưa đủ, chúng ta cần phải chỉ ra từ tổ nào đã được đoán nhận; và có thể một số trạng thái của $states[d]$ là kết thúc trong NFA. Trong trường hợp này chúng ta gán nhãn d bởi kiểu của từ tổ mà xuất hiện *đầu tiên* trong danh sách các biểu thức chính qui mà xây dựng nên đặc tả từ tổ. Điều này muốn nói lên qui tắc ưu tiên được cài đặt như thế nào.

Sau khi DFA xây dựng xong, mảng các trạng thái có thể được bỏ qua, và mảng chuyển được sử dụng để phân tích từ vựng.

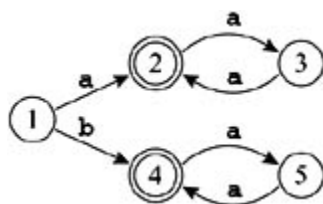
Áp dụng thuật toán xây dựng DFA cho otomat trong Hình 3.7 nhận được otomat trong Hình 3.8



Hình 3.8. Biến NFA thành DFA.

Otomat này gần tối ưu. Nó chưa phải là nhỏ nhất trong số đoán nhận cùng ngôn ngữ. Nói chung, chúng ta nói rằng trạng thái s_1 tương đương với trạng thái s_2 , khi máy xuất phát từ trạng thái s_1 chấp nhận xâu σ khi và chỉ khi máy xuất phát từ trạng thái s_2 chấp nhận xâu σ . Điều đó đúng với cặp trạng thái $\{5, 6, 8, 15\}$ và $\{6, 7, 8\}$; và cặp trạng thái $\{10, 11, 13, 15\}$ và $\{11, 12, 13\}$ trong Hình 3.8. Trong otomat với hai trạng thái s_1 và s_2 tương đương ta có thể cho mọi cạnh đến của s_2 hướng đến s_1 và xóa s_2 .

Làm sao chúng ta có thể tìm được trạng thái tương đương? Tất nhiên, s_1 và s_2 là tương đương nếu chúng cả hai là kết thúc hoặc không kết thúc và với mọi ký hiệu c , và thỏa mãn tiêu chuẩn: $\text{trans}[s_1, c] = \text{trans}[s_2, c]$. Nhưng điều kiện này nói chung là chưa đủ; hãy xét otomat sau



Ở đây, trạng thái 2 và 4 là tương đương, nhưng $\text{trans}[2, a] \neq \text{trans}[4, a]$.

Sau khi xây dựng DFA, rất có ích nếu áp dụng thuật toán tìm otomat tối thiểu bằng cách tìm các trạng thái tương đương; xem Bài tập 3.6.

3.4 BỘ PHÂN TÍCH TỪ VỰNG

Xây dựng DFA là nhiệm vụ cơ học dễ dàng thực hiện bởi máy tính, như vậy thật là có ý nghĩa nếu có bộ sinh phân tích từ vựng tự động để biến biểu thức chính qui thành DFA.

JavaCC và SableCC sinh ra phân tích từ vựng và phân tích cú pháp viết trong ngôn ngữ Java. Phân tích từ vựng được sinh từ đặc tả từ vựng; và như giải thích trong chương tiếp theo, phân tích cú pháp được sinh từ văn phạm.

Đối với cả JavaCC và SableCC đặc tả từ vựng và văn phạm chứa trong cùng một tệp.

JAVACC.

Từ tổ mô tả trong Hình 3.2 được đặc tả trong JavaCC trong Chương trình 3.9. Đặc tả JavaCC bắt đầu từ danh sách tùy chọn các lựa chọn tiếp theo phân dịch Java nằm giữa `PARSER_BEGIN(name)` và `PARSER_END(name)`. Cùng một tên tiếp sau `PARSER_BEGIN` và `PARSER_END`. Đó là cái tên của phân tích cú pháp (Myparser trong Chương trình 3.9). Phân dịch đầy đủ bao gồm khai báo lớp với cùng tên của phân tích cú pháp được sinh.

CHƯƠNG TRÌNH 3.9: Đặc tả JavaCC của các từ tổ trong Hình 3.2.

```
PARSER_BEGIN(MyParser)
    class MyParser {}
PARSER_END(MyParser)
/* For the regular expressions on the right, the token on the left will
be returned:*/
TOKEN : {
    < IF: "if" >
    | < #DIGIT: ["0"-"9"] >
    | < ID: ["a"-"z"] (["a"-"z"]|<DIGIT>) >
    | < NUM: (<DIGIT>)+ >
    | < REAL: ( (<DIGIT>)+ "." (<DIGIT>)* ) |
              ( (<DIGIT>)* "." (<DIGIT>)+ ) >
}
/* The regular expressions here will be skipped during lexical analysis:
*/
SKIP : {
    <"--" (["a"-"z"])* ("\n" | "\r" | "\r\n")>
    | ""
    | "\t"
    | "\n"
}
/* If we have a substring that does not match any of the regular
expressions in TOKEN or SKIP,
    JavaCC will automatically throw an error. */
void Start() :
{}
{ ( <IF> | <ID> | <NUM> | <REAL> )* }
```

Tiếp theo là danh sách các luật sinh của một số kiểu sau: luật biểu thức chính qui xác định từ tổ, khai báo quản trị từ tổ có thể được sử dụng bởi Phân tích từ vựng được sinh, và hai kiểu khác được sử dụng để định nghĩa văn phạm từ đó sinh ra Phân tích cú pháp.

Đặc tả từ tổ sử dụng các luật biểu thức chính qui; có bốn kiểu: `TOKEN`, `SKIP`, `MORE`, và `SPECIAL_TOKEN`. Chúng ta cần `TOKEN` và `SKIP` cho dự án chương trình dịch trong sách này. Kiểu `TOKEN` được sử dụng để đặc tả rằng xâu sánh cần phải được chuyển thành từ tổ để có thể trao đổi với Phân tích cú pháp. Kiểu `SKIP` được sử dụng để đặc tả rằng xâu sánh có thể bỏ đi.

Trong Chương trình 3.9, các đặc tả `ID`, `NUM`, và `REAL` sử dụng viết tắt `DIGIT`. Định nghĩa của `DIGIT` được khai báo trước bởi `#` để chỉ ra rằng nó có thể được sử dụng chỉ trong định nghĩa các từ tổ khác.

Trong phần cuối của Chương trình 3.9 bắt đầu bởi `void Start`. Đây là luật sinh, mà trong trường hợp này, cho phép Phân tích từ vựng được sinh đoán nhận từ tổ bất kỳ trong bốn loại được định nghĩa theo thứ tự tùy ý. Chương tiếp theo sẽ giải thích chi tiết các luật.

SABLECC

Các từ tổ được mô tả trong Hình 3.2 được đặc tả trong SableCC như chỉ ra trong Chương trình 3.10. File đặc tả SableCC có sáu phần (tất cả đều không bắt buộc):

1. Khai báo gói: đặc tả gói gốc cho mọi lớp sinh bởi SableCC.
2. Các khai báo trợ giúp: danh sách viết tắt.
3. Đặc tả trạng thái: hỗ trợ tìm các đặc trưng của trạng thái, chẳng hạn, GNU FLEX; khi mà Phân tích từ vựng ở một trạng thái nào đó, chỉ có các từ tổ liên kết với trạng thái đó được đoán nhận. Trạng thái có thể sử dụng cho nhiều mục đích, bao gồm cả việc phát hiện trạng thái bắt đầu dòng, với mục đích đoán nhận từ tổ chỉ khi chúng xuất hiện ở đầu dòng. Đối với chương trình dịch mô tả trong cuốn sách này, các trạng thái là không cần thiết.
4. Khai báo từ tổ: mỗi từ tổ được dùng để đặc tả rằng xâu sánh cần phải được chuyển thành từ tổ mà được trao đổi với Phân tích cú pháp.
5. Từ tổ bỏ qua: mỗi từ tổ được sử dụng để đặc tả rằng xâu sánh cần được bỏ qua.
6. Luật sinh: được sử dụng để định nghĩa văn phạm để từ đó sinh ra Phân tích cú pháp.

CHƯƠNG TRÌNH 3.10: Đặc tả SableCC cho các từ tổ trong Hình 3.2

```
Helpers
    digit = ['0'..'9'];
Tokens
    if = 'if';
    id = ['a'..'z'](['a'..'z' | (digit))*;
    number = digit+;
    real = ((digit)+ '.' (digit)*) |
           ((digit)* '.' (digit)+);
    whitespace = (' ' | '\t' | '\n')+;
    comments = ('--' ['a'..'z']* '\n');
Ignored Tokens
    whitespace,
    comments;
```

CHƯƠNG TRÌNH PHÂN TÍCH TỪ VỰNG

Viết phần phân tích từ vựng của đặc tả JavaCC hoặc SableCC cho MiniJava. Phụ lục A mô tả cú pháp cho MiniJava. Thư mục

\$MINIJAVA/chap2/javacc

chứa file kiểm tra `Main.java` mà gọi Phân tích từ vựng được sinh bởi `javacc`. Nó cũng chứa file `README` mà giải thích dùng `javacc` như thế nào. Các file tương tự đối với `sablecc` có thể được tìm trong `$MINIJAVA/chap2/sablecc`.

ĐỌC THÊM

Lex là bộ sinh Phân tích từ vựng dựa trên biểu thức chính qui đầu tiên [Lesk 1975]; nó vẫn còn được sử dụng rộng rãi.

Tính bao đóng ϵ có thể được thực hiện hiệu quả hơn nhờ hàng đợi hoặc ngăn xếp các trạng thái mà cạnh của chúng chưa được kiểm tra dịch chuyển $\in$ [Aho, 1986]. Các biểu thức chính qui có thể được chuyển trực tiếp thành DFA mà không cần qua NFA [McNaughton và Yamada 1960; Aho, 1986].

Các bảng chuyển DFA có thể rất lớn và thừa. Nếu được biểu diễn như ma trận hai chiều đơn giản (*trạng thái* $\times$ *ký hiệu*), chúng chiếm nhiều bộ nhớ. Trên thực tế, các bảng được nén lại; nó giảm bộ nhớ cần thiết, nhưng tăng thời gian tìm kiếm trạng thái tiếp theo [Aho, 1986].

Các bộ phân tích từ vựng, mà được tự động sinh hoặc viết tay, cần quản lý đầu vào của chúng một cách hiệu quả. Tất nhiên, đầu vào được lưu vào bộ đệm, sao cho gói lớn các ký tự được nhận một lần; khi đó Phân tích từ vựng có thể xử lý một ký tự một lần trong bộ đệm. Phân tích từ vựng cần kiểm tra, mỗi ký tự, xem đã đạt đến cuối bộ đệm chưa. Bằng cách đặt *lính canh* – là ký tự mà không thể là một phần của từ tố nào – ở cuối bộ đệm, nó có thể cho bộ phân tích từ vựng kiểm tra cuối bộ đệm chỉ một lần cho mỗi từ tố, thay vì một lần cho mỗi ký tự [Aho, 1986]. Gray [1988] sử dụng lược đồ mà yêu cầu kiểm tra một lần cho một dòng, thay vì một lần cho một từ tố, nhưng không thể gói các từ tố với cả ký tự kết thúc dòng. Bumbulis và Cowan [1993] kiểm tra chỉ một lần cho mỗi chu trình trong DFA; điều này làm giảm số lần kiểm tra (từ một cho mỗi ký tự) khi mà có đường dẫn dài trong DFA.

Các bộ Phân tích từ vựng sinh tự động thường bị phê phán là chậm. Về nguyên tắc, thao tác của otomat hữu hạn là rất đơn giản và cần phải hiệu quả, nhưng dịch từ bảng chuyển thêm gánh nặng. Gray [1988] chỉ ra rằng DFA dịch trực tiếp ra mã thực thi (cài đặt trạng thái như các lệnh case) có thể chạy nhanh như các bộ phân tích viết tay. “Bộ sinh Phân tích từ vựng nhanh” Flex [Paxson 1995] là nhanh hơn đáng kể so với Lex.

BÀI TẬP

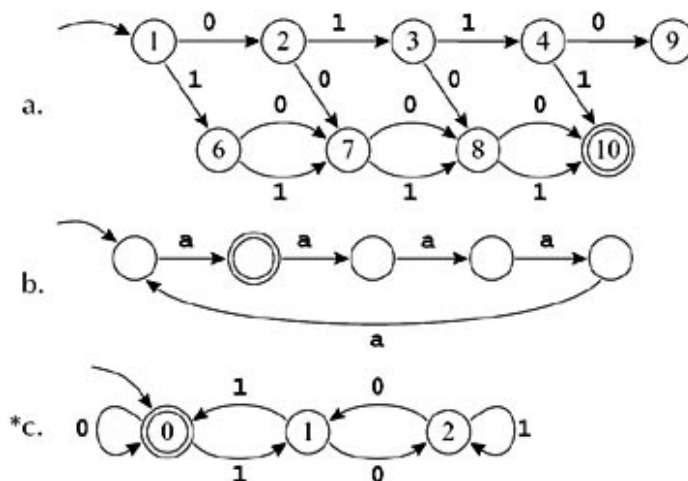
3.1 Viết biểu thức chính qui cho mỗi dạng xâu sau.

- Các xâu trên bảng chữ $\{a, b, c\}$ mà chữ a đầu tiên đứng trước chữ b đầu tiên.
- Các xâu trên bảng chữ $\{a, b, c\}$ với số chẵn số chữ a .
- Các số nhị phân mà là bội số của 4.
- Các số nhị phân lớn hơn 101001.
- Các xâu trên bảng chữ $\{a, b, c\}$ mà không chứa xâu con baa .
- Ngôn ngữ của các hằng số nguyên không âm trong C, ở đó các số mà bắt đầu bằng 0 là các hằng thuộc hệ 16, còn các số khác là các hằng số hệ thập phân.
- Các số nhị phân n sao cho tồn tại nghiệm nguyên của phương trình $a^n + b^n = c^n$.

3.2 Đối với mỗi tập các xâu sau, hãy giải thích vì sao bạn không ngạc nhiên là ở đây không có biểu thức chính qui định nghĩa chúng.

- Các xâu có các chữ a và b trong đó số chữ a nhiều hơn số chữ b .
- Các xâu có chữ a và b mà đối xứng qua tâm theo nghĩa đọc xuôi từ đầu hay đọc ngược từ đuôi đều như nhau.
- Các chương trình Java viết đúng cú pháp.

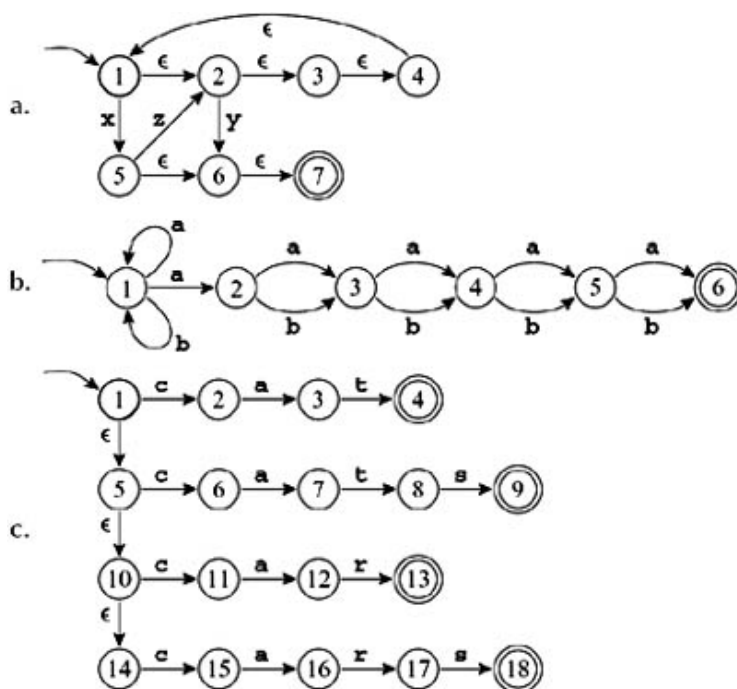
3.3 Giải thích bằng ngôn ngữ tự nhiên, mỗi otomat hữu hạn sau đoán nhận cái gì.



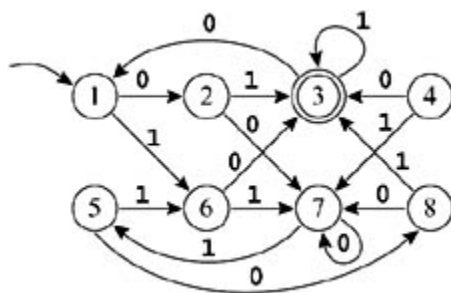
3.4 Chuyển các biểu thức chính qui sau thành otomat hữu hạn không tắt định.

- (if|then|else)
- $a((b|a^*c)x)^*jx^*a$

3.5 Chuyển các NFA sau thành otomat hữu hạn tất định.



3.6 Tìm hai trạng thái tương đương trong otomat sau, và nhập chúng thành một để tạo thành otomat nhỏ hơn mà đoán nhận cùng ngôn ngữ. Lặp lại cho đến khi không còn các trạng thái tương đương.

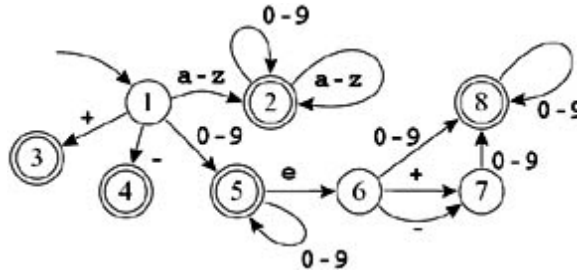


Trên thực tế, thuật toán tổng quan để tối thiểu trạng thái otomat hữu hạn làm việc theo qui trình ngược lại. Trước hết, tìm tất cả các cặp trạng thái không tương đương. Trạng thái X và Y là không tương đương, nếu X là kết thúc, còn Y thì không hoặc (bằng phép lặp) $X \xrightarrow{a} X'$ và $Y \xrightarrow{a} Y'$ và X', Y' là không tương đương. Sau phép lặp này nếu không tìm được cặp không tương đương, thì X, Y là tương đương. Xem Hopcroft và Ullman [1979], Định lý 3.10.

***3.7** Bất kỳ DFA nào mà đoán nhận ít nhất một xâu có thể được chuyển thành biểu thức chính qui. Hãy chuyển DFA trong bài tập 3.3.c thành biểu thức chính qui. Gợi ý: Trước hết, giả định trạng thái 1 là trạng thái bắt đầu. Sau đó viết biểu thức chính qui cho việc thực hiện đến trạng thái 2 và ngược lại, và tương tự cho

việc thực hiện đến trạng thái 0 và ngược lại. Hoặc xem Hopcroft và Ullman [1979], Định lý 2.4, cho thuật toán đó.

***3.8** Giả sử DFA này được sử dụng bởi Lex để tìm các từ tổ trong tệp đầu vào.



- Đi qua bao nhiêu ký tự sau khi kết thúc từ tổ mà Lex cần phải kiểm tra trước khi sánh từ tổ cuối cùng đó?
- Giả sử câu trả lời là k cho phần (a), hãy chỉ ra file đầu vào chứa ít nhất hai từ tổ sao cho lần gọi Lex thứ nhất sẽ kiểm tra k ký tự sau khi đi qua từ tổ đầu tiên trước khi trả về từ tổ đầu tiên đó. Nếu câu trả lời phần (a) là 0 thì chỉ ra file đầu vào chứa ít nhất hai từ tổ và chỉ ra điểm cuối của từng từ tổ.

3.9 Bộ phân tích từ vựng dựa trên DFA sử dụng hai bảng,

Bảng cạnh đánh số bởi trạng thái và ký hiệu đầu vào với trạng thái phân biệt bởi số và bảng kết thúc đánh số bởi trạng thái trả về 0 hoặc số - hành động.

Bắt đầu với đặc tả từ vựng sau,

```
(aba)+      (action 1);
(a(b*)a)    (action 2);
(a|b)       (action 3);
```

Hãy sinh ra bảng cạnh và bảng kết thúc cho Phân tích từ vựng đó.

Sau đó chỉ ra mỗi bước của từ vựng trên xâu đầu vào `abaabbaba`. Hãy chỉ ra các giá trị của các biến trong quan trọng của Đoán nhận đó. Có thể có lời gọi lặp đến Phân tích từ vựng để nhận được các từ tổ thành công.

****3.10** Lex có thao tác nhìn trước (*lookahead operator* /) sao cho biểu thức chính qui `abc/def` sánh `abc` chỉ khi sau đó được nối tiếp bởi `def` (nhưng `def` không là phần của xâu sánh, và sẽ là phần của từ tổ tiếp theo). Aho [1986] mô tả, và Lex [Lesk 1975] sử dụng thuật toán không đúng cho việc cài đặt xem trước (nó sai trên biểu thức `(a|ab)/ba` với đầu vào `aba`, sánh `ab` nhẽ ra phải sánh `a`). Flex [Paxson 1995] sử dụng cơ chế tốt hơn mà làm việc đúng đối với `(a|ab)/ba` nhưng sai (với cảnh báo) trên `zx*/xy*`.

Hãy thiết kế cơ chế xem trước tốt hơn.

CHƯƠNG 4: PHÂN TÍCH CÚ PHÁP (PARSING)

Cú pháp (syn-tax): cách mà các từ được đặt gần nhau để tạo thành các lệnh hay các câu.

TỔNG QUAN

Cơ chế viết tắt đã thảo luận ở chương trước, ở đó các ký hiệu thay cho biểu thức chính qui, là khá thuận lợi trong việc sử dụng:

$$\begin{aligned} \text{digits} &= [0 - 9]^+ \\ \text{sum} &= (\text{digits} "+")^* \text{digits} \end{aligned}$$

Các biểu thức chính qui này xác định tổng có dạng $28+301+9$.

Nhưng bây giờ ta xem xét

$$\begin{aligned} \text{digits} &= [0 - 9]^+ \\ \text{sum} &= \text{expr} "+" \text{expr} \\ \text{expr} &= "(" \text{sum} ")" \mid \text{digits} \end{aligned}$$

Nó được dùng để định nghĩa các biểu thức dạng:

$$\begin{aligned} &(109+23) \\ &61 \\ &(1+(250+3)) \end{aligned}$$

ở đó mọi dấu mở, đóng ngoặc được cân đối. Nhưng nó không thể dùng otomat hữu hạn để đoán nhận sự cân bằng các dấu ngoặc (vì máy với N trạng thái không thể nhớ các dấu ngoặc lồng nhau sâu hơn N), do đó rõ ràng sum và expr không thể là biểu thức chính qui.

Vậy phân tích từ vựng cài đặt viết tắt biểu thức chính qui như thế nào, chẳng hạn như số - digits ? Câu trả lời là về bên phải $[0-9]^+$ đơn giản được thế cho số digits bất cứ ở đâu mà nó xuất hiện trong biểu thức chính qui, trước khi chuyển đổi sang otomat hữu hạn.

Điều này là không thể đối với ngôn ngữ sum và expr ; trước hết chúng ta có thể thế sum thành tổng hai expr , được

$$\text{expr} = "(" \text{expr} "+" \text{expr} ")" \mid \text{digits}$$

nhưng bây giờ thử thay thế expr cho chính nó dẫn đến

$$\text{expr} = "(" ("(" \text{expr} "+" \text{expr} ")" \mid \text{digits}) "+" \text{expr} ")" \mid \text{digits}$$

và về phải bây giờ có rất nhiều sự xuất hiện của *expr* vì nó được thể như trước đó – trên thực tế, nó có nhiều hơn!

Như vậy, ký hiệu viết tắt cũng không làm tăng được khả năng thể hiện cho ngôn ngữ của các biểu thức chính qui - ở đây không có ngôn ngữ nào được thêm - trừ khi việc viết tắt là đệ qui (hoặc đệ qui nhiều chiều, như tổng *sum* và biểu thức *expr*).

Khả năng thể hiện tăng nhận được từ đệ qui chính là điều chúng ta cần để phân tích cú pháp. Vậy, mỗi lần chúng ta dùng viết tắt với đệ qui, chúng ta không cần nhiều lựa chọn ở mức cao nhất của biểu thức, vì định nghĩa

$$expr = ab(c \mid d)e$$

luôn có thể được viết lại sử dụng định nghĩa phụ như sau

$$\begin{aligned} aux &= c \mid d \\ expr &= a b aux e \end{aligned}$$

Thực tế, thay vì dùng ký hiệu tùy chọn nói chung, chúng ta có thể chỉ cần viết một số mở rộng cho phép cho cùng một ký hiệu:

$$\begin{aligned} aux &= c \\ aux &= d \\ expr &= a b aux e \end{aligned}$$

Bao đóng Kleene là không cần thiết, vì chúng ta có thể viết lại nó như sau, chẳng hạn biểu thức bao đóng

$$expr = (a b c)^*$$

trở thành

$$\begin{aligned} expr &= (a b c) expr \\ expr &= \epsilon \end{aligned}$$

Cái gì mà chúng ta đang có là một khái niệm rất đơn giản, gọi là văn phạm phi ngữ cảnh - (*context-free grammars*). Cũng như biểu thức chính qui có thể được sử dụng để định nghĩa cấu trúc từ vựng ở dạng khai báo tĩnh, văn phạm cũng định nghĩa cấu trúc cú pháp bằng khai báo. Nhưng chúng ta cần một cái gì đó mạnh hơn otomat hữu hạn để phân tích ngôn ngữ được mô tả bằng văn phạm.

Trên thực tế, văn phạm có thể được sử dụng để mô tả cấu trúc của các từ tổ từ vựng, mặc dù biểu thức chính qui là phù hợp cho mục đích đó.

4.1 VĂN PHẠM PHI NGỮ CẢNH

Như trước, ta nói ngôn ngữ là tập các xâu; mà mỗi xâu là một dãy hữu hạn các ký hiệu trên một bảng chữ hữu hạn. Đối với phân tích cú pháp, các xâu là các chương trình nguồn, các ký hiệu là các từ tổ từ vựng, và bảng chữ là tập hợp các kiểu từ tổ được trả về từ phân tích từ vựng.

Văn phạm phi ngữ cảnh cũng mô tả một ngôn ngữ. Văn phạm có một tập các luật sinh dạng

$$symbol \rightarrow symbol\ symbol\ \dots\ symbol$$

trong đó ở vế phải không có hoặc có nhiều ký hiệu. Mỗi ký hiệu hoặc là chưa tận (*nonterminal*), hoặc là tận cùng (*terminal*), có nghĩa đây là từ tổ trên bảng chữ các xâu trong ngôn ngữ đang xét, hoặc là chưa tận, tức là nó xuất hiện bên trái của một luật sinh nào đó. Cuối cùng, một chưa tận được đánh dấu đặc biệt là ký hiệu bắt đầu (*start symbol*) của văn phạm.

Văn phạm 4.1 là ví dụ về văn phạm của các chương trình dòng lệnh trực tiếp. Ký hiệu bắt đầu là S (khi ký hiệu bắt đầu không viết tường minh thì ta qui ước: chưa tận bên trái của luật sinh đầu tiên là ký hiệu bắt đầu). Các ký hiệu tận cùng là

```
id print num, + ( ) := ;
```

VĂN PHẠM 4.1: Cú pháp cho chương trình dòng lệnh trực tiếp.

1. $S \rightarrow S; S$
2. $S \rightarrow \text{id} := E$
3. $S \rightarrow \text{print } (L)$
4. $E \rightarrow \text{id}$
5. $E \rightarrow \text{num}$
6. $E \rightarrow E + E$
7. $E \rightarrow (S, E)$
8. $L \rightarrow E$
9. $L \rightarrow L, E$

và các chưa tận là S , E , và L . Chẳng hạn, một câu trong ngôn ngữ của văn phạm này là

```
id := num; id := id + (id := num + num, id)
```

trong đó văn bản nguồn (trước phân tích từ vựng) có thể có dạng

```
a := 7;  
b := c + (d := 5 + 6, d)
```

Kiểu từ tổ (ký hiệu tận cùng) là $id, num, :=, \dots$; các tên (a, b, c, d) và các số $(7, 5, 6)$ là các giá trị ngữ nghĩa liên kết với các từ tổ đó.

DẪN XUẤT (DERIVATIONS)

Để chỉ ra rằng một câu thuộc ngôn ngữ sinh bởi văn phạm, chúng ta phải thực hiện các phép dẫn xuất: bắt đầu từ ký hiệu xuất phát, sau đó lặp phép thay chưa tận bất kỳ bằng một trong những vế phải của nó, như chỉ ra trên Dẫn xuất 4.2.

- $\underline{S}$
- $S ; \underline{S}$
- $\underline{S} ; id := E$
- $id := \underline{E} ; id := E$
- $id := num ; id := \underline{E}$
- $id := num ; id := \underline{E} + E$
- $id := num ; id := \underline{E} + (S, E)$
- $id := num ; id := id + (\underline{S}, E)$
- $id := num ; id := id + (id := \underline{E}, E)$
- $id := num ; id := id + (id := E + E, \underline{E})$
- $id := num ; id := id + (id := \underline{E} + E, id)$
- $id := num ; id := id + (id := num + \underline{E}, id)$
- $id := num ; id := id + (id := num + num, id)$

DẪN XUẤT 4.2.

Có rất nhiều dẫn xuất của cùng một câu. Dẫn xuất trái nhất là dẫn xuất mà ở đó ký hiệu chưa tận bên trái nhất luôn được áp dụng để triển khai; trong dẫn xuất phải nhất, ký hiệu chưa tận phải nhất luôn được áp dụng để triển khai.

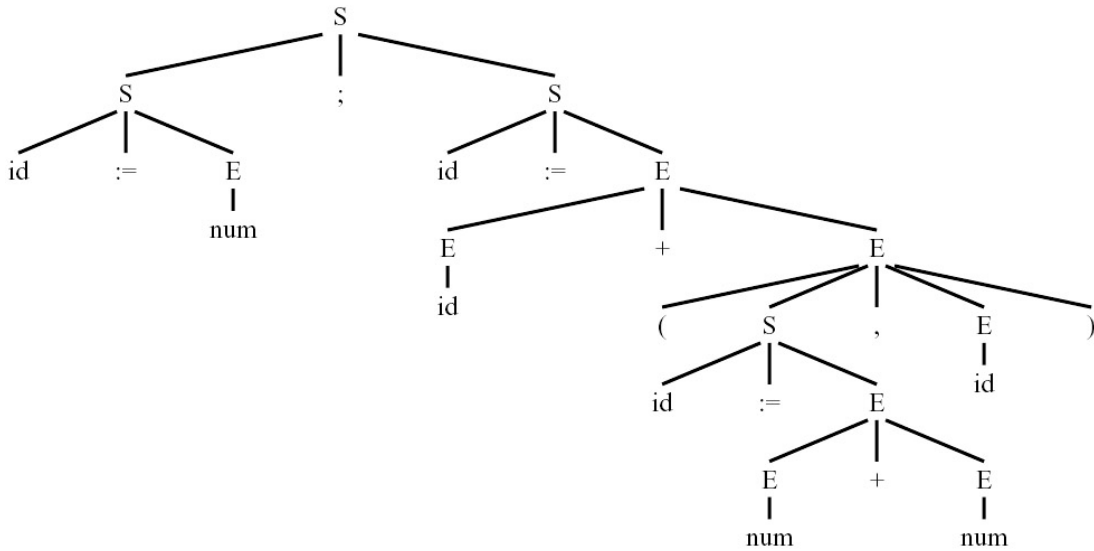
Dẫn xuất 4.2 không là trái nhất hoặc phải nhất; Dẫn xuất trái nhất cho câu này là dẫn xuất sau:

- $\underline{S}$
- $\underline{S} ; S$
- $id := \underline{E} ; S$
- $id := num ; \underline{S}$
- $id := num ; id := \underline{E}$
- $id := num ; id := \underline{E} + E$
- $\vdots$

4.2 PHÂN TÍCH VĂN PHẠM PHI NGỮ CẢNH

CÂY CÚ PHÁP - PARSE TREES

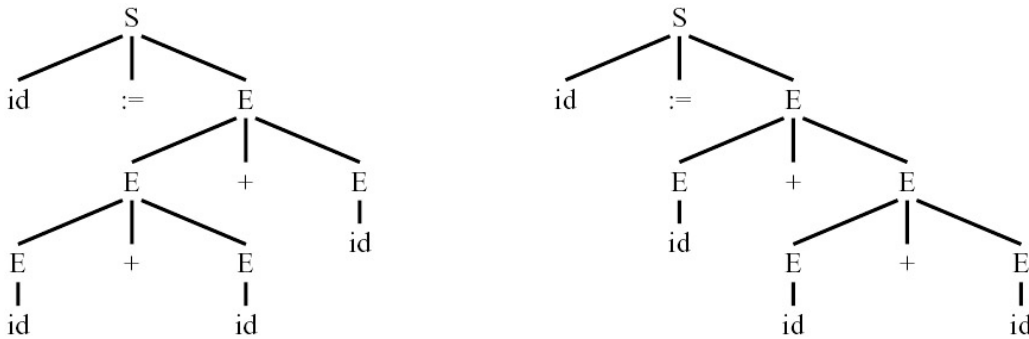
Cây cú pháp được tạo ra bởi việc liên kết mỗi đỉnh trong dẫn xuất với một luật sinh mà từ đó nó được suy diễn, như đã chỉ trên Hình 4.3. Hai dẫn xuất khác nhau có thể có cùng cây cú pháp.



Hình 4.3: Cây cú pháp.

VĂN PHẠM NHẬP NHẰNG - AMBIGUOUS GRAMMARS

Văn phạm được gọi là nhập nhằng nếu nó có thể dẫn xuất một câu với hai cây cú pháp khác nhau. Văn phạm 4.1 là nhập nhằng, vì câu $id := id + id + id$ có hai cây cú pháp khác nhau (Hình 4.4).



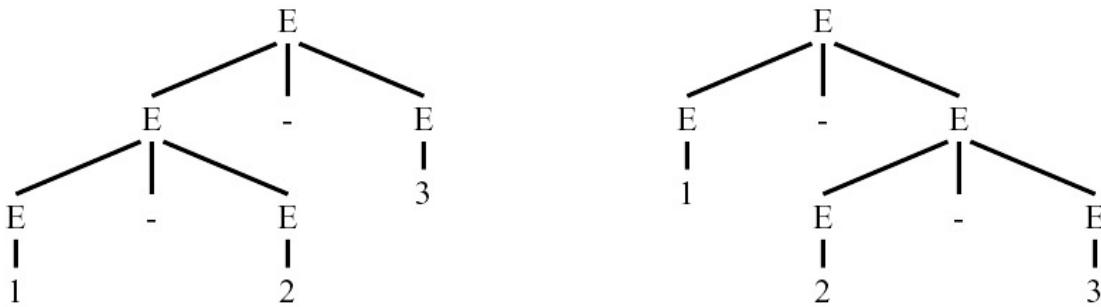
Hình 4.4: Hai cây cú pháp cho cùng một câu của văn phạm 4.1.

VĂN PHẠM 4.5

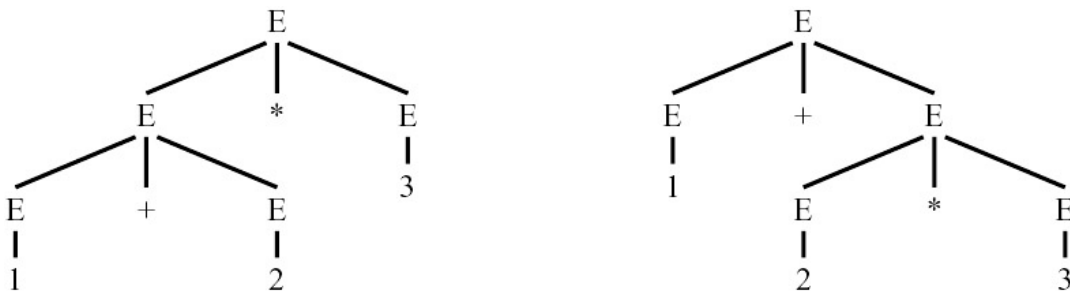
- $E \rightarrow id$
- $E \rightarrow num$
- $E \rightarrow E * E$

- $E \rightarrow E / E$
- $E \rightarrow E + E$
- $E \rightarrow E - E$
- $E \rightarrow (E)$

Văn phạm 4.5 cũng là nhập nhằng; Hình 4.6 chỉ ra hai cây cho câu $1-2-3$, và Hình 4.7 chỉ ra hai cây cho câu $1+2*3$. Rõ ràng, nếu chúng ta sử dụng cây cú pháp để dịch ý nghĩa của biểu thức, thì hai cây cú pháp của $1-2-3$ có nghĩa khác nhau: $(1-2)-3 = -4$ khác với $1-(2-3) = 2$. Tương tự, $(1+2)*3$ không cùng nghĩa với $1+(2*3)$. Trên thực tế, chương trình dịch sẽ dùng cây cú pháp để rút ra ý nghĩa của câu.



Hình 4.6: Hai cây cú pháp cho câu $1-2-3$ trong Văn phạm 4.5.



Hình 4.7: Hai cây cú pháp cho câu $1+2*3$ trong Văn phạm 4.5..

Như vậy, văn phạm nhập nhằng là vấn đề khó khăn khi dịch: nói chung, chúng ta mong muốn có văn phạm không nhập nhằng. May mắn, chúng ta có thể biến đổi văn phạm nhập nhằng thành văn phạm không nhập nhằng.

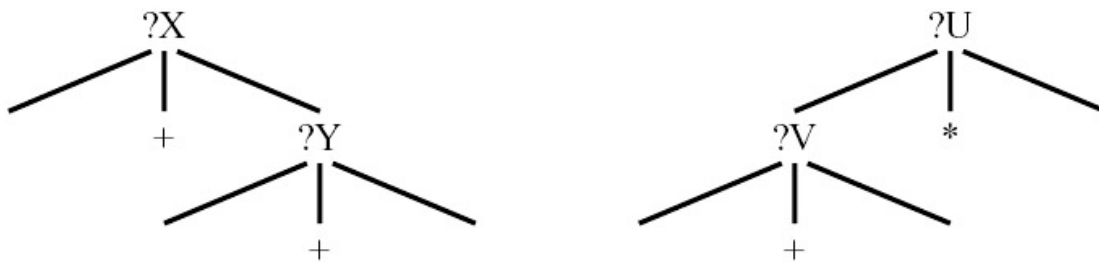
Bây giờ chúng ta tìm văn phạm không nhập nhằng mà đoán nhận cùng ngôn ngữ với văn phạm 4.5. Trước hết, chúng ta có thể nói rằng phép nhân $*$ gắn kết chặt hơn phép cộng $+$, hoặc có độ ưu tiên cao hơn. Thứ hai, chúng ta muốn nói rằng mỗi phép toán có tính kết hợp với bên trái, như chúng ta có $(1-2)-3$ thay vì $1-(2-3)$. Chúng ta sẽ làm điều đó bằng cách đưa vào thêm một ký hiệu chưa tận để nhận được Văn phạm 4.8.

VĂN PHẠM 4.8

- $E \rightarrow E + T$
- $E \rightarrow E - T$
- $E \rightarrow T$
- $T \rightarrow T * F$
- $T \rightarrow T / F$
- $T \rightarrow F$
- $F \rightarrow \text{id}$
- $F \rightarrow \text{num}$
- $F \rightarrow (E)$

Các ký hiệu E , T , và F tương ứng dùng cho *biểu thức (expression)*, *số hạng (term)*, và *thừa số (factor)*; tức là, thừa số dùng khi nhân và số hạng dùng khi cộng.

Văn phạm này đoán nhận cùng một tập câu giống như văn phạm nhập nhằng, nhưng bây giờ mỗi câu chỉ có đúng một cây cú pháp. Văn phạm 4.8 có thể không bao giờ sinh ra cây cú pháp ở dạng trong Hình 4.9 (xem Bài tập 4.17).



Hình 4.9: Cây cú pháp mà Văn phạm 4.8 không bao giờ sinh ra.

Nếu chúng ta muốn cho phép nhân có tính kết hợp phải, chúng ta cần phải viết luật sinh của nó dạng $T \rightarrow F * T$.

Thông thường chúng ta có thể khử tính nhập nhằng bằng cách biến đổi văn phạm. Tuy nhiên có một số ngôn ngữ (tập các xâu) mà có văn phạm nhập nhằng nhưng không có văn phạm không nhập nhằng, những ngôn ngữ như vậy là có vấn đề nếu xét theo khía cạnh ngôn ngữ lập trình, vì tính nhập nhằng trong cú pháp sẽ dẫn đến các vấn đề trong khi viết và hiểu chương trình.

DẤU KẾT THÚC TẬP (END-OF-FILE MARKER)

Phân tích cú pháp cần phải đọc không chỉ các ký hiệu tận cùng như $+$, $-$, số (num), ..., mà cả dấu kết thúc tệp. Chúng ta sẽ sử dụng ký hiệu $\$$ để thể hiện kết thúc tệp.

Giả sử S là ký hiệu khởi đầu trong văn phạm. Để chỉ ra $\$$ cần phải đi sau câu kết thúc sinh bởi S , chúng ta bổ sung vào văn phạm ký hiệu xuất phát mới S' và luật sinh mới $S' \rightarrow S\$$.

Trong Văn phạm 4.8, E là ký hiệu xuất phát và văn phạm phụ trợ là Văn phạm 4.10.

GRAMMAR 4.10

- $S \rightarrow E \$$
- $E \rightarrow E + T$
- $E \rightarrow E - T$
- $E \rightarrow T$
- $T \rightarrow T * F$
- $T \rightarrow T / F$
- $T \rightarrow F$
- $F \rightarrow \text{id}$
- $F \rightarrow \text{num}$
- $F \rightarrow (E)$

PHÂN TÍCH CÚ PHÁP DỰ BÁO (PREDICTIVE PARSING)

Một số văn phạm dễ phân tích dựa vào thuật toán đã biết là đệ qui lùi (*recursive descent*). Thực chất, mỗi luật sinh của văn phạm trở thành một mệnh đề của một hàm đệ qui. Chúng ta minh họa nó bằng việc viết Phân tích cú pháp đệ qui lùi cho Văn phạm 4.11.

VĂN PHẠM 4.11

- $S \rightarrow \text{if } E \text{ then } S \text{ else } S$
- $S \rightarrow \text{begin } S L$
- $S \rightarrow \text{print } E$
- $L \rightarrow \text{end}$
- $L \rightarrow ; S L$
- $E \rightarrow \text{num} = \text{num}$

Phân tích cú pháp đệ qui lùi cho ngôn ngữ này có một hàm cho mỗi chưa tận và một câu lệnh cho mỗi luật sinh.

```
final int IF=1, THEN=2, ELSE=3, BEGIN=4, END=5, PRINT=6,
        SEMI=7, NUM=8, EQ=9;

int tok = getToken();

void advance() {tok=getToken();}
void eat(int t) {if (tok==t) advance(); else error();}

void S() {switch(tok) {
    case IF: eat(IF); E(); eat(THEN); S();
```

```
        eat(ELSE); S(); break;
    case BEGIN: eat(BEGIN); S(); L(); break;
    case PRINT: eat(PRINT); E(); break;
    default: error();
    }}
void L() {switch(tok) {
    case END: eat(END); break;
    case SEMI: eat(SEMI); S(); L(); break;
    default: error();
    }}
void E() { eat(NUM); eat(EQ); eat(NUM); }
```

Với định nghĩa phù hợp cho báo lỗi `error` và nhận dạng từ tổ `getTokens`, chương trình này sẽ phân tích rất tốt.

Thành công với phương pháp đơn giản này, chúng ta thử thực hiện với văn phạm 4.10:

```
void S() { E(); eat(EQ); }
void E() {switch (tok) {
    case ?: E(); eat(PLUS); T(); break;
    case ?: E(); eat(MINUS); T(); break;
    case ?: T(); break;
    default: error();
    }}
void T() {switch (tok) {
    case ?: T(); eat(TIMES); F(); break;
    case ?: T(); eat(DIV); F(); break;
    case ?: F(); break;
    default: error();
    }}
```

Ở đây có sự mâu thuẫn: Hàm E không có cách nào để nhận biết mệnh đề nào đã sử dụng. Xét các xâu $(1*2-3)+4$ và $(1*2-3)$. Trong trường hợp đầu, lời gọi đầu tiên đến E sẽ cần phải sử dụng luật sinh $E \rightarrow E + T$, nhưng trường hợp sau sẽ cần phải sử dụng $E \rightarrow T$.

4.3 PHÂN TÍCH DẠNG CHUẨN TẮC CHOMSKY

Văn phạm phi ngữ cảnh ở dạng chuẩn Chomsky có dạng

- $A \rightarrow BC$ hoặc
- $A \rightarrow a$

Sau đây là đoạn văn trích từ “Ngôn ngữ dot net” nói về nhà bác học Noam Chomsky:

“Noam Chomsky sinh ngày 7-12-1928 trong một gia đình trí thức. Bố ông, William Chomsky, trốn quân dịch Sa hoàng, và di cư khỏi Nga năm 1913; ông trở thành học giả tiếng Sanskrit, chuyên về cú pháp thời trung cổ ở trường Sư phạm Gratz, và trường Cao đẳng Dropski ở Philadenphia.

Năm 10 tuổi, chú bé Chomsky đã được đọc bản in thử về ngữ pháp tiếng Sanskrit thể kỉ 13 của cha mình, điều này ảnh hưởng lớn tới lòng ham thích ngôn ngữ của N.Chomsky sau này.

Sau khi tốt nghiệp trung học ở Philadelphia, Chomsky học hành rất lang bang tại đại học Pensinvania. Đầu tiên, ông quan tâm tới tình hình chính trị Trung Đông hơn cả việc học đại học, ông đã từng có ý định thôi học để đi Palestin. Tìm cách khuyên can con, cha N.Chomsky giới thiệu Chomsky với Zellig Harris, lúc đó cũng rất quan tâm tới chính trị., và có vẻ như đồng quan điểm với Chomsky. Thế là chàng thanh niên Chomsky quyết định ở lại Pensinvania để học ngôn ngữ với Z. Harris. Đầu tiên chỉ là sửa bản in cho công trình sau này nổi tiếng toàn thế giới "*Các phương pháp ngôn ngữ học cấu trúc*", và Chomsky thật sự học ngôn ngữ qua lần sửa bản in đó. Z. Harris yêu cầu Chomsky tìm hiểu một cuốn ngữ pháp tiếng Sanskrit. Sau khi đọc, ông phát hiện ra những quy tắc tạo ra toàn bộ các câu của tài liệu này theo mô hình cải biến. Sự nghiên cứu này biệt lập với những giáo trình và bài giảng ở trường ốc. Về sau Chomsky cho rằng đi theo con đường *không chính thống* như thế thực là may, vì như vậy người sinh viên mới bước vào ngành này không bị đứng trước hàng loạt quan điểm đầy quyền uy và cực kì khó khăn cho người mới vào ngành đưa ra những kiến giải thách thức lại những ý kiến đấy.

Trong quá trình nghiên cứu, lúc đầu ông cố gắng cải tiến kĩ thuật phân bố và cải biến của Harris, đồng thời ông đã hình thức hoá chúng một cách triệt để. Chomsky biết ơn Harris về những bước đi ban đầu này. Sau khi tốt nghiệp ở đại học Pensinvania, ông đã bỏ ra hai năm nghiên cứu phương pháp cải biến, ông thấy rằng, đi theo con đường này sẽ có kết quả, chứ không đi vào ngõ cụt như những phương pháp khác.

Từ 1953, ông gấp rút hoàn thành công trình *Cấu trúc logic của ngôn ngữ tự nhiên* (1955), dày trên 300 trang, về sau đã được in. Một chương của nó "*Phép phân tích cải biến*" được đệ trình làm luận án tiến sĩ. Ông đỗ tiến sĩ nhưng bị đánh giá khá xoàng, một giáo sư phản biện viết: "Tôi không biết luận án này thuộc lĩnh vực gì, nhưng chắc chắn đây không phải là ngôn ngữ học". Thế là trong một hai năm tiếp theo, các bài báo và luận án của Chomsky bị các toà soạn và nhà xuất bản trả lại bản thảo. Thực ra ông có được công bố hai, ba bài trên các tạp chí không phải ngôn ngữ học(!). Những người ủng hộ và hoan nghênh hướng đi này của Chomsky lại là các nhà toán học nổi tiếng thế giới, như Y. Bar-Hillet và W. Quine. Dù sao, lúc này nhờ M. Haillet và R.Jakobson mà Chomsky được một chân ở học viện M.I.T. (Massachusetts Institute of Technology), tại đó ông dạy tiếng Pháp và tiếng Đức cho cán bộ khoa học-kĩ thuật. Thế là ông có điều kiện về tài chính để đeo đuổi điều mình quan tâm.

Năm 1957, quyển *Các cấu trúc cú pháp* được Nhà xuất bản Mouton ở Hà Lan ấn hành, và Robert Lees đã viết một bài báo dài hơn 30 trang trên tạp chí Language để bình luận và giới thiệu, nhờ đó mà cả thế giới biết tới ông. Chomsky đặc biệt biết ơn R. Lees, "người học trò" đầu tiên của mình, vì rằng số phận quyển sách này sẽ không được ai biết tới nếu không có bài trên của Lees. Trong quyển sách này có một câu rất nổi tiếng minh hoạ cho quan niệm "tính ngữ pháp" của ông; một câu vô nghĩa vẫn có thể có tính ngữ pháp. Câu đó là:

Colourless green ideas sleep furiously (Những tư tưởng xanh lục không màu sắc ngủ một cách giận dữ).

Câu này nổi tiếng đến nỗi người ta đã làm tặng Chomsky những bài thơ có chứa câu này.

N. Chomsky bắt đầu được dạy ngôn ngữ ở Học viện MIT từ những năm đầu của thập kỉ 60, khi Viện đó được thành lập bộ môn ngôn ngữ học. Theo ông, về một phương diện nào đó thì học viện này nằm ngoài hệ thống đại học Mỹ nên không có những khoa học xã

hội hoặc nhân văn lớn, "và chúng tôi – vẫn lời Chomsky – không bị vấp phải những cung cách "quan liêu hàn lâm" trong cách đào tạo chương trình ngôn ngữ học". Ở MIT, ông cùng với Lees, Mathews và Klima nghiên cứu chương trình dịch tự động của phòng thí nghiệm điện tử, nhưng chẳng ai hứng thú gì về đề tài này.

Khi MIT mở cao học về ngôn ngữ, ngoài M. Halle và nhóm trên còn có Jerry Fodor, Jerry Katz và Paul Postal. Nơi đây là cái nôi của lý thuyết ngữ pháp chuẩn, mà Chomsky là người đại diện, với quyển *Aspects of Theory of Syntax* lừng danh. Nơi đây đã lôi cuốn hàng loạt các nhà ngôn ngữ học trẻ và các nhà toán học chuyển sang nghiên cứu ngôn ngữ học.

Giai đoạn 3 của ngữ pháp tạo sinh được tuyên ngôn bằng bài "*Cấu trúc chìm, cấu trúc nổi và việc giải thích ngữ nghĩa*" (Deep structure, surface structure and semantic interpretation, 1970). Những mốc quan trọng trong giai đoạn này là những bài về dạng thức và giải thích ngữ nghĩa, đặc biệt là bài *Điều kiện về các phép cải biến* (Conditions on Transformations, 1973).

Chomsky là một người không dừng lại. Tháng 4-1979 ông đã trình bày một công trình cực kì quan trọng tại Trường đại học Sư phạm Pizza ở Ý, gây sự chú ý đặc biệt của giới ngôn ngữ học, và người ta gọi là "Hệ thống Pizza" của Chomsky. Dựa trên công trình đó, năm 1981, ông cho in *Những bài giảng về lý thuyết chi phối và liên kết* (Lectures on Government and Binding). Từ năm 1965 tới 1983, đây là công trình cơ bản nhất của Chomsky, thu tóm những quan điểm hiện nay của Chomsky về ngôn ngữ học.

Về đóng góp của Chomsky, phần lớn người ta đánh giá rất cao, ông đã làm "một cuộc cách mạng trong ngôn ngữ học" (J. Searle), nhưng không phải không có tiếng nói phủ định, "đó là một thứ ngôn ngữ học nguy tạo" (Andreev). Dù thế nào chăng nữa, N. Chomsky vẫn có một vị trí đặc biệt trong ngôn ngữ học 30 năm qua.

Có thể nói đến một Chomsky thứ hai, nhà tư tưởng và hoạt động xã hội. Trong lĩnh vực này ông cũng nổi tiếng không kém. Ông luôn đấu tranh cho tự do và hoà bình, phản đối chính sách đối ngoại hiếu chiến của chính quyền Mĩ, phản đối sự can thiệp vào nội bộ của các nước khác. Những quyển sách "*Về cuộc chiến tranh ở châu Á: bình luận về Đông Dương*"; "*Lực lượng Mĩ và những tên sen đầm mới*"; "*Vấn đề nhận thức và tự do*" là những quyển sách nổi tiếng.

Noam Chomsky đã sang giảng ở Hà Nội vào đầu năm 1970."

Thuật toán CYK

CYK viết tắt của từ Cocke-Younger-Kasami. CYK là một thuật toán dùng để xác định xem một xâu có được tạo ra (hay đoán nhận) bởi một văn phạm phi ngữ cảnh hay không (context-free grammar). Thuật toán này được sử dụng rất nhiều trong phân tích ngôn ngữ tự nhiên.

CYK là thuật toán bottom-up và chi phí là $O(n^3)$ trong trường hợp tồi nhất với n là độ dài xâu phân tích.

Giải thuật

Vào: Văn phạm $G = (N, T, S, P)$ dạng chuẩn Chomsky, không chứa sản xuất trống, xâu vào $w = a[1]a[2]... a[n] \in T^+$

Ra: Bảng phân tích T đối với w sao cho $t[i,j]$ chứa A khi và chỉ khi

$$A \rightarrow^+ a[i]a[i+1]\dots a[i+j-1]$$

Thuật toán

For $i = 1$ to n do $t[i,1] = \{ A | A \rightarrow a[i] \in P \}$

For $j = 2$ to n do

For $i = 1$ to $n - j + 1$ do

For $k = 1$ to $j - 1$ do

$$t[i,j] = \{ A | A \rightarrow BC \in P, B \rightarrow t[i,k] \text{ và } C \rightarrow t[i+k,j-k] \}$$

Nếu $S \in t[1,n]$ thì $w \in L(G)$.

4.4 BỘ PHÂN TÍCH BNF VÀ EBNF

Sau đây là đoạn văn được trích từ tài liệu trên Internet nói về nhà bác học John Backus:

“Backus–Naur Form (BNF): Dạng chuẩn Backus-Naur để biểu diễn văn phạm phi ngữ cảnh trong khoa học máy tính.

Nhiều mở rộng và biến thể của ký hiệu BNF gốc được sử dụng, một số được định nghĩa chính xác, trong đó có BNF mở rộng - *Extended Backus–Naur Form* (EBNF).

John Backus, mất ngày 17 tháng 3 năm 2007, là nhà tiên phong trong phát triển những ngôn ngữ lập trình máy tính, đồng thời cũng là nhà nghiên cứu hàng đầu về lập trình chức năng.

Backus chào đời ngày 3 tháng 12 năm 1924 tại Philadelphia và lớn lên ở Wilmington, Delaware. Bố ông được đào tạo để trở thành một nhà hóa học nhưng lại quyết định chuyển sang nghề môi giới nhà. Chính bởi vậy mà gia đình ông đã sớm trở nên giàu có và nổi lên nhanh chóng trong xã hội thời đó. Khi còn là một đứa trẻ, cậu bé Backus thích thú với công việc hàn cơ khí và rất yêu hóa học. Backus đã tỏ ra là một đứa trẻ có năng khiếu và ham học hỏi. Ông được cha mẹ gửi tới trường trung học Hill School, một trường tư thục dành riêng cho con cái nhà giàu ở Pottstown, Pennsylvania. Nhưng quả thực thành tích học tập không lấy gì làm tự hào và thường xuyên phải tham gia những khóa học hè để có thể giúp ông theo kịp chương trình học so với chúng bạn cùng trang lứa. Tình hình cũng không lấy gì sáng sủa hơn khi ông bước chân vào cánh cửa Đại học Virginia. Tại đây Backus theo học chuyên ngành về kỹ sư cơ khí vào năm 1942. Năm 1943, Backus gia nhập quân đội theo nghĩa vụ quân sự và được phân công vào chương trình chống máy bay. Những cuộc kiểm tra năng khiếu đã tiết lộ khả năng thông minh nổi bật của ông, và ngay lập tức, chàng trai này được gửi đi đào tạo tại một khóa học kỹ sư chuyên gia đặc biệt ở Đại học Pittsburgh, Pennsylvania. Trong khoảng thời gian đó, Backus cũng đã tham gia một khóa học đào tạo chuẩn bị y học. Do không hứng thú và chán nản nên đã bỏ dở giữa chừng. Rời quân ngũ năm 1946, chàng trai trẻ Backus tiếp tục theo học tại Đại học Columbia ở New York, ban đầu như một sinh viên dự bị do kết quả học tập quá kém cỏi. Nhưng ít nhất chàng trai này đã chứng tỏ được thành công

trong lĩnh vực toán học để tốt nghiệp với tấm bằng cử nhân vào năm 1949, để rồi năm tiếp sau ông nhận tiếp tấm bằng thạc sỹ.

Năm 1948, IBM hoàn thành xong chiếc máy tính điện tử thử nghiệm đầu tiên được biết với tên gọi SSEC. Đó là một cỗ máy đồ sộ với 13,000 đèn điện tử cùng với 23,000 rơ-le. IBM nhanh chóng đem giới thiệu cho công chúng sáng chế này tại phòng trưng bày ở quảng trường New York với mong muốn thu hút thật nhiều sự chú ý và tò mò của người dân và giới truyền thông. Backus đến xem, nhìn thấy “cỗ máy” SSEC với niềm phấn khích. Ngay lập tức ông quyết định thuyết phục IBM thuê mình trong vị trí của một lập trình viên. Kể từ đó, chàng trai Backus đã dành nhiều năm để tính toán vị trí của Mặt trăng, một công việc ông rất mực yêu thích.

Đến năm 1952, IBM giới thiệu sản phẩm máy tính thương mại đầu tiên, model 701. Chiếc máy tính đã được lập trình ở một cấp độ rất gần với ngôn ngữ máy tính nhị phân cơ bản, mà thực chất không khó nhưng tốn thời gian và dễ gặp phải lỗi. Bài toán này đã thôi thúc Backus phát minh ra một hệ thống lập trình tự động được gọi là Speedcoding cho model 701 giúp thực hiện được nhiệm vụ dễ dàng hơn nhiều. Cũng giống như những hệ thống lập trình tương tự đang được phát triển ở một vài nơi khác cùng thời gian đó, tuy nhiên, Speedcoding lại tạo ra những chương trình chậm không kinh tế. Chính vì vậy, mùa thu 1953, Backus đã đề nghị với người quản lý phát triển một hệ thống có tên Formula Translator-sau này gọi là Fortran- cho máy tính model 704. Sau đó không lâu chương trình này nhanh chóng được công bố. Đặc điểm độc đáo nhất của Fortran là khả năng tạo ra những chương trình tốt bằng 90% những chương trình được viết bởi một nhà lập trình. Backus đã hăng hái tích cực và chỉ định lập một nhóm gồm mười lập trình viên trong khoảng thời gian sáu tháng. Mục tiêu thiết kế một hệ thống chuyển đổi để tạo ra những chương trình hiệu quả thực sự đã trở thành một thách thức lớn. Lúc mà hệ thống được công bố vào tháng 4 năm 1957, sáu tháng đã trở thành ba năm.

Fortran Translation bao gồm 18,000 lệnh và chứa đựng những thuật toán cực kỳ phức tạp cho việc tối ưu hóa mã. Fortran được người sử dụng IBM 704 nồng nhiệt đón nhận và các nhà chế tạo máy tính khác cũng nỗ lực chạy đua để tạo ra những hệ thống Fortran, bởi vậy phần mềm máy tính của họ có thể tương tích với hệ thống phần mềm IBM. Ngày nay, nó đã trở thành ngôn ngữ chung cho tính toán khoa học. Tuy nhiên Fortran cũng đã thường bị chỉ trích vì cái “không trang nhã” của nó. Nhưng sau đó, Backus đã giải thích rằng, khi hệ thống này được thiết kế thì hiệu suất là vấn đề tối quan trọng đặt lên hàng đầu. Và tất nhiên, không một ai có thể mơ rằng, một ngôn ngữ sẽ tiếp tục mạnh mẽ trong nửa thế kỷ sau đó. Nhưng thực tế, Fortran đã làm được nhiều hơn điều đó. Cuối năm 1950, Backus đã trở thành một thành viên của Ủy ban Algol. Ở đây, các kỹ sư đang thiết kế một ngôn ngữ lập trình khoa học quốc tế với tên gọi Algol 60. Về nguyên tắc, ngôn ngữ này đã được định rõ trong BNF (Backus-Naur Form). Algol 60 là một ngôn ngữ lập trình phổ biến ở châu Âu trong những năm 1960 và 1970. Tuy nhiên, ngôn ngữ này không bao giờ thay thế được Fortran, nguyên do căn bản là bởi vì phát minh này đã được hoàn thiện trên mã Fortran. Nhưng dù sao Algol đã có những ảnh hưởng lớn trong thiết kế ngôn ngữ lập trình, và BNF đã trở thành một trong những nguyên tắc cơ bản của khoa học máy tính.

Sau những năm bị lôi cuốn bởi Fortran, Backus lặng lẽ trong công việc của một nhà khoa học. Và cho đến năm 1960, ông trở thành một nhân viên của IBM Research đặt tại New York. Ở IBM Research, Backus đã dẫn đầu một nhóm nghiên cứu về lập trình chức năng, một kiểu mẫu lập trình mới nhắm vào việc tính toán thông qua ước lượng những hàm toán. Ý tưởng này nhằm “tránh” cái mà Backus gọi là “cổ chai von Neumann” (John von Neumann, một trong những nhà phát minh máy tính). Và như Backus đã từng giải thích, một máy tính bao gồm một bộ xử lý và một bộ nhớ; mục đích của chương trình là thay đổi trạng thái của bộ nhớ, sử dụng bộ xử lý, một lệnh ở một thời điểm. Dù thế nào đi nữa, lập trình chức năng đã và sẽ tiếp tục trở thành một khoa học máy tính lớn. Nghỉ hưu vào năm 1991, sống hết quãng đời còn lại và ra đi trong vinh quang, John Backus đã tiên phong mở đường cho thế hệ ngôn ngữ lập trình khoa học phổ biến rộng rãi nhất, đồng thời cũng là người có nhiều đóng góp to lớn cho lĩnh vực khoa học máy tính.”

Sau đây là một số ký hiệu BNF về số và các lệnh về vòng lặp, phép gán, danh sách, số nguyên không dấu:

```
<number> ::= <digit> | <number> <digit>
<digit>  ::= 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9

• <while loop> ::= while ( <condition> )
  <statement>
• <assignment statement> ::= <variable> =
  <expression>
• <statement list> ::= <statement> |
  <statement list> <statement>
• <unsigned integer> ::= <digit> |
  <unsigned integer><digit>
```

BNF cho biểu thức

```
<expression> ::= <expression> + <term>
               | <expression> - <term>
               | <term>
<term> ::= <term> * <factor>
          | <term> / <factor>
          | <factor>
<factor> ::= <primary> ^ <factor>
           | <primary>
<primary> ::= <primary>
            | <element>
<element> ::= ( <expression> )
            | <variable>
            | <number>
```

BNF cho số thập phân:

```
<expr> ::= '-' <num> | <num>
<num> ::= <digits>
        | <digits> '.' <digits>
<digits> ::= <digit>
            | <digit> <digits>
<digit> ::= '0' | '1' | '2' | '3' |
            '4' | '5' | '6' | '7' | '8' | '9'
```

EBNF cho số thập phân:

```
<expr> := '-'? <digit>+ ('.' <digit>+)?
<digit> := '0' | '1' | '2' | '3' | '4' |
            '5' | '6' | '7' | '8' | '9'
```

Chuyển đổi BNF sang EBNF:

BNF:

```
<expr> ::= <digits>
<digits> ::= <digit>
            | <digit> <digits>
```

được chuyển thành:

```
<expr> ::= <digit>+
```

BNF:

```
<expr> ::= <digits> | empty
```

được chuyển thành:

```
<expr> ::= <digit>*
```

BNF

```
<id> ::= <letter>
        | <id><letter>
        | <id><digit>
```

chuyển thành:

```
<id> ::= <letter> (<letter> | <digit>)*
```

EBNF cho ngôn ngữ LISP:

```
s_expression ::= atomic_symbol
               | "(" s_expression "." s_expression ")"
               | list
list ::= "(" s_expression* ")"
atomic_symbol ::= letter atom_part
atom_part ::= empty
             | letter atom_part
             | number atom_part
letter ::= "a" | "b" | " ..." | "z"
number ::= "1" | "2" | " ..." | "9"
```

Biểu diễn BNF qua chính nó:

BNF in BNF

```
syntax      → production syntax | ε.
production  → identifier "→" expression ".".
expression  → term | expression "|" term.
term        → factor | term factor.
factor      → identifier | string.

identifier  → letter | identifier letter | identifier digit.
string      → stringhead "".
stringhead  → "" | stringhead character.
letter      → "A" | ... | "Z".
digit       → "0" | ... | "9".
```

Biểu diễn EBNF qua chính nó:

EBNF in EBNF

```

syntax      → { production }.
production → identifier "→" expression ".".
expression → term { "|" term }.
term       → factor { factor }.
factor     → identifier | string |
            "(" expression ")" |
            "[" expression "]" |
            "{" expression "}".

identifier  letter { letter | digit }.
string      "" { character } "".
letter      "A" | ... | "Z".
digit       "0" | ... | "9".
    
```

- Note that the use of EBNF is not only helpful for the definition of a grammar, but also essential for the construction of a recognizer.

BÀI TẬP

4.1 Dịch mỗi biểu thức chính qui sau sang văn phạm phi ngữ cảnh.

- $((xy^*x)(yx^*y))^?$
- $((0|1)^+ \cdot "(0|1)^*" | ((0|1)^* \cdot "(0|1)^+)$

***4.2** Viết văn phạm cho các câu tiếng Anh sử dụng các từ

time, arrow, banana, flies, like, a, an, the, fruit

và các dấu chấm phẩy. Tin tưởng là bao gồm mọi ý nghĩa (danh từ, động từ, v.v...) của mỗi từ. Sau đó chỉ ra rằng văn phạm này là nhập nhằng bằng cách chỉ ra nhiều hơn một cây phân tích cho câu "time flies like an arrow; fruit flies like a banana."

4.3 Viết văn phạm không nhập nhằng cho mỗi ngôn ngữ sau. Gợi ý: có một cách kiểm tra văn phạm là không nhập nhằng là chạy nó qua Yacc và không nhận được xung đột.

- Các xâu mà đọc xuôi từ đầu và đọc ngược từ cuối là như nhau (Palindromes) trên bảng chữ $\{a, b\}$
- Các xâu mà sánh với biểu thức chính qui a^*b^* và có số chữ a nhiều hơn số chữ b .
- Đóng mở ngoặc tròn và đóng mở ngoặc móc vuông cân bằng nhau . Ví dụ: $([] (() [()] []))$

*d. Đóng mở ngoặc tròn và đóng mở ngoặc móc cân bằng nhau, ở đó khi đóng ngoặc móc cũng đóng luôn mọi ngoặc tròn đang mở còn trong phạm vi của nó (cho đến tận ngoặc móc vuông mở trước đó). Ví dụ: `[ ( [ ] ( ( ) [ ( ) ] [ ] ) ]`. Gợi ý: Trước hết, tin tưởng là đã tạo ngôn ngữ ngoặc tròn và ngoặc móc cân bằng, ở đó ngoặc móc tròn mở có thể thừa; sau đó tin tưởng rằng chưa tận đó cần xuất hiện bên trong ngoặc móc.

e. Mọi tập con và hoán vị (không bị lặp) của các từ khóa `public final static synchronized transient`. (Khi đó đưa ra ý kiến về việc là làm sao có thể kiểm soát tốt nhất tình huống này trong chương trình dịch thực tế.)

f. Các khối lệnh trong Pascal và ML trong đó các dấu chấm phẩy tách các lệnh:

```
( statement ; ( statement ; statement ) ; statement )
```

g. Các khối lệnh trong C ở đó các dấu chấm phẩy kết thúc các lệnh:

```
{ expression; { expression; expression; } expression; }
```

CHƯƠNG 5:

PHÂN TÍCH CÚ PHÁP DỰA BẢNG

Phân tích cú pháp đệ qui lùi hay dự báo được dùng cho văn phạm mà ở đó ký tự tận cùng đầu tiên của mỗi biểu thức con cung cấp đầy đủ thông tin về việc chọn luật sinh nào. Để hiểu điều này rõ hơn, chúng ta sẽ hình thức hoá khái niệm tập FIRST, và sau đó sử dụng thuật toán đơn giản để nhận được Phân tích cú pháp đệ qui lùi không mâu thuẫn.

Cũng như phân tích từ vựng được xây dựng từ các biểu thức chính qui, có các công cụ sinh Phân tích cú pháp giúp xây dựng Phân tích cú pháp dự báo. Nếu chúng ta dự định sử dụng công cụ này, thì chúng ta cũng có thể dựa trên thuật toán phân tích cú pháp LR(1) mạnh hơn mà sẽ mô tả trong phần 5.3.

Đôi khi không thuận tiện hoặc cũng không thể sử dụng công cụ sinh Phân tích cú pháp. Ưu điểm của phân tích dự báo là thuật toán khá đơn giản đến mức chúng ta có thể sử dụng nó để xây dựng bộ phân tích cú pháp bằng tay mà không cần đến các công cụ tự động hỗ trợ.

5.1 PHÂN TÍCH TRÊN XUỐNG - VĂN PHẠM LL(1)

CÁC TẬP FIRST VÀ FOLLOW

Cho trước chuỗi γ gồm các ký hiệu tận cùng và chưa tận, $FIRST(\gamma)$ là tập các ký hiệu tận cùng mà là ký hiệu đầu tiên của một chuỗi nào đó được dẫn xuất từ γ . Chẳng hạn, giả sử $\gamma = T * F$. Mọi chuỗi tận cùng được dẫn xuất từ γ cần phải bắt đầu với các tận cùng `id`, `num`, hoặc `(`. Vậy, $FIRST(T * F) = \{id, num, (\}$.

Nếu hai luật sinh $X \rightarrow \gamma_1$ và $X \rightarrow \gamma_2$ có cùng ký hiệu ở vế trái là (X) và các vế phải của chúng có các tập FIRST đè lên nhau (tức là có phần tử chung của $FIRST(\gamma_1)$ và $FIRST(\gamma_2)$), thì văn phạm đó không thể được phân tích bằng phân tích dự báo. Nếu có ký hiệu tận cùng I thuộc $FIRST(\gamma_1)$ và cũng thuộc $FIRST(\gamma_2)$, thì hàm X trong đệ qui lùi sẽ không biết làm gì nếu từ tổ đầu vào là I , tức là không biết lựa chọn luật sinh nào để áp dụng.

Việc tính toán tập FIRST có vẻ đơn giản: nếu $\gamma = X Y Z$, thì trông có vẻ là có thể bỏ qua Y và Z , mà chỉ cần quan tâm đến $FIRST(X)$. Nhưng khi xét Văn phạm 5.1, vì Y có thể sinh chuỗi rỗng – và trước đó X cũng có thể sinh chuỗi rỗng – do đó chúng ta có $FIRST(X Y$

Z) cần phải chứa cả $FIRST(Z)$. Như vậy, khi tính toán các tập $FIRST$, chúng ta cần theo dõi các ký hiệu nào sinh xâu rỗng; chúng ta nói những ký hiệu như vậy là có thể nhận giá trị Null (*nullable*). Và chúng ta cần xét xem cái gì đi sau ký hiệu có thể null đó.

VĂN PHẠM 5.1

- $Z \rightarrow d$
- $Z \rightarrow X Y Z$
- $Y \rightarrow$
- $Y \rightarrow c$
- $X \rightarrow Y$
- $X \rightarrow a$

Với văn phạm cụ thể và một xâu γ gồm các ký hiệu tận cùng và chưa tận cho trước:

- $nullable(X)$ là đúng nếu X có thể dẫn xuất ra xâu rỗng.
- $FIRST(\gamma)$ là tập các tận cùng mà là ký hiệu bắt đầu của một xâu nào đó được dẫn xuất từ γ .
- $FOLLOW(X)$ là tập các ký hiệu tận cùng mà có thể đứng ngay sau X . Như vậy, $t \in FOLLOW(X)$ nếu có dẫn xuất chứa Xt . Điều đó có thể xảy ra nếu dẫn xuất đó chứa $X Y Zt$ mà ở đó cả Y và Z đều dẫn xuất xâu rỗng ϵ .

Định nghĩa chính xác của $FIRST$, $FOLLOW$ và $nullable$ là các tập nhỏ nhất mà có các tính chất đó:

Với mỗi ký hiệu tận cùng Z , $FIRST[Z] = \{Z\}$.

```
for each production  $X \rightarrow Y_1 Y_2 \dots Y_k$ 
  if  $Y_1 \dots Y_k$  are all nullable (or if  $k = 0$ )
    then  $nullable[X] = \text{true}$ 
  for each  $i$  from 1 to  $k$ , each  $j$  from  $i + 1$  to  $k$ 
    if  $Y_1 \dots Y_{i-1}$  are all nullable (or if  $i = 1$ )
      then  $FIRST[X] = FIRST[X] \cup FIRST[Y_i]$ 
    if  $Y_{i+1} \dots Y_k$  are all nullable (or if  $i = k$ )
      then  $FOLLOW[Y_i] = FOLLOW[Y_i] \cup FOLLOW[X]$ 
    if  $Y_{i+1} \dots Y_{j-1}$  are all nullable (or if  $i + 1 = j$ )
      then  $FOLLOW[Y_i] = FOLLOW[Y_i] \cup FIRST[Y_j]$ 
```

Thuật toán 5.2 để tính $FIRST$, $FOLLOW$ và $nullable$ được suy ra từ các nhận định trên; chúng ta đơn giản thay thế mỗi phương trình bằng lệnh gán và cho nó lặp.

THUẬT TOÁN 5.2: Tính toán lặp của $FIRST$, $FOLLOW$ và $nullable$.

Thuật toán tính $FIRST$, $FOLLOW$ và $nullable$.

Khởi tạo FIRST và FOLLOW là các tập rỗng và khả năng nullable của mọi ký hiệu là false.

```

for each terminal symbol  $Z$ 
    FIRST[ $Z$ ]  $\leftarrow \{Z\}$ 
repeat
    for each production  $X \rightarrow Y_1 Y_2 \dots Y_k$ 
        if  $Y_1 \dots Y_k$  are all nullable (or if  $k = 0$ )
            then nullable[ $X$ ]  $\leftarrow$  true
        for each  $i$  from 1 to  $k$ , each  $j$  from  $i + 1$  to  $k$ 
            if  $Y_1 \dots Y_{i-1}$  are all nullable (or if  $i = 1$ )
                then FIRST[ $X$ ]  $\leftarrow$  FIRST[ $X$ ]  $\cup$  FIRST[ $Y_i$ ]
            if  $Y_{i+1} \dots Y_k$  are all nullable (or if  $i = k$ )
                then FOLLOW[ $Y_i$ ]  $\leftarrow$  FOLLOW[ $Y_i$ ]  $\cup$  FOLLOW[ $X$ ]
            if  $Y_{i+1} \dots Y_{j-1}$  are all nullable (or if  $i + 1 = j$ )
                then FOLLOW[ $Y_i$ ]  $\leftarrow$  FOLLOW[ $Y_i$ ]  $\cup$  FIRST[ $Y_j$ ]
    until FIRST, FOLLOW, and nullable did not change in this iteration.
    
```

Tất nhiên, để làm cho thuật toán này hiệu quả, nên kiểm tra các luật sinh theo một trình tự đúng; đọc thêm tài liệu tham khảo ([1], xem mục 17.4). Cũng có thể ba quan hệ này không cần phải tính đồng thời; nullable có thể tính độc lập, sau đó đến FIRST, và cuối cùng là FOLLOW.

Đây không phải là lần đầu tiên mà một nhóm các phương trình trên tập hợp trở thành thuật toán tính các tập đó; có thể nhắc lại thuật toán tính bao đóng $\in$ khi biến đổi otomat không tắt định thành tắt định. Và cũng không là lần cuối cùng, vì kỹ thuật lặp đến điểm bất động được áp dụng trong phân tích luồng dữ liệu để tối ưu chương trình dịch, xem [1, ở phần cuối của cuốn sách].

Chúng ta có thể áp dụng thuật toán này cho Văn phạm 5.1. Ban đầu, ta có:

	nullable	FIRST	FOLLOW
X	no		
Y	no		
Z	no		

Trong lần lặp đầu tiên, chúng ta tìm thấy $a \in \text{FIRST}[X]$, Y là Null, $c \in \text{FIRST}[Y]$, $d \in \text{FIRST}[Z]$, $d \in \text{FOLLOW}[X]$, $c \in \text{FOLLOW}[X]$, $d \in \text{FOLLOW}[Y]$. Vậy:

	nullable	FIRST	FOLLOW
X	no	a	$c\ d$
Y	yes	c	d
Z	no	d	

Trong lần lặp thứ hai, chúng ta nhận được X là Null, $c \in \text{FIRST}[X]$, $\{a; c\} \subseteq \text{FIRST}[Z]$, $\{a, c, d\} \subseteq \text{FOLLOW}[X]$, $\{a, c, d\} \subseteq \text{FOLLOW}[Y]$. Vậy:

	nullable	FIRST	FOLLOW
X	yes	a c	a c d
Y	yes	c	a c d
Z	no	a c d	

Lần lặp thứ ba không nhận được thông tin gì thêm, và thuật toán dừng.

Rất có ích nếu chúng ta mở rộng quan hệ FIRST cho xâu các ký tự:

$$\begin{aligned} \text{FIRST}(X\gamma) &= \text{FIRST}[X] && \text{if not nullable}[X] \\ \text{FIRST}(X\gamma) &= \text{FIRST}[X] \cup \text{FIRST}(\gamma) && \text{if nullable}[X] \end{aligned}$$

Và tương tự, chúng ta nói rằng xâu γ là có thể Null được (Nullable), nếu mỗi ký hiệu trong γ là có thể Null được (Nullable).

XÂY DỰNG PHÂN TÍCH DỰ BÁO

Xét phân tích đệ qui lùi. Hàm phân tích cho mỗi chưa tận X có dòng lệnh cho mỗi luật sinh của X ; nó phải chọn một trong các dòng lệnh đề dựa vào từ tố tiếp T theo của đầu vào. Nếu chúng ta có thể chọn đúng luật sinh cho mỗi (X, T) , thì chúng ta có thể viết phân tích đệ qui lùi. Mọi thông tin chúng ta cần có thể được ghi trong bảng hai chiều của các luật sinh được đánh chỉ số bởi chưa tận X và tận cùng T . Đây được gọi là Bảng phân tích dự báo.

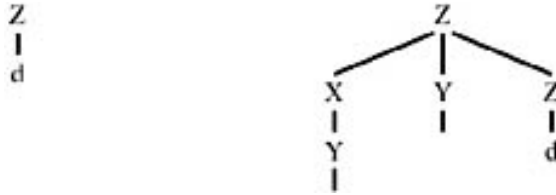
Để xây dựng bảng này, nhập luật sinh $X \rightarrow \gamma$ trên hàng X , cột T của Bảng cho mỗi $T \in \text{FIRST}(\gamma)$. Cũng như vậy, nếu γ là Null, nhập luật sinh trên hàng X , cột T cho mỗi $T \in \text{FOLLOW}[X]$.

Hình 5.3 chỉ ra Phân tích dự báo cho Văn phạm 5.1. Nhưng một số ô chứa nhiều hơn một luật sinh! Sự có mặt của nhiều hơn một luật ở một ô nói lên là không dùng Phân tích dự báo được cho Văn phạm 5.1.

	a	c	d
X	$X \rightarrow a$ $X \rightarrow Y$	$X \rightarrow Y$	$X \rightarrow Y$
Y	$Y \rightarrow$	$Y \rightarrow$ $Y \rightarrow c$	$Y \rightarrow$
Z	$Z \rightarrow XYZ$	$Z \rightarrow XYZ$	$Z \rightarrow d$ $Z \rightarrow XYZ$

Hình 5.3: Bảng cú pháp dự báo cho Văn phạm 5.1.

Nếu chúng ta kiểm tra Văn phạm cẩn thận hơn, chúng ta sẽ thấy là nó nhập nhằng. Câu d có nhiều hơn một cây cú pháp:



Văn phạm nhập nhằng sẽ luôn dẫn đến việc có hai luật sinh cho một ô trong Bảng phân tích dự báo. Nếu chúng ta sử dụng ngôn ngữ của Văn phạm 5.1 như ngôn ngữ lập trình, thì chúng ta phải tìm văn phạm không nhập nhằng.

Văn phạm mà Bảng phân tích cú pháp không có ô chứa nhiều hơn một luật sinh được gọi là LL(1). Điều này đúng cho phân tích trái qua phải, trái nhất và xem trước 1 ký hiệu (*Left-to-right parse*, *Leftmost-derivation*, *1-symbol lookahead*). Rõ ràng, phân tích dự báo đệ qui lùi kiểm tra đầu vào từ trái qua phải một lần (một số thuật toán phân tích không như vậy, nhưng chúng không có lợi cho chương trình dịch). Thứ tự mà theo đó phân tích dự báo triển khai các chưa tận ở về phải (khi mà phân tích đệ qui lùi gọi các hàm tương ứng với các chưa tận) là thứ tự mà suy diễn trái nhất triển khai các chưa tận. Và phân tích đệ qui lùi thực hiện nhiệm vụ của nó bằng cách xem trước một từ tổ của đầu vào, không khi nào phải xem trước nhiều hơn một từ tổ.

Chúng ta khái quát định nghĩa tập FIRST để mô tả k từ tổ đầu tiên của một xâu nào đó, và tạo nên bảng phân tích LL(k) mà các hàng của nó là các chưa tận và các cột là mọi dãy k từ tổ tận cùng. Điều này rất ít khi thực hiện (vì bảng như vậy rất lớn), nhưng đôi khi bạn viết phân tích đệ qui bằng tay, bạn cần phải xem trước nhiều hơn một từ tổ.

Văn phạm được phân tích bằng bảng phân tích LL(2) được gọi là văn phạm LL(2), và tương tự cho LL(3), v.v.... Mỗi văn phạm LL(1) là văn phạm LL(2), và cứ tiếp tục như vậy. Không có văn phạm nhập nhằng nào là LL(k) với k bất kỳ.

KHỦ ĐỆ QUI TRÁI

Giả sử chúng ta muốn xây dựng phân tích dự báo cho Văn phạm 4.10. Hai luật sinh

$$\begin{aligned} E &\rightarrow E + T \\ E &\rightarrow T \end{aligned}$$

tạo ra việc nhập nhằng nhiều hơn một luật sinh trong một ô trong bảng phân tích LL(1), vì mọi từ tổ trong FIRST(T) cũng sẽ ở trong FIRST($E + T$). Vấn đề là chính E xuất hiện như ký hiệu đầu tiên bên phải trong luật sinh của chính nó; điều đó gọi là đệ qui trái (*left recursion*). Văn phạm với đệ qui trái không thể là LL(1).

Để khử đệ qui trái, chúng ta sẽ viết lại sử dụng đệ qui phải. Chúng ta bổ sung chưa tận E' , và viết:

$$E \rightarrow T E'$$

$$E' \rightarrow + T E'$$

$$E' \rightarrow$$

Vấn phạm này cũng suy diễn cùng tập xâu (trên T và $+$) như hai luật sinh ban đầu, nhưng bây giờ không có đệ qui trái.

Nói chung, khi chúng ta có các luật sinh $X \rightarrow X\gamma$ và $X \rightarrow \alpha$, trong đó α không bắt đầu từ X , chúng ta biết chúng sẽ suy ra các xâu có dạng $\alpha\gamma^*$, tức là α được kế tiếp bởi không hay nhiều hơn γ . Vậy ta có thể viết lại biểu thức chính qui sử dụng đệ qui phải:

$$\left(\begin{array}{l} X \rightarrow X \gamma_1 \\ X \rightarrow X \gamma_2 \\ X \rightarrow \alpha_1 \\ X \rightarrow \alpha_2 \end{array} \right) \Rightarrow \left(\begin{array}{l} X \rightarrow \alpha_1 X' \\ X \rightarrow \alpha_2 X' \\ X' \rightarrow \gamma_1 X' \\ X' \rightarrow \gamma_2 X' \\ X' \rightarrow \end{array} \right)$$

Áp dụng biến đổi này cho Văn phạm 4.10, chúng ta nhận được Văn phạm 5.4.

VĂN PHẠM 5.4

- $S \rightarrow E \$$
- $E \rightarrow T E'$
- $E' \rightarrow + T E'$
- $E' \rightarrow - T E'$
- $E' \rightarrow$
- $T \rightarrow F T'$
- $T' \rightarrow * F T'$
- $T' \rightarrow / F T'$
- $T' \rightarrow$
- $F \rightarrow \text{id}$
- $F \rightarrow \text{num}$
- $F \rightarrow (E)$

Để xây dựng văn phạm dự báo, đầu tiên chúng ta tính nullable, FIRST và FOLLOW trong Bảng 5.5. Phân tích dự báo cho Văn phạm 5.4 được nêu trong Bảng 5.6.

Bảng 5.5: Nullable, FIRST và FOLLOW cho Văn phạm 5.4.

	nullable	FIRST	FOLLOW
S	no	(id num	
E	no	(id num	) \$
E'	yes	+ -	) \$
T	no	(id num	) + - \$
T'	yes	* /	) + - \$
F	no	(id num	) * / + - \$

Bảng 5.6: Bảng phân tích dự báo cho Văn phạm 5.4. Chúng ta bỏ qua các cột cho *num*, */*, và *-*, vì chúng tương tự những cái khác trong bảng.

	+	*	id	(	)	\$
S			$S \rightarrow ES$	$S \rightarrow ES$		
E			$E \rightarrow TE'$	$E \rightarrow TE'$		
E'	$E' \rightarrow +TE'$				$E' \rightarrow$	$E' \rightarrow$
T			$T \rightarrow FT'$	$T \rightarrow FT'$		
T'	$T' \rightarrow$	$T' \rightarrow *FT'$			$T' \rightarrow$	$T' \rightarrow$
F			$F \rightarrow id$	$F \rightarrow (E)$		

TẠO THỪA SỐ CHUNG TRÁI

Chúng ta đã thấy đệ qui trái gây rắc rối cho phân tích dự báo và nó có thể khử được. Vấn đề tương tự xảy ra khi hai luật sinh của cùng một chưa tận có hai vế phải được bắt đầu bởi cùng ký hiệu. Chẳng hạn:

$$S \rightarrow \text{if } E \text{ then } S \text{ else } S$$

$$S \rightarrow \text{if } E \text{ then } S$$

Trong trường hợp này, chúng ta có thể tách thừa số chung trái *if E then S* cho văn phạm - ở đây tách hai đuôi (*else S* và ϵ) và bổ sung chưa tận mới X cho chúng:

$$S \rightarrow \text{if } E \text{ then } SX$$

$$X \rightarrow$$

$$X \rightarrow \text{else } S$$

Các luật sinh nhận được không gây khó khăn cho phân tích dự báo nữa. Như vậy văn phạm ban đầu nhập nhằng - bảng phân tích có hai luật cho một ô - và chúng ta đã có thể giải quyết nhập nhằng bằng việc tạo thừa số chung và sử dụng thêm hành động *else S*.

KHẮC PHỤC LỖI

Với bảng phân tích dự báo, dễ dàng viết phân tích đệ qui lùi. Sau đây là một đoạn đại diện phân tích cú pháp cho Văn phạm 5.4:

```

void T() {switch (tok) {
    case ID:
    case NUM:
    case LPAREN: F(); Tprime(); break;
    default: error!
}}
void Tprime() {switch (tok) {
    case PLUS: break;
    case TIMES: eat(TIMES); F(); Tprime(); break;
    case EOF: break;
    case RPAREN: break;
    default: error!
}}
```

Ô trống trên hàng T , cột x của bảng phân tích LL(1) chỉ ra rằng hàm phân tích $T()$ không mong thấy từ tố x - đây sẽ là lỗi cú pháp. Lỗi có thể được kiểm soát như thế nào? Sẽ an toàn nếu xét ngoại lệ và thoát khỏi phân tích, nhưng điều đó không thân thiện với người sử dụng. Tốt hơn hết cần in ra thông báo lỗi và khắc phục nó, vì vậy các lỗi cú pháp có thể được tìm thấy trong khi dịch.

Lỗi cú pháp xảy ra khi xâu các từ tố đầu vào không là một câu của ngôn ngữ đó. Khôi phục lỗi là tìm câu đúng giống xâu các từ tố đầu vào đó. Nó có thể được tiếp tục bằng cách xoá, thay hoặc chèn thêm các từ tố.

Chẳng hạn, khôi phục lỗi cho T có thể được tiếp tục bằng cách chèn thêm từ tố số num . Không cần thiết phải chỉnh để có đầu vào đúng; mà chỉ cần ngụy tạo ra nhắc nhở rằng đây có thể là số, in thông báo và trở về bình thường.

```

void T() {switch (tok) {
    case ID:
    case NUM:
    case LPAREN: F(); Tprime(); break;
    default: print("expected id, num, or left-paren");
}}
```

Sẽ là nguy hiểm nếu khắc phục lỗi bằng cách chèn thêm, bởi vì nếu lỗi lại gây ra lỗi khác và quá trình này có thể lặp vô tận. Khắc phục lỗi bằng việc phát hiện an toàn hơn, vì vòng lặp cũng sẽ phải dừng khi đạt kết thúc tệp. Khôi phục đơn giản bằng cách xóa bớt khi bỏ qua các từ tố cho đến khi gặp từ tố trong tập FOLLOW. Chẳng hạn, khắc phục lỗi cho T' có thể làm việc như sau:

```

int Tprime_follow [] = {PLUS, RPAREN, EOF};

void Tprime() { switch (tok) {
    case PLUS: break;
    case TIMES: eat(TIMES); F(); Tprime(); break;
    case RPAREN: break;
    case EOF: break;
    default: print("expected +, *, right-paren,
                  or end-of-file");
              skipto(Tprime_follow);
}}
```

Cơ chế khắc phục lỗi cho phân tích cú pháp đệ qui lùi cần phải điều chỉnh để chống đưa ra nhiều thông báo sửa chữa lỗi chỉ từ một từ tổ ở một chỗ nào đó.

5.2 PHÂN TÍCH DƯỚI LÊN – VĂN PHẠM LR

Điểm yếu của các kỹ thuật phân tích LL(k) là chúng phải đoán cần sử dụng luật sinh nào, nếu như chỉ mới nhìn thấy k từ tổ đầu tiên của vế phải. Một kỹ thuật mạnh hơn, phân tích LR(k), là khả năng trì hoãn quyết định cho đến khi nó nhìn thấy các từ tổ tương ứng với toàn bộ vế phải của một luật sinh đang xem xét (khi đó có nhiều hơn k từ tổ ở đầu vào).

LR(k) có nghĩa là phân tích trái qua phải, suy diễn phải nhất, nhìn trước k từ tổ (*Left-to-right parse, Rightmost-derivation, k -token lookahead*). Việc sử dụng suy diễn phải nhất trông có vẻ lạ; Tương thích như thế nào so với phân tích trái qua phải? Hình 5.7 mô tả phân tích LR của chương trình sau

```
a := 7 ;
b := c + ( d := 5 + 6 , d )
```

Stack	Input	Action
1	a := 7 ; b := c + (d := 5 + 6 , d) \$	shift
1 id ₄	: = 7 ; b := c + (d := 5 + 6 , d) \$	shift
1 id ₄ := 6	7 ; b := c + (d := 5 + 6 , d) \$	shift
1 id ₄ := 6 num ₁₀	; b := c + (d := 5 + 6 , d) \$	reduce $E \rightarrow \text{num}$
1 id ₄ := 6 E ₁₁	; b := c + (d := 5 + 6 , d) \$	reduce $S \rightarrow \text{id} := E$
1 S ₂	; b := c + (d := 5 + 6 , d) \$	shift
1 S ₂ ; 3	b := c + (d := 5 + 6 , d) \$	shift
1 S ₂ ; 3 id ₄	: = c + (d := 5 + 6 , d) \$	shift
1 S ₂ ; 3 id ₄ := 6	c + (d := 5 + 6 , d) \$	shift
1 S ₂ ; 3 id ₄ := 6 id ₂₀	+ (d := 5 + 6 , d) \$	reduce $E \rightarrow \text{id}$
1 S ₂ ; 3 id ₄ := 6 E ₁₁	+ (d := 5 + 6 , d) \$	shift
1 S ₂ ; 3 id ₄ := 6 E ₁₁ + 16	(d := 5 + 6 , d) \$	shift
1 S ₂ ; 3 id ₄ := 6 E ₁₁ + 16 (8	d := 5 + 6 , d) \$	shift
1 S ₂ ; 3 id ₄ := 6 E ₁₁ + 16 (8 id ₄	: = 5 + 6 , d) \$	shift
1 S ₂ ; 3 id ₄ := 6 E ₁₁ + 16 (8 id ₄ := 6	5 + 6 , d) \$	shift
1 S ₂ ; 3 id ₄ := 6 E ₁₁ + 16 (8 id ₄ := 6 num ₁₀	+ 6 , d) \$	reduce $E \rightarrow \text{num}$
1 S ₂ ; 3 id ₄ := 6 E ₁₁ + 16 (8 id ₄ := 6 E ₁₁	+ 6 , d) \$	shift
1 S ₂ ; 3 id ₄ := 6 E ₁₁ + 16 (8 id ₄ := 6 E ₁₁ + 16	6 , d) \$	shift
1 S ₂ ; 3 id ₄ := 6 E ₁₁ + 16 (8 id ₄ := 6 E ₁₁ + 16 num ₁₀	, d) \$	reduce $E \rightarrow \text{num}$
1 S ₂ ; 3 id ₄ := 6 E ₁₁ + 16 (8 id ₄ := 6 E ₁₁ + 16 E ₁₇	, d) \$	reduce $E \rightarrow E + E$
1 S ₂ ; 3 id ₄ := 6 E ₁₁ + 16 (8 id ₄ := 6 E ₁₁	, d) \$	reduce $S \rightarrow \text{id} := E$
1 S ₂ ; 3 id ₄ := 6 E ₁₁ + 16 (8 S ₁₂	, d) \$	shift
1 S ₂ ; 3 id ₄ := 6 E ₁₁ + 16 (8 S ₁₂ , 18	d) \$	shift
1 S ₂ ; 3 id ₄ := 6 E ₁₁ + 16 (8 S ₁₂ , 18 id ₂₀	) \$	reduce $E \rightarrow \text{id}$
1 S ₂ ; 3 id ₄ := 6 E ₁₁ + 16 (8 S ₁₂ , 18 E ₂₁	) \$	shift
1 S ₂ ; 3 id ₄ := 6 E ₁₁ + 16 (8 S ₁₂ , 18 E ₂₁) 22	\$	reduce $E \rightarrow (S, E)$
1 S ₂ ; 3 id ₄ := 6 E ₁₁ + 16 E ₁₇	\$	reduce $E \rightarrow E + E$
1 S ₂ ; 3 id ₄ := 6 E ₁₁	\$	reduce $S \rightarrow \text{id} := E$
1 S ₂ ; 3 S ₅	\$	reduce $S \rightarrow S ; S$
1 S ₂	\$	accept

Hình vẽ 5.7: Phân tích đẩy – rút (Shift-reduce) của câu. Chỉ số dưới trong ngăn xếp *Stack* là số của trạng thái của otomat tất định DFA; xem bảng 5.8.

Sử dụng Văn phạm 4.1, bổ sung thêm luật khởi đầu $S' \rightarrow S\$$.

Bộ phân tích có ngăn xếp và đầu vào. k từ tổ đầu tiên của đầu vào được xem trước. Dựa trên nội dung của ngăn xếp và những từ tổ xem trước, bộ phân tích thực hiện hai kiểu hành động:

Đẩy - Shift: Đưa từ tổ đầu tiên của đầu vào vào đỉnh của ngăn xếp.

Rút gọn - Reduce: Chọn qui tắc văn phạm $X \rightarrow A B C$; rút C, B, A từ đỉnh của ngăn xếp; đẩy X vào ngăn xếp.

Khởi đầu, ngăn xếp rỗng và bộ phân tích ở đầu của đầu vào. Hành động đẩy dấu kết thúc tệp $\$$ được gọi là chấp nhận và đưa bộ phân tích đến kết thúc thành công.

Trong Hình 5.7, ngăn xếp và đầu vào được chỉ ra sau mỗi bước, cùng với việc chỉ ra hành động nào vừa được thực hiện. Ghép nối ngăn xếp với đầu vào luôn tạo thành một dòng của phân tích phải nhất; trên thực tế, Hình 5.7 cho thấy phân tích phải nhất của đầu vào, từ dưới lên trên (đọc ở cột hành động action từ dưới cùng ngược trở lại, để ý các hành động Reduce, ta nhận được cây cú pháp của đoạn chương trình đã cho).

CƠ CHẾ PHÂN TÍCH LR

Tại sao bộ phân tích LR biết khi nào thì đẩy và khi nào thì rút gọn? Bằng cách sử dụng otomat tất định hữu hạn! Otomat DFA không được áp dụng cho đầu vào – otomat hữu hạn quá yếu để phân tích văn phạm phi ngữ cảnh – nhưng nó có thể áp dụng cho ngăn xếp. Các cạnh của DFA được gán nhãn bởi các ký hiệu (tận cùng và chưa tận) mà có thể xuất hiện trong ngăn xếp. Bảng 5.8 là bảng chuyển cho Văn phạm 4.1.

Bảng 5.9: Bảng phân tích LR cho Văn phạm 4.1.

	id	num	print	;	,	+	:=	(	)	\$	<i>S</i>	<i>E</i>	<i>L</i>
1	s4		s7								g2		
2				s3						a			
3	s4		s7								g5		
4						s6							
5				r1	r1					r1			
6	s20	s10					s8				g11		
7							s9						
8	s4		s7								g12		
9	s20	s10					s8					g15	g14
10				r5	r5	r5				r5	r5		
11				r2	r2	s16				r2			
12				s3	s18								
13				r3	r3						r3		
14					s19					s13			
15					r8					r8			
16	s20	s10					s8				g17		
17				r6	r6	s16				r6	r6		
18	s20	s10					s8				g21		
19	s20	s10					s8				g23		
20				r4	r4	r4				r4	r4		
21										s22			
22				r7	r7	r7				r7	r7		
23					r9	s16				r9			

Các phần tử của bảng chuyển được gán nhãn với bốn kiểu hành động:

sn Đẩy vào trạng thái n ;

gn Đi đến trạng thái n ;

rk Rút gọn bởi luật k ;

a Chấp nhận;

Lỗi (ký hiệu dấu trống trong bảng).

Để sử dụng bảng này trong phân tích, xem các hành động đẩy - shift và đi đến - goto như các cạnh của DFA và tham khảo thêm ngăn xếp. Chẳng hạn, nếu ngăn xếp là $id := E$, thì DFA đi từ trạng thái 1 đến 4, đến 6, đến 11. Nếu từ tổ đầu vào tiếp theo là dấu chấm phẩy, khi đó cột ";" ở trạng thái 11 cho biết là rút gọn bởi luật 2. Luật thứ 2 của văn phạm là $S \rightarrow id := E$, vì vậy 3 từ tổ ở trên đỉnh được lấy ra khỏi ngăn xếp và S được đẩy vào.

Hành động đối với phép "+" ở trạng thái 11 là đẩy; vậy nếu từ tổ tiếp theo thay vào đó là dấu +, nó có thể được nuốt từ đầu vào và được đẩy vào ngăn xếp.

Thay vì quét lại ngăn xếp đối với từng từ tổ, bộ phân tích có thể nhớ lại bằng cách dùng trạng thái đạt được đối với mỗi phần tử của ngăn xếp. Thuật toán phân tích như sau:

Xem trạng thái của ngăn xếp, và ký hiệu đầu vào để nhận được hành động; Nếu hành động là

Shift(n): Nhảy tới từ tố tiếp theo của đầu vào; đẩy n vào ngăn xếp.

Reduce(k): Lấy ra từ ngăn xếp số từ tố cần thiết của vế phải của luật thứ k ;

Giả sử X là ký hiệu ở vế trái của luật thứ k ;

Ở trạng thái trên đỉnh ngăn xếp, xem X để nhận được hành động "goto n ";

Đẩy n vào đỉnh ngăn xếp.

Accept: Dừng phân tích, thông báo thành công.

Error: Dừng phân tích, thông báo thất bại.

SINH PHÂN TÍCH LR(0)

Phân tích LR(k) sử dụng nội dung của ngăn xếp và k từ tố tiếp theo của đầu vào để quyết định dùng hành động nào. Bảng 5.9 chỉ ra việc sử dụng một ký hiệu xem trước. Với $k = 2$, bảng có các cột với mỗi cặp hai từ tố và tiếp tục như vậy; trên thực tế, $k > 1$ không được sử dụng cho chương trình dịch. Điều đó một phần vì bảng trên có thể sẽ rất lớn, nhưng phần nhiều là do hầu hết các ngôn ngữ lập trình dùng hiện nay đều có thể mô tả như các văn phạm LR(1).

Các văn phạm LR(0) là văn phạm mà có thể được phân tích nhờ chỉ xem vào ngăn xếp, mà đưa ra quyết định đẩy/rút không cần xem trước ký hiệu. Mặc dù lớp văn phạm này quá yếu để có thể dùng được, thuật toán xây dựng bảng phân tích LR(0) là tập dượt tốt cho việc xây dựng thuật toán phân tích LR(1).

Chúng ta sẽ sử dụng Văn phạm 5.10 để mô tả việc sinh phân tích LR(0). Xét xem phân tích cú pháp cho văn phạm này sẽ thực hiện như thế nào. Khởi đầu, nó có ngăn xếp rỗng, và đầu vào là câu S hoàn chỉnh nối tiếp bởi $\$$; nó sẽ ở vế phải của luật S' . Chúng ta thể hiện bằng luật $S' \rightarrow .S\$$ trong đó dấu chấm chỉ vị trí hiện tại của bộ phân tích.

VĂN PHẠM 5.10

1. $S' \rightarrow S\$$
2. $S \rightarrow (L)$
3. $S \rightarrow x$
4. $L \rightarrow S$
5. $L \rightarrow L, S$

Trong trạng thái này, khi đầu vào bắt đầu từ S , nó có nghĩa là nó bắt đầu với mọi vế phải có thể của luật sinh S ; chúng ta chỉ ra rằng bằng cách

$$\begin{array}{l} S' \rightarrow .S\$ \\ S \rightarrow .x \\ S \rightarrow .(L) \end{array}^1$$

Gọi trạng thái này là 1. Một luật văn phạm, kết hợp với dấu chấm để chỉ ra vị trí trong vế phải của nó, được gọi là tay quai (*item* – hay *LR(0) item*). Trạng thái là một tập các tay quai.

Hành động đẩy - Shift actions. Trong trạng thái 1, xét xem cái gì xảy ra nếu chúng ta đẩy x vào. Chúng ta biết cuối ngăn xếp là x ; và chúng ta chỉ ra điều đó bằng cách đẩy dấu chấm vượt qua x trong luật $S \rightarrow x$. Các luật $S' \rightarrow .S\$$ và $S \rightarrow .(L)$ không liên quan đến hành động này, nên chúng ta bỏ qua chúng; chúng ta kết thúc ở trạng thái 2:

$$S \rightarrow x.^2$$

Hoặc trong trạng thái 1 chúng ta xét việc đẩy dấu mở ngoặc trái trong tay quai thứ ba tạo ra $S \rightarrow .(L)$, trong đó chúng ta biết rằng cần có dấu ngoặc trái ở trên đỉnh của ngăn xếp và đầu vào bắt đầu bởi một xâu nào đó suy diễn từ L , nối tiếp bởi dấu đóng ngoặc phải. Từ tổ nào có thể bắt đầu đầu vào bây giờ? Chúng ta tìm được bằng cách bổ sung mọi luật sinh từ L vào tập các tay quai. Nhưng trong một L tay quai này, dấu chấm đứng ngay trước S , vậy chúng ta cần bổ sung tiếp mọi luật sinh từ S :

$$\begin{array}{l} S \rightarrow .(L) \\ L \rightarrow .L, S \\ L \rightarrow .S \\ S \rightarrow .(L) \\ S \rightarrow .x \end{array}^3$$

Hành động đi đến - Goto actions. Ở trạng thái 1, xét tác động của bộ phân tích khi qua một xâu các từ tổ suy diễn từ chưa tận S . Điều này xảy ra khi x hoặc mở ngoặc trái được đẩy, tiếp theo (hiển nhiên) là rút gọn bởi luật sinh S . Mọi ký hiệu ở vế phải của luật sinh này sẽ được lấy ra, và bộ phân tích sẽ thực hiện hành động goto đối với S trong trạng thái 1. Tác động này có thể được mô tả bằng cách chuyển dấu chấm qua S trong tay quai thứ nhất của trạng thái 1, tạo ra trạng thái 4:

$$S' \rightarrow S.S^4$$

Hành động rút gọn - Reduce actions. Ở trạng thái 2 chúng ta tìm thấy dấu chấm cuối tay quai. Điều này có nghĩa là trên đỉnh ngăn xếp là toàn bộ vế phải của luật sinh tương ứng ($S \rightarrow x$), đã sẵn sàng để rút gọn. Ở trạng thái này bộ phân tích cần phải thực hiện hành động rút gọn (reduce action).

Các thao tác cơ bản mà chúng ta cần thực hiện ở các trạng thái là **closure**(I) và **goto**(I, X), trong đó I là tập các tay quay và X là ký hiệu văn phạm (tận cùng hoặc chưa tận). **Closure** bổ sung các tay quay vào tập tay quay khi mà có dấu chấm ở bên trái của chưa tận; **goto** dịch chuyển dấu chấm qua ký hiệu X trong mọi tay quay.

```

Closure( $I$ ) =
  repeat
    for any item  $A \rightarrow \alpha.X\beta$  in  $I$ 
      for any production  $X \rightarrow \gamma$ 
         $I \leftarrow I \cup \{X \rightarrow \gamma\}$ 
    until  $I$  does not change.
  return  $I$ 

Goto( $I, X$ ) =
  set  $J$  to the empty set
  for any item  $A \rightarrow \alpha.X\beta$  in  $I$ 
    add  $A \rightarrow \alpha.X\beta$  to  $J$ 
  return Closure( $J$ )

```

Dưới đây là thuật toán xây dựng bộ phân tích LR(0). Trước hết, thêm một luật sinh khởi đầu $S' \rightarrow S\$$. Giả sử T là tập các trạng thái đã có, và E là tập các cạnh (đẩy hoặc đi đến - shift or goto) đã được xây dựng.

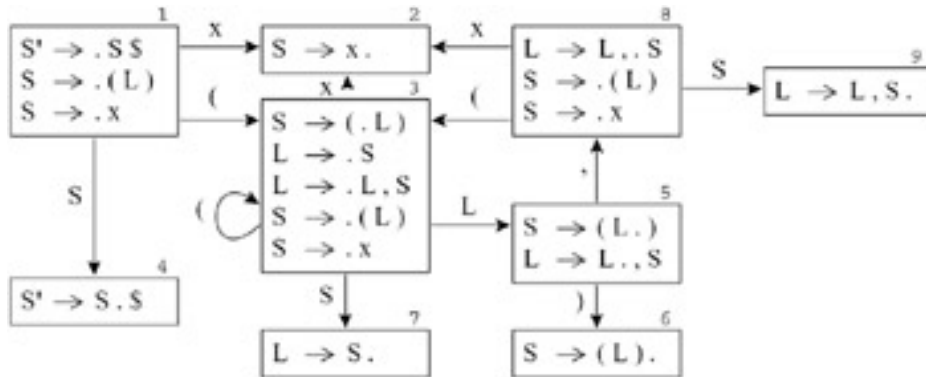
```

Initialize  $T$  to {Closure( $\{S' \rightarrow .S\}\})$ }
Initialize  $E$  to empty.
repeat
  for each state  $I$  in  $T$ 
    for each item  $A \rightarrow \alpha.X\beta$  in  $I$ 
      let  $J$  be Goto( $I, X$ )
       $T \leftarrow T \cup \{J\}$ 
       $E \leftarrow E \cup \{I \xrightarrow{X} J\}$ 
until  $E$  and  $T$  did not change in this iteration

```

Tuy nhiên, đối với ký hiệu $\$$ chúng ta không tính **Goto**($I, \$$); thay vào đó chúng ta sẽ thực hiện hành động chấp nhận (**accept action**).

Đối với Văn phạm 5.10 thuật toán xây dựng được mô tả trong Hình 5.11.



Hình 5.11: Các trạng thái LR(0) cho Văn phạm 5.10.

Bây giờ chúng ta tính tập R của các hành động LR(0) rút gọn:

```

 $R \leftarrow \{\}$ 
for each state  $I$  in  $T$ 
  for each item  $A \rightarrow \alpha.$  in  $I$ 

```

$$R \leftarrow R \cup \{ (I, A \rightarrow \alpha) \}$$

Sau đây chúng ta xây dựng bảng phân tích cho văn phạm này (Bảng 5.12). Đối với mỗi cạnh $I \xrightarrow{X} J$, trong đó X là tận cùng, chúng ta đặt hành động đẩy *shift* J vào vị trí (I, X) trong bảng; nếu X là chưa tận, chúng ta đặt *goto* J vào vị trí (I, X) . Với mỗi trạng thái I chứa tay quai $S' \rightarrow S.\$$ chúng ta đặt hành động chấp nhận vào ô $(I, \$)$. Cuối cùng, với mỗi trạng thái chứa tay quai $A \rightarrow \gamma$. (luật n với dấu chấm ở cuối), chúng ta đặt hành động rút gọn n - *reduce* n tại ô (I, Y) đối với mỗi từ tố Y .

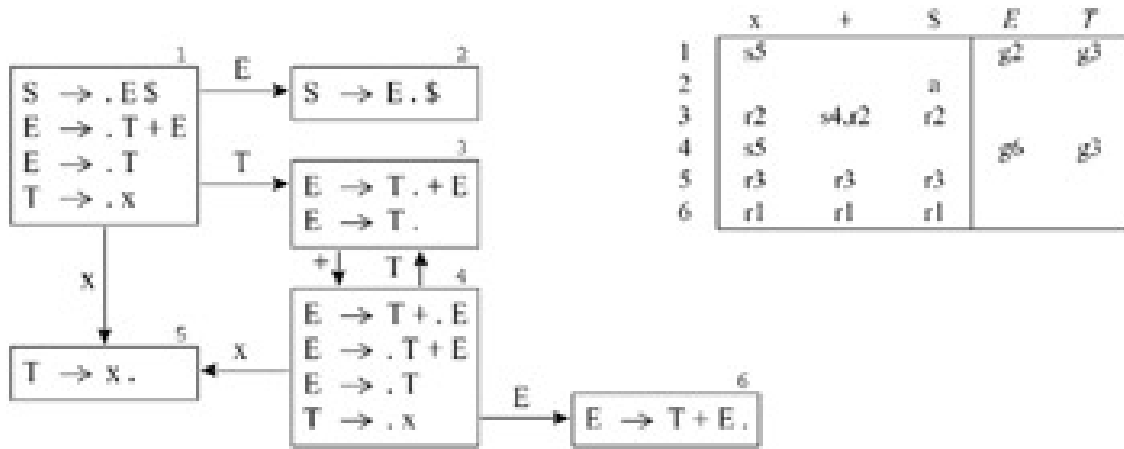
Bảng 5.12: Bảng phân tích LR(0) cho Văn phạm 5.10.

	(	)	x	,	\$	S	L
1	s3		s2			g4	
2	r2	r2	r2	r2	r2		
3	s3		s2			g7	g5
4					a		
5		s6		s8			
6	r1	r1	r1	r1	r1		
7	r3	r3	r3	r3	r3		
8	s3		s2			g9	
9	r4	r4	r4	r4	r4		

Về mặt nguyên tắc, vì LR(0) không cần xem trước, nên chúng ta chỉ cần một hành động duy nhất cho mỗi trạng thái: trạng thái sẽ đẩy (*shift*) hoặc rút gọn (*reduce*), nhưng không cả hai. Trên thực tế, vì chúng ta cần biết đầy đủ đến trạng thái nào, nên chúng ta có hàng bắt $S \rightarrow E \$$

VĂN PHẠM 5.13

0. $S \rightarrow E \$$
1. $E \rightarrow T + E$
2. $E \rightarrow T$
3. $T \rightarrow x$



Hình 5.14: Bảng phân tích và trạng thái LR(0) cho Văn phạm 5.13.

Trong trạng thái 3, trên ký hiệu +, có hai khả năng: Bộ phân tích cần đẩy đến trạng thái 4 và cũng rút gọn bởi luật 2. Điều này mâu thuẫn và chỉ ra rằng văn phạm này không là LR(0), nó không thể được duyệt bằng phân tích LR(0). Chúng ta cần có thuật toán phân tích mạnh hơn.

Một cách đơn giản xây dựng bộ duyệt tốt hơn LR(0) được gọi là SLR, tức là Simple LR. Xây dựng bộ phân tích SLR cũng gần giống với LR(0), ngoại trừ chúng ta đặt hành động rút gọn vào trong bảng chỉ ở nơi nào được chỉ dẫn bởi tập FOLLOW.

Sau đây là thuật toán đặt hành động rút gọn (reduce) vào bảng SLR:

```

R ← {}
for each state I in T
    for each item A → α. in I
        for each token X in FOLLOW(A)
            R ← R ∪ {(I, X, A → α)}
    
```

Hành động $(I, X, A \rightarrow \alpha)$ chỉ ra rằng ở trạng thái I , trên ký hiệu xem trước X , bộ phân tích sẽ rút gọn bởi luật $A \rightarrow \alpha$.

Chẳng hạn, đối với Văn phạm 5.13 chúng ta cũng sử dụng sơ đồ trạng thái LR(0) (Hình 5.14), nhưng chúng ta đặt ít hành động rút gọn hơn vào bảng SLR, như trên Hình 5.15.

	x	+	\$	E	T
1	s5			g2	g3
2			a		
3		s4	r2		
4	s5			g6	g3
5		r3	r3		
6			r1		

Hình 5.15: Bảng phân tích SLR cho Văn phạm 5.13.

Lớp các văn phạm SLR chính là bao gồm các văn phạm mà bảng phân tích SLR không chứa tranh chấp (không có hai khả năng cho một ô nào đó). Văn phạm 5.13 thuộc lớp này, cũng như nhiều văn phạm của các ngôn ngữ lập trình có ích khác.

MỤC LR(1), BẢNG PHÂN TÍCH LR(1)

Mạnh hơn hẳn SLR là thuật toán phân tích LR(1). Hầu hết các ngôn ngữ lập trình mà cú pháp của nó được mô tả bằng văn phạm phi ngữ cảnh đều có văn phạm LR(1).

Thuật toán để xây dựng bảng phân tích LR(1) tương tự như đối với LR(0), nhưng định nghĩa *mục item* là phức tạp hơn. Mục LR(1) gồm luật văn phạm, vị trí về phải (biểu diễn qua dấu chấm), và ký hiệu xem trước *lookahead symbol*. Ý tưởng là mục $(A \rightarrow \alpha.\beta, x)$ chỉ ra rằng dãy α đang ở trên đỉnh ngăn xếp, và ở đầu của đầu vào là xâu suy diễn được từ βx .

Trạng thái LR(1) là tập các mục LR(1), và có hai thao tác **Closure** và **Goto** cho LR(1) mà tương tác với ký hiệu xem trước:

<pre> Closure (I) = repeat for any item $(A \rightarrow \alpha.X\beta, z)$ in I for any production $X \rightarrow \gamma$ for any $w \in \text{FIRST}(\beta z)$ $I \leftarrow I \cup \{(X \rightarrow .\gamma, w)\}$ until I does not change return I </pre>	<pre> Goto (I, X) = $J \leftarrow \{\}$ for any item $(A \rightarrow \alpha.X\beta, z)$ in I add $(A \rightarrow \alpha.X.\beta, z)$ to J return Closure (J) . </pre>
---	--

Trạng thái bắt đầu là bao đóng của mục $(S' \rightarrow .S \$, ?)$, trong đó ký hiệu xem trước $?$ là không quan trọng, vì dấu kết thúc file không bao giờ được chuyển.

Các hành động rút gọn được chọn bởi thuật toán này:

```

 $R \leftarrow \{\}$ 
for each state  $I$  in  $T$ 
  for each item  $(A \rightarrow \alpha., z)$  in  $I$ 
     $R \leftarrow R \cup \{(I, z, A \rightarrow \alpha)\}$ 

```

Hành động $(I, z, A \rightarrow \alpha)$ chỉ ra rằng ở trạng thái I , với ký hiệu xem trước z , bộ phân tích sẽ rút gọn bởi luật $A \rightarrow \alpha$.

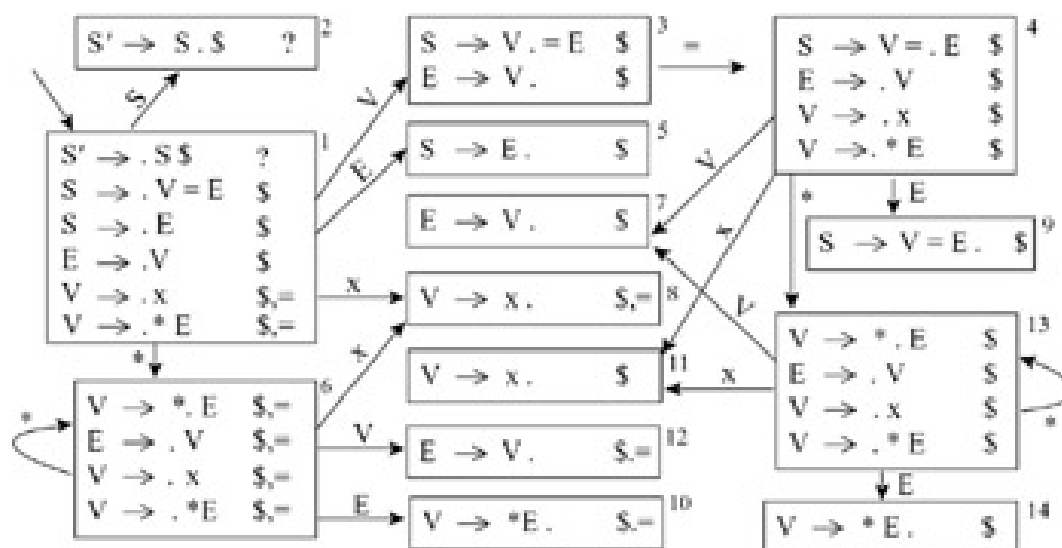
Văn phạm 5.16 không là SLR (xem Bài tập 5.6), nhưng nó nằm trong lớp văn phạm LR(1). Hình 5.17 chỉ ra các trạng thái LR(1) cho văn phạm này; Trên hình vẽ, có một số mục với cùng một luật sinh nhưng các ký hiệu xem trước khác nhau, như ở bên trái dưới đây, chúng ta có viết tắt như ở bên phải:

$S' \rightarrow . S \$$	?
$S \rightarrow . V = E$	\$
$S \rightarrow . E$	\$
$E \rightarrow . V$	\$
$V \rightarrow . x$	\$
$V \rightarrow . * E$	\$
$V \rightarrow . x$	=
$V \rightarrow . * E$	=

$S' \rightarrow . S \$$	?
$S \rightarrow . V = E$	\$
$S \rightarrow . E$	\$
$E \rightarrow . V$	\$
$V \rightarrow . x$	\$, =
$V \rightarrow . * E$	\$, =

VĂN PHẠM 5.16: Văn phạm bao quát thực chất các biểu thức, các biến và các con trỏ (ký hiệu bởi $*$) trong ngôn ngữ C.

1. $S' \rightarrow S \$$
2. $S \rightarrow V = E$
3. $S \rightarrow E$
4. $E \rightarrow V$
5. $V \rightarrow x$
6. $V \rightarrow * E$



Hình 5.17: Các trạng thái LR(1) cho Văn phạm 5.16.

Bảng cú pháp LR(1) suy ra từ đồ thị trạng thái này là Bảng 5.18a. Ở đâu dấu chấm ở cuối luật sinh (như trong trạng thái 3 của Bảng 5.17, ở đó là cuối luật sinh $E \rightarrow V$), thì ở đó có hành động rút gọn đối với luật sinh đó trong bảng LR(1), trên hàng tương ứng với số của trạng thái đó và cột tương ứng với ký hiệu xem trước của mục đó (trong trường hợp này là \$). Ở đâu dấu chấm ở bên trái ký hiệu kết thúc hoặc chưa kết, ở đó có hành động shift hoặc goto tương ứng trong bảng phân tích cú pháp LR(1), giống như trong bảng LR(0).

Bảng 5.18: Các bảng phân tích cú pháp LR(1) và LALR(1) cho Văn phạm 5.16.

	x	*	=	\$	S	E	F
1	s8	s6			g2	g5	g3
2				a			
3			s4	r3			
4	s11	s13				g9	g7
5				r2			
6	s8	s6				g10	g12
7				r3			
8			r4	r4			
9				r1			
10			r5	r5			
11				r4			
12			r3	r3			
13	s11	s13				g14	g7
14				r5			

(a) LR(1)

	x	*	=	\$	S	E	F
1	s8	s6			g2	g5	g3
2				a			
3			s4	r3			
4	s8	s6				g9	g7
5				r2			
6	s8	s6				g10	g7
7			r3	r3			
8			r4	r4			
9				r1			
10			r5	r5			

(b) LALR(1)

CÁC BẢNG PHÂN TÍCH CÚ PHÁP LALR(1)

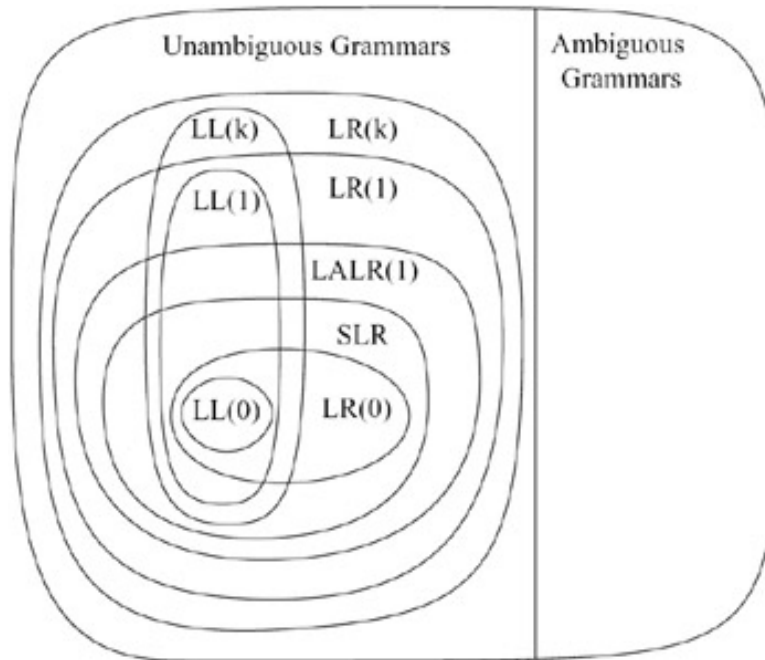
Các bảng phân tích cú pháp LR(1) có thể rất lớn, với nhiều trạng thái. Bảng nhỏ hơn có thể được tạo bằng cách nhập hai trạng thái mà các mục của chúng là giống nhau trừ các tập xem trước. Bảng kết quả được gọi là Bảng phân tích cú pháp LALR(1), đối với LR(1) xem trước.

Chẳng hạn, các mục trong các trạng thái 6 và 13 của Bảng LR(1) cho Văn phạm 5.16 (Hình 5.6) là đồng nhất nếu bỏ qua các tập xem trước. Cũng như vậy, các trạng thái 7 và 12 là đồng nhất trừ các tập xem trước, và cũng như vậy với cặp trạng thái 8 và 11, cặp trạng thái 10 và 14. Nhập các cặp trạng thái này được Bảng phân tích cú pháp LALR(1) như đã nêu trong Bảng 5.18b.

Đối với một số văn phạm, bảng LALR(1) chứa xung đột rút gọn - rút gọn trong khi đó bảng LR(1) lại không có, nhưng trong thực tế bản chất sự việc có khác nhau đôi chút. Bảng LALR(1) đòi hỏi ít bộ nhớ để biểu diễn hơn bảng LR(1), vì rằng có ít hơn khá nhiều các trạng thái.

PHÂN LOẠI CÁC LỚP VĂN PHẠM

Văn phạm được gọi là LALR(1) nếu bảng phân tích cú pháp LALR(1) không chứa xung đột. Mọi văn phạm SLR là LALR(1), nhưng ngược lại không đúng. Hình 5.19 chỉ ra quan hệ giữa một số lớp văn phạm.



Hình 5.19: Phân loại các lớp văn phạm.

Hầu như bất cứ ngôn ngữ lập trình nào cũng có văn phạm LALR(1), và có nhiều công cụ sinh bộ phân tích cú pháp có sẵn cho các văn phạm LALR(1). Với lý do này, LALR(1) trở thành chuẩn cho ngôn ngữ lập trình và bộ sinh phân tích cú pháp tự động.

PHÂN TÍCH CÚ PHÁP LR CỦA CÁC VĂN PHẠM NHẬP NHẰNG

Nhiều ngôn ngữ lập trình có các qui tắc văn phạm như sau

- $S \rightarrow \text{if } E \text{ then } S \text{ else } S$
- $S \rightarrow \text{if } E \text{ then } S$
- $S \rightarrow \text{other}$

Mà cho phép có các chương trình như

```
if a then if b then s1 else s2
```

Chương trình trên có thể được hiểu theo hai cách khác nhau:

- (1)

```
if a then { if b then s1 else s2 }
```
- (2)

```
if a then { if b then s1 } else s2
```

Trong hầu hết các ngôn ngữ lập trình, phần lệnh `else` cần được sánh với `then` gần nhất có thể, như vậy thông dịch (1) là đúng. Trong bảng phân tích cú pháp LR sẽ có xung đột dịch – rút gọn:

$S \rightarrow \text{if } E \text{ then } S .$	else
$S \rightarrow \text{if } E \text{ then } S . \text{ else } S$	(any)

Đẩy (Shifting) tương ứng với thông dịch (1) và rút gọn (reducing) tương ứng với thông dịch (2).

Tính nhập nhằng có thể được loại bỏ bằng cách đưa thêm các chưa tận phụ trợ M (đối với lệnh sánh) và U (đối với lệnh không sánh):

- $S \rightarrow M$
- $S \rightarrow U$
- $M \rightarrow \text{if } E \text{ then } M \text{ else } M$
- $M \rightarrow \text{other}$
- $U \rightarrow \text{if } E \text{ then } S$
- $U \rightarrow \text{if } E \text{ then } M \text{ else } U$

Nhưng thay vì viết lại văn phạm, chúng ta có thể cứ để văn phạm không thay đổi và giải quyết xung đột dịch – đẩy. Khi xây dựng bảng phân tích cú pháp mâu thuẫn này sẽ được giải quyết bằng việc dịch (shifting), vì chúng ta mong muốn cách dịch (1).

Nhiều khi có thể sử dụng văn phạm nhập nhằng bằng cách giải quyết mâu thuẫn dịch – đẩy theo hướng ưu tiên dịch hoặc đẩy tùy theo. Nhưng tốt nhất sử dụng kỹ thuật này một cách tiết kiệm, nhưng chỉ trong trường hợp (như trường hợp `else` đưa mô tả ở trên, và độ ưu tiên các phép toán mô tả ở phần sau) mà được hiểu một cách đúng đắn. Hầu hết mâu thuẫn dịch – rút gọn, và xác suất mọi mâu thuẫn rút gọn – rút gọn, không được giải quyết bằng cách thêm vào bảng phân tích cú pháp. Chúng là triệu chứng của văn phạm được đặc tả lỗi, và chúng cần được giải quyết bằng cách loại bỏ nhập nhằng.

SỬ DỤNG BỘ SINH PHÂN TÍCH CÚ PHÁP

Nhiệm vụ xây dựng Phân tích cú pháp là tương đối đơn giản đến mức có thể tự động hóa được. Trong chương trước chúng ta đã mô tả các khía cạnh Phân tích từ vựng của JavaCC và SableCC. Ở đây chúng ta tiếp tục trao đổi về các khía cạnh bộ sinh Phân tích cú pháp của các công cụ này. Tài liệu về JavaCC và SableCC được cung cấp qua trang Web của tài liệu [1].

JAVACC

JavaCC là bộ sinh Phân tích cú pháp LL(k). Các luật sinh có dạng:

```
void Assignment() : {} { Identifier() "=" Expression() ";" }
```

trong đó về trái là phép gán `Assignment()`; hai vế phải được bao quanh bên trong hai dấu ngoặc móc; `Assignment()`, `Identifier()`, và `Expression()` là các ký hiệu chưa tận; và "=" và ";" là các ký hiệu tận cùng.

Văn phạm 5.10 có thể được thể hiện như văn phạm JavaCC được nêu trong Văn phạm 5.21. Nhận thấy rằng, nếu chúng ta đã viết luật sinh cho `StmList()` theo phong cách của Văn phạm 5.20, thì đó là

```
void StmList() :  
{  
  { Stm()  
  | StmList( ) ";" Stm()  
  }
```

VĂN PHẠM 5.20

1. $P \rightarrow L$
2. $S \rightarrow \text{id} := \text{id}$
3. $S \rightarrow \text{while id do } S$
4. $S \rightarrow \text{begin } L \text{ end}$
5. $S \rightarrow \text{if id then } S$
6. $S \rightarrow \text{if id then } S \text{ else } S$
7. $L \rightarrow S$
8. $L \rightarrow L ; S$

Khi đó Văn phạm này sẽ là đệ qui trái. Trong trường hợp này, JavaCC cho thông báo lỗi sau:

```
Left recursion detected: "StmList... --> StmList..."
```

VĂN PHẠM 5.21: Phiên bản JavaCC cho Văn phạm 5.20.

```
PARSER_BEGIN(MyParser)  
  public class MyParser {}  
PARSER_END(MyParser)
```

```
SKIP :  
{ " " | "\t" | "\n" }
```

```
TOKEN :  
{ < WHILE: "while" >  
  | < BEGIN: "begin" >  
  | < END: "end" >  
  | < DO: "do" >  
  | < IF: "if" >  
  | < THEN: "then" >
```

```
| < ELSE: "else" >
| < SEMI: ";" >
| < ASSIGN: "=" >
| < ID: ["a"-"z"] (["a"-"z"] | ["0"-"9"])* >
}

void Prog() :
{}
{ StmList() <EOF> }

void StmList() :
{}
{ Stm() StmListPrime() }

void StmListPrime() :
{}
{ ( ";" Stm() StmListPrime() )? }

void Stm() :
{}
{ <ID> "=" <ID>
| "while" <ID> "do" Stm()
| "begin" StmList() "end"
| LOOKAHEAD(5) /* we need to lookahead till we see "else" */
"if" <ID> "then" Stm()
| "if" <ID> "then" Stm() "else" Stm()
}
```

Chúng ta sử dụng kỹ thuật đã nêu trước đây để loại bỏ đệ qui trái và nhận được Văn phạm 5.21.

SABLECC

SableCC là bộ sinh Phân tích cú pháp LALR(1). Các luật sinh có dạng:

```
assignment = identifier assign expression semicolon ;
```

trong đó về trái là *assignment*; về phải được bao quanh giữa dấu = và *;; assignment*, *identifier*, và *expression* là các ký hiệu chưa tận; và *assign* và *semicolon* là các ký hiệu tận cùng mà đã được định nghĩa ở phần trước của đặc tả cú pháp đó.

Văn phạm 5.20 có thể được biểu diễn như Văn phạm SableCC được nêu trong Văn phạm 5.22. Khi có thêm một lựa chọn nữa, SableCC đòi hỏi đặt tên cho mỗi lựa chọn. Tên được gán cho một lựa chọn của Văn phạm bằng cách thêm tiếp đầu ngữ cho lựa chọn với định danh trong ngoặc móc. Cũng như vậy, nếu cùng một ký hiệu văn phạm xuất hiện hai lần trong cùng một lựa chọn của một luật sinh, thì SableCC yêu cầu đặt tên cho ít nhất một trong hai phần tử đó. Các tên của phần tử được đặc tả bởi tiếp đầu ngữ cho phần tử với định danh nằm giữa các ngoặc vuông đứng sau là dấu hai chấm.

VĂN PHẠM 5.22: Phiên bản SableCC cho Văn phạm 5.20.

```

Tokens
  while = 'while';
  begin = 'begin';
  end = 'end';
  do = 'do';
  if = 'if';
  then = 'then';
  else = 'else';
  semi = ';';
  assign = '=';
  whitespace = (' ' | '\t' | '\n')+;
  id = ['a'..'z'](['a'..'z' | ['0'..'9']]*)

Ignored Tokens
  whitespace;

Productions
  prog = stmlist;

  stm = {assign} [left]:id assign [right]:id |
        {while} while id do stm |
        {begin} begin stmlist end |
        {if_then} if id then stm |
        {if_then_else} if id then [true_stm]:stm else
[false_stm]:stm;

  stmlist = {stmt} stm |
            {stmtlist} stmlist semi stm;

```

SableCC sẽ đưa ra thông báo về xung khắc đẩy - rút gọn (shift-reduce) và rút gọn – rút gọn (reduce-reduce). Xung khắc đẩy – rút gọn là lựa chọn giữa đẩy và rút gọn; xung khắc rút gọn – rút gọn là lựa chọn giữa hai rút gọn khác nhau.

SableCC sẽ thông báo Văn phạm 5.22 có xung khắc đẩy – rút gọn. Xung khắc này có thể kiểm tra bằng cách thông báo lỗi chi tiết mà SableCC tạo ra như được nêu trên Hình 5.23.

```

shift/reduce conflict in state [stack: TIf TId TThen PStm *] on TElse
in {
    [ PStm = TIf TId TThen PStm * TElse PStm ] (shift),
    [ PStm = TIf TId TThen PStm * ] followed by TElse (reduce)
}

```

Hình 5.23: Thông báo lỗi shift-reduce của SableCC cho Văn phạm 5.22.

SableCC shift-reduce bổ sung tiếp đầu ngữ cho các luật sinh với chữ hoa ‘P’ và các từ tổ với chữ hoa ‘T’, và thay các chữ đầu với chữ hoa khi chúng tạo các đối tượng cho các từ tổ và các luật sinh. Chúng ta nhìn thấy điều đó trên ngăn xếp thông báo lỗi trong Hình 5.23. Thật vậy trên ngăn xếp mà chúng ta có cho các từ tổ *if*, *id*, *then*, và luật sinh mà sánh với *stm*, và cả từ tổ *else*. Rõ ràng ở đó chỉ ra rằng mâu thuẫn được gây ra bởi *else* đu đưa.

Để giải quyết mâu thuẫn này chúng ta viết lại Văn phạm, loại bỏ tính nhập nhằng trong Văn phạm 5.14.

VĂN PHẠM 5.24: Các luật sinh dạng SableCC của Văn phạm 5.22 khi đã giải quyết xung đột.

Productions

```

prog = stmlist;

stm = {stm_without_trailing_substm}
      stm_without_trailing_substm |
      {while} while id do stm |
      {if_then} if id then stm |
      {if_then_else} if id then stm_no_short_if
                      else [false_stm]:stm;

stm_no_short_if = {stm_without_trailing_substm}
                  stm_without_trailing_substm |
                  {while_no_short_if}
                  while id do stm_no_short_if |
                  {if_then_else_no_short_if}
                  if id then [true_stm]:stm_no_short_if
                      else [fals_stm]:stm_no_short_if;

stm_without_trailing_substm = {assign} [left]:id assign [right]:id
|
                              {begin} begin stmlist end ;

stmlist = {stmt} stm | {stmtlist} stmlist semi stm;

```

CHỈ DẪN ĐỘ ƯU TIÊN

Không có văn phạm nhập nhằng là LR(k) đối với k bất kỳ; Bảng phân tích cú pháp LR(k) của văn phạm nhập nhằng luôn có xung đột. Tuy nhiên, văn phạm nhập nhằng có thể vẫn còn có ích, nếu chúng ta tìm cách giải quyết các xung đột.

Chẳng hạn, Văn phạm 4.5 là nhập nhằng mức độ cao. Sử dụng văn phạm này để mô tả ngôn ngữ lập trình, chúng ta muốn phân tích sao cho $*$ và $=$ gắn kết chặt hơn $+$ và $-$, và mỗi một thao tác có tính kết hợp trái. Chúng ta muốn biểu diễn nó bằng cách viết lại Văn phạm không ngập nhằng 4.8.

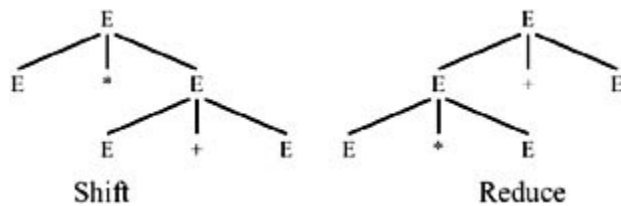
Nhưng chúng ta có thể tránh đưa thêm các ký hiệu T và F và các phép rút gọn tầm thường $E \rightarrow T$ và $T \rightarrow F$. Thay vào đó, chúng ta bắt đầu xây dựng Bảng phân tích cú pháp LR(1) cho Văn phạm 4.5 như đã nêu trong Bảng 5.25. Chúng ta thấy có nhiều xung đột. Chẳng hạn, ở trạng thái 13 với ký hiệu xem trước $+$ ta thấy có xung đột giữa dịch chuyển *shift sang trạng thái 8* và rút gọn *reduce bởi luật 3*. Hai mục ở trạng thái 13 là

$$\begin{array}{l} E \rightarrow E * E . \quad + \\ E \rightarrow E . + E \quad (any) \end{array}$$

Bảng 5.25: Bảng phân tích cú pháp cho Văn phạm 4.5.

	id	num	+	-	*	/	(	)	\$	E
1	s2	s3					s4			g7
2			r1	r1	r1	r1		r1	r1	
3			r2	r2	r2	r2		r2	r2	
4	s2	s3					s4			g5
5								s6		
6			r7	r7	r7	r7		r7	r7	
7			s8	s10	s12	s14			a	
8	s2	s3					s4			g9
9			s8,r5	s10,r5	s12,r5	s14,r5		r5	r5	
10	s2	s3					s4			g11
11			s8,r6	s10,r6	s12,r6	s14,r6		r6	r6	
12	s2	s3					s4			g13
13			s8,r3	s10,r3	s12,r3	s14,r3		r3	r3	
14	s2	s3					s4			g15
15			s8,r4	s10,r4	s12,r4	s14,r4		r4	r4	

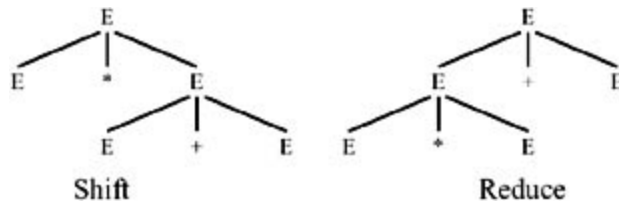
Trong trạng thái này đỉnh của ngăn xếp là $\dots E * E$. Dịch chuyển Shifting sẽ dẫn đến ngăn xếp $\dots E * E +$ và sau đó hiển nhiên $\dots E * E + E$ sẽ rút gọn $E + E$ thành E . Còn rút gọn reducing bây giờ sẽ dẫn đến ngăn xếp $\dots E$ và sau đó $+$ sẽ được dịch chuyển. Các cây phân tích cú pháp nhận được bởi dịch chuyển và rút gọn như sau



Nếu chúng ta muốn $*$ gắn kết chặt hơn $+$, chúng ta sẽ rút gọn thay vì dịch chuyển. khi đó chúng ta điền vào ô (13, $+$) trong Bảng với nội dung r3 và bỏ qua hành động s8.

Ngược lại, trong trạng thái 9 với ký hiệu xem trước $*$, chúng ta sẽ dịch chuyển thay vì rút gọn, khi đó ta giải quyết xung đột bằng cách điền vào ô (9, $*$) với nội dung s12.

Trường hợp cho trạng thái 9, ký hiệu xem trước $+$ là



Dịch chuyển sẽ làm cho phép toán có tính kết hợp phải; rút gọn sẽ làm cho nó có tính kết hợp trái. Vì chúng ta muốn có kết hợp trái, ta sẽ điền vào ô (9, +) với r5.

Xét biểu thức $a - b - c$. Trong hầu hết các ngôn ngữ lập trình, nó có tính kết hợp trái, như viết dạng $(a - b) - c$. Nhưng giả sử chúng ta cho rằng biểu thức trên bản chất là không rõ ràng và chúng ta buộc lập trình viên phải viết ngoặc tường minh, hoặc $(a - b) - c$ hoặc $a - (b - c)$. Khi đó ta nói phép toán trừ là không kết hợp, và ta có thể sẽ điền vào ô (11, -) với nội dung là lỗi error.

Kết quả các quyết định trên sẽ là Bảng phân tích cú pháp với mọi xung đột được giải quyết (Bảng 5.26).

Bảng 5.26: Các xung đột của Bảng 5.25 được giải quyết.

	+	-	*	/
			⋮	
9	r5	r5	s12	s14
11	...		s12	s14
13	r3	r3	r3	r3
15	r4	r4		
			⋮	

Yacc có chỉ dẫn độ ưu tiên để giải quyết kiểu xung đột dịch chuyển – rút gọn nêu trên. (Không may, SableCC không có chỉ dẫn độ ưu tiên.) Dãy các khai báo như sau

```
precedence nonassoc EQ, NEQ;
precedence left PLUS, MINUS;
precedence left TIMES, DIV;
precedence right EXP;
```

chỉ rõ rằng + và - là kết hợp trái và có độ gắn kết cao bằng nhau; rằng * và / là kết hợp trái và gắn kết cao hơn +; rằng lũy thừa $\wedge$ là kết hợp phải và gắn kết cao nhất; và rằng = và $\neq$ không kết hợp và có độ gắn kết yếu hơn +.

Khi kiểm tra xung đột dịch chuyển – rút gọn như

$E \rightarrow E * E .$	+
$E \rightarrow E . + E$	(any)

ở đây có lựa chọn giữa dịch chuyển *từ tổ* tiếp theo và rút gọn bằng *luật*. Luật hoặc từ tổ được cho cần phải có độ ưu tiên cao hơn? Khai báo độ ưu tiên (ưu tiên trái, v.v...) cấp độ ưu tiên cho các từ tổ; độ ưu tiên của các luật được cho bởi từ tổ cuối cùng xuất hiện ở vế phải của luật này. Như vậy lựa chọn ở đây giữa luật với độ ưu tiên nhân $*$ và từ tổ với độ ưu tiên $+$; ở đây luật có độ ưu tiên cao hơn, nên xung đột được giải quyết theo hướng rút gọn.

Khi luật và từ tổ có độ ưu tiên như nhau, thì độ ưu tiên trái có xu hướng rút gọn, phải có xu hướng dịch chuyển, và không kết hợp sẽ đưa ra thông báo hành động lỗi.

Thay vì sử dụng mặc định "luật có độ ưu tiên của từ tổ cuối cùng", chúng ta có thể cấp độ ưu tiên cho luật sử dụng chỉ dẫn `%prec`. Qui tắc thường dùng này giúp giải quyết vấn đề "dấu trừ một ngôi" ("unary minus" problem). Trong nhiều ngôn ngữ lập trình dấu trừ một ngôi được gắn chặt hơn bất cứ phép toán hai ngôi nào, như vậy $-6 * 8$ được hiểu là $(-6) * 8$, không phải $-(6 * 8)$. Văn phạm 5.27 cho ví dụ này.

VĂN PHẠM 5.27: Văn phạm Yacc với chỉ dẫn độ ưu tiên.

```
{ declarations of yylex and yyerror }
%token INT PLUS MINUS TIMES UMINUS
%start exp

%left PLUS MINUS
%left TIMES
%left UMINUS
%%

exp : INT
    | exp PLUS exp
    | exp MINUS exp
    | exp TIMES exp
    | MINUS exp %prec UMINUS
```

Từ tổ UMINUS không bao giờ được trả về bởi Phân tích từ vựng; Nó đơn giản là chỗ chứa trong dãy các khai báo độ ưu tiên. Chỉ dẫn `%prec UMINUS` cho luật `exp ::= MINUS exp` có độ ưu tiên cao nhất, vậy rút gọn theo luật này có độ ưu tiên cao hơn mọi thao tác đẩy, ngay cả dấu trừ.

Các qui tắc ưu tiên có ích trong việc giải quyết xung đột, nhưng chúng cần phải không được mập mờ. Nếu bạn có lo lắng khi sử dụng các qui tắc ưu tiên, thì có lẽ bạn nên viết lại văn phạm cho nó không nhập nhằng.

CÚ PHÁP VÀ NGỮ NGHĨA

Xét ngôn ngữ lập trình với các biểu thức số học như $x + y$ và các biểu thức bool như $x + y = z$ hoặc $a \& (b = c)$. Các phép toán số học gắn kết chặt hơn các phép toán bool; có một số biến số học và một số biến bool; và các biểu thức bool không thể cộng với biểu thức số học. Văn phạm 5.28 cho cú pháp đối với ngôn ngữ này.

VĂN PHẠM 5.28: Văn phạm Yacc với các chỉ dẫn độ ưu tiên.

```
%token ID ASSIGN PLUS MINUS AND EQUAL
%start stm
%left OR
%left AND
%left PLUS
%%

stm : ID ASSIGN ae
    | ID ASSIGN be

be  : be OR be
    | be AND be
    | ae EQUAL ae
    | ID

ae  : ae PLUS ae
    | ID
```

Văn phạm có xung đột rút gọn – rút gọn. Chúng ta sẽ viết lại văn phạm như thế nào để loại bỏ xung đột này?

Vấn đề ở đây là khi bộ phân tích cú pháp nhìn thấy định danh như a , nó không có cách nào để biết đây là biến số học hay là biến bool – về cú pháp chúng giống nhau. Giải pháp để dành cho pha Phân tích ngữ nghĩa của chương trình dịch; Nó không là vấn đề vì đã được xử lý một cách tự nhiên bởi với văn phạm phi ngữ cảnh. Văn phạm thích hợp hơn là

- $S \rightarrow id : E$
- $E \rightarrow id$
- $E \rightarrow E \& E$
- $E \rightarrow E = E$
- $E \rightarrow E + E$

Bây giờ biểu thức $a + 5 \& b$ là cú pháp hợp lệ, và pha về sau của chương trình dịch sẽ từ chối nó và in ra thông báo lỗi ngữ nghĩa.

KHẮC PHỤC LỖI

Các bảng LR(k) chứa các hành động đẩy, rút gọn, chấp nhận và lỗi. Ở phần trước chúng ta đã tuyên bố khi phân tích cú pháp LR đối đầu với hành động lỗi nó sẽ dừng phân tích và đưa ra thông báo lỗi. Hành vi này không phải là hay đối với lập trình viên, người mà muốn đưa ra mọi lỗi chứ không chỉ lỗi đầu tiên.

KHẮC PHỤC SỬ DỤNG KÝ HIỆU LỖI

Cơ chế khắc phục lỗi *cục bộ* làm việc theo cách chỉnh sửa ngăn xếp phân tích và đầu vào tại điểm mà lỗi được phát hiện dọc đường và cho phép bộ phân tích tổng kết lại. Một cơ chế khắc phục cục bộ - mà có trong nhiều phiên bản của bộ sinh Phân tích cú pháp Yacc - sử dụng ký hiệu lỗi *error* để kiểm soát quá trình xử lý khôi phục. Khi ký hiệu lỗi chuyên dùng này xuất hiện trong luật sinh của Văn phạm, dây từ tổ đầu vào có lỗi cần được sánh.

Chẳng hạn, trong văn phạm Yacc chúng ta có các luật sinh như sau

- $exp \rightarrow ID$
- $exp \rightarrow exp + ext$
- $exp \rightarrow (exps)$
- $exps \rightarrow exp$
- $exps \rightarrow exps ; exp$

Về mặt không hình thức, chúng ta có thể đặc tả là nếu lỗi cú pháp gặp phải ở giữa biểu thức, bộ phân tích cần bỏ qua đến dấu chấm phẩy tiếp theo hoặc ngoặc ở bên phải (chúng được gọi là đồng bộ các từ tổ *synchronizing tokens*) và tổng kết phân tích. Chúng ta làm việc đó bằng cách bổ sung các luật khắc phục lỗi như sau

- $exp \rightarrow (error)$
- $exps \rightarrow error ; exp$

Bộ sinh phân tích sẽ làm gì với ký hiệu lỗi *error*? Trong khi sinh bộ phân tích, lỗi *error* được coi là ký hiệu tận cùng, và hành động dịch chuyển được nhập vào bảng phân tích cho nó như đối với từ tổ bình thường.

Khi bộ phân tích LR đạt đến trạng thái lỗi, nó sẽ thực hiện các hành động sau:

1. Dỡ ngăn xếp (nếu cần thiết) đến trạng thái đạt được mà ở đó từ tổ hành động lỗi *error* được đẩy *shift*.
2. Dịch chuyển từ tổ lỗi *error*.
3. Bỏ qua các ký hiệu đầu vào (nếu cần thiết) đến khi từ xem trước đạt được có hành động không phải lỗi ở trạng thái hiện tại.
4. Báo cáo phân tích bình thường.

Trong hai luật lỗi *error* nêu trên, chúng ta có quan tâm theo dõi ký hiệu lỗi với từ tổ đồng bộ thích hợp – trong trường hợp này, là dấu đóng ngoặc hoặc dấu chấm phẩy. Như vậy, "hành động không lỗi" được lấy ở bước 3 sẽ luôn là đẩy *shift*. Nếu thay vào đó chúng ta dùng luật $exp \rightarrow error$, "hành động không lỗi" có thể được rút gọn *reduce*, và (trong phân tích SLR hoặc LALR) có thể là ký hiệu xem trước gốc (có lỗi) có thể gây ra lỗi khác sau hành động rút gọn, mà không tăng tiếp được đầu vào. Bởi vậy, các luật văn phạm mà chứa *error* không được nối tiếp bởi từ tổ cần được sử dụng khi không có lựa chọn tốt nào.

Nhắc Ta có thể đính kèm hành động ngữ nghĩa vào các luật của Yacc; khi các luật được **nhỏ:** rút gọn, hành động ngữ nghĩa được thực hiện. Chương 6 sẽ giải thích việc sử dụng

các hành động ngữ nghĩa. Các trạng thái dỡ ra từ ngăn xếp có thể dẫn đến hành động ngữ nghĩa tương chừng như “không thể”, đặc biệt nếu hành động có tác dụng phụ. Xét đoạn chương trình sau:

```
statements: statements exp SEMICOLON
           | statements error SEMICOLON
           | /* empty */

exp : increment exp decrement
    | ID

increment: LPAREN      {: nest=nest+1; :}
decrement: RPAREN      {: nest=nest-1; :}
```

"Hiển nhiên" là khi nào dấu chấm phẩy đạt được, thì giá trị của *nest* bằng không, vì nó tăng và giảm một cách cân bằng tùy thuộc vào văn phạm của các biểu thức. Nhưng nếu lỗi cú pháp được tìm thấy sau khi một số mở ngoặc trái được phân tích, thì các trạng thái được dỡ từ ngăn xếp mà chưa hoàn thành thì sẽ dẫn đến giá trị khác không của *nest*. Giải pháp tốt nhất cho vấn đề này là có các hành động ngữ nghĩa không có tác dụng phụ mà xây dựng các cây cú pháp trừu tượng như mô tả trong chương sau.

Không may, cả JavaCC và SableCC đều không hỗ trợ phương pháp khắc phục lỗi nhờ ký hiệu *error*, và cả kiểu chữa lỗi tổng thể mô tả dưới đây.

CHỮA LỖI TỔNG THỂ

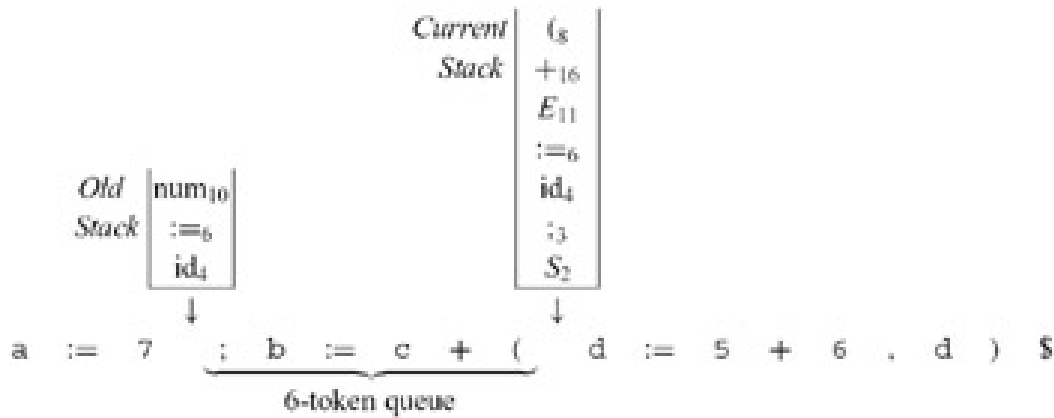
Chữa lỗi tổng thể tìm tập nhỏ nhất việc chèn và xóa mà có thể biến đổi cấu trúc thành cấu trúc cú pháp, ngay cả nếu việc chèn và xóa không xảy ra tại điểm mà ở đó phân tích LL hoặc LR lần đầu thông báo lỗi.

Chữa lỗi Burke-Fisher. Chúng ta sẽ mô tả dạng hạn chế nhưng có ích chữa lỗi tổng thể, mà tìm cách chèn, xóa hoặc thay thế từ tổ có thể duy nhất tại mỗi điểm mà xảy ra không sớm hơn *K* từ tổ trước điểm mà bộ phân tích báo lỗi. Như vậy, với *K* = 15, nếu cơ chế phân tích có trục trặc ở tại từ tổ thứ 100 của đầu vào, thì nó sẽ thử chữa mọi cách có thể giữa các từ tổ thứ 85 đến 100.

Điều chỉnh mà cho phép bộ phân tích tiếp tục phân tích phần vừa qua chỗ lỗi trở đi là chữa lỗi tốt nhất. Như vậy, nếu thay thế từ tổ kiểu `var` cho kiểu từ tổ thứ 98 cho phép cơ chế phân tích xử lý qua từ tổ thứ 104 mà không có trục trặc, sửa lỗi này là thành công. Nói chung, nếu sửa lỗi xử lý tiếp *R* = 4 từ tổ từ chỗ nó có trục trặc, đã là "đủ tốt".

Ưu thế của kỹ thuật này là các văn phạm *LL(k)* hoặc *LR(k)* (hoặc *LALR*, v.v...) không cần điều chỉnh gì cả (*không có luật sinh lỗi*), và *bảng phân tích cú pháp cũng không cần thay đổi*. Chỉ cơ chế phân tích, mà dịch bảng phân tích, là điều chỉnh.

Cơ chế phân tích cần phải có khả năng quay lại K từ tổ và phân tích lại. Để làm được điều đó, nó cần phải nhớ ngăn xếp phân tích trông như thế nào với K từ tổ trước đó. Như vậy, thuật toán duy trì hai ngăn xếp phân tích: ngăn xếp hiện tại và ngăn xếp cũ. Hàng đợi K từ tổ cũng được giữ; như mỗi từ tổ mới được dịch chuyển, nó sẽ đẩy vào ngăn xếp hiện tại và cũng đặt nó vào đuôi hàng đợi; đồng thời, đầu hàng đợi được xóa bỏ và dịch chuyển ngăn xếp cũ. Với mỗi dịch chuyển trên ngăn xếp cũ, các hành động rút gọn thích hợp cũng được thực hiện. Hình 5.29 mô tả hai ngăn xếp và hàng đợi đó.



Hình 5.29: Phân tích Burke-Fisher, với hàng đợi chứa lỗi. Hình 5.7 nêu phân tích hoàn chỉnh của xâu này dựa trên Bảng 5.8.

Bây giờ giả sử lỗi cú pháp được phát hiện tại từ tổ *hiện tại*. Với mỗi phép chèn, xóa hoặc thay thế có thể của từ tổ tại vị trí bất kỳ trong hàng đợi, sửa lỗi Burke-Fisher sẽ thay đổi bên trong (bản sao) của hàng đợi, sau đó thử phân tích lại từ ngăn xếp *cũ*. Thành công của phép sửa sẽ là có bao nhiêu từ tổ sau từ tổ hiện tại có thể phân tích được; nói chung, nếu có ba hoặc bốn từ tổ mới phân tích được, thì nó được xem là sửa thành công hoàn toàn.

Trong ngôn ngữ với N kiểu từ tổ, ở đây có $K + K \cdot N + K \cdot N$ phép chèn, xóa, và thay thế có thể trong cửa sổ K – từ tổ. Thử với nhiều phép sửa nhưng không trả giá nhiều, đặc biệt xét để ý rằng nó chỉ xảy ra khi lỗi cú pháp được phát hiện, chứ không phải phân tích thông thường.

Các hành động ngữ nghĩa đẩy Shift và rút gọn reduce được thử lặp đi lặp lại và bỏ qua trong khi tìm kiếm sửa lỗi tốt nhất. Các bộ sinh Phân tích thường thực hiện các hành động ngữ nghĩa của người lập trình dọc theo mỗi hành động rút gọn, nhưng lập trình viên không mong muốn các hành động này được thực hiện lặp đi lặp lại và bỏ qua – chúng có thể có tác động phụ. Như vậy, phân tích Burke-Fisher không thực hiện bất cứ hành động ngữ nghĩa nào khi rút gọn được thực hiện trên ngăn xếp hiện tại, nhưng chờ cho đến khi cũng phép rút gọn đó được thực hiện (ổn định) trên ngăn xếp cũ.

Điều đó có nghĩa là phân tích từ vựng có thể lên đến $K + R$ từ tổ trước điểm mà hành động ngữ nghĩa thực hiện đến đó. Nếu các hành động ngữ nghĩa tác động lên phân tích từ vựng – như chúng thực hiện trong C, khi dịch đặc tính `typedef` – nó có thể là vấn đề với

cách tiếp cận Burke-Fisher. Đối với các ngôn ngữ có cách tiếp cận cú pháp là văn phạm phi ngữ cảnh thuần túy, làm trễ các hành động ngữ nghĩa không gây ra vấn đề gì.

Các giá trị ngữ nghĩa cho việc chèn. Trong khi sửa lỗi bằng việc chèn, phân tích cần phải cung cấp giá trị ngữ nghĩa cho mỗi từ tố chèn, sao cho các hành động ngữ nghĩa có thể được thực hiện khi từ tố đến từ phân tích từ vựng. Đối với các từ tố phân cách không giá trị nào cần thiết, nhưng khi các từ tố như các số hoặc các định danh được chèn, thì giá trị được lấy từ đâu? Bộ sinh phân tích ML-Yacc, mà dùng chỉnh lỗi Burke-Fischer, có chỉ định `%value`, cho phép lập trình viên đặc tả giá trị nào cần được sử dụng khi chèn mỗi kiểu từ tố:

```
%value ID ("bogus")
%value INT (1)
%value STRING ("")
```

Phép thế đặc tả bởi lập trình viên. Một số kiểu lỗi chung không thể sửa được bằng việc chèn hoặc xóa từ tố đơn, và đôi khi phép chèn hoặc thay thế từ tố đơn được yêu cầu và cần được thử trước. Như vậy, trong đặc tả văn phạm ML-Yacc lập trình viên có thể sử dụng chỉ định `%change` để đề xuất điều chỉnh lỗi cần được thử trước, trước khi sử dụng mặc định "xóa hoặc chèn mỗi từ tố có thể".

```
%change      EQ -> ASSIGN | ASSIGN -> EQ
              | SEMICOLON ELSE -> ELSE | -> IN INT END
```

Ở đây lập trình viên đưa ra giả thiết là các người sử dụng thường viết `"; else"` thay vì viết đúng là `"else"`. Chỉnh lỗi cụ thể này được dùng trong phân tích ngôn ngữ lập trình ML.

Việc chèn `in 0 end` là kiểu sửa lỗi cụ thể được biết như gần hơn phạm vi (*scope closer*). Các chương trình nói chung có thừa các đóng ngoặc trái hoặc phải, hoặc thừa ngoặc móc trái hoặc phải. Trong ML, một kiểu khác của cấu trúc lồng nhau là `let ... in ... end`. Nếu lập trình viên quên đóng phạm vi mà được mở bằng ngoặc trái, thì việc chèn từ tố đơn tự động theo cảm tính như đóng các phạm vi nếu cần. Nhưng để đóng phạm vi của `let` cần chèn ba từ tố, mà không được thực hiện tự động trừ khi người viết chương trình đề xuất "không thay đổi gì cho `in 0 end`" như mô tả trong lệnh `%change` ở trên.

PHÂN TÍCH CÚ PHÁP CHƯƠNG TRÌNH

Sử dụng JavaCC hoặc SableCC để cài đặt bộ phân tích cú pháp cho ngôn ngữ MiniJava. Thực hiện điều đó bằng việc mở rộng đặc tả từ bài tập tương ứng trong chương trước và phụ lục cuối bài giảng mô tả cú pháp của MiniJava.

ĐỌC THÊM

Conway [1963] mô tả phân tích dự báo (đệ qui lùi), với khái niệm tập FIRST và rút thừa số chung trái. Lý thuyết phân tích LL(k) được hình thức hóa bởi Lewis và Stearns [1968].

Phân tích LR(k) được phát triển bởi Knuth [1965]; các kỹ thuật SLR và LALR – bởi DeRemer [1971]; phân tích LALR(1) được phổ cập bởi sự phát triển và phân bố Yacc [Johnson 1975] (mà không là bộ sinh phân tích cú pháp đầu tiên, hoặc "chương trình dịch – chương trình dịch" ("compiler-compiler")).

Hình 5.19 tóm tắt nhiều định lý trên quan hệ tập con giữa các lớp văn phạm. Heilbrunner [1981] chỉ ra chứng minh một số định lý đó, bao gồm $LL(k) \subset LR(k)$ và $LL(1) \subsetneq LALR(1)$ (xem Bài tập 3.14). Backhouse [1979] là đề xuất tốt cho các khía cạnh lý thuyết của phân tích LL và LR.

Aho [1975] chỉ ra rằng các cơ chế phân tích LL hoặc LR tất định có thể kiểm soát các văn phạm nhập nhằng, với việc nhập nhằng được phân giải bằng chỉ dẫn độ ưu tiên (như được mô tả trong 5.4).

Burke và Fisher [1987] phát kiến chiến thuật chữa lỗi mà giữ hàng đợi K từ tổ và hai ngăn xếp phân tích.

BÀI TẬP

5.1 Viết văn phạm mà chấp nhận cùng một ngôn ngữ với Văn phạm 4.1, nhưng phù hợp cho Phân tích LL(1). Vậy cần phải khử nhập nhằng, khử đệ qui trái và (nếu cần thiết) rút thừa số chung trái.

5.2 Tìm các tập có thể nullable, FIRST và FOLLOW của Văn phạm sau; sau đó xây dựng Bảng phân tích cú pháp LL(1).

1. $S' \rightarrow S \$$
2. $S \rightarrow$
3. $S \rightarrow XS$
4. $B \rightarrow \backslash \text{ begin } \{ \text{ WORD } \}$
5. $E \rightarrow \backslash \text{ end } \{ \text{ WORD } \}$
6. $X \rightarrow BSE$
7. $X \rightarrow \{ S \}$
8. $X \rightarrow \text{ WORD }$
9. $X \rightarrow \text{ begin }$
10. $X \rightarrow \text{ end }$
11. $X \rightarrow \backslash \text{ WORD }$

5.3 Thực hiện các yêu cầu sau:

- a. Tính các tập nullable, FIRST và FOLLOW cho văn phạm:

- $S \rightarrow u B D z$
- $B \rightarrow B v$
- $B \rightarrow w$
- $D \rightarrow E F$
- $E \rightarrow y$
- $E \rightarrow$
- $F \rightarrow x$
- $F \rightarrow$

- b. Xây dựng bảng phân tích cú pháp LL(1).
- c. Cho mình chứng là văn phạm này không phải LL(1).
- d. Điều chỉnh Văn phạm **ít nhất có thể** để tạo thành văn phạm LL(1) chấp nhận cùng một ngôn ngữ.

*5.4

- a. Rút thừa số chung trái của văn phạm sau.

1. $S \rightarrow G \$$
2. $G \rightarrow P$
3. $G \rightarrow PG$
4. $P \rightarrow \text{id} : R$
5. $R \rightarrow$
6. $R \rightarrow \text{id} R$

- c. Chỉ ra rằng văn phạm kết quả là LL(2). Bạn có thể làm điều đó bằng cách xây dựng các tập FIRST chứa xâu hai ký hiệu; nhưng là đơn giản hơn việc xây dựng bảng phân tích cú pháp LL(1) và sau đó chứng tỏ mọi xung đột đã được giải quyết bằng cách xem trước một ký tự nhiều hơn.
- d. Chỉ ra biến `tok` và hàm `advance` function cần phải thay đổi để phân tích đệ quy lùi với hai ký hiệu xem trước như thế nào.
- e. Sử dụng phân lớp văn phạm (Hình 5.19) để chỉ ra rằng Văn phạm rút thừa số chung trái đó là LR(2).
- f. Chứng tỏ rằng không xâu nào có hai cây cú pháp theo Văn phạm rút thừa số chung trái đó.

5.5 Tạo văn phạm nhỏ chứa đệ quy trái, và sử dụng nó để chỉ ra rằng đệ quy trái không là vấn đề gì đối với phân tích cú pháp LR. Sau đó chỉ ra ví dụ nhỏ so sánh độ tăng của ngăn xếp phân tích LR có đệ quy trái so với có đệ quy phải.

5.6 Vẽ sơ đồ các trạng thái LR(0) cho Văn phạm 5.16, xây dựng bảng phân tích cú pháp SLR, và chỉ ra các xung đột.

5.7 Vẽ sơ đồ các trạng thái LR(1) cho văn phạm của Bài tập 5.4 (không có thừa số chung trái), và xây dựng bảng phân tích cú pháp LR(1). Chỉ ra mọi xung đột một cách rõ ràng.

5.8 Xây dựng các trạng thái LR(0) cho văn phạm sau, và sau đó xác định xem nó có phải là văn phạm SLR không.

- a. $S \rightarrow B \$$
- b. $B \rightarrow \text{id } P$
- c. $B \rightarrow \text{id } *(E]$
- d. $P \rightarrow$
- e. $P \rightarrow (E)$
- f. $E \rightarrow B$
- g. $E \rightarrow B, E$

5.9

a. Xây dựng Otomat tất định DFA của bảng LR(0) cho văn phạm sau:

- 1. $S \rightarrow E \$$
- 2. $E \rightarrow \text{id}$
- 3. $E \rightarrow \text{id } (E)$
- 4. $E \rightarrow E + \text{id}$

- b. Nó có là văn phạm LR(0) không? Cho minh chứng.
- c. Nó có là văn phạm SLR(không? Cho minh chứng.
- d. Nó có là văn phạm LR(1) không? Cho minh chứng.

5.10 Chỉ ra rằng văn phạm sau là LALR(1) nhưng không phải SLR:

- 1. $S \rightarrow X \$$
- 2. $X \rightarrow Ma$
- 3. $X \rightarrow bMc$
- 4. $X \rightarrow dc$
- 5. $X \rightarrow bda$
- 6. $M \rightarrow d$

5.11 Chỉ ra rằng văn phạm sau là LL(1) nhưng không phải LALR(1):

- 1. $S \rightarrow (X$
- 2. $S \rightarrow E]$
- 3. $S \rightarrow F)$
- 4. $X \rightarrow E)$
- 5. $X \rightarrow F]$
- 6. $E \rightarrow A$
- 7. $F \rightarrow A$
- 8. $A \rightarrow$

***5.12** Cung cấp văn phạm này cho Yacc; từ file mô tả đầu ra, xây dựng bảng phân tích cú pháp LALR(1) cho văn phạm này, với bản nhập kép ở đâu có xung đột.

Đối với mỗi xung đột, chỉ rõ cần phải chọn dịch chuyển hay rút gọn sao cho các kiểu biểu thức khác nhau có độ ưu tiên "đúng như thường lệ". Sau đó nêu rõ các thí thị ưu tiên theo phong cách Yacc để giải quyết mâu thuẫn theo cách đó.

1. $S \rightarrow E \$$
2. $E \rightarrow \text{while } E \text{ do } E$
3. $E \rightarrow \text{id} := E$
4. $E \rightarrow E + E$
5. $E \rightarrow \text{id}$

***5.13** Giải thích giải quyết xung đột như thế nào trong văn phạm sau, sử dụng các chỉ định độ ưu tiên, hoặc biến đổi văn phạm, hoặc cả hai. Sử dụng Yacc hoặc SableCC như công cụ trong nghiên cứu của bạn, nếu bạn muốn.

1. $E \rightarrow \text{id}$
2. $E \rightarrow EBE$
3. $B \rightarrow +$
4. $B \rightarrow -$
5. $B \rightarrow \times$
6. $B \rightarrow /$

***5.14** Chứng tỏ rằng Văn phạm 4.8 không thể sinh ra cây cú pháp dạng nêu trên Hình 4.9. Gợi ý: Các chưa tận nào có thể ở đó? X có được nêu ra không? Cái gì có thể ở đó cho ta biết điều gì? Y có được nêu ra không?

CHƯƠNG 6: PHÂN TÍCH NGŨ NGHĨA

TỔNG QUAN

Chương trình dịch làm nhiều việc hơn là chỉ xem chương trình có thuộc ngôn ngữ sinh bởi văn phạm không – nó cần phải làm điều gì đó có ích cho chương trình. Các hành động ngữ nghĩa của bộ phân tích sẽ làm các điều có ích đó với các câu đã duyệt qua.

Trong phân tích đệ qui lùi, mã của hành động ngữ nghĩa được đặt xen cùng với dòng điều khiển của các hành động phân tích. Trong bộ phân tích với JavaCC, các hành động ngữ nghĩa là các đoạn chương trình Java đính kèm với các phép suy diễn văn phạm. SableCC, thì lại khác, tự động sinh cây cú pháp mà nó phân tích.

6.1 LƯỢC ĐỒ DỊCH

Lược đồ dịch là văn phạm có hành động ngữ nghĩa được đính kèm bên phải luật sinh, tức là dịch biểu thức trung tố sang hậu tố:

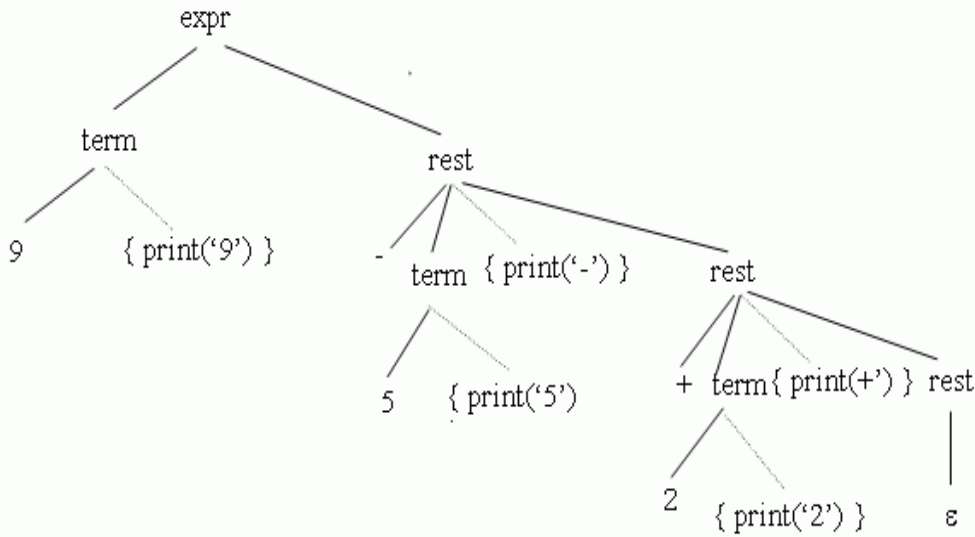
```
expr -> expr + term { print ('+') }
expr -> expr - term { print ('-') }
expr -> term
term -> 0 { print ('0') }
term -> 1 { print ('1') }
...
term -> 9 { print ('9') }

expr -> term rest

rest -> + term { print ('+') } rest |
      - term { print ('-') } rest | e

term -> 0 { print ('0') }
term -> 1 { print ('1') }
...
term -> 9 { print ('9') }
```

Cây cú pháp trong Lược đồ dịch gọi là cây chủ giải. Duyệt cây chủ giải cho biểu thức: $9 - 5 + 2$ với các hành động ngữ nghĩa ta được biểu thức hậu tố: $9\ 5\ -\ 2\ +$



CÚ PHÁP TRỪU TƯỢNG

HÀNH ĐỘNG NGỮ NGHĨA

Một ký tự tận cùng và chưa tận có thể được gắn kết với một kiểu giá trị ngữ nghĩa của nó. Chẳng hạn, trong máy tính bỏ túi sử dụng văn phạm 5.17, kiểu mà gắn kết với `exp` và `INT` có thể là `int`; các từ tổ khác có thể không cần mang giá trị. Kiểu mà gắn kết với một từ tổ cũng cần phải phù hợp với kiểu kết hợp từ các thành phần mà từ tổ này suy diễn.

Đối với qui tắc $A \rightarrow B\ C\ D$, hành động ngữ nghĩa cần phải trả về giá trị mà kiểu của nó được gắn kết với chưa tận A . Nhưng nó cũng cần tạo nên giá trị này từ các giá trị gắn kết với các tận cùng và chưa tận B, C, D .

Trong phân tích đệ qui lùi, các hành động ngữ nghĩa là các giá trị được trả về từ các hàm phân tích, hoặc các hành động phụ của các hàm đó, hoặc cả hai. Với mỗi ký tự tận cùng hoặc chưa tận, chúng ta gắn kết với kiểu (từ ngôn ngữ cài đặt của chương trình dịch) của các giá trị ngữ nghĩa biểu diễn các câu suy diễn từ ký tự đó.

Chương trình 6.1 là thông dịch đệ qui lùi cho một phần của văn phạm 5.4. Các từ tổ `ID` và `NUM` cần mang giá trị kiểu `string` và `int`, tương ứng. Chúng ta sẽ giả thiết rằng Bảng tìm kiếm ánh xạ các định danh vào các số nguyên. Kiểu gắn kết với $E; T; F; \dots$ là `int`, và các hành động ngữ nghĩa khá dễ dàng cài đặt.

CHƯƠNG TRÌNH 6.1: Thông dịch đệ qui lùi một phần văn phạm 5.4.

```

class Token {int kind; Object val;
             Token(int k, Object v) {kind=k; val=v;}
             }
final int EOF=0, ID=1, NUM=2, PLUS=3, MINUS=4, ...

int lookup(String id) { ... }

int F_follow[] = { PLUS, TIMES, RPAREN, EOF };

int F() {switch (tok.kind) {
        case ID:  int i=lookup((String)(tok.val)); advance(); return
i;
        case NUM: int i=((Integer)(tok.val)).intValue();
                advance(); return i;
        case LPAREN: eat(LPAREN);
                int i = E();
                eatOrSkipTo(RPAREN, F_follow);
                return i;
        case EOF:
        default:  print("expected ID, NUM, or left-paren");
                skipto(F_follow); return 0;
        }}

int T_follow[] = { PLUS, RPAREN, EOF };

int T() {switch (tok.kind) {
        case ID:
        case NUM:
        case LPAREN: return Tprime(F());
        default: print("expected ID, NUM, or left-paren");
                skipto(T_follow);
                return 0;
        }}

int Tprime(int a) {switch (tok.kind) {
        case TIMES: eat(TIMES); return Tprime(a*F());
        case PLUS:
        case RPAREN:
        case EOF: return a;
        default: ...
        }}

void eatOrSkipTo(int expected, int[] stop) {
    if (tok.kind==expected)
        eat(expected);
    else {print(...); skipto(stop);}
}

```

Hành động ngữ nghĩa cho ký tự nhân tạo như T' (được đưa vào để khử đệ qui trái) cần phải khéo léo một chút. Vì có phép suy diễn $T \rightarrow T * F$, nên hành động ngữ nghĩa có thể là

```
int a = T(); eat(TIMES); int b=F(); return a*b;
```

Với việc sắp xếp lại văn phạm, phép suy diễn $T' \rightarrow *FT'$ bỏ sót toán hạng trái của $*$. Một giải pháp cho T truyền toán hạng trái như một đối số của T' , được nêu trong chương trình 6.1.

CÁC BỘ PHÂN TÍCH ĐƯỢC SINH TỰ ĐỘNG

Đặc tả bộ phân tích cho JavaCC bao gồm tập các qui tắc văn phạm, mỗi qui tắc được chú dẫn với hành động ngữ nghĩa như trong lệnh Java. Khi bộ phân tích được sinh rút gọn bởi một luật nào đó, nó sẽ thực hiện đoạn hành động ngữ nghĩa tương ứng.

Chương trình 6.2 chỉ ra rằng nó thực hiện cho một phương án của văn phạm 5.4 như thế nào. Mỗi tận cùng `INTEGER_CONSTANT` và mỗi chưa tận (ngoại trừ `Start`) mang một giá trị nào đó. Để truy cập giá trị này, gán tên cho tận cùng và chưa tận trong mỗi văn phạm (như `i` trong chương trình 6.2), và truy cập đến tên này như biến trong hành động ngữ nghĩa.

CHƯƠNG TRÌNH 6.2: Phương án JavaCC cho một biến thể của văn phạm 5.4.

```
void Start() :
{ int i; }
{ i=Exp() <EOF> { System.out.println(i); }
}
int Exp() :
{ int a,i; }
{ a=Term()
  ( "+" i=Term() { a=a+i; }
  | "-" i=Term() { a=a-i; }
  )*
  { return a; }
}
int Term() :
{ int a,i; }
{ a=Factor()
  ( "*" i=Factor() { a=a*i; }
  | "/" i=Factor() { a=a/i; }
  )*
  { return a; }
}
int Factor() :
{ Token t; int i; }
{ t=<IDENTIFIER> { return lookup(t.image); }
| t=<INTEGER_LITERAL> { return Integer.parseInt(t.image); }
| "(" i=Exp() ")" { return i; }
}
```

SableCC, không giống như JavaCC, không dùng việc gán đoạn mã hành động cho các luật sinh. Tuy nhiên, SableCC sẽ tự động sinh ra lớp cây cú pháp, và bộ phân tích được sinh bởi SableCC sẽ xây dựng cây cú pháp sử dụng các lớp này. Đối với JavaCC, có một

số công cụ đi kèm, bao gồm JJTree và JTB (the Java Tree Builder), cũng như SableCC, sinh ra các lớp cây cú pháp và chèn đoạn mã hành động vào trong văn phạm để xây dựng cây cú pháp.

CÁC CÂY CÚ PHÁP TRỪU TƯỢNG

Có thể viết cả chương trình dịch mà phù hợp với các câu hành động ngữ nghĩa của bộ phân tích JavaCC hoặc SableCC. Tuy nhiên, chương trình dịch như vậy rất khó đọc và bảo trì, và cách tiếp này buộc chương trình dịch phân tích chương trình chính xác theo thứ tự mà nó duyệt.

Để hoàn thiện tính dễ lắp ghép, tốt hơn hết là tách các vấn đề cú pháp (như phân tích) ra khỏi các vấn đề ngữ nghĩa (như kiểm tra kiểu và dịch ra mã máy). Một cách làm điều đó là cho bộ phân tích tạo ra cây cú pháp - một cấu trúc dữ liệu mà các giai đoạn sau của chương trình dịch cần duyệt qua. Về mặt kỹ thuật, cây cú pháp sẽ có chính xác một lá cho mỗi từ tổ của đầu vào và mỗi đỉnh bên trong cho mỗi quy tắc văn phạm rút gọn trong quá trình phân tích.

Cây cú pháp như vậy, mà chúng ta sẽ gọi là *cây cú pháp cụ thể*, biểu diễn cú pháp cụ thể của ngôn ngữ nguồn, có thể không thuận tiện để sử dụng trực tiếp. Nhiều từ tổ để phân cách là thừa và không mang thông tin gì – chúng có ích trong xâu đầu vào, nhưng khi cây cú pháp đã xây dựng xong, cấu trúc của cây đã mang thông tin cấu trúc thuận tiện hơn.

Hơn nữa, cấu trúc của cây cú pháp có thể phụ thuộc nhiều vào văn phạm! Các phép biến đổi văn phạm nêu trong chương 3 như rút thừa số chung, khử đệ qui trái, khử nhập nhằng, lôi kéo thêm nhiều ký hiệu chưa tận và các luật sinh vì các mục đích kỹ thuật. Các chi tiết này có thể chỉ dùng cho khâu phân tích, chứ không giúp ích cho phân tích ngữ nghĩa.

Cú pháp trừu tượng tạo giao diện gọn hơn giữa giai đoạn phân tích và các giai đoạn sau của chương trình dịch. Cây cú pháp trừu tượng mang cấu trúc của chương trình nguồn, với mọi vấn đề cú pháp đã được giải quyết mà không cần thông dịch ngữ nghĩa.

Mọi chương trình dịch trước kia không dùng cấu trúc dữ liệu cú pháp trừu tượng vì máy tính khi đó không đủ bộ nhớ để thể hiện toàn bộ cây cú pháp. Máy tính ngày nay không khó khăn giải quyết vấn đề này. Nhiều ngôn ngữ lập trình hiện đại (ML, Modula-3, Java) cho phép tham chiếu các định danh được định nghĩa về sau trong cùng một module; mà sử dụng cây cú pháp trừu tượng làm cho chương trình dịch trở nên dễ dàng hơn đối với chúng. Có thể Pascal và C yêu cầu các khai báo trước nặng nề vì các nhà thiết kế của chúng muốn tránh cho chương trình duyệt lại một lần nữa trên các máy của những năm 1970.

Văn phạm 6.3 cho biết cú pháp trừu tượng của ngôn ngữ biểu thức trong văn phạm 5.4. Văn phạm này hoàn toàn không dùng được để phân tích trên thực tế: nó nhập nhằng, không ưu tiên thứ tự các phép toán.

VĂN PHẠM 6.3: Văn phạm trừu tượng của biểu thức.

- $E \rightarrow E + E$
- $E \rightarrow E - E$
- $E \rightarrow E * E$
- $E \rightarrow E / E$
- $E \rightarrow \text{id}$
- $E \rightarrow \text{num}$

Tuy nhiên, văn phạm 6.3 không dùng để phân tích. Bộ phân tích cụ thể dùng cú pháp cụ thể để xây dựng cây cú pháp cho cú pháp trừu tượng. Giai đoạn phân tích ngữ nghĩa dùng cây cú pháp trừu tượng này, nó sẽ không nhập nhằng, vì nó đã là cây phân tích.

Chương trình dịch sẽ cần để thể hiện và thao tác trên cây cú pháp trừu tượng như một cấu trúc dữ liệu. Trong Java cấu trúc dữ liệu này được tổ chức theo các nguyên tắc nêu ở mục 1.3: mỗi lớp trừu tượng cho mỗi chưa tận, mỗi lớp con cho mỗi luật sinh, và cứ như vậy. Trên thực tế các lớp của chương trình 6.4 là các lớp cú pháp trừu tượng của văn phạm 6.3. Một sự sắp xếp xen kẽ với mọi phép toán hai ngôi khác nhau được nhóm vào `OpExp` class, cũng có thể khả thi.

Giả sử chúng ta viết chương trình dịch cho ngôn ngữ biểu thức trong văn phạm 5.4 bằng việc trước hết xây dựng cây cú pháp và sau đó dịch các cây đó. Chương trình 6.4 là văn phạm JavaCC với các hành động ngữ nghĩa mà tạo ra cây cú pháp. Mỗi lớp của một đỉnh cây cú pháp chứa một hàm eval; khi gọi hàm đó sẽ trả về giá trị của biểu thức thể hiện.

CHƯƠNG TRÌNH 6.4: Xây dựng cây cú pháp cho biểu thức.

```
Exp Start() :
{ Exp e; }
{ e=Exp() { return e; }
}
Exp Exp() :
{ Exp e1,e2; }
{ e1=Term()
  ( "+" e2=Term() { e1=new PlusExp(e1,e2); }
  | "-" e2=Term() { e1=new MinusExp(e1,e2); }
  ) *
  { return e1; }
}
Exp Term() :
{ Exp e1,e2; }
{ e1=Factor()
  ( "*" e2=Factor() { e1=new TimesExp(e1,e2); }
  | "/" e2=Factor() { e1=new DivideExp(e1,e2); }
  ) *
  { return e1; }
}
Exp Factor() :
```



```

{ Token t; Exp e; }
{ ( t=<IDENTIFIER>          { return new Identifier(t.image); } |
  t=<INTEGER_LITERAL> { return new IntegerLiteral(t.image); } |
  "(" e=Exp() ")"        { return e; } )
}

```

CÁC VỊ TRÍ

Trong chương trình duyệt một lần, phân tích từ vựng, cú pháp và phân tích ngữ nghĩa (kiểm tra kiểu) được thực hiện tất cả đồng thời. Nếu có lỗi kiểu thì nó báo ngay cho người sử dụng, vị trí hiện tại của phân tích từ vựng là vị trí gần đúng với vị trí của lỗi. Trong chương trình dịch như vậy, bộ phân tích từ vựng giữ biến tổng thể “vị trí hiện tại”, và chương trình báo lỗi sẽ in ra giá trị của biến này kèm theo thông báo.

Chương trình dịch mà sử dụng cấu trúc dữ liệu cây-cú pháp-trừu tượng không cần làm tất cả phân tích cú pháp và ngữ nghĩa trong cùng một lần. Điều đó làm cho dễ dàng hơn nhiều, tuy nhiên hơi phức tạp hơn trong việc tạo ra thông báo lỗi. Bộ từ vựng đạt đến cuối file trước khi phân tích ngữ nghĩa bắt đầu; như vậy nếu lỗi ngữ nghĩa được phát hiện khi duyệt cây cú pháp trừu tượng, thì vị trí hiện tại của phân tích từ vựng (cuối file) sẽ không có ích sinh ra số dòng báo lỗi. Vì vậy, vị trí của mỗi đỉnh trên cây cú pháp trừu tượng trong file nguồn cần phải được nhớ, trong trường hợp đỉnh này chứa lỗi ngữ nghĩa sẽ dùng đến.

Để nhớ vị trí chính xác, cấu trúc dữ liệu cú pháp trừu tượng cần phải chứa trường pos. Nó chỉ ra vị trí trong file nguồn của các ký tự mà từ đó cấu trúc cú pháp trừu tượng được suy diễn. Khi đó kiểm tra kiểu có thể đưa ra các thông báo lỗi có ích. (Các hàm tạo cú pháp mà chúng ta chỉ ra trong Hình 6.9 không có trường pos, mọi chương trình dịch mà sử dụng chính xác các vị trí đó sẽ cần nhiều thời gian để đưa ra thông báo chính xác vị trí lỗi.)

```
package syntaxtree;
```

```
Program(MainClass m, ClassDeclList cl)
```

```
MainClass(Identifier i1, Identifier i2, Statement s)
```

```
abstract class ClassDecl
```

```
ClassDeclSimple(Identifier i, VarDeclList vl, MethodDeclList ml)
```

```
ClassDeclExtends(Identifier i, Identifier j,
                  VarDeclList vl, MethodDeclList ml)
```

```
VarDecl(Type t, Identifier i)
```

```
MethodDecl(Type t, Identifier i, FormalList fl, VarDeclList vl,
            StatementList sl, Exp e)
```

```
Formal(Type t, Identifier i)
```

```
abstract class Type
```

```
IntArrayType() BooleanType() IntegerType() IdentifierType(String s)
```

```
abstract class Statement
```

```
Block(StatementList sl)
```

```

If(Exp e, Statement s1, Statement s2)
While(Exp e, Statement s)
Print(Exp e)
Assign(Identifier i, Exp e)
ArrayAssign(Identifier i, Exp e1, Exp e2)

abstract class Exp
And(Exp e1, Exp e2)
LessThan(Exp e1, Exp e2)
Plus(Exp e1, Exp e2) Minus(Exp e1, Exp e2) Times(Exp e1, Exp e2)
ArrayLookup(Exp e1, Exp e2)
ArrayLength(Exp e)
Call(Exp e, Identifier i, ExpList e1)
IntegerLiteral(int i)
True()
False()
IdentifierExp(String s)
This()
NewArray(Exp e)
NewObject(Identifier i)
Not(Exp e)

Identifier(String s)
list classes ClassDeclList() ExpList() FormalList() MethodDeclList()
StatementList() VarDeclList()

```

Hình 6.9: Cú pháp trừu tượng cho ngôn ngữ MiniJava.

Bộ phân tích từ vựng truyền vị trí đầu và cuối mỗi từ tố trong file nguồn đến bộ phân tích. Chúng ta có thể bổ sung cho các kiểu Exp, v.v... với trường vị trí; khi đó mỗi hàm tạo cần lấy biến vị trí này để khởi tạo trường đó. Các vị trí của các đỉnh lá của cây cú pháp có thể nhận được từ các từ tố trả về bởi bộ phân tích từ vựng. Các vị trí của các đỉnh bên trong có thể suy ra từ các vị trí cây con của chúng. Điều đó là công kênh nhưng dễ dàng thực hiện.

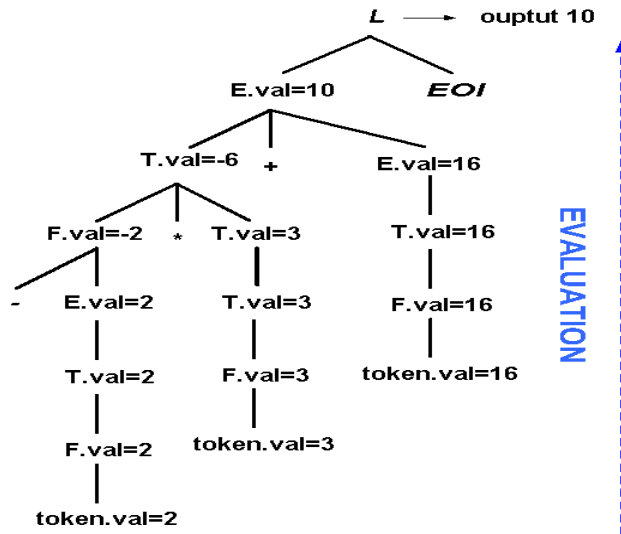
6.2 VĂN PHẠM THUỘC TÍNH TỔNG HỢP

Nếu một thuộc tính ở vế trái được tính qua các thuộc tính ở vế phải thì ta gọi đó là thuộc tính tổng hợp. Và văn phạm có các thuộc tính ở vế trái là tổng hợp, được gọi là văn phạm tổng hợp.

$F \rightarrow \text{number}$	$F.\text{val} = \text{tokenval} /*\text{by scanner}*/$
$F \rightarrow (E)$	$F.\text{val} = E.\text{val}$
$F \rightarrow -E$	$F.\text{val} = -E.\text{val}$
$T \rightarrow F$	$T.\text{val} = F.\text{val}$
$T_1 \rightarrow F * T_2$	$T_1.\text{val} = F.\text{val} * T_2.\text{val}$
$T_1 \rightarrow F / T_2$	$T_1.\text{val} = F.\text{val} / T_2.\text{val}$
$E_1 \rightarrow T$	$E_1.\text{val} = T.\text{val}$
$E_1 \rightarrow T + E_2$	$E_1.\text{val} = T.\text{val} + E_2.\text{val}$

$E \rightarrow T - E$ $E.val = T.val - E.val$
 $L \xrightarrow{1} E \xrightarrow{2} EOI$ $\text{output } E.val$ /* bổ sung tạo luật sinh khởi đầu */

Xây dựng cây chú giải cho biểu thức: $-2 * 3 + 16$



CÁC DU KHÁCH - VISITORS

Mỗi lớp cú pháp trừu tượng trong chương trình 6.5 có hàm tạo để xây dựng cây cú pháp, và phương thức `eval` để trả về giá trị của biểu thức thể hiện. Đây là kiểu lập trình hướng đối tượng. Bây giờ chúng ta xem xét cách khác.

CHƯƠNG TRÌNH 6.5: Lớp Exp cho Chương trình 6.4.

```

public abstract class Exp {
    public abstract int eval();
}

public class PlusExp extends Exp {
    private Exp e1,e2;
    public PlusExp(Exp a1, Exp a2) { e1=a1; e2=a2; }
    public int eval() {
        return e1.eval()+e2.eval();
    }
}

public class MinusExp extends Exp {
    private Exp e1,e2;
    public MinusExp(Exp a1, Exp a2) { e1=a1; e2=a2; }
    public int eval() {
        return e1.eval()-e2.eval();
    }
}

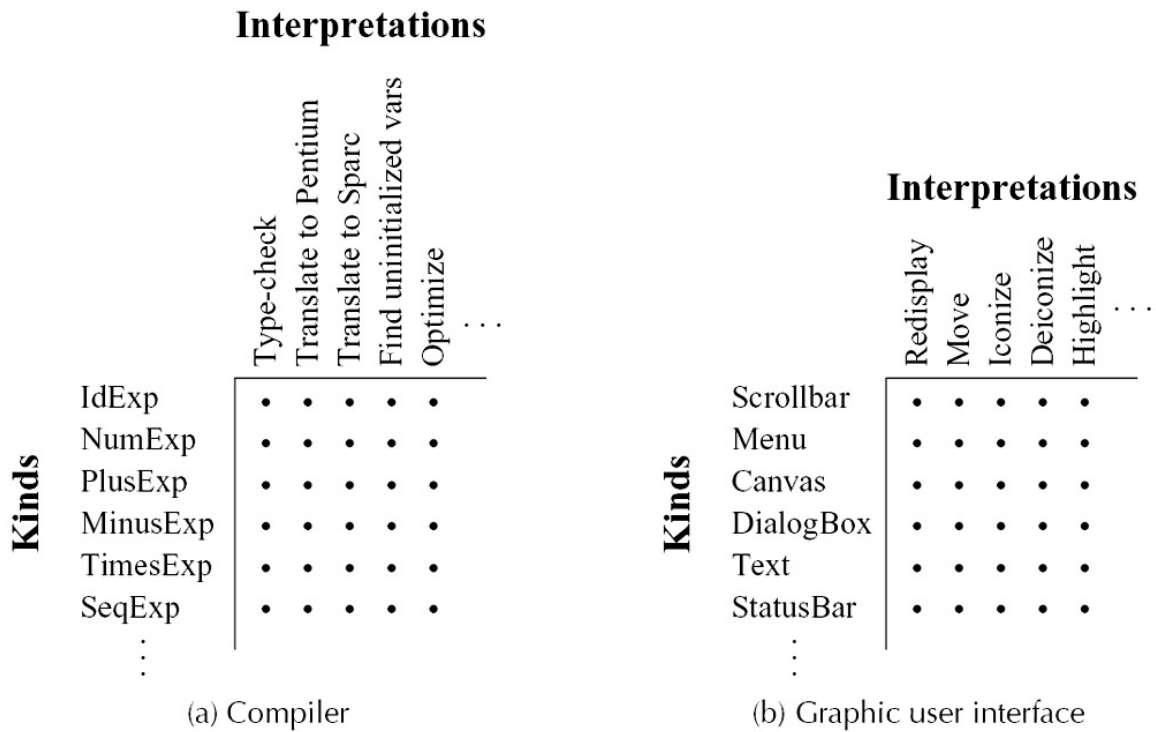
public class TimesExp extends Exp {
    private Exp e1,e2;
    
```

```
    public TimesExp(Exp a1, Exp a2) { e1=a1; e2=a2; }
    public int eval() {
        return e1.eval()*e2.eval();
    }
}
public class DivideExp extends Exp {
    private Exp e1,e2;
    public DivideExp(Exp a1, Exp a2) { e1=a1; e2=a2; }
    public int eval() {
        return e1.eval()/e2.eval();
    }
}
public class Identifier extends Exp {
    private String f0;
    public Identifier(String n0) { f0 = n0; }
    public int eval() {
        return lookup(f0);
    }
}
public class IntegerLiteral extends Exp {
    private String f0;
    public IntegerLiteral(String n0) { f0 = n0; }
    public int eval() {
        return Integer.parseInt(f0);
    }
}
```

Giả sử đoạn mã để tính giá trị các biểu thức được viết riêng ra khỏi các lớp cú pháp trừu tượng. Chúng ta có thể làm điều đó bằng việc kiểm tra cấu trúc dữ liệu cây cú pháp mà sử dụng `instanceof` và bằng việc bổ sung các biến lớp công khai để biểu diễn các cây con. Đây là phong cách lập trình cú pháp tách khỏi dịch.

Việc chọn kiểu lập trình sẽ tác động đến cấu trúc module của chương trình dịch. Giả sử trong tình huống mà chúng ta có một số kiểu đối tượng: lệnh phức, lệnh gán, lệnh in, v.v... Và chúng ta có một số cách thông dịch khác nhau cho các đối tượng này: kiểm tra kiểu, dịch sang mã Pentium, dịch sang mã Sparc, tối ưu, v.v...

Mỗi thông dịch cần áp dụng cho mỗi kiểu; nếu chúng ta bổ sung kiểu mới, thì chúng ta cần phải cài đặt mỗi thông dịch cho nó; Và nếu chúng ta thêm thông dịch mới, thì chúng ta phải cài đặt nó cho mỗi kiểu. Hình 6.6 mô tả tính trực giao của các kiểu và các thông dịch cho chương trình dịch, và cho giao diện đồ họa với người sử dụng, ở đó kiểu là đối tượng đồ họa và thông dịch là các kiểu thể hiện.



Hình 6.6: Các hướng trực giao của tính Module.

Nếu dùng kiểu cú pháp tách với thông dịch, thì dễ dàng thêm module bản dịch mới: một hàm mới với các câu lệnh cho các kiểu khác nhau được nhóm chung vào một modul. Mặt khác, lại khó mà module hóa khi thêm một kiểu mới, vì câu lệnh cho kiểu cần phải bổ sung cho mọi hàm thông dịch.

Với kiểu hướng đối tượng, mỗi thông dịch là một phương thức *method* trong mọi lớp. Điều đó dễ dàng module hóa khi bổ sung kiểu mới: mọi thông dịch của kiểu này được nhóm lại như phương thức của kiểu mới đó. Nhưng lại khó module hóa khi thêm bản dịch mới: Phương thức mới cần được bổ sung cho từng lớp.

Đối với giao diện đồ họa cho người sử dụng, mỗi ứng dụng muốn có kiểu đối tượng đồ họa riêng, rất khó định nghĩa trước tập tất cả các kiểu cho mọi ứng dụng. Mặt khác tập các thao tác chung (các thông dịch) là cố định. Quản trị cửa sổ yêu cầu mỗi khung hỗ trợ giao diện nào đó. Như vậy, kiểu lập trình hướng đối tượng là phù hợp, và kiểu lập trình cú pháp tách thông dịch khó module hóa được.

Đối với các ngôn ngữ lập trình, mặt khác sẽ tốt hơn nếu cố định cú pháp và cung cấp nhiều thông dịch cho cú pháp đó. Nếu chúng ta có chương trình dịch mà ở đó có thông dịch là dịch sang *Pentium* và chúng ta muốn chương trình dịch hỗ trợ dịch sang Sparc, thì chúng ta không chỉ cần phải bổ sung các thao tác sinh mã Sparc mà còn muốn loại bỏ các hàm sinh mã Pentium. Điều này rất không thuận tiện trong kiểu lập trình hướng đối tượng, đòi hỏi mỗi lớp cần phải được sửa. Trong kiểu lập trình cú pháp tách thông dịch, thay đổi đó có thể được module hoá: Chúng ta bỏ module dùng cho Pentium và bổ sung module dùng cho Sparc.

Như vậy chúng ta sẽ dùng kiểu tách cú pháp khỏi thông dịch. May mắn, chúng ta có thể sử dụng kiểu này mà không cần khai thác biểu thức `instanceof` cho việc truy cập cây cú pháp. Thay vào đó chúng ta có thể dùng kỹ thuật đã biết là mẫu Du khách (Visitor pattern). Mỗi Visitor cài đặt một thông dịch; đây là đối tượng mà chứa phương thức `visit` method cho mỗi lớp cây cú pháp (syntax-tree class). Mỗi lớp cây cú pháp cần chứa phương thức `accept`. Phương thức `accept` dùng làm chỗ chứa mọi thông dịch. Nó được gọi bởi visitor và nó có một nhiệm vụ: nó truyền điều khiển ngược lại cho phương thức tương ứng của visitor. Như vậy, điều khiển truyền tới và lui giữa visitor và các lớp cây cú pháp.

Trực quan, visitor gọi phương thức `accept` của đỉnh và hỏi "lớp của anh là gì?" Và phương thức `accept` trả lời bằng cách gọi phương thức `visit` tương ứng của visitor đó. Mã để chạy ví dụ, sử dụng visitors, được cho trong Chương trình 6.7 và 6.8. Mỗi visitor cài đặt một interface `Visitor`. Lưu ý rằng mỗi phương thức `accept` lấy một visitor như đối số, và mỗi một phương thức `visit` lấy một đối tượng đỉnh cây cú pháp làm đối số.

CHƯƠNG TRÌNH 6.7: Các lớp Syntax với các phương thức `accept`.

```
public abstract class Exp {
    public abstract int accept(Visitor v);
}
public class PlusExp extends Exp {
    public Exp e1,e2;
    public PlusExp(Exp a1, Exp a2) { e1=a1; e2=a2; }
    public int accept(Visitor v) {
        return v.visit(this);
    }
}
public class MinusExp extends Exp {
    public Exp e1,e2;
    public MinusExp(Exp a1, Exp a2) { e1=a1; e2=a2; }
    public int accept(Visitor v) {
        return v.visit(this);
    }
}
public class TimesExp extends Exp {
    public Exp e1,e2;
    public TimesExp(Exp a1, Exp a2) { e1=a1; e2=a2; }
    public int accept(Visitor v) {
        return v.visit(this);
    }
}
public class DivideExp extends Exp {
    public Exp e1,e2;
    public DivideExp(Exp a1, Exp a2) { e1=a1; e2=a2; }
    public int accept(Visitor v) {
        return v.visit(this);
    }
}
public class Identifier extends Exp {
    public String f0;
    public Identifier(String n0) { f0 = n0; }
    public int accept(Visitor v) {
```

```
        return v.visit(this);
    }
}
public class IntegerLiteral extends Exp {
    public String f0;
    public IntegerLiteral(String n0) { f0 = n0; }
    public int accept() {
        return v.visit(this);
    }
}
```

CHƯƠNG TRÌNH 6.8: Khách du lịch – thông dịch (An interpreter visitor).

```
public interface Visitor {
    public int visit(PlusExp n);
    public int visit(MinusExp n);
    public int visit(TimesExp n);
    public int visit(DivideExp n);
    public int visit(Identifier n);
    public int visit(IntegerLiteral n);
}
public class Interpreter implements Visitor {
    public int visit(PlusExp n) {
        return n.e1.accept(this)+n.e2.accept(this);
    }
    public int visit(MinusExp n) {
        return n.e1.accept(this)-n.e2.accept(this);
    }
    public int visit(TimesExp n) {
        return n.e1.accept(this)*n.e2.accept(this);
    }
    public int visit(DivideExp n) {
        return n.e1.accept(this)/n.e2.accept(this);
    }
    public int visit(Identifier n) {
        return lookup(n.f0);
    }
    public int visit(IntegerLiteral n) {
        return Integer.parseInt(n.f0);
    }
}
```

Trong Chương trình 6.7 và 6.8 các phương thức `visit` và `accept` luôn trả về `int`. Giả sử chúng ta muốn thay vào đó là trả về `String`. Trong trường hợp này, chúng ta có thể bổ sung phương thức `accept` tương ứng cho mỗi lớp cây cú pháp, và viết lớp visitor mới trong đó phương thức `visit` trả về `String`.

Sự khác biệt chính giữa kiểu hướng đối tượng và kiểu cú pháp tách thông dịch là ở chỗ, chẳng hạn đoạn mã thông dịch (interpreter code) trong Chương trình 6.5 là ở phương thức `eval` còn trong Chương trình 6.8 là ở `Interpreter visitor`.

Tóm lại, với Mẫu khách du lịch (Visitor pattern) chúng ta có thể bổ sung thông dịch mới mà không cần sửa và dịch lại các lớp đã có, đảm bảo rằng mỗi lớp tương ứng có một phương thức `accept`. Bảng sau tổng kết các ưu điểm của Visitor pattern:

Thiên hướng đổi kiểu? Thường xuyên dịch lại?

Instanceof and type casts	Yes	No
Dedicated methods	No	Yes
The Visitor pattern	No	No

CÚ PHÁP TRỪU TƯỢNG CHO MiniJava

Hình 6.9 nêu các lớp cho cú pháp trừu tượng của MiniJava. Ngữ nghĩa của mỗi hàm tạo trong cú pháp trừu tượng có thể trở nên rõ ràng hơn sau khi nghiên cứu kỹ phụ lục A, nhưng ở đây cũng có một số điểm có thể giải thích được.

Chỉ có các hàm tạo được chỉ ra trên Hình 6.9; các biến trường của đối tượng tương ứng chính xác với các tên của các đối số hàm tạo. Mỗi một trong sáu lớp *list* được cài đặt giống nhau, chẳng hạn:

```
public class ExpList {  
  
    private Vector list;  
    public ExpList() {  
        list = new Vector();  
    }  
    public void addElement(Exp n) {  
        list.addElement(n);  
    }  
    public Exp elementAt(int i) {  
        return (Exp)list.elementAt(i);  
    }  
    public int size() {  
        return list.size();  
    }  
}
```

Mỗi một lớp không phải là danh sách (nonlist) có phương thức `accept` để sử dụng với visitor pattern. Giao diện interface `Visitor` được nêu trong Chương trình 6.10.

CHƯƠNG TRÌNH 6.10: MiniJava visitor

```
public interface Visitor {  
    public void visit(Program n);  
    public void visit(MainClass n);  
    public void visit(ClassDeclSimple n);  
    public void visit(ClassDeclExtends n);  
}
```



```
public void visit(VarDecl n);
public void visit(MethodDecl n);
public void visit(Formal n);
public void visit(IntArrayType n);
public void visit(BooleanType n);
public void visit(IntegerType n);
public void visit(IdentifierType n);
public void visit(Block n);
public void visit(If n);
public void visit(While n);
public void visit(Print n);
public void visit(Assign n);
public void visit(ArrayAssign n);
public void visit(And n);
public void visit(LessThan n);
public void visit(Plus n);
public void visit(Minus n);
public void visit(Times n);
public void visit(ArrayLookup n);
public void visit(ArrayLength n);
public void visit(Call n);
public void visit(IntegerLiteral n);
public void visit(True n);
public void visit(False n);
public void visit(IdentifierExp n);
public void visit(This n);
public void visit(NewArray n);
public void visit(NewObject n);
public void visit(Not n);
public void visit(Identifier n);
}
```

Chúng ta có thể xây dựng cây cú pháp bằng cách sử dụng *new expressions* lồng nhau. Chẳng hạn, chúng ta sẽ xây dựng cây cú pháp cho lệnh MiniJava:

```
x = y.m(1,4+5);
```

sử dụng đoạn mã Java sau:

```
ExpList el = new ExpList();
el.addElement(new IntegerLiteral(1));
el.addElement(new Plus(new IntegerLiteral(4),
    new IntegerLiteral(5)));
Statement s = new Assign(new Identifier("x"),
    new Call(new IdentifierExp("y"),
        new Identifier("m"),
        el));
```

SableCC sẽ cho tự động sinh đoạn mã cho các lớp cây cú pháp, mã để xây dựng cây cú pháp, và mã cho các template visitors. Đối với JavaCC, các công cụ đi kèm gọi là Java Tree Builder (JTB) cho phép sinh ra mã tương tự. Ưu điểm sử dụng các công cụ này là một khi văn phạm được viết ra, người ta có thể viết các *visitors* mà thao tác trên các cây

cú pháp. Nhược điểm là các cây cú pháp được hỗ trợ bởi các mã được sinh ra có thể kém trừu tượng hơn so với thiết kế.

CÚ PHÁP TRỪU TƯỢNG CHƯƠNG TRÌNH

Bổ sung hành động ngữ nghĩa cho bộ phân tích để tạo ra cú pháp trừu tượng cho ngôn ngữ MiniJava. Các lớp cây cú pháp có ở `$MINIJAVA/chap4`, cùng với `PrettyPrintVisitor`. Nếu bạn sử dụng JavaCC, bạn có thể dùng JTB để sinh tự động các đoạn mã cần thiết. Tương tự, với SableCC, các mã cần thiết sẽ được tự động sinh.

ĐỌC THÊM

Nhiều chương trình dịch trộn mã Phân tích đệ quy lùi với mã hành động ngữ nghĩa, như chỉ ra trong Chương trình 6.1; Gries [1971], Fraser và Hanson [1995] là người tiên phong và những người đương thời có đóng góp. Các bộ phân tích máy sinh với các hành động ngữ nghĩa (trong những “ngôn ngữ nhỏ hành động ngữ nghĩa” chuyên dụng) được định với các luật sinh văn phạm đã được thử nghiệm trong những năm 1960 [Feldman và Gries 1968]; Yacc [Johnson 1975] là những người đầu tiên cho phép các đoạn hành động ngữ nghĩa được viết trong ngôn ngữ đa năng thông thường.

Khái niệm *cú pháp trừu tượng* thuộc về McCarthy [1963], người đã thiết kế cú pháp trừu tượng cho *Lisp* [McCarthy 1962]. Cú pháp trừu tượng được đề xuất để sử dụng viết chương trình cho đến khi các nhà thiết kế cần phải tránh việc tạo cú pháp cụ thể với các dấu phân cách người có thể đọc được, nhưng các lập trình viên nhanh chóng sử dụng để lập trình trực tiếp trong cú pháp trừu tượng.

Việc nghiên cứu lý thuyết ngữ nghĩa cho ngôn ngữ lập trình và khái niệm biểu diễn ngữ nghĩa trong chương trình dịch – chương trình dịch, dẫn đến ý tưởng như ngữ nghĩa ký hiệu (*denotational semantics*) [Stoy 1977]. Thông dịch ngữ nghĩa được chỉ ra trong Chương trình 6.4 và 6.5 được truyền cảm hứng từ ngữ nghĩa ký hiệu, là ý tưởng về việc tách cú pháp cụ thể khỏi ngữ nghĩa sử dụng cú pháp trừu tượng như một giao diện thuần túy.

6.3 VĂN PHẠM THUỘC TÍNH KẾ THỪA

Khi các thuộc tính được tính từ trái qua phải, thì văn phạm được gọi là thuộc tính kế thừa:

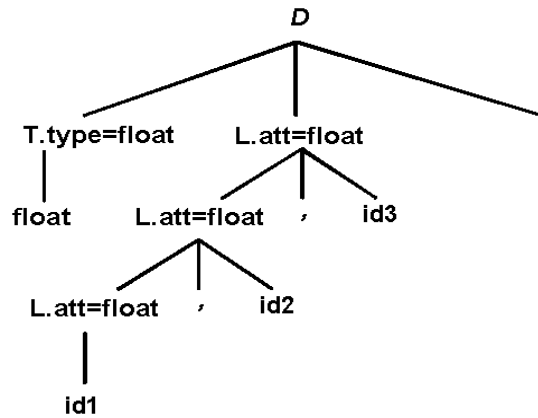
$D \rightarrow T L ;$	$L.att = T.type$
$T \rightarrow \text{int}$	$T.type = \text{integer}$
$T \rightarrow \text{float}$	$T.type = \text{float}$
$L1 \rightarrow L2 , id$	$inserttype(L.att, id)$
	$L2.att = L1.att$
$L \rightarrow id$	$inserttype(L.att, id)$

inserttype() = insert type into symbol table

Ở đây sử dụng Bảng ký hiệu để lưu trữ và tính các thuộc tính.

Cây chú giải cho khai báo biến:

float id1, id2, id3;



PHÂN TÍCH NGŨ NGHĨA

TỔNG QUAN

se-man-tic: ngữ nghĩa hoặc liên quan đến ý nghĩa trong ngôn ngữ

Giai đoạn Phân tích ngữ nghĩa của Chương trình dịch kết nối các định nghĩa biến để sử dụng chúng, kiểm tra xem mỗi biểu thức có đúng kiểu không, và dịch cú pháp trừu tượng sang thể hiện đơn giản hơn phù hợp để sinh mã máy.

BẢNG KÝ HIỆU

Giai đoạn này được đặc trưng bởi việc duy trì Bảng ký hiệu (hay còn gọi là môi trường) ánh xạ các định danh vào các kiểu và vị trí của chúng. Như khai báo các kiểu, các biến, và các hàm được xử lý, các định danh này được gắn với ngữ nghĩa của chúng trong Bảng ký hiệu. Khi sử dụng (xuất hiện khác với định nghĩa) các định danh được nhận diện, chúng được tìm trong Bảng ký hiệu.

Mỗi biến cục bộ trong chương trình có phạm vi mà ở đó nó nhìn thấy. Chẳng hạn, trong MiniJava phương thức m , mọi tham số hình thức và các biến cục bộ được khai báo trong m được nhìn thấy (visible) chỉ đến cuối m . Như vậy khi phân tích ngữ nghĩa đạt đến cuối mỗi phạm vi, sự gắn kết giữa định danh và phạm vi cục bộ của nó sẽ biến mất.

Môi trường là tập các gán kết được ký hiệu bởi mũi tên $\mapsto$. Chẳng hạn, chúng ta có thể nói môi trường σ_0 chứa các gán kết $\{g \mapsto \text{string}, a \mapsto \text{int}\}$, có nghĩa là định danh a được gán kết với biến nguyên và g là biến xâu.

Xét ví dụ đơn giản sau trong ngôn ngữ Java:

```
1 class C {
2     int a; int b; int c;
3     public void m() {
4         System.out.println(a+c);
5         int j = a+b;
6         String a = "hello";
7         System.out.println(a);
8         System.out.println(j);
9         System.out.println(b);
10    }
11 }
```

Giả sử chúng ta dịch lớp này trong môi trường σ_0 . Khai báo trường trên dòng 2 cho chúng ta bảng σ_1 bằng $\sigma_0 + \{a \mapsto \text{int}, b \mapsto \text{int}, c \mapsto \text{int}\}$, như vậy, σ_0 được mở rộng với các gán kết mới cho a , b , và c . Các định danh trên dòng có thể được tìm thấy trong σ_1 . Trên dòng 5, bảng $\sigma_2 = \sigma_1 + \{j \mapsto \text{int}\}$ được tạo ra; và trên dòng 6, $\sigma_3 = \sigma_2 + \{a \mapsto \text{String}\}$ được thiết lập.

Phép toán $+$ đối với bảng được làm việc như thế nào khi hai môi trường được cộng hai gán kết khác nhau cho cùng một ký hiệu? Khi σ_2 và $\{a \mapsto \text{String}\}$ ánh xạ a to int và String , tương ứng? Để làm cho các qui tắc phạm vi làm việc theo đúng cách như chúng ta mong muốn trong ngôn ngữ lập trình, chúng ta muốn $\{a \mapsto \text{String}\}$ được ưu tiên hơn. Khi đó chúng ta nói rằng $X + Y$ đối với bảng không giống như $Y + X$; gán kết của bảng ở vế phải ghi đè gán kết ở vế trái.

Các định danh ở các dòng 7, 8, và 9 có thể tìm trong σ_3 . Cuối cùng, trên dòng 10, chúng ta bỏ qua σ_3 và quay lại σ_1 . Và trên dòng 11 chúng ta bỏ qua σ_1 và quay trở lại σ_0 .

Điều trên cần được cài đặt như thế nào? Thực tế có hai lựa chọn. Trong lập trình hàm, chúng ta yên tâm giữ σ_1 trong điều kiện ban đầu trong khi tạo σ_2 và σ_3 . Và khi chúng ta lại cần σ_1 , nó được lấy lại và sẵn sàng.

Trong kiểu lập trình theo mệnh lệnh, chúng ta cần thay đổi σ_1 cho nó trở thành σ_2 . Kiểu cập nhật phá bỏ này huỷ diệt σ_1 ; khi σ_2 còn tồn tại, chúng ta không thể tìm kiếm trong σ_1 . Nhưng khi chúng ta sử dụng xong với σ_2 , chúng ta có thể khôi phục lại các thay đổi để nhận trở lại σ_1 . Vậy, có môi trường tổng thể duy nhất σ mà trở thành $\sigma_0, \sigma_1, \sigma_2, \sigma_3, \sigma_1, \sigma_0$ ở những thời điểm khác nhau và gỡ bỏ ngăn xếp undo với vừa đủ thông tin để loại bỏ các cập nhật phá bỏ. Khi các ký hiệu được bổ sung vào môi trường, nó được bổ sung vào ngăn xếp undo; ở cuối phạm vi (như tại dòng 10), các ký hiệu được gỡ bỏ từ ngăn xếp undo để loại bỏ các gán kết cuối cùng từ σ (và để khôi phục các gán kết trước đó).

Kiểu lập trình mệnh lệnh hay hàm để quản trị môi trường đều có thể được sử dụng bất chấp ngôn ngữ được dịch hay ngôn ngữ cài đặt chương trình dịch là ngôn ngữ lập trình hàm hay mệnh lệnh hay hướng đối tượng.

NHIỀU BẢNG KÝ HIỆU

Trong một số ngôn ngữ lập trình đồng thời có thể có nhiều môi trường hoạt động: mỗi module, hoặc lớp, hoặc bản ghi trong chương trình có bảng ký hiệu riêng của mình.

Trong phân tích 6.11, giả sử σ_0 là môi trường cơ bản chứa các hàm được định nghĩa trước và giả sử

$$\begin{aligned}\sigma_1 &= \{a \mapsto \text{int}\} \\ \sigma_2 &= \{E \mapsto \sigma_1\} \\ \sigma_3 &= \{b \mapsto \text{int}, a \mapsto \text{int}\} \\ \sigma_4 &= \{N \mapsto \sigma_3\} \\ \sigma_5 &= \{d \mapsto \text{int}\} \\ \sigma_6 &= \{D \mapsto \sigma_5\} \\ \sigma_7 &= \sigma_2 + \sigma_4 + \sigma_6\end{aligned}$$

<pre>structure M = struct structure E = struct val a = 5; end structure N = struct val b = 10 val a = E.a + b end structure D = struct val d = E.a + N.a end end</pre>	<pre>package M; class E { static int a = 5; } class N { static int b = 10; static int a = E.a + b; } class D { static int d = E.a + N.a; }</pre>
--	--

(a) Ví dụ trong ML

(b) Ví dụ trong Java

Hình 6.11: Một số môi trường hoạt động cùng một lúc.

Trong ML, N được dịch sử dụng môi trường $\sigma_0 + \sigma_2$ để tìm các định danh; D được dịch sử dụng $\sigma_0 + \sigma_2 + \sigma_4$, và kết quả của phân tích là $\{M \mapsto \sigma_7\}$.

Trong Java, tham chiếu tới phía trước được cho phép (do đó bên trong N biểu thức $D.d$ sẽ là hợp lệ), vậy E , N , và D tất cả được dịch trong môi trường σ_7 ; đối với chương trình này kết quả vẫn là $\{M \mapsto \sigma_7\}$.

CÁC BẢNG KÝ HIỆU MỆNH LỆNH HIỆU QUẢ

Vì chương trình lớn có thể chứa hàng ngàn định danh khác nhau, các bảng ký hiệu cần phải cho phép tìm kiếm hiệu quả.

Môi trường phong cách mệnh lệnh thường được cài đặt sử dụng bảng băm một cách rất hiệu quả. Thao tác $\sigma' = \sigma + \{a \mapsto \tau\}$ được cài đặt bằng cách chèn τ vào bảng băm với khóa a . Bảng băm đơn giản với các dãy bổ sung ngoài làm việc rất tốt và hỗ trợ xóa dễ dàng (chúng ta sẽ cần phải xóa để khôi phục σ ở cuối tầm nhìn của a).

Chương trình 6.12 cài đặt bảng băm đơn giản. Giỏ thứ i là danh sách móc nối của mọi phần tử mà bản băm khóa của chúng là $i \bmod \text{SIZE}$.

CHƯƠNG TRÌNH 6.12: Bảng băm với móc nối ngoài

```
class Bucket {String key; Object binding; Bucket next;
    Bucket(String k, Object b, Bucket n) {key=k; binding=b;
next=n;}
}

class HashT {
    final int SIZE = 256;
    Bucket table[] = new Bucket[SIZE];

    private int hash(String s) {
        int h=0;
        for(int i=0; i<s.length(); i++)
            h=h*65599+s.charAt(i);
        return h;
    }

    void insert(String s, Binding b) {
        int index=hash(s)%SIZE
        table[index]=new Bucket(s,b,table[index]);
    }

    Object lookup(String s) {
        int index=hash(s)%SIZE
        for (Binding b = table[index]; b!=null; b=b.next)
            if (s.equals(b.key)) return b.binding;
        return null;
    }

    void pop(String s) {
        int index=hash(s)%SIZE
        table[index]=table[index].next;
    }
}
```

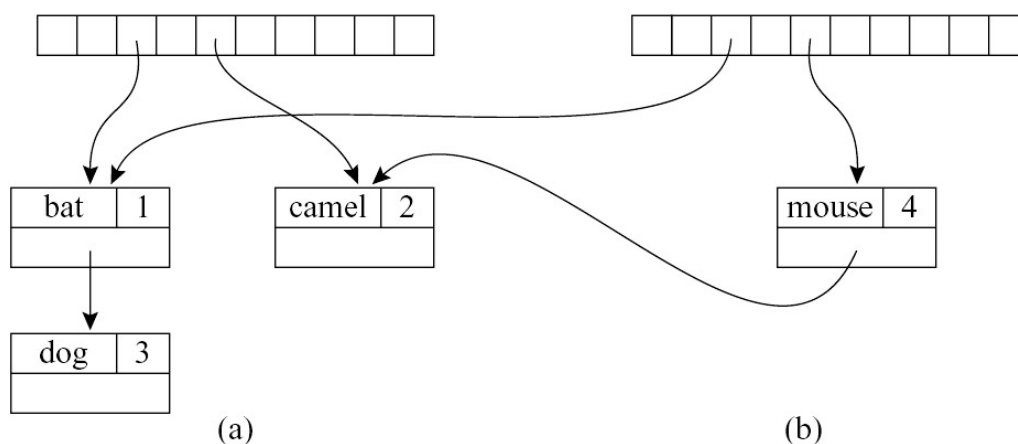
Xét $\sigma + \{a \mapsto \tau_2\}$ trong đó σ đã chứa $a \mapsto \tau_1$. Hàm `insert` để nguyên $a \mapsto \tau_1$ trong bucket và đặt $a \mapsto \tau_2$ trước đó trong list. Sau đó, khi `pop(a)` được thực hiện xong ở cuối phạm vi của a , σ được khôi phục lại. Tất nhiên, `pop` làm việc chỉ khi liên kết được chèn và gỡ ra theo kiểu ngăn xếp.

Cài đặt có sức mạnh công nghiệp có thể được cải tiến về điểm này theo một số cách; xem trong Bài tập 6.7.

BẢNG KÝ HIỆU DẠNG HÀM HIỆU QUẢ

Theo kiểu hàm, chúng ta muốn tính $\sigma' = \sigma + \{a \mapsto \tau\}$ theo cách mà chúng ta vẫn có σ được dùng để tìm kiếm các định danh. Như vậy, thay vì sửa bảng bằng cách bổ sung gắn kết mới, chúng ta sẽ tạo bảng mới bằng “tổng” bảng cũ với gắn kết mới đó. Tương tự, khi cộng $7 + 8$, ta không sửa 7 bằng cách cộng 8 vào nó; ta tạo giá trị mới, 15 và 7 vẫn được dùng cho tính toán khác.

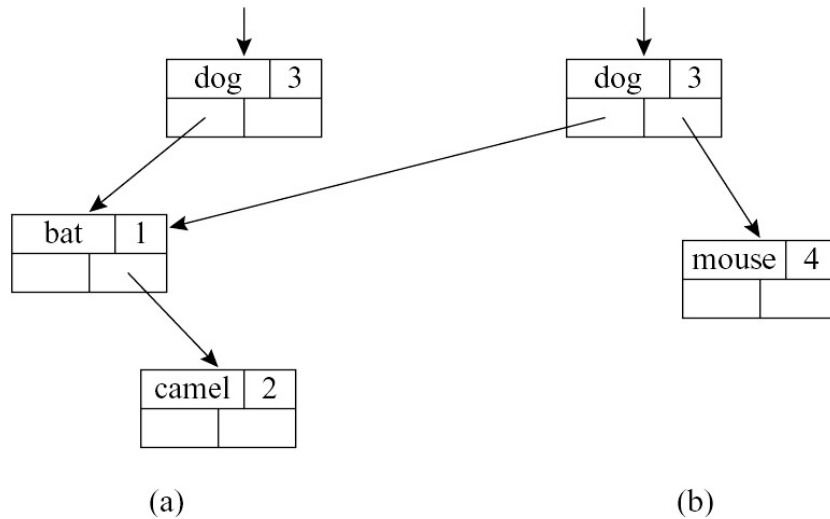
Tuy nhiên, việc cập nhật không phá hủy lại tỏ ra không hiệu quả cho các bảng băm. Hình 6.13a cho biết bảng băm cài đặt ánh xạ m_1 . Nó nhanh và hiệu quả nếu bổ sung *mouse* vào khe thứ năm; chỉ việc cho bản ghi *mouse* trở đến bảng (cũ) tại danh sách liên kết thứ năm, và làm cho khe thứ năm trở đến bản ghi *mouse*. Nhưng khi đó chúng ta sẽ không còn ánh xạ m_1 : chúng ta đã huỷ nó để tạo nên m_2 . Một khả năng khác là sao mảng, nhưng vẫn chia sẻ mọi giỏ cũ, như trên Hình 6.13b. Nhưng nó không hiệu quả: Mảng trong bảng băm có thể rất lớn, tỷ lệ với số phần tử, và chúng ta không đủ sức sao nó cho mỗi lần thêm mới vào bảng.



Hình 6.13: Các bảng băm.

Bằng cách sử dụng các cây tìm kiếm nhị phân chúng ta có thể thực hiện các bổ sung hàm vào cây tìm kiếm một cách hiệu quả. Xét, chẳng hạn, cây tìm kiếm trên Hình 6.14, mà thể hiện ánh xạ

$$m_1 = \{bat \mapsto 1, camel \mapsto 2, dog \mapsto 3\}.$$



Hình 6.14: Các cây tìm kiếm nhị phân.

Chúng ta có thể bổ sung $mouse \mapsto 4$, tạo nên ánh xạ m_2 mà không cần hủy ánh xạ m_1 , như chỉ ra trên Hình 6.14b. Nếu chúng ta bổ sung đỉnh mới tại độ sâu d của cây, chúng ta cần tạo d đỉnh mới – nhưng không cần sao toàn bộ cây. Như vậy tạo cây mới (mà chia sẻ cấu trúc với cây cũ) có thể được thực hiện hiệu quả như tìm một phần tử: trong thời gian $\log(n)$ đối với cây cân bằng n đỉnh. Đây là cấu trúc dữ liệu thường trực; cây đồ đen thường trực có thể giữ cân bằng để đảm bảo thời gian truy cập $\log(n)$ (xem Bài tập 1.1c).

CÁC KÝ HIỆU - SYMBOLS

Bảng băm trong Chương trình 6.12 cần kiểm tra mỗi ký tự trong chuỗi s cho thao tác `hash`, và mỗi lần nó lại so sánh s với chuỗi trong giỏ thứ i . Để tránh so sánh chuỗi không cần thiết, chúng ta biến đổi mỗi chuỗi thành ký hiệu, và mọi xuất hiện khác của cùng một chuỗi sẽ được biến đổi thành cùng một đối tượng ký hiệu.

Module `Symbol` sẽ cài đặt các ký hiệu và có các tính chất quan trọng sau:

- So sánh các ký hiệu xem có bằng nhau hay không là nhanh (chỉ so sánh con trỏ hoặc số nguyên).
- Triết xuất khoá băm nguyên là nhanh (trong trường hợp chúng ta muốn bản băm ánh xạ các ký hiệu vào điều gì đó khác).
- So sánh hai ký hiệu xem cái nào “lớn hơn” (theo một thứ tự nào đó) là nhanh (trong trường hợp chúng ta tạo ra các cây tìm kiếm nhị phân).

Ngay cả nếu chúng ta muốn tạo các môi trường kiểu hàm ánh xạ các ký hiệu vào các gán kết, chúng ta cần sử dụng bảng băm cập nhật kiểu hủy để ánh xạ chuỗi vào ký hiệu: chúng ta cần điều đó để tin tưởng rằng lần xuất hiện thứ hai của “abc” ánh xạ đến cùng ký hiệu như lần xuất hiện thứ nhất. Chương trình 6.15 nêu giao diện của module `Symbol`.

CHƯƠNG TRÌNH 6.15: Giao diện của package `Symbol`.

```
package Symbol;

public class Symbol {
    public String toString();
    public static Symbol symbol(String s);
}

public class Table {
    public Table();
    public void put(Symbol key, Object value);
    public Object get(Symbol key);
    public void beginScope();
    public void endScope();
    public java.util Enumeration keys();
}
```

Môi trường được cài đặt trong `Symbol`. Lớp `Table` như các bảng `Tables` ánh xạ `Symbols` vào các gắn kết. Chúng ta muốn các định nghĩa khác nhau của gắn kết (`binding`) cho các mục đích khác nhau trong chương trình dịch – gắn kết kiểu cho kiểu, gắn kết giá trị cho các biến và các hàm – như vậy chúng ta giả thiết rằng gắn kết là `Object`, mặc dù trong bảng bất kỳ mỗi gắn kết cần phải là gắn kết kiểu hoặc mỗi gắn kết cần là gắn kết giá trị, v.v...

Để cài đặt lớp `Symbol` class (Chương trình 6.16), chúng ta dựa trên phương thức `intern()` của lớp `java.lang.String` cho đối tượng duy nhất đối với mỗi dãy ký tự; chúng ta cần ánh xạ từ `Symbol` vào `String` bằng cách mỗi ký hiệu chứa biến xâu, nhưng ánh xạ ngược cần phải thực hiện nhờ bảng băm (chúng ta dùng `java.util.Hashtable`).

CHƯƠNG TRÌNH 6.16: Cài đặt bảng `Symbol`.

```
package Symbol;

public class Symbol {
    private String name;
    private Symbol(String n) {name=n; }
    private static java.util.Dictionary dict = new java.util.Hashtable();

    public String toString() {return name;}

    public static Symbol symbol(String n) {
        String u = n.intern();
        Symbol s = (Symbol)dict.get(u);
        if (s==null) {s = new Symbol(u); dict.put(u,s); }
        return s;
    }
}
```

Để đáp ứng yêu cầu "undo" của cập nhật kiểu hủy, hàm interface `beginScope` nhớ trạng thái hiện tại trong bảng, và `endScope` lưu lại bảng tại chỗ mà `beginScope` vừa ở đó mà nó chưa kết thúc.

Bảng mệnh lệnh được cài đặt nhờ bảng băm. Khi gắn kết $x \mapsto b$ được đưa vào (`table.put(x, b)`), x được băm vào chỉ số i , và đối tượng `Binder` $x \mapsto b$ được đặt vào đầu của danh sách móc nối cho giỏ thứ i . Nếu `table` đã có chứa gắn kết $x \mapsto b'$, nó vẫn ở trong giỏ, được che giấu bởi $x \mapsto b$. Điều này quan trọng vì nó sẽ hỗ trợ cài đặt *undo* (`beginScope` và `endScope`).

Khóa x không là xâu ký tự, nhưng bản thân nó là đối tượng `Symbol`.

Ở đây cần có thêm ngăn xếp, chỉ ra rằng các ký hiệu được đưa vào bảng theo thứ tự nào. Khi $x \mapsto b$ được đưa vào, thì x được đẩy vào ngăn xếp. Thao tác `beginScope` đẩy ký hiệu đánh dấu đặc biệt vào ngăn xếp. Sau đó, để cài đặt `endScope`, các ký hiệu được lấy ra từ ngăn xếp cho đến khi gặp đánh dấu trên cùng. Khi mỗi ký hiệu được lấy ra, đầu gắn kết trong giỏ được xoá bỏ.

Ngăn xếp bổ sung cần được tích hợp với `Binder` bởi biến tổng thể `top` chỉ ra ký hiệu `Symbol` vừa mới nhất đưa vào bảng. Sau đó "hành động đẩy" được thực hiện cùng với việc sao chép `top` vào trường `prevtop` của `Binder`. Vậy, “ngăn xếp” được xâu lại qua `binders`.

Nếu chúng ta muốn sử dụng môi trường kiểu hàm ánh xạ ký tự vào gắn kết, thì giao diện bảng `Table` có dạng sau:

```
public class Table {
    public Table();
    public Table put(Symbol key, Object value);
    public Object get(Symbol key);
    public java.util.Enumeration keys();
}
```

Hàm `put` cần trả về bảng mới mà không thay đổi bảng cũ. Chúng ta không cần `beginScope` và `endScope`, vì chúng ta cần giữ phiên bản cũ của bảng ngay cả khi đang sử dụng phiên bản mới.

BÀI TẬP

- 6.1 Viết `package` của các lớp Java biểu diễn cú pháp trừu tượng cho các biểu thức chính qui.
- 6.2 Mở rộng Văn phạm 5.4 sao cho chương trình là dãy các lệnh gán hoặc lệnh in. Mỗi lệnh gán sẽ gán biểu thức cho biến khai báo ẩn; mỗi lệnh in sẽ in giá trị của biểu thức. Mở rộng chương trình dịch trong Chương trình 6.1 để xử lý được ngôn ngữ này.

- 6.3 Viết phiên bản JavaCC của văn phạm trong Bài tập 6.2. Chèn đoạn mã Java để dịch các chương trình theo phong cách của Chương trình 6.2.
- 6.4 Chỉnh sửa Văn phạm JavaCC trong Bài tập 6.3 để chứa mã Java xây dựng cây cú pháp, theo phong cách của Chương trình 6.1. Viết hai bộ dịch cho ngôn ngữ này: một theo phong cách hướng đối tượng và một sử dụng các du khách visitors.
- 6.5 Trong `$MINIJAVA/chap4/handcrafted/visitor`, có một file với visitor `PrettyPrintVisitor.java` cho cây cú pháp in đẹp. Hoàn thiện thêm việc in đẹp của các lệnh `if` và `while` lồng nhau.
- 6.6 Mẫu visitor pattern trong Chương trình 6.7 có các phương thức `accept` mà trả về `int`. Nếu một người muốn viết một số visitors mà trả về integers, các visitors khác trả về lớp `A`, và những cái khác trả về lớp `B`, người đó cần phải điều chỉnh mọi lớp trong Chương trình 6.7 để thêm hai hoặc nhiều hơn phương thức `accept`, nhưng điều này có thể không mang tính modular. Một cách khác là làm cho visitor trả về `Object` và chuyển kiểu mỗi kết quả, nhưng nó mất lợi ích kiểm tra kiểu trong thời gian dịch. Nhưng còn có cách thứ ba.

Điều chỉnh Chương trình 6.7 sao cho mọi phương thức `accept` trả về `void`, và viết hai mở rộng của lớp `Visitor`: một tính `int` cho mỗi `Exp`, và cái khác tính `float` cho mỗi `Exp`. Vì phương thức `accept` sẽ trả về `void`, đối tượng visitor cần phải có biến riêng mà phương thức sẽ đặt kết quả của mình vào đó. Giải thích tại sao, nếu một người muốn viết visitor tính đối tượng của lớp `C` cho mỗi `Exp`, mà không cần phải sửa các lớp con `Exp`.

6.7 Hoàn thiện cài đặt bảng băm trong Chương trình 6.12. Tăng gấp đôi kích thước mảng khi độ dài giỏ trung bình tăng lớn hơn 2 (như vậy `table` bây giờ là con trỏ đến mảng cấp phát động). Để tăng gấp đôi mảng, cấp mảng lớn hơn nhiều và băm lại nội dung mảng cũ, rồi sau đó loại bỏ mảng cũ.

*****6.8** Trong nhiều ứng dụng, chúng ta muốn phép toán $+$ cho môi trường thực hiện nhiều hơn là bổ sung một gắn kết mới; thay vì $\sigma' = \sigma + \{a \mapsto \tau\}$, ta muốn $\sigma' = \sigma_1 + \sigma_2$, trong đó σ_1 và σ_2 là hai môi trường bất kỳ (có thể chồng nhau, trong trường hợp đó các gắn kết trong σ_2 sẽ được ưu tiên).

Chúng ta muốn thuật toán hiệu quả và cấu trúc dữ liệu cho việc “cộng” môi trường. Cây cân bằng có thể cài đặt $\sigma + \{a \mapsto \tau\}$ một cách hiệu quả trong thời gian $\log(N)$, trong đó N là cỡ của σ , nhưng có $O(N)$ để tính toán $\sigma_1 + \sigma_2$ nếu σ_1 và σ_2 cả hai đều cỡ N .

Để trừu tượng hóa vấn đề, giải bài toán hợp hai tập nguyên rời nhau tổng quát. Đầu vào là tập các lệnh dạng,

```
s1 = {4}    (define singleton set)
s2 = {7}
s3 = s1 ∪ s2 (nondestructive union)
6 ? ∈ s3    (membership test)
s4 = s1 ∪ s3
s5 = {9}
s6 = s4 ∪ s5
7 ? ∈ s2
```

Thuật toán hiệu quả là thuật toán mà có thể xử lý đầu vào của N lệnh, trả lời mọi câu hỏi có phải thành viên không, ít hơn thời gian $o(N^2)$.

*a. Cài đặt thuật toán mà là hiệu quả khi hợp tập hợp tiêu biểu $a \leftarrow b \cup c$ có b nhỏ hơn nhiều so với c [Brown và Tarjan 1979].

***b. Thiết kế thuật toán mà là hiệu quả ngay cả trong trường hợp xấu nhất, hoặc chứng minh rằng nó không thể được thực hiện (xem Lipton v.v... [1997] đối với cận dưới trong mô hình thu hẹp hơn).

CHƯƠNG 7: KIỂM TRA KIỂU

TỔNG QUAN

Lập trình sử dụng phạm vi rất rộng các cấu trúc tính toán, như các cấu trúc dữ liệu, các hàm, các đối tượng, các kênh truyền và các luồng điều khiển. Vì ngôn ngữ lập trình được thiết kế giúp các lập trình viên tổ chức các cấu trúc tính toán và sử dụng chúng một cách đúng đắn. Nhiều ngôn ngữ lập trình tổ chức dữ liệu và cấu trúc tính toán thành các kiểu. Trong chương này, chúng ta sẽ xem xét các lý do sử dụng kiểu trong ngôn ngữ lập trình và các phương pháp kiểm tra kiểu, và một số vấn đề về kiểu như đa hình, ghi đè, và bằng nhau kiểu. Một mục lớn của chương dành cho suy diễn kiểu, quá trình xác định kiểu các biểu thức dựa trên các kiểu đã biết của các ký hiệu xuất hiện trong chúng. Suy diễn kiểu là tổng quát của kiểm tra kiểu, với rất nhiều điểm giống nhau, và các ví dụ tiêu biểu về các thuật toán kiểu này được sử dụng trong chương trình dịch và môi trường lập trình để xác định các tính chất của chương trình. Suy diễn kiểu cũng đưa ra khái niệm đa hình, mà cho phép một biểu thức có nhiều kiểu.

7.1 KIỂM TRA KIỂU

Trong nhiều ngôn ngữ lập trình, kiểm tra kiểu được sử dụng để ngăn ngừa một số hoặc tất cả các lỗi kiểu. Một số ngôn ngữ sử dụng ràng buộc kiểu trong định nghĩa chương trình hợp lệ. Cài đặt các ngôn ngữ này kiểm tra kiểu trong thời gian dịch, trước khi chương trình bắt đầu. Trong các ngôn ngữ này, các chương trình vi phạm ràng buộc kiểu sẽ không được dịch và không thể chạy. Trong các ngôn ngữ khác, kiểm tra lỗi kiểu được thực hiện khi chương trình đang chạy.

Kiểm tra thời gian chạy. Trong các ngôn ngữ lập trình có kiểm tra kiểu trong thời gian chạy, chương trình dịch sinh ra mã sao cho khi mã được thực hiện, sẽ kiểm tra để tin tưởng rằng các toán hạng có kiểu đúng đắn. Ưu điểm của kiểm tra kiểu thời gian chạy là nó bắt được lỗi kiểu. Nhược điểm là thời gian chạy dành cho các kiểm tra này.

Kiểm tra thời gian dịch. Nhiều ngôn ngữ lập trình hiện đại được thiết kế sao cho có khả năng kiểm tra các biểu thức có tiềm năng lỗi kiểu. Trong các ngôn ngữ này, nói chung là từ chối các chương trình không qua được kiểm tra kiểu thời gian dịch. Ưu điểm của kiểm tra kiểu thời gian dịch là nó bắt lỗi sớm hơn so với kiểm tra kiểu thời gian chạy: người phát triển chương trình được cảnh báo về lỗi trước khi chương trình được đưa cho người khác hoặc chuyển giao như sản phẩm. Vì kiểm tra kiểu trong thời gian dịch có thể loại bỏ

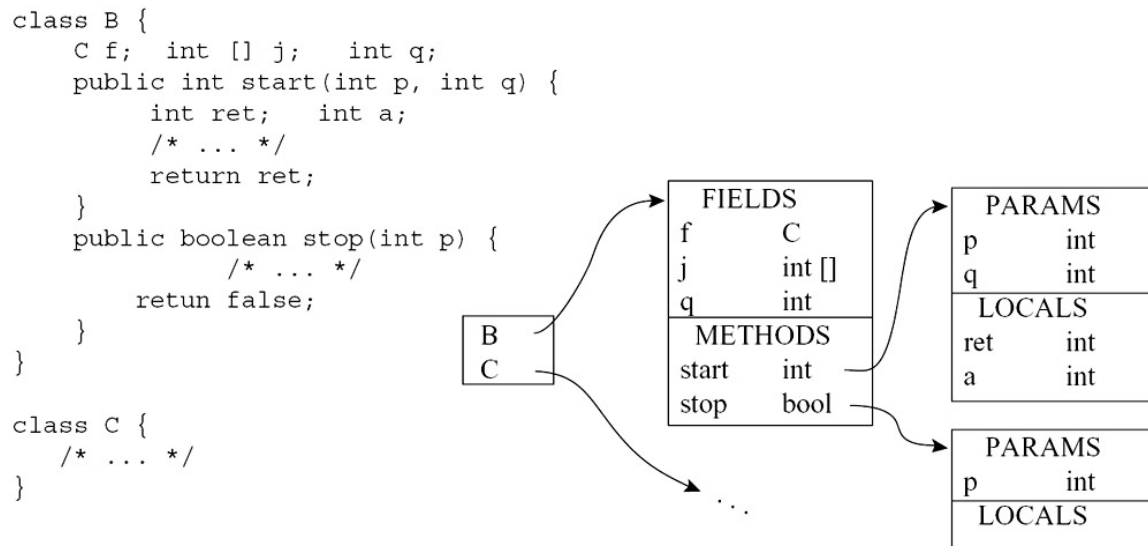
sự cần thiết để kiểm tra một số lỗi trong thời gian chạy, kiểm tra thời gian dịch làm cho nó tạo mã hiệu quả hơn. Chẳng hạn, mã dịch ML là nhanh hơn Lisp từ hai đến bốn lần. Lý do chính tăng tốc độ này là kiểm tra kiểu tĩnh của chương trình ML giảm đáng kể sự cần thiết phải kiểm tra thời gian chạy.

Kiểm tra kiểu MiniJava

Để hỗ trợ kiểm tra kiểu cho các chương trình MiniJava, bảng ký hiệu cần chứa thông tin mọi kiểu khai báo:

- mỗi tên biến và mỗi tên tham số hình thức cần phải được gắn với kiểu của nó;
- mỗi tên phương thức cần phải gắn với các tham số, kiểu kết quả, và các biến cục bộ của nó; và
- mỗi tên lớp phải gắn với các khai báo biến và phương thức của nó.

Chẳng hạn, xét Hình 7.1, ở đó nêu chương trình và bảng ký hiệu của nó. Tên hai lớp B và C được ánh xạ đến hai bảng cho các trường và phương thức. Tiếp theo, mỗi phương thức lại ánh xạ vào cả hai kiểu kết quả và các bảng với các tham số hình thức và các biến cục bộ của nó.



Hình 7.1: Chương trình MiniJava và bảng ký hiệu của nó

Các kiểu nguyên sơ trong MiniJava là `int` và `boolean`; mọi kiểu khác là kiểu mảng nguyên, được viết dạng `int []`, hoặc các tên lớp. Để đơn giản, chúng ta chọn cách để biểu diễn mỗi kiểu như string, thay vì symbol; nó cho phép chúng ta kiểm tra bằng nhau của kiểu bằng cách so sánh xâu.

Kiểm tra kiểu của chương trình MiniJava được tiến hành theo hai giai đoạn. Trước hết, chúng ta xây dựng bảng ký hiệu, và sau đó chúng ta kiểm tra kiểu các lệnh và các biểu

thức. Trong giai đoạn hai, bảng ký hiệu được dùng để kiểm tra mỗi định danh khi nó được xác định. Rất thuận tiện nếu dùng hai giai đoạn vì, trong Java và MiniJava, các lớp là đệ qui đa chiều. Nếu chúng ta thử kiểm tra kiểu trong một giai đoạn, thì chúng ta cần kiểm tra kiểu các lời gọi phương thức mà chưa có trong bảng ký hiệu. Để tránh tình huống như vậy, chúng ta sử dụng cách tiếp cận hai giai đoạn.

Giai đoạn đầu của kiểm tra kiểu có thể cài đặt bằng visitor mà viếng thăm các đỉnh trên cây cú pháp MiniJava và xây dựng bảng ký hiệu. Chẳng hạn, phương thức visit trong Chương trình 7.2 quản lý các khai báo biến. Nó sẽ bổ sung tên và kiểu biến vào cấu trúc dữ liệu của lớp hiện thời mà sau này sẽ được bổ sung vào bảng ký hiệu. Nhận thấy rằng phương thức visit sẽ kiểm tra biến có được khai báo nhiều hơn một lần không, nếu có sẽ in ra thông báo lỗi tương ứng.

CHƯƠNG TRÌNH 7.2: Phương thức visit cho khai báo biến

```
class ErrorMsg {
    boolean anyErrors;
    void complain(String msg) {
        anyErrors = true;
        System.out.println(msg);
    }
}

// Type t;
// Identifier i;
public void visit(VarDecl n) {

    Type t = n.t.accept(this);
    String id = n.i.toString();

    if (currMethod == null) {
        if (!currClass.addVar(id,t))
            error.complain(id + "is already defined in " +
currClass.getId());
    } else if (!currMethod.addVar(id,t))
        error.complain(id + "is already defined in "
            + currClass.getId() + "." + currMethod.getId());
}
```

Giai đoạn hai của kiểm tra kiểu có thể được cài đặt bằng visitor mà sẽ kiểm tra mọi lệnh và biểu thức. Kiểu kết quả của mỗi phương thức visit là `String` để biểu diễn kiểu của MiniJava. Ý tưởng là khi visitor viếng thăm biểu thức, thì nó trả về kiểu của biểu thức. Nếu biểu thức không qua được kiểm tra kiểu, thì kiểm tra kiểu sẽ được kết thúc với thông báo lỗi.

Giả sử xét trường hợp đơn giản: biểu thức cộng $e_1 + e_2$. Trong MiniJava, cả hai toán hạng cần phải là số nguyên (kiểm tra kiểu cần phải kiểm tra điều này) và kết quả sẽ là số nguyên (kiểm tra kiểu sẽ trả về kiểu này). Phương thức visit đối với cộng sẽ dễ dàng được cài đặt; xem Chương trình 7.3.

CHƯƠNG TRÌNH 7.3: Phương thức `visit` cho biểu thức cộng

```
// Exp e1,e2;
public Type visit(Plus n) {
    if (! (n.e1.accept(this) instanceof IntegerType) )
        error.complain("Left side of LessThan must be of type integer");
    if (! (n.e2.accept(this) instanceof IntegerType) )
        error.complain("Right side of LessThan must be of type integer");
    return new IntegerType();
}
```

Trong hầu hết các ngôn ngữ lập trình, phép cộng được ghi đè: phép `+` dùng cho cộng số nguyên hoặc cộng số thực. Nếu cả hai toán hạng là nguyên, thì kết quả nguyên, nếu cả hai toán hạng là thực, thì kết quả là thực. Và trong nhiều ngôn ngữ, nếu một toán hạng là nguyên và toán hạng kia là thực, thì số nguyên được chuyển kiểu ẩn thành số thực, và kết quả là thực. Tất nhiên, chương trình dịch sẽ chuyển tường minh trong mã máy mà nó sinh ra.

Đối với lệnh gán, nó cần được kiểm tra vế trái và vế phải có cùng kiểu hay không. Khi chúng ta mở rộng lớp, yêu cầu này không còn chặt chẽ như trước nữa: chỉ cần kiểm tra rằng vế phải có phải là kiểu con của vế trái không.

Đối với lời gọi hàm, cần phải tìm tên phương thức trong bảng ký hiệu để lấy danh sách các biến và kiểu kết quả. Đối với lời gọi `e.m(...)`, trong đó `e` có kiểu `C`, ta tìm định nghĩa của `m` trong lớp `C`. Các kiểu của các tham số cần phải sánh với các đối số của biểu thức gọi hàm. Kiểu kết quả của phương thức trở thành kiểu của lời gọi phương thức.

Mỗi loại lệnh và biểu thức có qui tắc kiểm tra kiểu của riêng nó, nhưng trong mọi trường hợp chúng ta còn chưa mô tả hết, các qui tắc có thể được suy diễn bằng cách tham chiếu đến đặc tả ngôn ngữ Java.

XỬ LÝ LỖI

Khi bộ kiểm tra kiểu phát hiện lỗi kiểu hoặc định danh không khai báo, nó sẽ in ra thông báo lỗi và tiếp tục – vì lập trình viên muốn được biết mọi lỗi trong chương trình. Để khắc phục sau lỗi, thông thường cần xây dựng cấu trúc dữ liệu để đảm bảo như không có lỗi xảy ra và biểu thức cần tính vẫn đúng. Ví dụ, kiểm tra kiểu

```
{int i = new C();
 int j = i + i;
 ...
}
```

ngay cả khi biểu thức `new C()` không sánh kiểu yêu cầu là khởi tạo số nguyên, vẫn còn có ích khi nhập `i` vào bảng ký hiệu như số nguyên sao cho để phần sau của chương trình cần được kiểm tra kiểu.

Nếu pha kiểm tra kiểu phát hiện lỗi, thì chương trình dịch không sinh ra chương trình được dịch như đầu ra. Điều đó có nghĩa là các pha về sau của chương trình dịch như thông dịch, cấp thanh ghi, v.v... sẽ không được thực hiện. Sẽ là dễ dàng cài đặt các pha về sau của chương trình dịch nếu chúng không được yêu cầu xử lý đầu vào không đúng. Như vậy, nếu mọi việc có thể, thì mọi lỗi của chương trình đầu vào cần được phát hiện ở kỳ đầu của chương trình dịch (phân tích cú pháp và kiểm tra kiểu).

KIỂM TRA KIỂU CHƯƠNG TRÌNH

Thiết kế tập các visitors mà kiểm tra kiểu chương trình MiniJava và sinh ra các thông báo lỗi thích hợp về lỗi kiểu hoặc các định danh không khai báo.

7.2 XÁC ĐỊNH PHÉP TOÁN

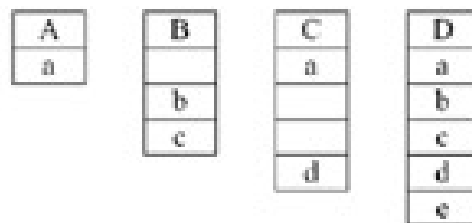
Do tính chất đa hình của lập trình hướng đối tượng, do đó việc xác định đúng phương thức dựa vào tên, các kiểu tham số và đối tượng hoặc lớp sở hữu phương thức đó đòi hỏi một số cơ chế hỗ trợ của Chương trình dịch.

ĐA KẾ THỪA (MULTIPLE INHERITANCE)

Trong các ngôn ngữ mà cho phép lớp D được mở rộng từ một số lớp cha A, B, C (mà ở đó A không là lớp con của B, và ngược lại), tìm lè các trường và các bản phương thức là khó hơn. Không thể đặt tất cả các trường của A ở đầu của D và đặt tất cả các trường của B ở đầu của D.

Tô màu đồ thị tổng thể (Global graph coloring). Một giải pháp cho vấn đề này phân tích đồng thời tất cả các lớp, tìm lè nào đó cho mỗi tên trường để có thể được sử dụng trong mỗi bản ghi chứa trường đó. Chúng ta cần mô hình nó như bài toán tô màu đồ thị: ở đây có đỉnh cho mỗi tên trường khác nhau, và cạnh cho mỗi hai trường mà cùng tồn tại (chẳng hạn bởi kế thừa) trong cùng một lớp. Các lè 0, 1, 2;... là các màu. Hình 7.4 cho một ví dụ.

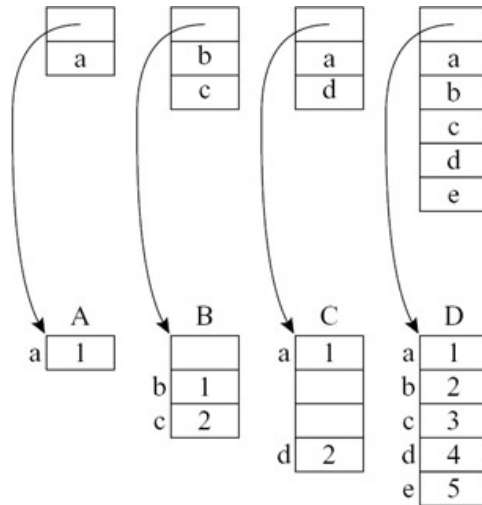
```
class A { int a = 0; }
class B { int b = 0;
         int c = 0; }
class C extends A { int d = 0; }
class D extends A,B,C { int e = 0; }
```



Hình 7.4: Đa kế thừa từ các trường dữ liệu.

Vấn đề với cách tiếp cận này là nó để lại các rãnh rỗng ở giữa các đối tượng trung gian, vì nó không thể bao giờ cũng tô màu N trường của mỗi lớp với các màu của N màu đầu tiên.

Để loại bỏ các rãnh rỗng này trong các đối tượng, chúng ta bó các trường của mỗi đối tượng và có mô tả lớp nói trường nào ở đâu. Hình 7.5 cho ví dụ. Chúng ta có đồ thị tô màu trên tất cả các tên trường, như trước, nhưng bây giờ các màu không phải là các lề của các trường đó bên trong đối tượng *objects* mà ở bên trong mô tả *descriptors*. Để tìm trường a của đối tượng x , ta tìm từ- a trong mô tả của x ; từ này chứa các số nguyên nhỏ nói lên vị trí của dữ liệu thực tế a trong x .



Hình 7.5: Các lề của các trường trong các mô tả đa kế thừa.

Trong sơ đồ này, các mô tả lớp có các rãnh rỗng, nhưng các đối tượng thì không, điều này là chấp nhận được, vì hệ thống với hàng triệu đối tượng trong khi chỉ có hàng chục các mô tả lớp. Nhưng việc tìm (hoặc lưu) mỗi dữ liệu đòi hỏi ba chỉ lệnh thay vì một:

1. Tìm con trỏ mô tả từ đối tượng
2. Tìm giá trị lề của trường từ mô tả
3. Tìm (hoặc lưu) dữ liệu tại lề thích hợp của đối tượng

Trên thực tế, một số thao tác khác trên đối tượng đã tìm được con trỏ mô tả rồi, và các thao tác lặp trên cùng một trường (như tìm và sau đó lưu) sẽ không cần tìm lại lề của mô tả; việc xóa bỏ các biểu thức con chung sẽ loại bỏ nhiều công việc thừa.

Tìm kiếm phương thức (Method lookup). Tìm các bản phương thức trong ngôn ngữ với đa kế thừa là một việc phức tạp như tìm các lề của trường. Tiếp cận tô màu đồ thị tổng thể làm việc tốt: các tên phương thức có thể hòa lẫn với các tên trường tạo thành các đỉnh của đồ thị suy diễn lớn. Các bản nhập mô tả cho các trường cho vị trí bên trong các đối tượng; các bản nhập mô tả các phương thức cho địa chỉ mã máy của các bản phương thức.

Các vấn đề với liên kết động (Problems with dynamic linking). Mọi tiếp cận tổng thể phải trả giá từ vấn đề trên là tô màu (và thể hiện qua các mô tả lớp) có thể được thực hiện chỉ tại thời điểm gắn kết link; Công việc này tất nhiên nằm trong khả năng của bộ link chuyên dụng.

Tuy nhiên, nhiều hệ thống hướng đối tượng có khả năng tải các lớp mới và các hệ thống đang chạy; các lớp này có thể là mở rộng của các lớp đã sử dụng. Tô màu đồ thị thời gian link đưa ra nhiều vấn đề cho hệ thống mà cho phép liên kết động tăng lên.

Băm (Hashing). Thay vì tô màu đồ thị tổng thể, chúng ta có thể đặt bảng băm trong mỗi mô tả lớp, ánh xạ tên các trường vào lẻ và tên các phương thức vào các bản phương thức. Điều này làm việc tốt với việc dịch tách biệt và liên kết động.

Các ký tự của các tên trường không được băm trong thời gian chạy. Thay vào đó, mỗi tên trường a được băm trong thời gian dịch vào một số nguyên hash_a nằm trong khoảng $[0, N - 1]$. Cũng như vậy, đối với mỗi tên trường, bản ghi thời gian chạy duy nhất (con trỏ) ptr_a được tạo nên.

Mỗi mô tả lớp có một bảng lẻ trường (field-offset table) Ftab cỡ N chứa các lẻ trường và các bản phương thức, và (để phát hiện các xung đột) một bảng khóa song song (key table) Ktab chứa các con trỏ tên trường. Nếu lớp có trường x , thì số rãnh hash_x của bảng lẻ trường chứa lẻ cho x , và số rãnh bảng khóa hash_x chứa con trỏ ptr_x .

Để tìm trường x của đối tượng c , chương trình dịch sẽ sinh code cho việc

1. Tìm mô tả lớp d tại lẻ 0 của đối tượng c .
2. Tìm tên trường f từ lẻ của địa chỉ $d + \text{Ktab} + \text{hash}_x$.
3. Kiểm tra $f = \text{ptr}_x$; nếu đúng như vậy
4. Tìm lẻ k của trường từ $d + \text{Ftab} + \text{hash}_x$.
5. Tìm nội dung của trường từ $c + k$.

Thuật toán này có bốn chỉ lệnh ở trên, mà có thể chịu được. Một thuật toán tương tự làm việc cho tìm kiếm bản phương thức động.

Thuật toán như đã mô tả không cho biết cái gì cần làm nếu kiểm tra ở dòng 3 bị lỗi. Kỹ thuật phân giải va chạm bất kỳ của bảng băm được sử dụng.

KIỂM TRA THÀNH VIÊN CỦA LỚP

Một số ngôn ngữ hướng đối tượng cho phép chương trình kiểm tra một đối tượng có là thành viên của một lớp trong thời gian chạy hay không, như tóm tắt trong bảng 7.6.

Bảng 7.6 Tiện ích để kiểm tra kiểu và ép kiểu an toàn.

	Modula-3	Java
Kiểm tra đối tượng x có thuộc lớp C , hoặc một lớp con của C .	ISTYPE(x, C)	x instanceof C
Cho biến x của lớp C , trong đó x thực tế trở đến đối tượng của lớp D mà mở rộng C , tạo biểu thức mà kiểu trong thời gian dịch là lớp D .	NARROW(x, D)	$(D) x$

Vì mỗi đối tượng trở đến mô tả lớp của nó, địa chỉ mô tả lớp có thể coi như “thẻ-kiểu” "type-tag." Tuy nhiên, nếu x là đối tượng của D , và D mở rộng C , thì x cũng là bản tạo của C . Giả sử không có đa kế thừa, cách đơn giản để cài đặt x instanceof C là sinh ra mã mà thực hiện vòng lặp sau tại thời gian chạy:

```

     $t_1 \leftarrow x.descriptor$ 
 $L_1$  : if  $t_1 = C$  goto true
     $t_1 \leftarrow t_1.super$ 
    if  $t_1 = nil$  goto false

```

goto L_1

trong đó $t_1.super$ là lớp cha của lớp t_1 .

Tuy nhiên, có cách tiếp cận nhanh hơn sử dụng *display* của các lớp cha. Giả sử rằng độ sâu lồng lớp được giới hạn bởi một hằng số nào đó, chẳng hạn 20. Lăn ngược khỏi 20 từ trong mô tả của mỗi lớp. Trong mô tả lớp D độ sâu lồng nhau là j , đặt con trỏ vào mô tả D tại rãnh thứ j , con trỏ đến $D.super$ ở rãnh thứ $(j - 1)$, và con trỏ đến $D.super.super$ ở rãnh $j - 2$, và tiếp tục như vậy đến Object ở rãnh 0. Trong mọi rãnh được đánh số lớn hơn j , đặt nil.

Bây giờ, nếu x là bản tạo của D , hoặc lớp con bất kỳ của D , thì rãnh j của mô tả lớp x 's sẽ trở đến mô tả lớp D . Ngược lại, nó sẽ không. Vì vậy x instanceof D đòi hỏi

1. Đặt mô tả lớp D tại lè 0 của đối tượng c .
2. Đặt con trỏ lớp rãnh j trong d .
3. So sánh với mô tả lớp D .

Điều này làm việc vì độ sâu lồng lớp của D được biết trong thời gian dịch.

Ép kiểu (Type coercions). Cho trước biến c của kiểu C , luôn là hợp lệ khi xử lý c như kiểu cha bất kỳ của C – nếu C mở rộng B , và biến b có kiểu B , thì phép gán $b \leftarrow c$ là hợp lệ và an toàn.

Nhưng ngược lại không đúng. Phép gán $c \leftarrow b$ là an toàn chỉ khi nếu b thực tế (trong thời gian chạy) là bản tạo của C , mà không phải khi nào cũng vậy. Nếu ta có $b \leftarrow \text{new } B$, $c \leftarrow b$, ngay tiếp theo khi tìm thấy một trường nào đó của c mà là một phần của lớp C chứ không phải lớp B , thì việc tìm thấy này có thể dẫn tới hành vi không đoán được.

Vì vậy, các ngôn ngữ hướng đối tượng an toàn (như Modula-3 và Java) đi kèm ép kiểu bất kỳ từ lớp cha về lớp con với việc kiểm tra kiểu thời gian chạy mà đưa ra ngoại lệ, trừ khi giá trị thực tế là bản tạo của lớp con (tức là, trừ khi `b instanceof C`).

Đây là cách diễn đạt chung

```
Modula-3:
IF ISTYPE(b, C)
  THEN f(NARROW(b, C))
  ELSE ...
```

```
Java:
if (b instanceof C)
  f((C)b)
else ...
```

Bây giờ có hai kiểm tra kiểu liên tiếp: một tường minh (ISTYPE hoặc `instanceof`) và một ẩn (trong NARROW hoặc chuyển kiểu). Một chương trình dịch tốt sẽ thực hiện phân tích luồng đủ tốt để báo trước là mệnh đề **then** đạt được chỉ khi nếu b thực tế là bản tạo của C , sao cho kiểm tra kiểu làm hẹp thao tác có thể được loại bỏ.

C++ là ngôn ngữ hướng đối tượng không an toàn. Nó có cơ chế chuyển kiểu tĩnh (*static cast*) không cần kiểm tra thời gian chạy; việc sử dụng không thận trọng của cơ chế này có thể làm chương trình chạy sai theo cách không dự báo được. C++ cũng có chuyển kiểu động `dynamic_cast` với kiểm tra kiểu thời gian chạy, mà giống cơ chế trong Modula-3 và Java.

Trường hợp kiểu (Typecase). Kiểm tra `instanceof` tường minh, tiếp theo làm hẹp kiểu về lớp con, là không được khuyến khích trong phong cách hướng đối tượng. Thay vì sử dụng cách thức này, các lập trình viên thường sử dụng các phương thức động mà thực hiện đúng trong mỗi lớp con. Tuy vậy, cách kiểm tra làm hẹp vẫn được coi là được.

Modula-3 có tiện ích **typecase** mà làm cho cách thức này đẹp và hiệu quả hơn (nhưng vẫn không "hướng đối tượng" hơn)

```
TYPECASE expr
OF C1 (v1) => S1
  | C2 (v2) => S2
  :
  | Cn (vn) => Sn
ELSE S0
END
```

Nếu *expr* được tính cho bản tạo của lớp C_i , thì biến mới v_i kiểu C_i trở đến kết quả của *expr*, và lệnh S_i được thực hiện. Khai báo của v_i là ẩn trong TYPECASE, và phạm vi C_i của nó chỉ đủ cho S_i .

Nếu nhiều hơn một trong số C_i được sánh (mà có thể xảy ra nếu, chẳng hạn, một là lớp cha của lớp khác), thì chỉ phép sánh đầu tiên của câu lệnh được xét. Nếu không thỏa mãn nào của C_i , thì câu lệnh ELSE sẽ được xét (lệnh S_0 được thực thi).

Typecase có thể chuyển đổi trực tiếp vào dãy các **else-ifs**, với mỗi **if** thực hiện kiểm tra bản tạo, thu hẹp, và khai báo biến cục bộ. Tuy nhiên, nếu ở đây có rất nhiều mệnh đề, thì nó có thể mất nhiều thời gian trải qua mọi **else-ifs**. Do đó thuận tiện hơn là xử lý chúng như các trường hợp số nguyên, sử dụng bước nhảy có chỉ số.

Đây là lệnh rẽ nhánh thông thường trên số nguyên:

ML:	C, Java:
case i	switch (i) {
of 0 => s_0	case 0: s_0 ; break;
1=> s_1	case 1: s_1 ; break;
2=> s_2	case 2: s_2 ; break;
3=> s_3	case 3: s_3 ; break;
4=> s_4	case 4: s_4 ; break;
_=> s_d	default: s_d ;
	}

được dịch như sau: Trước hết so sánh kiểm tra phạm vi được thực hiện để tin tưởng rằng i nằm trong phạm vi của các nhãn (0-4, trong trường hợp này); sau đó địa chỉ của lệnh i sẽ được đưa vào rãnh thứ i trong bảng, và điều khiển nhảy đến s_i .

Tiếp cận này sẽ không làm việc đối với **typecase**, vì có lớp con. Như vậy, ngay cả nếu chúng ta có thể tạo mô tả lớp là các số nguyên nhỏ thay vì các con trỏ, chúng ta không thể thực hiện các bước nhảy chỉ số dựa trên lớp của đối tượng, vì chúng ta sẽ bỏ qua các câu lệnh mà sánh các lớp cha của lớp đó. Vì vậy, Modula-3 **typecase** được cài đặt như dãy các **else-ifs**.

Gán các số nguyên cho các lớp là không hiển nhiên, vì các module được dịch riêng rẽ có thể định nghĩa các lớp của mình, và chúng ta không muốn số nguyên xung đột nhau. Nhưng liên kết có tính toán hơn có thể có khả năng gán cho số nguyên trong thời gian liên kết.

Nếu mọi lớp trong **typecase** là các lớp *final* (theo nghĩa sử dụng bởi Java, là chúng không thể được mở rộng), thì vấn đề này không nên áp dụng. Modula-3 không có các lớp; và Java không có **typecase**. Nhưng hệ thống Java thông minh hơn có thể có khả năng nhận biết được dãy **else-ifs** mà thực hiện các kiểm tra **instanceof** cho tập các lớp *final*, và sinh ra bước nhảy được đánh số.

CÁC TRƯỜNG VÀ CÁC PHƯƠNG THỨC RIÊNG

Các ngôn ngữ hướng đối tượng thực sự có thể bảo vệ các trường của đối tượng khỏi thao tác trực tiếp của các phương thức của đối tượng khác. Trường riêng (*private field*) là

trường mà không thể được lấy và chỉnh sửa từ các hàm hoặc phương thức bất kỳ khai báo bên ngoài đối tượng; phương thức riêng (private method) là phương thức không thể gọi từ bên ngoài đối tượng.

Tính riêng tư là bắt buộc trong pha kiểm tra kiểu của chương trình dịch. Trong bảng ký hiệu của C, với lẽ của mỗi trường và lẽ của mỗi phương thức, là một cờ Bool chỉ ra trường đó có là private không. Khi dịch biểu thức $c.f()$ hoặc $c.x$, việc đơn giản là kiểm tra trường đó và từ chối truy cập từ bất cứ phương thức nào ở bên ngoài khai báo của đối tượng.

Có nhiều phương án về riêng tư và bảo vệ. Các ngôn ngữ khác nhau cho phép

- Các trường và các phương thức mà có thể truy cập được đến lớp mà khai báo chúng.
- Các trường và các phương thức truy cập được đến các lớp khai báo và đến lớp con bất kỳ của lớp đó.
- Các trường và các phương thức truy cập được chỉ bên trong cùng một Module (gói package, không gian tên namespace) như lớp khai báo.
- Các trường mà chỉ đọc được từ bên ngoài lớp khai báo, nhưng được viết bởi các phương thức của lớp.

Nói chung, các phương án bảo vệ này có thể buộc tuân thủ tĩnh bằng kiểm tra kiểu trong thời gian dịch, đối với các ngôn ngữ dựa trên lớp.

CÁC NGÔN NGỮ KHÔNG CÓ LỚP

Một số ngôn ngữ hướng đối tượng không sử dụng khái niệm lớp class. Trong ngôn ngữ đó, mỗi đối tượng cài đặt các phương thức và có các trường tùy ý mà nó muốn có. Kiểm tra kiểu đối với ngôn ngữ như vậy thường là động (thực hiện trong thời gian chạy) thay vì tĩnh (thực hiện trong thời gian dịch).

Nhiều đối tượng được tạo ra bằng cách nhân bản (*cloning*): copy đối tượng đã có (hoặc đối tượng nháp *template*) và sau đó thay đổi một vài trường. Như vậy, ngay cả trong ngôn ngữ không có lớp ở đây cũng có giả lớp ("pseudo-classes") của các đối tượng giống nhau mà chia sẻ mô tả. Khi b được tạo ra bằng nhân bản a , nó chia sẻ mô tả với a . Chỉ nếu khi một trường mới được bổ sung hoặc một trường phương thức được cập nhật (ghi đè) thì b đòi hỏi có mô tả mới.

Kỹ thuật này được sử dụng trong việc dịch các ngôn ngữ không lớp tương tự như các ngôn ngữ dựa trên lớp với đa kế thừa và liên kết động: mô tả giả lớp chứa các bảng băm mà thỏa mãn các lẽ trường và bản tạo phương thức.

Các dạng phân tích và tối ưu chương trình tổng thể được sử dụng đối với các ngôn ngữ dựa trên lớp để tìm bản tạo phương thức nào được gọi từ lời gọi động cũng là hữu ích cho các ngôn ngữ không lớp.

TỐI ƯU CÁC CHƯƠNG TRÌNH HƯỚNG ĐỐI TƯỢNG

Tối ưu có tầm quan trọng đặc biệt cho các ngôn ngữ hướng đối tượng (mà cũng đem lại lợi ích từ hầu hết các tối ưu mà áp dụng cho ngôn ngữ lập trình nói chung) là chuyển lời gọi phương thức động sang lời gọi bản tạo phương thức tĩnh.

So sánh với lời gọi hàm thông thường, với mỗi lần gọi phương thức cần phải tìm kiếm phương thức động để xác định bản tạo phương thức. Đối với ngôn ngữ đơn kế thừa, tìm kiếm phương thức chỉ thực hiện hai chỉ lệnh. Điều này có vẻ có giá nhỏ, nhưng

- Máy hiện đại có thể nhảy đến các địa chỉ cố định hiệu quả hơn là các địa chỉ lấy từ các bảng. Khi địa chỉ là rõ ràng trong dòng chỉ lệnh, bộ xử lý có thể lưu cache chỉ lệnh trước ở đích và hướng cơ chế đưa ra chỉ lệnh lấy ở đích của bước nhảy. Các bước nhảy không dự báo trước được làm chậm đưa ra chỉ lệnh và thực thi một vài chu trình.
- Chương trình dịch tối ưu mà mở rộng trực tiếp hoặc phân tích bên trong thủ tục gặp rắc rối khi phân tích các hệ quả của lời gọi nếu không biết trước bản tạo phương thức nào được gọi khi đó.

Đối với đa kế thừa và ngôn ngữ không lớp, giá tìm kiếm phương thức động thậm chí còn cao hơn.

Như vậy, tối ưu chương trình dịch cho ngôn ngữ hướng đối tượng thực hiện phân tích chương trình tổng thể để xác định các vị trí ở đó lời gọi phương thức luôn gọi cùng bản tạo phương thức, sau đó lời gọi phương thức động được thay thế bằng lời gọi phương thức tĩnh.

Chẳng hạn, lời gọi phương thức $c.f()$, trong đó c là đối tượng lớp C , *phân tích phân cấp kiểu* (*type hierarchy analysis*) được dùng để xác định lớp con nào của C chứa các phương thức f mà có thể đề $C.f$. Nếu không có phương thức nào như vậy, thì bản tạo phương thức cần phải là $C.f$.

Ý tưởng này kết hợp với việc lan truyền kiểu (*type propagation*), là dạng phân tích dữ liệu tĩnh. Sau khi gán $c \leftarrow_{\text{new}} C$, lớp chính xác của c đã được biết. Thông tin này có thể lan truyền đến phép gán $d \leftarrow c$, và tiếp tục. Khi $d.f()$ được xem xét, thông tin lan truyền kiểu sẽ hạn chế phạm vi của phân cấp kiểu mà xác định bản tạo phương thức cho d .

Giả sử phương thức f được định nghĩa trong lớp C gọi phương thức g trên `this`. Nhưng g là phương thức động và có thể được đề, do đó lời gọi này đòi hỏi tìm kiếm động. Tối ưu chương trình dịch có thể tạo bản sao của bản tạo phương thức $C.f$ cho mỗi lớp con (chẳng hạn, D, E) mà mở rộng C . Sau đó khi (new copy) $D.f$ gọi g , Chương trình dịch biết phải gọi bản tạo $D.g$ mà không cần tìm kiếm động.

KIỂM TRA KIỂU ĐA HÌNH

Kiểm tra kiểu cho ngôn ngữ đa hình là không trực tiếp như đối với ngôn ngữ đơn hình. Trước khi bắt tay vào cài đặt, chúng ta cần phải rõ ràng về các qui tắc kiểu đang có.

Các kiểu được sử dụng trong chương trình dịch MiniJava cơ bản cơ bản cần được biểu diễn như các xâu không tách rời được: `int`, `boolean`, `int[]`, và các định danh lớp như `IntList`. Trong GJ chúng ta có ba dạng biểu thức kiểu:

Các kiểu nguyên thủy (Primitive types) như `int` và `boolean`;

Áp dụng kiểu (Type applications) có dạng $c \langle t_1, t_2, \dots, t_n \rangle$, trong đó c là cấu trúc tạo kiểu (*type constructor*) – một lớp đa hình như `List` trong ví dụ của chúng ta – và t_1 cho đến t_n là các biểu thức kiểu; và **Các biến kiểu (Type variables)** như x trong biểu thức kiểu `ListX`.

Tất cả các định danh lớp sẽ được xem xét là đa hình, nhưng những cái mà không có đối số kiểu có thể được xem như các kiểu có dạng $c<>$. Trong chương này chúng ta bỏ qua việc xét các kiểu mảng, mặc dù trong chương trình dịch "MiniGJ", `int[]` có thể được xử lý như kiểu nguyên thủy.

Các qui tắc cú pháp (Syntactic conventions). Thuật toán kiểm tra kiểu sẽ có một số dạng các biến:

- T, T_i, U, V ký hiệu là các biểu thức kiểu;
- N, N_i, P, Q ký hiệu là các biểu thức kiểu không thay đổi (mà không chứa các biến kiểu);
- X, X_i, Y, Z ký hiệu là các biến kiểu;
- C ký hiệu là các tên lớp;
- m ký hiệu là các tên lớp;
- f ký hiệu là tên các trường;
- e ký hiệu là các biểu thức chương trình; và
- x ký hiệu là các biến chương trình (các biến cục bộ của các phương thức).

Chúng ta sẽ thường sử dụng viết tắt từ khóa `extends` bằng ký hiệu $\triangleleft$. Trong khi bàn luận thuật toán kiểm tra kiểu chúng ta sẽ yêu cầu là mỗi *TyParams* có gắn kết tường minh, như vậy thay vì viết `class List<X>` chúng ta sẽ viết `class List<X $\triangleleft$ Object>`.

Các kiểu phương thức (Method types). Sẽ là thuận tiện nếu tách kiểu của phương thức ra khỏi tên của phương thức. Chẳng hạn, xét khai báo

```
<X extends Printable> GPair<X> firstTwo(List<X> x) { ... }
```

Đối với kiểu X bất kỳ mà là kiểu con của `Printable`, phương thức `firstTwo` nhận danh sách các phần tử kiểu X , và trả về một cặp phần tử kiểu X . Kiểu của phương thức `firstTwo` là $\langle X \triangleleft \text{Printable} \rangle \text{ List } \langle X \rangle \rightarrow \text{GPair } \langle X \rangle$. (Lưu ý rằng xuất hiện gắn kết của X đòi hỏi mệnh đề gắn kết $\langle X \triangleleft \text{Printable} \rangle$, nhưng các xuất hiện áp dụng của X không sử dụng ký hiệu $\triangleleft$). Nói chung, dạng của kiểu phương thức là

$$MethodTy \rightarrow TyParams \ TyList \rightarrow Ty$$

nghĩa là, với tham số kiểu *TyParams* và các tham số giá trị mà kiểu của chúng là *TyList*, phương thức trả về giá trị của kiểu *Ty*.

Phép thế (Substitution). Giả sử chúng ta muốn biết kiểu của phương thức `firstTwo` khi áp dụng nó cho danh sách của `PrintableInt`. Chúng ta lấy `List <X> → GPair <X>` và thế `PrintableInt` cho `x`, nhận được kiểu của phương thức

`List <PrintableInt> → GPair <PrintableInt>`

Chúng ta có thể viết hàm thế trong kiểm tra kiểu của chương trình dịch để làm điều này; trong chương này chúng ta sẽ viết $[V_1, \dots, V_k/X_1, \dots, X_k]U$ nghĩa là thế kiểu V_i cho mỗi xuất hiện của biến kiểu X_i trong biểu thức kiểu (hoặc mẫu hàm) U . Vì tham số hóa lớp không được lồng trong GJ – tức là, U không được chứa khai báo lớp – chúng ta không cần quan tâm về các khai báo lại bên trong của các biến kiểu X_i ; vấn đề tránh xác định biến (*avoiding variable capture* – mô tả trong mục trước) không xảy ra.

Bảng lớp, bảng kiểu và bảng các biến. Vì các lớp có thể đa chiều tham chiếu nhau, kiểm tra kiểu cần diễn ra trong hai pha: Trước hết, xây dựng bảng ký hiệu (*bảng lớp*) ánh xạ tên lớp vào khai báo lớp; thứ hai, kiểm tra kiểu bên trong khai báo lớp. Tiếp cận tương tự làm việc đối với GJ. Pha thứ nhất hoàn toàn trực tiếp; phần còn lại của mục này giải thích thuật toán cho pha thứ hai. Pha này sẽ sử dụng bảng lớp và duy trì bảng kiểu ánh xạ các tham số kiểu hình thức vào các gắn kết của chúng và bảng var ánh xạ các biến giá trị thông thường vào các kiểu của chúng. Chẳng hạn, trong khi xử lý phương thức `firstTwo` (mô tả bên trên), bảng kiểu sẽ ánh xạ `x` vào `Printable`, và bảng var ánh xạ `x` vào `List<X>`.

Các kiểu ở dạng tốt. Một kiểu ở dạng tốt nếu mỗi biến kiểu được gắn kết trong bảng kiểu tương ứng. Như vậy, kiểu `List<X>` là ở dạng tốt khi xử lý khai báo của `firstTwo` vì `x < Printable` là ở trong bảng kiểu. Nói chung, chúng ta cũng cần tính đến các phép thế. Thuật toán 7.5 đưa ra thủ tục *checktype* để kiểm tra dạng tốt của các kiểu.

THUẬT TOÁN 7.5: Kiểm tra dạng tốt của các kiểu và kiểu con.

```

checkType(T) =
  if T is primitive (int, boolean, etc.) then OK
  else if T is Object then OK
  else if (T ↦ ...) is in the type table then OK
  else if T is C <T1, ..., Tn>
    look up C in the class table, yielding class C <X1 < N1, ..., Xn < Nn> < N{...}>
    for each Ti do checkType(Ti)
    for each Ti do checkSubtype(Ti, [T1, ..., Tm] X1, ..., Xn] Ni)

checkSubtype(T, U) =
  if T = U then OK

```

```

else if  $(T \mapsto T')$  is in the type table then subtype( $T'$ ,  $U$ )
else if  $T$  is  $C \langle T_1, \dots, T_n \rangle$ 
    look up  $C$  in the class table, yielding class  $C \langle X_1 \triangleleft N_1, \dots, X_n \triangleleft N_n \rangle \triangleleft N\{\dots\}$ 
    subtype( $[T_1, \dots, T_n/X_1, \dots, X_n]N$ ,  $U$ )
else error

```

Kiểu con. Trong kiểm tra kiểu Java (và GJ) thường là cần thiết xét xem một kiểu có là kiểu con của kiểu khác không: A là kiểu con của B nếu, ở đâu giá trị kiểu B được yêu cầu, thì giá trị kiểu A sẽ đáp ứng được. Một kiểu con A cũng không chính xác như lớp con, nhưng trong các ngôn ngữ dựa trên lớp như Java, nếu lớp A mở rộng lớp B , thì ở đây cũng có quan hệ kiểu con. Kiểm tra kiểu con trong GJ được thực hiện phức tạp hơn bởi cần thiết thực hiện phép thế kiểu và cần thiết tìm biến kiểu trong bảng kiểu; Thuật toán 7.5 chỉ ra điều đó thực hiện như thế nào.

Tìm gắn kết của kiểu. Các tham số kiểu hình thức của một lớp hay một phương thức đa hình là có dạng $hX \triangleleft N_i$. Bên trong lớp hoặc phương thức đó, giả sử có khai báo biến cục bộ $X \ x$, ở đó X là kiểu và x là biến. Khi kiểm tra kiểu biểu thức như truy cập trường $x.f$, chúng ta cần biết x là đối tượng của lớp nào. Chúng ta không cần biết chính xác x thuộc lớp nào – nó thuộc lớp X nào đó chưa biết – nhưng chúng ta biết rằng X là lớp con của N . Vì thế, chúng ta nhận kiểu X và tìm kiểm phạm vi của nó, N , trong bảng kiểu. Hàm `getBound( $T$ )` tìm kiểm phạm vi kiểu.

Tìm kiếm các kiểu của trường và phương thức. Khi kiểm tra kiểu truy cập đến trường $e.f$ hoặc lời gọi phương thức $e.m \langle T \rangle ()$, người ta cần phải tìm kiếm trường f hoặc phương thức m trong lớp của e . Điều này có thể bao gồm việc tìm kiếm qua các lớp cha, và trong quá trình này, các kiểu cần được thay thế cho các biến kiểu. Các tìm kiếm này được điều khiển bởi `fieldType` và `methodType` trong Thuật toán 7.6. Thuật toán 7.7 chỉ ra cách kiểm tra kiểu các biểu thức sử dụng tất cả các thủ tục hỗ trợ được định nghĩa ở đó.

THUẬT TOÁN 7.6: Tìm kiếm trường và phương thức.

```

getBound( $T$ ) =
    if  $T$  is a type variable  $X$ 
    then if  $(X \mapsto N)$  is in the type table then  $N$  else error
    else if  $T$  is a nonvariable type  $N$  then  $N$ 
    else error

fieldType( $f$ ; ) =
    if  $T$  is  $C \langle T_1, \dots, T_n \rangle$ 
    look in the class table for class  $C \langle X_1 \triangleleft N_1, \dots, X_n \triangleleft N_n \rangle \triangleleft N\{fields, \dots\}$ 
    if field  $f$  with type  $S$  is in  $fields$ 
        return  $[T_1, \dots, T_n/X_1, \dots, X_n]S$ 
    else return fieldType( $f$ ,  $[T_1, \dots, T_n/X_1, \dots, X_n]N$ )
    else error

methodType( $m$ ,  $T$ ) =
    if  $T$  is  $C \langle T_1, \dots, T_n \rangle$ 

```

```

    look in the class table for class  $C \langle X_1 \triangleleft N_1, \dots, X_n \triangleleft N_n \rangle \triangleleft N\{\dots, \text{methods}\}$ 
    if method  $m$  is in  $\text{methods}$ ; with the declaration
         $\langle Y_1 \triangleleft P_1, \dots, Y_k \triangleleft P_k \rangle \quad U \ m(U_1 \ x_1 \ \dots \ U_l \ x_l) \{\text{return } e\}$ 
    then return  $[T_1, \dots, T_n/X_1, \dots, X_n] ( \langle Y_1 \triangleleft P_1, \dots, Y_k \triangleleft P_k \rangle \ (U_1 \ x_1, \dots, U_l \ x_l) \rightarrow U)$ 
    else return  $\text{methodType}(m, [T_1, \dots, T_n/X_1, \dots, X_n]N)$ 

```

THUẬT TOÁN 7.7: Kiểm tra kiểu biểu thức. Các biểu thức với kiểu nguyên được bỏ qua, vì chúng được kiểm tra giống như trong MiniJava

```

checkExp( $e$ ) =
    if  $e$  is a variable  $x$ 
        look up  $(x \mapsto T)$  in the var table; return  $T$ 
    else if  $e$  is a field-selection  $e_0.f$ 
        return  $\text{fieldType}(f; \text{getBound}(\text{checkExp}(e_0)))$ 
    else if  $e$  is a method-call  $e_0.m \langle V_1, \dots, V_n \rangle (e_1, \dots, e^l)$ 
        call  $\text{methodType}(m, \text{getBound}(\text{checkExp}(e_0))),$  yielding
             $\langle Y_1 \triangleleft P_1, \dots, Y_k \triangleleft P_k \rangle \ (U_1 \ x_1, \dots, U^l \ x_l) \rightarrow U)$ 
        for each  $V_i$  do  $\text{checkType}(V_i)$ 
        for each  $V_i$  do  $\text{checkSubtype}(V_i, [V_1, \dots, V^k/Y_1, \dots, Y_k]P^i)$ 
        for  $i \in 1 \dots l$  do  $\text{checkSubtype}(\text{checkExp}(e_i), [V_1, \dots, V_k/Y_1, \dots, Y_k]U_i)$ 
        return  $[V_1, \dots, V_k/Y_1, \dots, Y_k]U$ 
    else if  $e$  is new  $N$ 
         $\text{checktype}(N/)$ 
    return  $N$ 

```

Thuật toán 7.8 chỉ ra cách kiểm tra các khai báo lớp và phương thức. Các tham số kiểu hình thức của lớp được bổ sung vào bảng kiểu, sau đó mọi phương thức được kiểm tra. Đối với mỗi phương thức, các tham số kiểu hình thức và các tham số giá trị thông thường của phương thức được bổ sung vào bảng kiểu và bảng giá trị, sau đó thân phương thức sẽ được kiểm tra kiểu. Bốn dòng kết thúc tại lời gọi cuối cùng đến checkSubtype được tạo để tin tưởng rằng phương thức ghi đè là đúng đắn; trong GJ cũng như trong Java, người ta chỉ có thể ghi đè phương thức với cùng đối số và kết quả trả về như trong lớp cha, nhưng đối với GJ ở đây có nhiều phép thực hiện để kiểm tra điều đó.

THUẬT TOÁN 7.8: Kiểm tra kiểu các khai báo lớp.

```

checkClass( $cdecl$ ) =
    suppose  $cdecl$  is class  $C \langle X_1 \triangleleft N_1, \dots, X_n \triangleleft N_n \rangle \triangleleft N\{\text{fields}, \text{methods}\}$ 
    add  $(X_1 \mapsto N_1, \dots, X_k \mapsto N_k)$  to the type table
     $\text{checkType}(N)$ 
    for each  $N_i$  do  $\text{checkType}(N_i)$ 
    for each  $mdecl$  in  $\text{methods}$ 
        suppose  $mdecl$  is  $\langle Y_1 \triangleleft P_1, \dots, Y_j \rangle \triangleleft P_j \quad T \ m(T_1 \ x_1 \ \dots \ T_l \ x_l)$ 
    {return  $e$ }
        add  $(Y_1 \mapsto P_1, \dots, Y_j \mapsto P_j)$  to the type table
         $\text{checkType}(T)$ 
        for each  $T_i$  do  $\text{checkType}(T_i)$ 
        for each  $P_i$  do  $\text{checkType}(P_i)$ 
        add  $(\text{this} \mapsto C \langle X_1, \dots, X_n \rangle)$  to the var table

```

```

    add ( $x_1 \mapsto T_1, \dots, x_l \mapsto T_l$ ) to the var table
    checkSubtype(checkType( $e$ ),  $T$ )
    suppose methodType( $m$ ,  $N$ ) is  $\langle Z_1 \triangleleft Q_1, \dots, Z_j \triangleleft Q_j \rangle$  ( $U_1 x_1, \dots, U_l$ 
 $x_l$ )  $\rightarrow U$ 
    for each  $T_i$  check that  $T_i = ([Y_1, \dots, Y_j/Z_1, \dots, Z_j] U^i)$ 
    for each  $P_i$  check that  $P_i = ([Y_1, \dots, Y_j/Z_1, \dots, Z_j] Q_i)$ 
    checkSubtype( $T$ ,  $[Y_1, \dots, Y_j/Z_1, \dots, Z_j] U$ )
    pop  $j$  most recent bindings from the type table
    pop  $l + 1$  most recent bindings from the var table
    pop  $k$  most recent bindings from the type table

```

ĐỌC THÊM

Ngôn ngữ Simula-67 của Dahl và Nygaard [Birtwistle và một số người khác. 1973] đưa ra khái niệm lớp, đối tượng, đơn kế thừa, phương thức tĩnh, kiểm tra bản tạo, typecase, và kỹ thuật tiền tố (*prefix*) để cài đặt đơn kế thừa tĩnh. Sau đó bổ sung chương trình song song coroutines và thu dọn rác.

Cohen [1991] đề xuất thể hiện (*display*) để kiểm tra thành viên lớp thời gian hằng số.

Các phương thức động và đa kế thừa xuất hiện trong Smalltalk [Goldberg và một số người khác. 1983], nhưng các cài đặt đầu tiên được sử dụng tìm kiếm chậm các lớp cha để xác định bản tạo phương thức. Rose [1988] và Connor và một số người khác [1989] tranh luận thuật toán tìm kiếm nhanh dựa trên bản băm trường và phương thức cho đa kế thừa. Việc sử dụng tô màu đồ thị trong cài đặt đa kế thừa thuộc về Dixon và một số người khác [1989]. Lippman [1996] chỉ ra đa kế thừa kiểu C++ được cài đặt như thế nào.

Chambers và một số người khác [1991] mô tả một số kỹ thuật để tạo ngôn ngữ kiểu động, không lớp thực thi hiệu quả: mô tả pseudo-class, đa phiên bản của bản tạo phương thức, và các tối ưu khác. Diwan và một số người khác [1996] mô tả các giải pháp tối ưu cho các ngôn ngữ kiểu tĩnh mà có thể thay thế các lời gọi phương thức động bằng các lời gọi hàm tĩnh.

Các ngôn ngữ hướng đối tượng truyền thống chọn bản tạo phương thức cho lời gọi $a.f(x, y)$ chỉ dựa trên lớp của phương thức *receiver* (a) mà không dựa trên các đối số (x, y) . Các ngôn ngữ với đa phương thức *multimethods* [Bobrow và một số người khác 1989] cho phép tìm kiếm động dựa trên kiểu của mọi đối số. Chambers và Leavens [1995] chỉ ra rằng làm sao có thể kiểm tra kiểu tĩnh cho đa phương thức; Amiel và một số người khác [1994] và Chen và Turau [1994] chỉ ra làm sao để tìm kiếm động đa phương thức hiệu quả.

Nelson [1991] mô tả Modula-3, Stroustrup [1997] mô tả C++, và Arnold và Gosling [1996] mô tả Java.

7.3 GIẢI QUYẾT KIỂM NGHIỆM

Bây giờ chúng ta xét đến khả năng, một chương trình có thể chạy đối với nhiều kiểu dữ liệu đầu vào khác nhau, đó là lập trình tổng quan (Generic programming)

DỊCH CÁC CHƯƠNG TRÌNH ĐA KIỂU

Sau khi chương trình đa hình được kiểm tra kiểu, nó cần được dịch sang mã máy. Ở đây có một số cách có thể làm được, bốn trong số đó chúng ta sẽ bàn đến ở phần sau:

Mở rộng: Không sinh mã cho các lớp tổng quan như `Cons<X>`; thay vào đó, tạo lớp `Cons` mới cho mỗi lớp mà ở đó `<X>` được khởi tạo.

Ép kiểu (Casting): Sinh chỉ một lớp `Cons` duy nhất, và sử dụng áp kiểu kiểm tra thời gian chạy kiểu Java để thao tác trên đó.

Tẩy xóa (Erasure): Sinh chỉ một lớp `Cons` duy nhất, và thao tác trực tiếp trên đó.

Truyền kiểu (Type-passing): Sinh mã cho lớp mẫu (template class) `Cons`, và truyền tham số kiểu trong thời gian chạy.

Mở rộng và ép kiểu có ưu điểm là chúng tương thích với máy ảo Java chuẩn; Tẩy xóa hiệu quả hơn nhưng không tương thích; và truyền kiểu có ưu điểm thú vị và nhược điểm của nó.

Mở rộng. Hoàn toàn có thể mở rộng các khởi tạo của lớp đa hình thành các lớp Java thông thường. Điều này được gọi là dịch không đồng nhất (*heterogenous*) của GJ sang Java, bởi vì lớp `Cons_Int` sẽ hoàn toàn khác và về bản chất không liên quan đến lớp `Cons_Bool`, v.v Các mẫu Templates trong C++ làm việc theo cách này khá tốt: Chúng được mở rộng thành các lớp thông thường. Mở rộng cũng giống như mở rộng trên hàng của các hàm thông thường; nhưng mở rộng trên hàng của các hàm không thể được làm đầy đủ như xóa mọi định nghĩa hàm, vì hàm đệ quy hoặc lời gọi hàm bên trong vòng lặp có thể mở rộng chương trình ra vô hạn. Ngược lại, khởi tạo của các lớp tổng quan không bao giờ phụ thuộc vào các biến của chương trình và không là đệ quy.

Các ưu điểm của mở rộng là các lớp kết quả nói chung tương thích với Java thông thường (vì vậy GJ có thể chạy trên máy ảo Java thông thường), và mã được dịch tương đối hiệu quả (tương tự như Java thông thường). Các nhược điểm là nó tạo ra nhiều bản sao của cùng một mã – trong trường hợp xấu nhất, nó có thể gây ra bùng nổ hàm mũ của chương trình, nhưng trên thực tế hành vi hàm mũ này rất hiếm khi thấy (mở rộng của các mẫu C++ templates là có thể chịu được). Cũng vậy, mở rộng tương tác kém với cơ chế gói của Java.

Ép kiểu. Trong bản dịch thuần nhất (*homogenous*) từ GJ sang Java, mọi tham số kiểu được xóa đi. Khi biến kiểu được sử dụng trong khai báo biến hoặc tham số cục bộ, nó được thay thế bởi gắn kết của nó. Như vậy, bản dịch của `GPair` có thể là

```
class GPair {
    Printable a;
    Printable b;
    void print_me() { a.print_me(); b.print_me(); }
}
```

mà là mã Java ở dạng tốt. Tuy nhiên, việc sử dụng `GPair` như

```
int sum(GPair<PrintableInt> p) {
    return p.a.x + p.b.x;
}
```

cần phải dịch với chuyển kiểu để trở thành mã Java hợp lệ:

```
int sum(GPair p) {
    return ((PrintableInt)(p.a)).x + ((PrintableInt)(p.b)).x;
}
```

Không may, việc ép kiểu từ lớp cha về lớp con đòi hỏi kiểm tra thời gian chạy trong Java thông thường, mặc dù (nếu chương trình Java nhận được từ việc dịch chương trình GJ ở dạng tốt) việc ép kiểu này luôn luôn thành công. Một nhược điểm nhỏ khác của việc dịch thuần nhất là cấu trúc lớp không thể áp dụng cho biến kiểu; tức là, `new X()`, trong đó `X` là biến kiểu, không thể được thay thế bằng `new C()`, trong đó `C` gắn kết với `X`, bởi vì lớp không đúng sẽ được tạo nên.

Tẩy xóa. Nếu chương trình GJ được dịch trực tiếp sang mã máy, bản dịch đồng nhất có thể được tạo an toàn mà không cần chèn ép kiểu: đơn giản chỉ xóa tham số kiểu khỏi chương trình. Ưu điểm là ở đây không có lặp của mã và không có ép kiểu thời gian chạy. Không may, Java đi vòng này cũng có nghĩa là Ngôn ngữ máy ảo Java (mã JVMCL hoặc “Java byte code”) cũng cần phải được bỏ qua, vì bộ kiểm tra Java bytecode sử dụng hệ thống kiểu Java, không phải hệ thống kiểu GJ. Kỹ thuật dịch này do đó không tương thích với máy ảo Java đang có mà nhận chương trình trong JVMCL.

Truyền kiểu. Thay vì xóa tham số kiểu, chúng có thể được chuyển thành tham số giá trị. Một phương thức đa hình

```
<X1 extends C1> int m (X1 x, int y)
```

có thể được dịch (xấp xỉ) thành

```
int m (Class X1, X1 x, int y)
```

trong đó mô tả lớp được truyền thực tế như đối số thời gian chạy. Một ưu điểm của bản dịch này là bây giờ cấu trúc lớp có thể được áp dụng cho biến kiểu.

Một ưu điểm quan trọng hơn là, về nguyên tắc, có thể tách mô tả lớp ra khỏi các đối tượng mà chúng thuộc vào các lớp đó. Mặt khác, thay vì đối tượng của lớp `PrintableInt` yêu cầu hai từ thể hiện nó – mô tả lớp và trường `x` – bây giờ chỉ có một từ bị đòi hỏi. Mọi chỗ trong chương trình mà thao tác `PrintableInt` cũng có thể có tham số lớp tường minh được truyền cho nó trong biến cục bộ liên kết; từ tham số này, bảng phương thức ảo có thể được truy cập nếu cần thiết, và thu dọn rác có thể học xem cái gì cần để biết về cách trình bày đối tượng.

Nhược điểm của truyền kiểu là ở đó phải trả giá thời gian chạy (nhỏ) cho việc truyền kiểu, và nó không tương thích với Java và máy ảo Java chuẩn.

CON TRỎ, SỐ NGUYÊN VÀ ĐÓNG HỘP

Đa hình trong GJ làm việc đối với các kiểu đối tượng, nhưng không cho `int` và `boolean`. Ngay cả trong Java thông thường, cơ chế mở rộng lớp làm việc cho đối tượng chứ không phải số nguyên. Giải pháp trong Java và GJ đối với các lập trình viên mà muốn có lợi ích từ lớp con và đa hình đối với số nguyên là tạo vỏ bọc là lớp số nguyên `Integer` mà chứa biến khởi tạo cho kiểu `int`. Điều này được gọi là số nguyên đóng hộp, so sánh với kiểu `int` thô, mà là giá trị *không đóng hộp*. Các giá trị đóng hộp – được cài đặt bằng các con trỏ đến các đối tượng chứa các giá trị thô – là dễ hơn nhiều các giá trị không đóng hộp để dịch đa hình:

- Chúng cùng một kích cỡ (kích thước con trỏ) sao cho cùng một mã máy có thể thao tác các giá trị đóng hộp của các kiểu khác nhau.
- Chúng có thể chứa kiểu hoặc mô tả lớp (trong đối tượng được trỏ) sao cho bộ thu dọn rác có thể hiệu duyệt chúng như thế.

Một số ngôn ngữ lập trình (như ML và C#) tự động đóng hộp các giá trị như việc cần thiết để hỗ trợ đa hình. Như vậy, các lập trình viên không bao giờ cần phân biệt giữa `int` và `Integer` vì chương trình dịch chèn chuyển kiểu tự động. Ở đây vẫn còn giá phải trả thời gian chạy cho đóng hộp và mở hộp, nhưng nó là cùng một giá mà phải trả nếu lập trình viên đóng tường minh các số nguyên vào các hộp.

PHÂN GIẢI TẢI CHỒNG TÍNH

Một số ngôn ngữ cho phép *tải chồng*: các hàm khác nhau với cùng tên nhưng các kiểu đối số khác nhau. Chương trình dịch cần chọn giữa các thân hàm dựa trên các kiểu của các tham số thực tế. Điều này đôi khi được biết đến như đa hình *đặc biệt*, trái ngược với đa hình *tham số* mô tả ở mục trước.

Tải chồng tính không khó cài đặt. Khi xử lý khai báo của hàm tải chồng f , một gán kết mới b_n cần phải không che các định nghĩa cũ $b_1, \dots, b_{n-1}$. Thay vào đó, gán kết mới ánh xạ f vào danh sách các cài đặt khác

nhau, $f \mapsto [b_1, \dots, b_n]$. Phụ thuộc vào ngữ nghĩa ngôn ngữ, nó có thể cần thiết cho thông báo lỗi nếu b_n có cùng kiểu tham số với một trong số b_i .

Sau đó, khi tìm kiếm f tại chỗ mà được gọi với các tham số thực tế, các kiểu tham số thực tế sẽ xác định gắn kết b_i cần được sử dụng.

Một số ngôn ngữ cho phép các hàm với cùng kiểu đối số (nhưng khác kiểu kết quả) được phép tải chồng; một số ngôn ngữ cho phép các dạng tải chồng động; xem mục đọc thêm.

ĐỌC THÊM

Ngôn ngữ “đa hình” đầu tiên là Lisp [McCarthy 1960], mà không có hệ thống kiểu tĩnh (tĩnh tức là, được kiểm tra trong thời gian dịch). Do đó, cài đặt đóng hộp hoàn toàn của dữ liệu được sử dụng, sao cho dữ liệu cần phải tự mô tả tốt nó cho kiểm tra kiểu thời gian chạy cũng như cho bộ thu dọn rác.

Ngôn ngữ lập trình đầu tiên sử dụng đa hình tham số được kiểm tra kiểu tĩnh là ML, mà bắt nguồn từ *MetaLanguage* của bộ chứng minh định lý Edinburgh [Gordon, 1978] nhưng sau đó phát triển thành ngôn ngữ lập trình đa năng [Milner, 1990]. Cardelli [1984] mô tả cài đặt đóng hộp đầy đủ của ML.

Trong ngôn ngữ lập trình Ada [Ada 1980], cơ chế *tổng quan* cho phép hàm (trên thực tế, cả gói) được tham số hóa trên các kiểu; nhưng kiểm tra kiểu đầy đủ được thực hiện tại mỗi lời gọi sau khi tổng quan được áp dụng cho các tham số thực tế, và kỹ thuật *mở rộng* của cài đặt cần phải được sử dụng. Ngược lại, Thuật toán 7.8 cần kiểm tra lớp tổng quan không phụ thuộc vào việc các tham số hình thức sẽ được khởi tạo như thế nào.

Pierce [2002] cung cấp tổng quan toàn diện về các hệ thống kiểu, bao gồm các kiểu đa hình, theo các ký hiệu hiện đại. Bracha và một số người khác. [1998] mô tả Generic Java (GJ) và cài đặt của nó. Thuật toán kiểm tra kiểu trong mục trên của chương này được hiệu chỉnh từ "Featherweight Generic Java" [Igarashi, 2001], mà cần được đọc bởi bất cứ ai định cài đặt bộ kiểm tra kiểu như vậy.

Tải chồng. Ada cho phép các hàm khác nhau với các kiểu tham số như nhau được tải chồng, khi các kiểu kết quả là khác nhau. Khi đầu ra của hàm như trên là đối số của hàm tải chồng khác, thì có thể có không, một hoặc nhiều cài đặt khác nhau một biểu thức; ngữ nghĩa Ada cho biết rằng một biểu thức là hợp lệ chỉ khi chỉ có chính xác một bản dịch. Aho [1986, mục 6.5] bàn luận vấn đề này và đưa ra thuật giải. Nhưng tải chồng phong cách Ada không được làm theo rộng rãi trong thiết kế các ngôn ngữ hiện tại, có thể vì nó gây rắc rối cho lập trình viên.

Tải chồng động cho phép các cài đặt khác nhau của một hàm được chọn dựa trên kiểu trong thời gian chạy của tham số thực tế; nó là một dạng của *cấp phát động*. Cấp phát động cũng được sử dụng để cài đặt phương thức ghi đè, một khái niệm cơ bản của lập trình hướng đối tượng – ghi đè là một dạng cấp phát động dựa trên tham số `this`, trong khi tải động tổng quan cần phụ thuộc vào bất kỳ hay mọi tham số của hàm. *Các lớp kiểu*

trong ngôn ngữ Haskell cho phép đa hình tải chồng và tham số có thể tương tác theo cách có ích và tùy chọn [Hall, 1996].

CÁC BIẾN KHÔNG THAY ĐỔI (IMMUTABLE VARIABLES)

Ngôn ngữ FunJava có các hàm bậc cao với phạm vi lồng nhau, nhưng vẫn không có khả năng thực tế sử dụng suy luận đẳng thức cho các chương trình FunJava. Chẳng hạn, $f(3)$ có thể mỗi lần trả về giá trị khác nhau. Để cứu chữa tình huống này, chúng ta cấm các tác động phụ của các hàm số: Khi hàm được gọi, nó cần phải trả về kết quả mà không thay đổi “thế giới” theo mọi cách quan sát.

Như vậy, chúng ta tạo ra một *ngôn ngữ lập trình hàm thuần túy* mới PureFunJava, ở đó các điều sau đây là cấm:

- ⊗ Gán cho các biến (trừ khởi tạo trong khai báo biến);
- ⊗ Gán cho các trường của các bản ghi cấp trên heap (trừ khởi tạo trong hàm tạo các lớp);
- ⊗ Các lời gọi đến các hàm bên ngoài mà có tác động nhìn thấy được: `println`.

Để phân biệt rõ ràng giữa khởi tạo các biến bản tạo (mà cho phép) và cập nhật các biến bản tạo (mà không cho phép), chúng ta đòi hỏi rằng mỗi lớp có hàm tạo ở dạng in đặc biệt mà khởi tạo mọi biến bản tạo:

```
ClassDecl  → class id { VarDecl* MethodDecl* Constructor }
Constructor → public id ( FormalList ){ Init* }
Init       → this . id = id
```

Điều này trông có vẻ hà khắc: thế thì làm sao chương trình có thể làm việc được? Để lập trình mà không có lệnh gán, trong phong cách lập trình hàm, bạn tạo ra các giá trị mới thay vì cập nhật các giá trị cũ. Chẳng hạn, Chương trình 7.9 cho biết việc cài đặt cây tìm kiếm nhị phân theo phong cách mệnh lệnh và hàm. Như giải thích trong phân tích cú pháp, chương trình mệnh lệnh cập nhật đỉnh của cây, nhưng chương trình hàm trả về cây mới giống như cây cũ, dường như đường đi từ gốc đến lá “mới” được copy. Nếu ta giả thiết t_1 là cây trên Hình 6.14a, ta có thể nói

CHƯƠNG TRÌNH 7.9: Các cây tìm kiếm nhị phân được cài đặt theo hai cách

```
class tree {
    String key; int binding; tree left; tree right;

    public tree(String key, int binding, tree left, tree right) {
        this.key=key; this.binding=binding;
        this.left=left; this.right=right;
    }
}
```

```
public int look(String k) {
    int c = key.compareTo(k);
    if (c < 0) return left.look(k);
    else if (c > 0) return right.look(k);
    else return binding;
}

public void enter(String k, int b) {
    int c = key.compareTo(k);
    if (c < 0)
        if (left==null)
            left = new tree(k,b,null,null);
        else left.enter(k,b);
    else if (c > 0)
        if (right==null)
            right = new tree(k,b,null,null);
        else right.enter(k,b);
    else binding=b;
}
}
```

(a) Imperative, object-oriented Java

```
// Alternative implementation of enter
public tree enter(String k, int b) {
    int c = key.compareTo(k);
    if (c < 0)
        if (left==null)
            return new tree(k,b,null,null);
        else return left.enter(k,b);
    else if (c > 0)
        if (right==null)
            return new tree(k,b,null,null);
        else return right.enter(k,b);
    else return new tree(k,b,left,right);
}
```

(b) Functional, object-oriented Java

```
int t2 = t1.enter("mouse",4);
```

và bây giờ `t1` và `t2` cả hai đều sẵn sàng để chương trình sử dụng. Mặt khác, nếu chương trình trả về `t2` như kết quả của hàm và bỏ qua `t1`, thì đỉnh gốc của `t1` sẽ được giải phóng bởi dọn rác (các đỉnh khác của `t1` sẽ không được giải phóng, vì chúng còn được sử dụng bởi cây `t2`).

Các kỹ thuật tương tự có thể cho phép chương trình hàm biểu diễn cùng sắc thái phong phú của các thuật toán mà chương trình mệnh lệnh có thể, và thường là rõ ràng, chính xác hơn.

VÀO RA DỰA TRÊN TIẾP DIỄN (CONTINUATION-BASED I/O)

Tạo ra các cấu trúc dữ liệu mới thay vì cập nhật các cái cũ làm cho nó có thể tuân thủ nguyên tắc “không phép gán”, nhưng chương trình thực hiện vào ra như thế nào? Kỹ

thuật vào ra “*dựa trên tiếp diễn*” thể hiện trong khung hàm. Như chỉ ra trong Chương trình 7.10, các kiểu và hàm định nghĩa trước trong PureFunJava dựa trên khái niệm trả lời (*answer*): Đây là “kết quả” trả về bởi *toàn bộ chương trình*.

CHƯƠNG TRÌNH 7.10: Các kiểu và hàm đã định nghĩa trước cho PureFunJava.

```
type answer    // a special, built-in type
type intConsumer = int -> answer
type cont = () -> answer

class ContIO {
    public answer readByte (intConsumer c);
    public answer putByte (int i, cont c);
    public answer exit();
}
```

MiniJava không có hàm nhập dữ liệu, nhưng nếu có thực hiện, kiểu sẽ là trực tiếp: một cái gì đó giống như `int readByte()`. Để biểu diễn điều này mà không có tác động phụ, `readByte` của PureFunJava nhận đối số mà là `int-Consumer` và truyền số nguyên mới đọc cho khách hàng (consumer) đó. Cho dù trả lời thế nào mà khách hàng tạo ra, sẽ cũng là trả lời *answer* của `readByte`.

Tương tự, `putByte` nhận ký tự để in cũng tốt như *continuation* (`cont`); `putByte` xuất ra ký tự và sau đó gọi `cont` để tạo ra trả lời.

Điểm được của việc sắp xếp lại này là cho phép thực hiện *input/output* trong khi vẫn giữ được suy luận đẳng thức. Thử ví là, *input/output* bây giờ được “nhìn thấy” cho kiểm tra kiểu (typechecker): Hàm bất kỳ mà thực hiện I/O sẽ có *answer* trong kiểu trả về của nó.

CÁC THAY ĐỔI NGÔN NGỮ

Các điều chỉnh sau của FunJava tạo thành ngôn ngữ mới PureFunJava:

- Bổ sung các kiểu và lớp định nghĩa trước chỉ ra trong Chương trình 7.10, và loại bỏ `System.out.println`.
- Các lệnh gán và vòng lặp **while** được xóa bỏ khỏi ngôn ngữ, và mỗi lệnh phức (trong ngoặc móc) có thể có chỉ một lệnh sau khai báo biến của nó.

Chương trình 7.11 chỉ ra chương trình hoàn toàn trong PureFunJava program mà lặp, đọc số nguyên và in giai thừa của mỗi số nguyên, cho đến khi có số nguyên lớn hơn 12 như đầu vào.

CHƯƠNG TRÌNH 7.11: Chương trình PureFunJava để: read i, print i!.

```
class Factorial {
    boolean isDigit (int c) {
        return c >= 48 && c <= 48+9; // 48 == (int)'0'
    }

    public answer getInt(intConsumer done) {
```

```
public answer nextDigit(int accum) {
    public answer eatChar(int dig) {
        return if (isDigit(dig))
            nextDigit(accum*10+dig-48)
        else done(accum);
    }
    return ContIO.readByte(eatChar);
}
return nextDigit(0);
}

answer putInt(int i, cont c) {
    return if (i==0)
        c()
    else {
        int rest = i/10;
        int dig = i - rest * 10;
        public answer doDigit() { return ContIO.putByte(dig,c); }
        return putInt(rest, doDigit);
    };
}

int factorial (int i) {
    return if (i==0) 1 else i * factorial(i-1);
}

answer loop (int i) {
    return if (i > 12) ContIO.exit()
    else {
        public answer next() { return getInt(loop); }
        return putInt(factorial(i), next);
    };
}

public static answer main (String [] argv) {
    return getInt(loop);
}
}
```

TỐI ƯU NGÔN NGỮ HÀM THUẦN TÚY

Vì chúng ta chỉ xóa một số đặc trưng của FunJava, và không bổ sung bất cứ cái gì mới (ngoại trừ thay đổi một số kiểu định nghĩa trước), chương trình dịch FunJava của chúng ta có thể dịch PureFunJava vẫn đúng. Và, nói chung, chương trình dịch ngôn ngữ hàm có thể sử dụng các kiểu tối ưu như chương trình dịch ngôn ngữ mệnh lệnh: mở rộng inline, lựa chọn chỉ lệnh, phân tích bất biến vòng lặp, cấp thanh ghi đồ thị tô màu, lan truyền copy, và tiếp tục như vậy. Tính đồ thị điều khiển luồng có thể phức tạp hơn một chút, tuy nhiên, vì phần nhiều điều khiển luồng biểu diễn qua các lời gọi hàm, và một số trong các lời gọi đó có thể là biến hàm thay vì các hàm được định nghĩa tĩnh.

Chương trình dịch PureFunJava có thể tạo một số kiểu tối ưu mà chương trình dịch FunJava không thể làm được, vì nó có ưu điểm của suy luận đẳng thức.

Xét đoạn chương trình sau, mà xây dựng bản ghi `r` và sau đó lấy các trường từ nó:

```
class recrd {int a; int b;
    public recrd(int a, int b) {this.a=a; this.b=b;}
}

int a1 = 5;
int b1 = 7;
recrd r = new recrd(a1,b1);

int x = f(r);

int y = r.a + r.b;
```

Trong ngôn ngữ hàm thuần túy, chương trình dịch biết được là khi nào tính toán `y` tham chiếu đến `r.a` và `r.b`, nó đến để nhận các giá trị `a1` và `b1`. Trong ngôn ngữ mệnh lệnh (hoặc hàm không thuần túy), tính toán `f(r)` có thể gán giá trị mới cho các trường của `r`, nhưng không phải trong PureFunJava.

Vì vậy, trong phạm vi của `r` mỗi lần xuất hiện `r.a` có thể được thay bằng `a1`, và tương tự `b1` có thể được thay cho `r.b`. Cũng vậy, vì không có phần khác của chương trình có thể gán giá trị mới cho `a1`, nó sẽ chứa cùng giá trị (5) mọi lúc. Vậy, 5 có thể thế cho `a1` mọi nơi và 7 cho `b1`. Do đó, chúng ta kết thúc với `int y = 5+7`, mà có thể đổi thành `int y = 12`; nên, 12 có thể thế cho `y` trong suốt phạm vi của nó.

Cùng kiểu phép thế cũng làm việc cho các ngôn ngữ mệnh lệnh; chỉ có điều là chương trình dịch cho ngôn ngữ mệnh lệnh thường xuyên không tin tưởng là một trường hoặc biến có được cập nhật giữa điểm định nghĩa và điểm sử dụng không. Do đó, cần phải giả thiết cẩn thận là biến có thể được cập nhật và vì vậy trong đa số trường hợp, phép thế không thể được thực hiện.

Ngôn ngữ ML có bản ghi hàm thuần túy, mà không thể bị cập nhật và ở đó phép thế luôn luôn đúng, và cũng có các ô tham chiếu cập nhật được, mà có thể được gán và chúng làm việc giống như bản ghi trong các ngôn ngữ mệnh lệnh truyền thống.

MỞ RỘNG TRÊN DÒNG (INLINE EXPANSION)

Vì các chương trình hàm hướng tới việc sử dụng các hàm nhỏ, và đặc biệt chúng truyền hàm từ một chỗ sang chỗ khác, một kỹ thuật tối ưu quan trọng là mở rộng trên dòng của lời gọi hàm: thay lời gọi hàm bằng bản sao của thân hàm.

Chẳng hạn, trong Chương trình 7.12, `observeInt` là một hàm bất kỳ (giống như `putInt` của Chương trình 7.11) mà "quan sát" số nguyên và sau đó sẽ tiếp tục. `doList` là hàm mà áp dụng quan sát `f` đến danh sách `l`, và sau đó tiếp tục. Trong trường hợp này, quan sát không phải là `putInt` mà là `printDouble`, mà in `i` được nối tiếp bởi `2i`. Do đó, `printTable` in bảng các số nguyên, mỗi số được nối tiếp bởi nhân đôi của nó.

CHƯƠNG TRÌNH 7.12: printTable trong PureFunJava.

```
class list {int head; list tail;
    public list (int head, list tail) {
        this.head=head; this.tail=tail;
    }
}

type observeInt = (int,cont) -> answer;

class PrintT {
    public answer doList (observeInt f, list l, cont c) {
        return if (l==null)
            c();
        else {
            public answer doRest() {return doList(f, l.tail, c);}
            return f(l.head, doRest);
        }
    }

    public int double(int j) {return j+j;}

    public answer printDouble(int i, cont c) {
        public answer again() {return putInt(double(i), c);}
        return putInt(i, again);
    }

    public answer printTable(list l, cont c) {
        return doList(printDouble, l, c);
    }

    public static void main(string argv[]) {
        list mylist = ... ;
        return printTable(mylist, IO.exit);
    }
}
```

Để so sánh, Chương trình 7.13a là chương trình Java bình thường mà cũng làm cùng một việc như trên.

CHƯƠNG TRÌNH 7.13: Cài đặt Java của printTable.

```
class list {int head; int tail;}

class PrintT {
    int double(int j) {return j+j;}

    void printDouble(int i) {
        putInt(i); putInt(double(i));
    }

    void printTable(list l) {
        while (l != null) {
            int i = l.head;
            putInt(i);
            putInt(i+i);
        }
    }
}
```

<pre> printDouble(l.head); l = l.tail; } public static void main(...){ printTable(mylist); } </pre>	<pre> l = l.tail; } public static void main(...){ printTable(mylist); } </pre>
(a) As written	(b) Optimized

Chương trình 7.12 sử dụng duyệt danh sách tổng quát (generic list-traverser), `doList`, mà mọi hàm có thể dính vào. Tuy trong trường hợp này `printDouble` được sử dụng, cũng chương trình như vậy có thể tái sử dụng `doList` cho mục đích khác như hoặc "quan sát" mọi số nguyên trong danh sách. Nhưng Chương trình 7.13a mất sự linh hoạt này – nó gọi trực tiếp `printDouble`.

Nếu dịch một cách ngây thơ, chương trình hàm thuần túy – mà truyền `printDouble` như một đối số - sẽ thực hiện được rất nhiều các lời gọi hàm hơn so với chương trình chỉ lệnh. Bằng cách sử dụng mở rộng dòng lệnh và tối ưu lời gọi đuôi, Chương trình 7.12 có thể được tối ưu thành các chỉ lệnh máy tương đương với vòng lặp hiệu quả của Chương trình 7.13b.

Loại bỏ nắm bắt biến (Avoiding variable capture). Chúng ta cần quan tâm về các tên biến khi thực hiện dòng lệnh trong MiniJava (hoặc Java), ở đó khai báo cục bộ tạo nên "lỗ hổng" trong phạm vi của các biến ngoài cùng tên:

```

class A {
1 int x = 5
2 int function g(int y) {
3     return y+x;
4 }
4 int f(int x) {
5     return g(1)+x;
6 }
6 void main() { ... f(2)+x ... }
}

```

Tham số hình thức `x` trên dòng 4 tạo lỗ hổng trong phạm vi của biến `x` khai báo ở dòng 1, do đó `x` ở dòng 5 tham chiếu đến tham số hình thức, chứ không phải biến. Nếu chúng ta mở rộng trên dòng cho lời gọi hàm `g(1)` trên dòng 5 bằng việc thay thân hàm `g` cho lời gọi, chúng ta không đơn giản chỉ viết `1+x`, mà cần phải viết như sau

```

4 int f(int x) {
5     return return 1+x;+x; }

```

nhưng `x` đầu tiên trên dòng 5 bây giờ tham chiếu không đúng đến tham số của `f` thay vì đến biến khai báo ở dòng 1.

Để giải quyết vấn đề này, chúng ta cần trước hết đổi tên, hoặc α -convert, tham số hình thức của f , sau đó thực hiện phép thế:

<pre> 2 int function g(int y) { 3 return y+x; 4 } 4 int f(int a) { 5 return g(1)+a; 6 }</pre>	<pre> int function g(int y) { return y+x; } int f(int a) { return {return 1+x;}+a; }</pre>
---	--

Có cách khác, chúng ta cần đổi tên các tham số thực sự thay vì các tham số hình thức, và định nghĩa hàm thế để tránh thế x bên trong phạm vi định nghĩa mới của x .

Nhưng giải pháp tốt nhất cho việc bao quát biến là trước khi dịch đổi tên tất cả các biến sao cho cùng một tên biến không bao giờ khai báo hai lần. Nó làm đơn giản việc suy luận và tối ưu chương trình.

Bằng cách như vậy, biểu thức `{return 1+x;}` trên dòng 5 hoàn toàn tương đương với biểu thức `(1+x)`.

Các qui tắc trên dòng (Rules for inlining). Thuật toán 7.14 cho các qui tắc để mở rộng trên dòng, mà có thể áp dụng cho lập trình mệnh lệnh hoặc lập trình hàm. Thân hàm B được sử dụng ở vị trí lời gọi hàm $f(\dots)$, nhưng bên trong bản sao của B , mỗi tham số thực tế được thay cho tham số hình thức tương ứng. Khi tham số thực tế chỉ là biến hoặc hằng số, thì phép thế rất đơn giản (Thuật toán 7.14a). Nhưng khi tham số thực tế là biểu thức không tầm thường, chúng ta cần trước hết gán nó cho biến mới (Thuật toán 7.14b).

THUẬT TOÁN 7.14: Mở rộng trên dòng của thân hàm. Chúng ta giả thiết rằng không có hai khai báo khai báo cùng một tên.

(a) Khi các tham số thực tế là các biến đơn giản $i_1, \dots, i_n$. Bên trong phạm vi của:	(b) Khi các tham số thực tế là các biểu thức không tầm thường, không chỉ là biến. Bên trong phạm vi của:
<code>int $f(a_1, \dots, a_n)B$</code>	<code>int $f(a_1, \dots, a_n)B$</code>
(where B is a <i>Compound</i>) the expression	the expression <code>$f(E_1, \dots, E_n)$</code> rewrites to
<code>$f(i_1, \dots, i_n)$</code>	{ <code>int $i_1 = E_1$;</code>
rewrites to	:
<code>$B[a_1 \mapsto i_1, \dots, a_n \mapsto i_n]$</code>	<code>int $i_n = E_n$;</code>
	<code>return $B[a_1 \mapsto i_1, \dots, a_n \mapsto i_n]$;</code>
	}
	ở đây $i_1, \dots, i_n$ là các tên chưa được sử dụng trước đây.

Chẳng hạn, trong Chương trình 7.13 lời gọi hàm `double(i)` được thay bằng bản sao `j+j` ở đó mỗi `j` được thay bằng tham số hình thức `i`. Ở đây chúng ta sử dụng Thuật toán 7.14a, vì `i` là biến, không phải là biểu thức phức tạp.

Giả sử chúng ta mở rộng trên hàng `double(g(x))`; nếu ta sử dụng không đúng Thuật toán 7.14a, ta nhận được `g(x)+g(x)`, mà tính `g(x)` hai lần. Ngay cả suy luận đẳng thức đảm bảo rằng chúng ta sẽ tính ra cùng một kết quả trong mỗi lần, chúng ta không muốn làm chậm tính toán bằng cách tính lặp hàm `g(x)` (có thể trả giá đắt). Thay vào đó, Thuật toán 7.14b thực hiện:

```
{int i = g(x); return i+i;}
```

mà tính `g(x)` chỉ một lần.

Trong lập trình mệnh lệnh, không chỉ `g(x)+g(x)` chậm hơn

```
{int i = g(x); return i+i;}
```

mà – còn vì `g` có thể có tác động phụ - có thể tính ra kết quả khác! Một lần nữa, Thuật toán 7.14b lại làm đúng.

Loại bỏ hàm chết. Nếu mọi lời gọi đến một hàm (như `double`) cần phải mở rộng trên hàm, và nếu hàm không truyền như đối số hoặc được tham chiếu theo bất kỳ cách nào khác, bản thân hàm đó có thể được xóa.

Các hàm đệ qui trên hàng. Viết `doList` dạng trên hàng vào `printTable` tạo thành phiên bản mới của `printTable`:

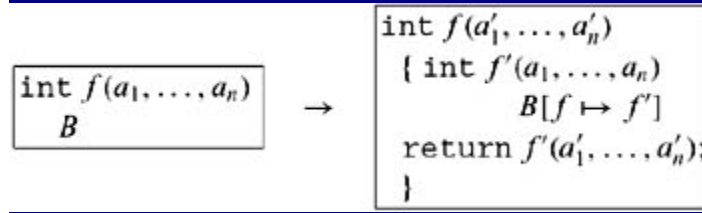
```
public answer printTable(list l, cont c) {
    return if (l==null)
        c();
    else {
        public answer doRest() {
            return doList(printDouble, l.tail, c);
        }
        return printDouble(l.head, doRest);
    }
};
```

Điều này không tốt lắm: `printTable` gọi `printDouble` trên `l.head`, nhưng đến tiến trình `l.tail` nó gọi `doList` như trước kia. Do đó, chúng ta phải mở rộng trên hàng *chỉ phần đầu của vòng lặp*. Chúng ta có thể có phương án riêng đầy đủ của `doRest`; tuy nhiên, chúng ta không thực hiện mở rộng trên dòng theo cách này.

Đối với các hàm đệ qui chúng ta sử dụng phép biến đổi trước vòng lặp (*loop-preheader transformation*) (Thuật toán 7.15). Ý tưởng là tách *f* thành hai hàm: *prelude* được gọi từ bên ngoài, và *loop header* được gọi từ bên trong. Mỗi lời gọi đến đầu vòng lặp sẽ là lời

gọi đệ qui từ bên trong bản thân nó, ngoại trừ đối với lời gọi đơn từ prelude. Áp dụng phép biến đổi này cho `doList` như sau

THUẬT TOÁN 7.15: Phép biến đổi Loop-preheader.



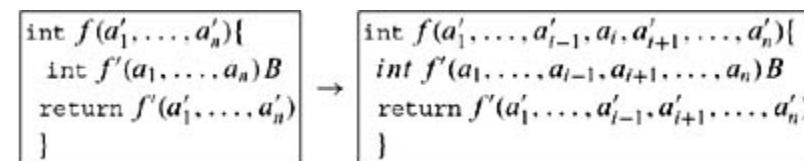
```
public answer doList (observeInt fX, list lX, cont cX) {
  public answer doListX(observeInt f, list l, cont c) {
    return if (l==null)
      c();
    else {
      public answer doRest() {return doListX(f, l.tail, c);}
      return f(l.head, doRest);
    };
  }
  return doListX(fX, lX, cX);
}
```

trong đó `doList` mới là prelude, và `doListX` là loop header. Nhận thấy rằng hàm function chứa cả vòng lặp như hàm bên trong, sao cho khi lời gọi bất kỳ đến `doList` là mở rộng trên hàng, bản copy mới `doListX` sẽ đi kèm cùng nó.

Các đối số bất biến vòng lặp. Trong ví dụ này, hàm `doListX` được truyền quanh các giá trị `f` và `c` mà là bất biến – chúng là không đổi trong mỗi lời gọi đệ qui. Trong mỗi trường hợp, `f` là `fX` và `c` là `cX`. Phép biến đổi kéo bất biến vòng lặp (*loop-invariant hoisting*) (Thuật toán 7.16) cần phải thay mỗi lần sử dụng `f` bằng `fX`, và `c` bằng `cX`.

THUẬT TOÁN 7.16: Loop-invariant hoisting.

Nếu mỗi f' bên trong B có dạng $f'(E_1, \dots, E_{i-1}, a_i, E_{i+1}, \dots, E_n)$ sao cho đối số thứ i luôn là a_i , thì viết lại



Trong mỗi lời gọi $f'(E_1, \dots, E_{i-1}, a_i, E_{i+1}, \dots, E_n)$ bên trong B được viết lại $f'(E_1, \dots, E_{i-1}, E_{i+1}, \dots, E_n)$.

Áp dụng phép biến đổi này cho `doList` ta được

```
public answer doList (observeInt f, list lX, cont c) {
  public answer doListX(list l) {
    return if (l==null)
      c();
    else {
      public answer doRest() {return doListX(l.tail);}
      return f(l.head, doRest);
    };
  }
  return doListX(lX);
}
```

Cuối cùng, trong `printTable`, khi lời gọi `doList(printDouble, l, c)` được viết trên dòng, ta nhận được:

```
public answer printTable(list l, cont c) {
  public answer doListX(list l) {
    return if (l==null)
      c();
    else {
      public answer doRest() {return doListX(l.tail);}
      return printDouble(l.head, doRest);
    };
  }
  return doListX(l);
}
```

Viết trên dòng truyền tiếp. Trong phiên bản này của `printTable`, chúng ta có `printDouble` được áp dụng cho các đối số (thay vì chỉ truyền cho `doList`), do đó chúng ta có thể mở rộng trên dòng lời gọi đó

```
public answer printTable(list l, cont c) {
  public answer doListX(list l) {
    return if (l==null)
      c();
    else {
      public answer doRest() {return doListX(l.tail);}
      return {
        int i = l.head;
        public answer again() {return putInt(i+i, doRest);}
        return putInt(i, again);
      };
    };
  }
  return doListX(l);
}
```

Tránh bùng nổ code. Mở rộng trên dòng sẽ nhân bản các thân hàm. Điều này nói chung làm cho chương trình to hơn. Nếu làm một cách tùy ý, kích thước của chương trình sẽ bùng nổ; trên thực tế, dễ dàng xây dựng các trường hợp mà mở rộng một lời gọi hàm tạo nên các khởi tạo mới mà có thể mở rộng ra vô hạn.

Có một số chỉ dẫn trực quan có thể được sử dụng để kiểm soát trên dòng:

1. Mở rộng chỉ các lời gọi hàm mà thường xuyên thực hiện; xác định tần suất hoặc qua đánh giá tĩnh (độ sâu vòng lặp lồng nhau) hoặc bằng phản hồi từ hồ sơ thực thi.
2. Mở rộng các hàm với các thân hàm rất nhỏ, sao cho thân hàm được nhân bản không lớn hơn các chỉ dẫn mà phải gọi hàm.
3. Mở rộng các hàm chỉ được gọi một lần; sau đó *cơ chế loại hàm chết* sẽ xóa bản copy gốc của thân hàm.

Các ngoặc không lồng nhau. Vì biểu thức FunJava

```
{ Decl1 return { Decl2 return Exp }}
```

Là chính xác tương đương với

```
{ Decl1 Decl2 return Exp }
```

Chúng ta kết thúc với Chương trình 7.17.

CHƯƠNG TRÌNH 7.17: `printTable` được đặc tả tự động.

```
1 public answer printTable(list l, cont c) {  
2     public answer doListX(list l) {  
3         return if (l==null) c()  
4         else {public answer doRest() {  
5             return doListX(l.tail);}  
6             int i = l.head;  
7             public answer again() {  
8                 return putInt(i+i,doRest); }  
9                 return putInt(i,again);  
10        }  
11        return doListX(l);  
12    }  
}
```

Tối ưu buộc chương trình được viết với trừu tượng (với `doList` đa năng) và biến đổi nó cho hiệu quả hơn, chương trình đa năng (với `doListX` chuyên dụng mà gọi trực tiếp `putInt`).

BÀI TẬP

***7.1** Vấn đề với kỹ thuật hiển thị *display* (như giải thích trong chương) cho việc kiểm tra thành viên của lớp là độ sâu tối đa lồng nhau lớp N cần được cố định trước, và mỗi

mô tả lớp cần N từ của không gian ngay cả đa số lớp không lồng nhau sâu. Thiết kế phương án khác của kỹ thuật hiển thị mà không phải chịu đựng vấn đề này; nó sẽ có một cấp chỉ lệnh có giá cao hơn chỉ lệnh mô tả trong mục 7.1.

7.2 Kỹ thuật bảng băm tìm lẻ trường và bản nhập phương thức vì có đa kế thừa được chỉ ra một cách không đầy đủ trường hợp $f \neq ptr_x$ không được giải quyết. Chọn kỹ thuật giải quyết xung đột, giải thích nó làm việc như thế nào và phân tích giá bổ sung (về chỉ lệnh) trong trường hợp $f = ptr_x$ (không xung đột) và $f \neq ptr_x$ (xung đột).

***7.3** Xét phân cấp lớp sau, mà chứa năm nơi gọi hàm. Nhiệm vụ là chỉ ra nơi gọi phương thức nào gọi bản tạo phương thức đã biết và trong từng trường hợp chỉ ra bản tạo phương thức nào. Chẳng hạn, bạn có thể nói rằng "bản tạo phương thức x_g luôn gọi y_f ; phương thức z_g có thể gọi nhiều hơn một bản tạo của f ."

```
class A { int f() { return 1; } }
class B extends A { int g() { this.f(); return 2; } }
class C extends B { int f() { this.g(); return 3; } }
class D extends C { int g() { this.f(); return 4; } }
class E extends A { int g() { this.f(); return 5; } }
class F extends E { int g() { this.f(); return 6; } }
```

Phân tích mỗi một giả thiết sau:

- Đây là toàn bộ chương trình, và ở đây không còn lớp con nào của các module này.
- Đây là một phần của chương trình lớn, và một trong các lớp này có thể được mở rộng ở đâu đó.
- Các lớp C và E là cục bộ đối với module này, và không thể được mở rộng ở chỗ khác; các lớp khác có thể được mở rộng.

***7.4** Sử dụng nhân bản phương thức để làm tốt hơn phân tích chương trình của bạn trong bài tập 7.3. Như là tạo cho mỗi lớp ghi đề f và g . Chẳng hạn, trong lớp B (mà đang chưa ghi đề f), đặt bản sao phương thức A_f , và trong D đặt bản sao C_F :

```
class B extends A { ...int f() { return 1; } }
class D extends C { ...int f() { this.g(); return 3; } }
```

Tương tự, bổ sung bản tạo mới E_f , F_f , và C_g . Bây giờ, đối với mỗi tập các giả thiết (a), (b), và (c), chỉ ra các lời gọi phương thức đến các bản tạo tĩnh đã biết.

****7.5** Nghĩ ra cơ chế cài đặt hiệu quả cho bất kỳ **typecase** mà chỉ đề cập đến các lớp **final**. Lớp **final** là lớp mà không mở rộng được. (Trong Java, có từ khóa **final**; nhưng ngay cả trong các ngôn ngữ hướng đối tượng khác, lớp mà không được xuất từ một module nào đó là **final** cách hiệu quả, và phân tích cả chương trình trong thời gian liên kết có thể phát hiện các lớp nào không bao giờ được mở rộng, mặc dù có khai báo **final** hay không).

Bạn có thể đưa ra bất cứ giả thiết nào sau đây, nhưng khẳng định giả thiết nào mà bạn cần sử dụng:

- a. Bộ liên kết Linker kiểm soát việc thay thế các bản ghi mô tả lớp.
- b. Mô tả là các số nguyên được quản lý bởi bộ liên kết linker mà đánh chỉ số vào bảng các bản ghi mô tả.
- c. Chương trình dịch đánh dấu tường minh các lớp `final` (trong mô tả của chúng).
- d. Mã cho **typecase** có thể được sinh trong thời gian liên kết.
- e. Sau khi chương trình chạy, không lớp nào và lớp con nào khác được liên kết động đến chương trình.

7.6 Nêu các bước trong kiểm tra kiểu khai báo của `append` sử dụng Thuật toán 7.8.

***7.7** Đọc mục Phân tích cú pháp dự báo 3.2 của Igarashi [2001] và chỉ ra cách mở rộng các Thuật toán 7.7 và 7.8 để kiểm soát việc phân tích kiểu của các biểu thức chuyển kiểu và của các hàm tạo lớp.

7.8 Sử dụng phép chuyển không đồng nhất (tức là, mở rộng) để chuyển chương trình GJ sau thành Java thông thường.

```
package p;  
  public class C <X extends Object> { X a; B b; }  
  class B { }  
package q;  
  class D { }  
  class E { p.C<D> y; p.C<Object> z; }
```

- a. Trước hết thực hiện phép chuyển loại bỏ các khai báo `package`, từ khóa `public`, và định tính `p.` bên trong lớp class E. **Hướng dẫn:** Phép chuyển sẽ không có lớp C, nhưng sẽ có các lớp `C_D` và `C_Object`.
- b. Bây giờ, thử chuyển giữ nguyên cấu trúc gói. Chỉ ra rằng không thể có B là phạm vi gói trong p tại cùng thời điểm D là phạm vi gói trong q.

CHƯƠNG 8: MÔI TRƯỜNG TÍNH TOÁN

TỔNG QUAN

Trong chương này quản lý bộ nhớ đối với các ngôn ngữ lập trình sẽ được mô tả bằng cấu trúc dữ liệu thời gian thực mà được sử dụng trong cài đặt tham chiếu đơn giản. Các đặc trưng của ngôn ngữ lập trình mà tạo nên liên kết giữa các tên trong chương trình với các vị trí bộ nhớ quan tâm là phạm vi, mà cho phép hai tên cú pháp khác nhau tham chiếu đến hai vị trí khác nhau, và các lời gọi hàm, mà mỗi lời gọi yêu cầu một vùng nhớ mới ở đó lưu trữ các tham số của hàm và các biến cục bộ. Một số chủ đề quan trọng trong chương này là truyền tham số, truy cập biến tổng thể và tối ưu bộ nhớ gắn kết với kiểu gọi hàm chuyên biệt gọi là lời gọi đuôi. Chúng ta sẽ thấy quản lý bộ nhớ trở nên phức tạp trong các ngôn ngữ có các khai báo hàm lồng nhau mà cho phép hàm được truyền như các tham số hoặc trả về như kết quả của lời gọi hàm.

8.1 TỔ CHỨC BỘ NHỚ

BẢN GHI KÍCH HOẠT

Trong hầu hết ngôn ngữ lập trình hiện đại, hàm có thể có các biến cục bộ mà được tạo khi gọi hàm. Một số lời gọi hàm có thể xảy ra cùng một thời điểm, và mỗi lời gọi có các khởi tạo các biến cục bộ của nó.

Trong phương thức Java

```
int f(int x) {  
    int y = x+x;  
    if (y<10)  
        return f(y);  
    else  
        return y-1;  
}
```

khởi tạo x được thực hiện (cùng với hàm f) mỗi lần hàm f được gọi. Vì có các lời gọi đệ qui, nên có thể có nhiều x đồng thời cùng tồn tại. Tương tự, khởi tạo mới của y được tiến hành mỗi lần hàm f được thực hiện.

Trong nhiều ngôn ngữ (bao gồm C, Pascal, và Java), các biến cục bộ được loại bỏ mỗi khi hàm trả về. Vì hàm trả về chỉ sau khi mọi hàm mà nó gọi đã được trả về, chúng ta nói các lời gọi hàm diễn ra theo kiểu vào sau ra trước (LIFO). Các biến cục bộ được tạo khi bắt đầu thực hiện hàm và loại bỏ khi hàm kết thúc, nên chúng ta sử dụng cấu trúc dữ liệu LIFO là ngăn xếp (stack) để lưu chúng.

CÁC HÀM BẬC CAO

Nhưng trong ngôn ngữ hỗ trợ cả hàm lồng nhau và trả về giá trị là hàm, có thể vẫn cần thiết giữ các biến cục bộ sau khi hàm trả về! Xét Chương trình 8.1: điều này là hợp lệ trong ML, nhưng trong C ta không thể lồng hàm g bên trong hàm f .

CHƯƠNG TRÌNH 8.1: Ví dụ các hàm bậc cao.

<pre>fun f(x) = let fun g(y) = x+y in g end</pre>	<pre>int (*)() f(int x) { int g(int y) {return x+y;} return g; }</pre>
<pre>val h = f(3) val j = f(4)</pre>	<pre>int (*h)() = f(3); int (*j)() = f(4);</pre>
<pre>val z = h(5) val w = j(7)</pre>	<pre>int z = h(5); int w = j(7);</pre>

(a) Viết trong ngôn ngữ ML (b) Viết trong C giả mã

Khi $f(3)$ được thực hiện, biến cục bộ mới x được tạo ra để kích hoạt hàm f . Sau đó g được trả về như kết quả của $f(x)$; nhưng g vẫn chưa được gọi, nên y vẫn chưa được tạo.

Tại thời điểm f trả về, còn quá sớm để xóa x , bởi vì khi $h(5)$ được thực hiện nó cần giá trị $x = 3$. Giữa hai thời điểm đó, $f(4)$ được đưa vào tính, tạo nên giá trị khác của x , và nó trả về giá trị khác của g ở đó $x = 4$.

Đây là sự kết hợp các hàm lồng nhau (ở đó hàm bên trong có thể sử dụng các biến được định nghĩa trong hàm ngoài) và các hàm trả về như kết quả (hoặc được lưu vào biến) nó làm cho biến cục bộ cần thêm thời gian sống lâu hơn so với khởi tạo hàm bao chúng.

Pascal có hàm lồng nhau, nhưng nó không có hàm như giá trị trả về. C có hàm như giá trị trả về, nhưng nó không có hàm lồng nhau. Vậy các ngôn ngữ này có thể sử dụng ngăn xếp để giữ các biến cục bộ.

ML, Scheme, và một số ngôn ngữ khác có thể có cả hàm lồng nhau và các hàm như giá trị trả về (sự kết hợp này gọi là hàm bậc cao - *higher-order functions*). Vậy chúng không thể dùng ngăn xếp để giữ các biến cục bộ. Điều này làm phức tạp việc cài đặt ML và

Scheme – nhưng khả năng biểu diễn bổ sung của các hàm bậc cao sẽ bù đắp các nỗ lực cài đặt thêm đó.

Trong phần còn lại của chương này chúng ta sẽ xét các ngôn ngữ với các biến cục bộ lưu ngăn xếp được. Lưu ý trong khi Java cho phép hàm lồng nhau (qua các lớp trong), thì MiniJava lại không.

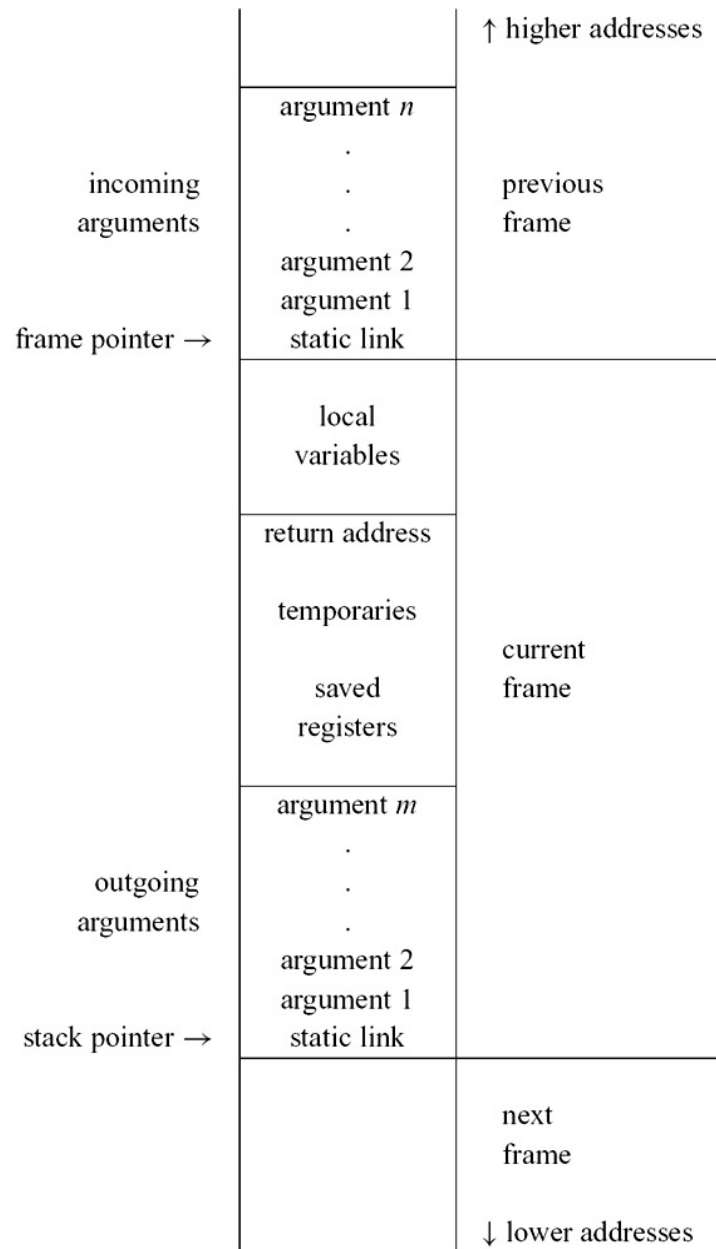
CÁC KHUNG NGĂN XẾP

Khái niệm đơn giản nhất của ngăn xếp - *stack* là cấu trúc dữ liệu mà hỗ trợ hai phép toán đẩy vào - *push* và lấy ra - *pop*. Tuy nhiên, trên thực tế các biến cục bộ được đẩy vào dạng bó lớn (khi hàm được gọi) và lấy ra dạng bó lớn (khi trả về). Hơn nữa, khi các biến cục bộ được tạo, chúng không phải khi nào cũng được khởi tạo ngay. Cuối cùng, sau khi nhiều biến được đẩy vào, chúng ta muốn tiếp tục truy cập đến các biến ở sâu trong ngăn xếp. Như vậy mô hình đẩy và lấy ra trừu tượng không còn phù hợp.

Thay vào đó, chúng ta coi ngăn xếp như mảng lớn, với thanh ghi đặc biệt – con trỏ ngăn xếp (the *stack pointer*) – mà trỏ đến vị trí nào đó. Mọi vị trí dưới con trỏ ngăn xếp được xem như là thùng rác, và mọi vị trí trước con trỏ ngăn xếp được xem như đã cấp. Ngăn xếp thường chỉ lớn lên khi gọi hàm nào đó, bằng cách tăng kích thước đủ lớn để giữ mọi biến cục bộ của hàm, và chỉ đến khi rời bỏ hàm đó, nó sẽ co lại đúng bằng kích thước đó. Dung lượng trên ngăn xếp dùng cho các biến cục bộ, các tham số, địa chỉ trả về, các giá trị tạm thời khác cho hàm đó được gọi là bản ghi kích hoạt - *activation record* hoặc khung ngăn xếp *stack frame*. Vì những lý do lịch sử, các ngăn xếp thời gian thực thường bắt đầu ở địa chỉ cao của bộ nhớ và tăng đến các địa chỉ nhỏ hơn. Điều này có vẻ gây nhầm lẫn: ngăn xếp tăng xuống và co lên.

Thiết kế giao diện khung cần chú ý đến các đặc thù của kiến trúc tập lệnh và ngôn ngữ lập trình đang dịch. Tuy nhiên, các nhà sản xuất máy tính thông thường mô tả giao diện khung “chuẩn” được dùng trên kiến trúc đó, ở đâu có thể, bởi mọi chương trình dịch cho mọi ngôn ngữ lập trình. Đôi khi giao diện này không thuận tiện lắm cho một ngôn ngữ lập trình hoặc một chương trình dịch cụ thể. Nhưng bằng việc sử dụng giao diện chuẩn, chúng ta thu được lợi ích đáng kể là các hàm viết trên một ngôn ngữ lập trình có thể gọi các hàm viết trên ngôn ngữ khác.

Hình 8.2 nêu giao diện khung ngăn xếp tiêu biểu. Khung này có tập các đối số đến *incoming arguments* (về kỹ thuật đó là phần khung trước, mà chúng ở offset cho trước của con trỏ khung) được truyền đến bởi nơi gọi. Địa chỉ trả về được tạo bởi lệnh CALL (trong lời gọi hàm) và cho biết điều khiển cần trả về đâu khi kết thúc hàm hiện tại. Một số biến cục bộ đang ở trên khung này; một số biến cục bộ khác được lưu giữ trên các thanh ghi của máy. Đôi khi một biến cục bộ được giữ trên thanh ghi cần phải lưu vào khung để giải phóng chỗ trên thanh ghi cho việc khác; cần có chỗ dùng cho việc đó trên khung. Cuối cùng, khi hàm hiện tại gọi hàm khác, nó cần sử dụng không gian các đối ra (the *outgoing argument space*) để truyền tham số.



Hình 8.2: Khung ngăn xếp.

CON TRỎ KHUNG NGĂN XẾP

Giả sử hàm $g(\dots)$ gọi hàm $f(a_1, \dots, a_n)$. Ta nói g là hàm gọi – the *caller* và f là hàm được gọi – the *callee*. Khi bắt đầu f , con trỏ ngăn xếp trỏ đến đối số đầu tiên mà g truyền cho f . Khi vào thân hàm, f phân phối khung bằng cách đơn giản trừ bớt kích thước khung từ con trỏ ngăn xếp SP (stack pointer).

Con trỏ ngăn xếp SP cũ trở thành con trỏ khung hiện tại – *frame pointer* FP. Trong một số giao diện khung, FP là thanh ghi riêng biệt; giá trị cũ của FP được lưu trong bộ nhớ (trong khung) và FP mới trở thành SP cũ. Khi f thoát ra, nó lưu FP ngược trở lại cho SP.

và lấy lại FP đã lưu. Tổ chức như vậy có ích khi kích thước khung của f có thể thay đổi, hoặc nếu khung không phải khi nào cũng liền nhau trên ngăn xếp. Nhưng nếu kích thước khung cố định, thì đối với mỗi hàm f con trỏ FP sẽ luôn khác SP một hằng số biết trước, và không cần phải dùng thanh ghi cho FP nữa – FP là thanh ghi “cho rằng” giá trị của nó luôn là SP + kích thước khung (*framesize*).

Tại sao ta phải bàn về con trỏ khung? Sao không nói đến mọi biến, tham số, v.v..., bằng khoảng cách của chúng từ SP, nếu kích thước khung là hằng số? Kích thước khung không biết trước, cho đến khá muộn về sau trong quá trình dịch, khi số các giá trị tạm thời trong bộ nhớ và thanh ghi được lưu hoàn toàn xác định. Nhưng nó có ích để biết khoảng cách của các tham số hình thức và các biến cục bộ khá sớm. Vì vậy, để thuận tiện, chúng ta nói về con trỏ khung. Và chúng ta sẽ đặt tham số hình thức và biến cục bộ ngay bên phải con trỏ khung ở khoảng cách mà đã được biết trước; các giá trị tạm thời và các thanh ghi lưu được xếp tiếp theo ở khoảng cách sẽ được biết về sau.

THANH GHI – REGISTERS

Máy tính hiện đại có tập khá lớn các thanh ghi (khoảng 32). Để chương trình dịch chạy nhanh rất có ích nếu giữ một số biến cục bộ, các kết quả trung gian của các biểu thức, và các giá trị khác trên thanh ghi thay vì trên khung ngăn xếp. Thanh ghi có thể được truy cập trực tiếp bởi các lệnh số học; trên hầu hết các máy tính, việc truy cập đến bộ nhớ yêu cầu hai lệnh riêng biệt *load* và *store*. Ngay cả trên máy các lệnh số học có thể truy cập trực tiếp đến bộ nhớ, thì việc truy cập đến thanh ghi vẫn nhanh hơn.

Máy (thông thường) chỉ có một tập các thanh ghi, nhưng nhiều thủ tục và hàm khác nhau cần dùng thanh ghi. Giả sử hàm f sử dụng thanh ghi r để giữ biến cục bộ và gọi thủ tục g , mà cũng sử dụng r để thực hiện tính toán của mình. Sau đó r cần phải lưu lại (cất vào khung ngăn xếp) trước khi g sử dụng nó và khôi phục (lấy lại giá trị từ khung) sau khi g kết thúc dùng nó. Nhưng trách nhiệm lưu và khôi phục thanh ghi là của f hay g ? Chúng ta nói rằng r là thanh ghi *caller-save* (*gọi – lưu*) nếu hàm gọi (trong trường hợp này là f) cần phải lưu và khôi phục thanh ghi; và r được gọi là *callee-save* (*được gọi – lưu*) nếu trách nhiệm đó là của hàm được gọi (trong trường hợp này là g).

Trong hầu hết kiến trúc máy, các khái niệm thanh ghi *gọi – lưu* hoặc *được gọi – lưu* không phải gắn với phần cứng, có những định nghĩa được mô tả trong sách tham khảo. Trên máy tính MIPS, chẳng hạn, thanh ghi 16-23 được dành trước cho các lời gọi thủ tục (*callee-save*), và mọi cái khác không dành trước cho các lời gọi thủ tục (*caller-save*).

Đôi khi việc lưu trữ và khôi phục không cần thiết. Nếu f biết rằng giá trị của biến x nào đó không cần sau lời gọi, nó có thể đặt x vào thanh ghi *caller-save* và không lưu nó khi gọi g . Ngược lại, nếu f có biến cục bộ i mà cần trước và sau khi một số lời gọi hàm, nó sẽ đặt i vào thanh ghi *callee-save* r_i , và lưu r_i ngay mỗi lần (mà gọi hàm f) và gắn nó trở lại ngay mỗi lần (trước khi trả về từ f). Vậy, sự lựa chọn thông minh thanh ghi *caller-save* hoặc thanh ghi *callee-save* cho mỗi biến cục bộ và tạm thời sẽ làm giảm số giá trị lưu và gắn trở lại cần thực hiện. Chúng ta sẽ dựa trên bộ phân phối thanh ghi của chúng ta để chọn kiểu thanh ghi phù hợp cho mỗi biến cục bộ và giá trị tạm thời.

TRUYỀN THAM SỐ

Trên hầu hết các máy tính, mà những lời gọi hàm được thiết kế từ những năm 1970, các đối số của hàm được truyền vào ngăn xếp. Nhưng điều này sinh ra việc truyền bộ nhớ không cần thiết. Nghiên cứu các chương trình thực tế chỉ ra rằng rất ít hàm cần nhiều hơn 4 biến và hầu hết không có nhiều hơn 6 biến. Vì vậy, việc truyền tham số cho máy tính hiện đại được đặc tả sao cho k đối số đầu tiên (với $k = 4$ hoặc $k = 6$) của hàm được truyền vào các thanh ghi $r_p, \dots, r_{p+k-1}$, và những đối số còn lại được truyền vào bộ nhớ.

Bây giờ, giả sử $f(a_1, \dots, a_n)$ (mà nhận các tham số của nó trong $r_1, \dots, r_n$, chẳng hạn) gọi $h(z)$. Nó truyền đối số z trong r_1 ; vậy f lưu nội dung cũ của r_1 (giá trị) vào khung ngăn xếp trước khi gọi h . Nhưng vẫn còn việc truyền bộ nhớ mà đã được hỗ trợ tránh nhờ truyền đối số vào thanh ghi! Tại sao việc sử dụng thanh ghi lại tiết kiệm thời gian?

Có bốn câu trả lời, mỗi một hoặc tất cả trong số chúng có thể được sử dụng đồng thời:

1. Một số thủ tục không gọi các thủ tục khác – chúng được gọi là thủ tục lá. Tỷ lệ phần trăm thủ tục lá là bao nhiêu? Được, nếu chúng ta giả thiết (lạc quan) rằng, trung bình các lời gọi thủ tục hoặc không gọi thủ tục khác hoặc gọi ít nhất hai thủ tục khác, thì chúng ta có thể mô tả “cây” lời gọi thủ tục trong đó lá nhiều hơn đỉnh trong. Điều đó có nghĩa là hầu hết các thủ tục là lá.

Thủ tục lá không cần viết các đối số đến vào bộ nhớ. Trên thực tế, thông thường chúng không cần đến khung ngăn xếp. Đây là sự tiết kiệm đáng kể.

2. Một số chương trình dịch tối ưu sử dụng cấp thanh ghi liên kết giữa các thủ tục, phân tích mọi hàm trong toàn bộ chương trình một lần. Sau đó chúng gán các thủ tục cho các thanh ghi khác nhau ở đó nhận và giữ các biến cục bộ. Vì vậy hàm $f(x)$ có thể nhận x trong r_1 , nhưng gọi $h(z)$ với z trong r_7 .
3. Ngay cả nếu f không là thủ tục lá, nó có thể kết thúc với mọi việc sử dụng đối số x tại cùng thời điểm gọi h (về mặt kỹ thuật, x là biến chết tại điểm mà h được gọi). Nên f có thể viết đè r_1 mà không lưu nó.
4. Một số kiến trúc có cửa sổ thanh ghi (*register windows*), sao cho mỗi lời gọi hàm cần được cấp tập mới các thanh ghi không cần dịch chuyển bộ nhớ.

Nếu f cần viết các tham số đến vào khung, thì cần viết vào đâu trong khung? Tốt nhất, giao diện khung của f cần dùng cho cài đặt chỉ của f . Cách tiếp cận trực tiếp đối với nơi gọi là truyền các đối số $a_1, \dots, a_k$ vào các thanh ghi và $a_{k+1}, \dots, a_n$ vào cuối khung riêng của nó - chỗ mà đánh dấu các đối số ra (*outgoing arguments*) trong Hình 8.2 - mà trở thành các đối số vào (*incoming arguments*) cho nơi được gọi. Nếu nơi được gọi cần viết bất cứ đối số nào vào bộ nhớ, nó cần viết chúng vào vùng có đánh dấu biến cục bộ (*local variables*).

Trong ngôn ngữ lập trình C thực tế cho phép bạn lấy địa chỉ của các tham số hình thức và đảm bảo rằng mọi tham số hình thức của hàm nằm ở các địa chỉ nối tiếp nhau! Đây là đặc tính `varargs` mà `printf` sử dụng. Cho phép lập trình lấy địa chỉ của các tham số có thể

dẫn đến tham chiếu đu đưa (*dangling reference*) nếu địa chỉ vượt khỏi khung - như địa chỉ của x ở trong `int *f(int x){return &x;}` - và ngay cả khi nó không dẫn đến lỗi, qui tắc địa chỉ liên tiếp đối với tham số ràng buộc chương trình dịch và làm cho giao diện khung phức tạp hơn. Để giải quyết mâu thuẫn này, các tham số được truyền vào thanh ghi, nhưng cũng có địa chỉ, k tham số đầu truyền vào thanh ghi; nhưng bất cứ tham số nào mà địa chỉ của nó bị lấy cần được viết đè vào bộ nhớ khi bắt đầu gọi hàm. Để thỏa `printf`, vị trí bộ nhớ mà các đối số thanh ghi được viết đè vào cần phải liên tiếp nhau với các vị trí mà ở đó các đối số $k + 1, k + 2, \text{v.v...}$, được viết. Vì vậy, chương trình C không thể có một số đối số được lưu ở một chỗ, và một số khác ở chỗ khác - tất cả chúng cần được lưu liên tiếp kề nhau.

Vậy trong việc biến đổi lời gọi hàm chuẩn của nhiều máy hiện đại, hàm gọi dành không gian cho các đối số thanh ghi trên khung của nó, tiếp theo vị trí đó là nơi viết đối số $k + 1$. Nhưng hàm gọi không viết gì ở đây, không gian này được viết bởi hàm được gọi, và chỉ nếu hàm được gọi cần viết các đối số vào bộ nhớ vì lý do nào đó.

Có một cách đáng nói hơn là dùng địa chỉ của biến cục bộ để sử dụng gọi theo tham chiếu (*call-by-reference*). Với lời gọi theo tham số, lập trình viên không thao tác tường minh địa chỉ của biến x . Thay vào đó, nếu x được truyền như đối số cho $f(y)$, trong đó y là tham số tham chiếu, chương trình dịch sinh ra mã để truyền địa chỉ của x thay vì nội dung của x . Trong bất cứ sử dụng nào của y trong hàm, chương trình dịch sẽ tạo ra tham chiếu con trỏ bổ sung. Với lời gọi theo tham chiếu, cần phải không có tham chiếu “treo”, vì y cần biến mất khi f trả về, và f trả về trước khi phạm vi của x kết thúc.

ĐỊA CHỈ TRẢ VỀ

Khi hàm g gọi hàm f , tất nhiên f cần phải trả về. Cần biết phải quay ngược trở lại đâu. Nếu lệnh gọi trong g ở địa chỉ a , thì (thông thường) vị trí trả đúng là $a + 1$, lệnh tiếp theo trong g . Đây được gọi là địa chỉ trả về (*return address*).

Trong các máy năm 1970, địa chỉ trả về được đẩy vào ngăn xếp bởi lệnh *gọi*. Khoa học hiện đại đã chỉ ra rằng sẽ nhanh và linh hoạt hơn nếu truyền địa chỉ trả về trong thanh ghi, tránh việc truyền bộ nhớ và cũng tránh sự cần thiết phải tạo ngăn xếp riêng nào đó trong phần cứng.

Trong các máy hiện đại, lệnh gọi đơn giản đặt lệnh trả lại vào thanh ghi chỉ định trước. Thủ tục không là lá thì sẽ viết nó lên ngăn xếp (trừ khi thanh ghi chung cho các thủ tục được sử dụng), nhưng thủ tục lá thì không.

8.2 LƯU TRỮ THÔNG TIN

CÁC BIẾN THƯỜNG TRỰC TRÊN KHUNG

Do đó qui ước lời gọi thủ tục hiện đại sẽ truyền các tham số hàm số trong thanh ghi, truyền địa chỉ trả về trong thanh ghi, và trả về kết quả hàm số trong thanh ghi. Nhiều biến cục bộ được lưu trong thanh ghi, cũng như các kết quả trung gian của việc tính toán các biểu thức. Các giá trị được viết lên bộ nhớ (trong khung ngăn xếp) chỉ khi cần thiết vì một trong các lý do sau:

- Biến sẽ được truyền theo tham chiếu, vậy nó cần phải có địa chỉ bộ nhớ (hoặc trong ngôn ngữ C phép toán & sẽ được áp dụng ở mọi nơi cho biến);
- Biến được truy cập bởi thủ tục lồng bên trong thủ tục hiện tại
- Biến quá lớn để lưu trong một thanh ghi
- Biến là mảng, đối với nó việc tính toán địa chỉ cần được thực hiện qua các thành phần.
- Thanh ghi giữ biến cần thiết cho mục đích riêng khác, chẳng hạn như truyền tham số (như mô tả ở bên trên), mặc dù bộ dịch có thể dịch chuyển các giá trị này đến các thanh ghi khác thay vì lưu trữ chúng trong bộ nhớ.
- Hoặc ở đây có quá nhiều biến cục bộ và các giá trị trung gian, mà không thể giữ tất cả trong các thanh ghi, trong trường này một số trong số chúng bị tràn khung.

Chúng ta sẽ nói rằng biến biến mất nếu nó được truyền theo tham chiếu mà địa chỉ của nó bị lấy mất (sử dụng phép toán & trong C), hoặc nó được truy cập từ các hàm lồng.

Khi các tham số hình thức hoặc biến cục bộ được khai báo, chúng thường được gắn với vị trí – hoặc trên thanh ghi hoặc trên khung ngăn xếp – ngay tại chỗ mà chương trình đang xử lý. Do đó, khi sự xuất hiện của biến này được tìm thấy trong biểu thức, chúng có thể được phiên dịch sang mã máy để chỉ rõ vị trí đúng của nó. Không may, các điều kiện trong danh sách của chúng ta không bộc lộ đủ sớm. Khi chương trình dịch lần đầu gặp khai báo của biến, nó cũng chưa biết biến được truyền theo tham chiếu hay được truy cập trong thủ tục lồng nhau, hay là có địa chỉ bị lấy mất; và nó không biết có bao nhiêu thanh ghi cần thiết cho việc tính toán giá trị biểu thức (có thể muốn đặt một số biến trong khung thay vì trên thanh ghi). Bộ dịch có sức mạnh như trong công nghiệp cần phải gán tạm thời mọi vị trí cho các biến hình thức và cục bộ, và sẽ quyết định về sau cái nào trong số chúng cần phải ghi trên thanh ghi.

LIÊN KẾT TÍNH

Trong các ngôn ngữ mà cho phép khai báo các hàm lồng nhau (như Pascal, ML và Java), hàm bên trong có thể sử dụng biến được khai báo trong hàm bên ngoài. Đặc tính này của ngôn ngữ được gọi là cấu trúc khối. Chẳng hạn xem chương trình 8.3, mà được viết trong cú pháp giống Pascal. Hàm `write` dùng biến bên ngoài `output`, và hàm `indent` dùng các biến bên ngoài `n` và `output`. Để thực hiện công việc này, hàm `indent` cần phải truy cập không chỉ khung của mình (đối với các biến `i` và `s`) mà còn phải truy cập khung của hàm `show` (đối với biến `n`) và hàm `prettyprint` (đối với biến `output`).

CHƯƠNG TRÌNH 8.3: Các hàm lồng nhau

```

.
1      type tree = {key: string, left: tree, right: tree}
2
3      function prettyprint(tree: tree) : string =
4          let
5              var output := ""
6
7              function write(s: string) =
8                  output := concat(output,s)
9
10             function show(n:int, t: tree) =
11                 let function indent(s: string) =
12                     (for i := 1 to n
13                     do write(" "));
14                     output := concat(output,s); write("\n"))
15                 in if t=nil then indent(".")
16                     else (indent(t.key);
17                           show(n+1,t.left);
18                           show(n+1,t.right))
19             end
20
21         in show(0,tree); output
22     end

```

Có một số phương pháp giải quyết vấn đề này:

- Khi nào hàm f được gọi, nó sẽ được truyền như con trỏ đến khung tĩnh của hàm số chứa f ; con trỏ này là liên kết tĩnh *static link*.
- Mảng tổng thể có thể được duy trì, chứa tại vị trí i con trỏ đến khung của thủ tục vừa mới thực hiện mà độ sâu lồng tĩnh là i . Mảng này gọi là thể hiện *display*.
- Khi g gọi f , mỗi biến của g mà được truy cập thực tế bởi f (hoặc bởi mọi hàm lồng bên trong f) được truyền đến f như một biến phụ. Điều này được gọi là leo thang *lambda lifting*.

Chúng ta sẽ mô tả chi tiết phương pháp liên kết tĩnh. Phương pháp nào được sử dụng trên thực tế? Xem bài tập 8.6.

Khi hàm f được gọi, nó sẽ được truyền như con trỏ đến khung ngăn xếp của hành động hiện thời của hàm g mà chứa trực tiếp f trong văn bản của chương trình.

Chẳng hạn, trong chương trình 8.3:

Số dòng#

21 `prettyprint` gọi `show`, truyền con trỏ khung của riêng `prettyprint` như liên kết tĩnh của `show`.

10 `show` lưu liên kết tĩnh của nó (địa chỉ của khung `prettyprint`) vào trong khung riêng của nó.

15 `show` gọi `indent`, truyền con trỏ của khung riêng của nó như liên kết tĩnh của `indent`.

17 `show` gọi `show`, truyền liên kết tĩnh của riêng nó (không phải con trỏ đến khung của riêng nó) như liên kết tĩnh.

12 `indent` sử dụng giá trị n từ khung của `show`. Để làm như vậy, nó đánh offset phù hợp từ liên kết tĩnh của `indent` (mà trỏ đến khung của `show`).

13 `indent` gọi `write`. Nó cần truyền con trỏ khung của `prettyprint` như liên kết tĩnh. Để nhận được điều đó, nó đánh offset từ liên kết tĩnh của mình (từ khung của `show`), liên kết tĩnh này sẽ được truyền cho `show`.

14 `indent` sử dụng biến `output` từ khung của `prettyprint`. Để làm như vậy, nó bắt đầu từ liên kết tĩnh của mình, sau đó đánh của `show`, và cuối cùng đánh `output`.^[4]

Cứ tiếp tục như vậy, mỗi lời gọi thủ tục hoặc truy cập biến, có thể đòi hỏi nhiều giá trị đánh kèm; độ dài của dãy các giá trị đánh kèm đó chính là sự *chênh lệch* về độ sâu lồng tĩnh giữa hai hàm liên quan.

KHUNG TRONG BỘ DỊCH MiniJava

Kiểu khung ngăn xếp nào mà chương trình dịch MiniJava sử dụng? Ở đây chúng ta đối mặt với thực tế rằng mỗi kiến trúc máy đích sẽ có giao diện khung ngăn xếp chuẩn khác nhau. Nếu chúng ta muốn các hàm của MiniJava có khả năng gọi các hàm của C, chúng ta cần phải sử dụng giao diện chuẩn. Nhưng chúng ta không muốn một số đặc thù của một máy riêng biệt nào đó lại xâm nhập vào việc cài đặt module dịch của chương trình dịch MiniJava.

Như vậy chúng ta muốn sử dụng việc trừu tượng hóa. Cũng như module `Symbol` cung cấp một giao diện sạch, và giấu đi các biểu diễn bên trong của `Symbol.Table`, chúng ta cần phải sử dụng biểu diễn trừu tượng cho khung.

Giao diện sẽ trông giống như sau:

```
package Frame;
import Temp.Temp; import Temp.Label;

public abstract class Access { ... }
public abstract class AccessList {...head;...tail;... }
public abstract class Frame {
    abstract public Frame newFrame(Label name,
                                   Util.BoolList formals);
```

```
    public Label name;
    public AccessList formals;
    abstract public Access allocLocal(boolean escape);
    /* ... other stuff, eventually ... */
}
```

Lớp trừu tượng `Frame` được cài đặt bởi module chuyên dụng cho máy đích xác định. Chẳng hạn, nếu dịch sang kiến trúc MIPS, nó có thể như sau

```
package Mips;
class Frame extends Frame.Frame { ... }
```

Nói chung, chúng ta có thể giả thiết rằng phần độc lập với máy của chương trình dịch cần phải dùng đến phần cài đặt này của khung `Frame`; chẳng hạn,

```
// in class Main.Main:
Frame.Frame frame = new Mips.Frame(...);
```

Theo cách này phần còn lại của chương trình dịch có thể truy cập đến `frame` mà không cần biết máy đích là gì (ngoài việc xuất hiện từ `Mips` ở đâu đó).

Lớp `Frame` giữ thông tin về các tham số hình thức và các biến cục bộ được đặt trong frame đó. Để tạo frame mới cho hàm f với k tham số hình thức, gọi `newFrame( $f$ ,  $l$ )`, trong đó l là danh sách k biến bool: `true` cho mỗi tham số mà biến mất được và `false` cho mỗi tham số mà không biến mất được. (Trong MiniJava, không tham số nào biến mất được). Kết quả này sẽ là đối tượng `Frame`. Chẳng hạn, xét hàm ba biến g mà biến đầu tiên của nó biến mất được (cần được giữ trong bộ nhớ). Khi đó

```
frame.newFrame( $g$ , new BoolList(true,
                                new BoolList(false,
                                new BoolList(false, null))))
```

trả về đối tượng frame mới.

Lớp `Access` mô tả các tham số hình thức hoặc biến cục bộ mà có trong khung frame hoặc trong các thanh ghi registers. Đây là kiểu dữ liệu trừu tượng, do đó cài đặt của nó như một cặp các lớp con mà chỉ nhìn thấy được bên trong module `Frame`:

```
package Mips;
class InFrame extends Frame.Access {int offset; ... }
class InReg    extends Frame.Access {Temp temp; ... }
```

`InFrame( $X$ )` chỉ ra vị trí bộ nhớ tại X từ con trỏ khung; `InReg( $t_{84}$ )` chỉ ra rằng nó được giữ trong thanh ghi "register" t_{84} . `Frame.Access` là kiểu dữ liệu trừu tượng, do đó bên ngoài module `InFrame` và `InReg` các hàm tạo không được nhìn thấy. Các module khác thao tác truy cập sử dụng các hàm giao diện được mô tả trong tài liệu tham khảo.

Các trường hình thức `formals` là danh sách của k "accesses" ký hiệu các vị trí ở đó các tham số hình thức sẽ được giữ trong thời gian chạy, như được nhìn thấy từ bên trong nơi

được gọi. Các tham số có thể được nhìn thấy khác nhau giữa nơi gọi và nơi được gọi. Chẳng hạn, nếu các tham số được truyền trên ngăn xếp, người gọi có thể đặt tham số tại offset 4 từ con trỏ ngăn xếp, nhưng người được gọi nhìn thấy nó tại offset 4 từ con trỏ khung. Hoặc người gọi có thể đặt tham số vào thanh ghi 6, nhưng người được gọi có thể muốn chuyển nó ra khỏi chỗ đó và luôn muốn truy cập nó từ thanh ghi 13. Trên kiến trúc Sparc, với cửa sổ thanh ghi, người gọi đặt tham số vào register `o1`, nhưng chỉ thị `save` dịch chuyển cửa sổ thanh ghi sao cho người gọi nhìn thấy tham số này trong thanh ghi register `i1`.

Vì việc dịch chuyển tầm nhìn này phụ thuộc vào qui ước gọi của máy đích, nó cần được kiểm soát trong module `Frame`, bắt đầu với `newFrame`. Đối với mỗi tham số hình thức, `newFrame` cần phải tính hai việc sau:

- Tham số sẽ được nhìn thấy từ bên trong hàm số như thế nào (trong thanh ghi, hoặc trong vị trí khung).
- Các chỉ lệnh nào cần đưa ra để cài đặt việc dịch chuyển tầm nhìn "view shift."

Chẳng hạn, tham số cư trú ở khung nhìn sẽ được nhìn thấy như "bộ nhớ tại offset X từ con trỏ khung", và việc dịch chuyển tầm nhìn sẽ được cài đặt bằng việc sao lưu con trỏ ngăn xếp sang con trỏ khung khi bắt đầu thủ tục.

BIỂU DIỄN MÔ TẢ KHUNG

Module cài đặt `Frame` coi như giữ được biểu diễn thông tin mật của các đối tượng `Frame` khỏi mọi khách hàng của module `Frame`. Nhưng thực tế nó là đối tượng giữ:

- Vị trí của mọi tham số hình thức,
- Chỉ thị yêu cầu để cài đặt việc dịch chuyển tầm nhìn "view shift",
- Số các biến cục bộ sử dụng,
- Và nhãn `label` mà ở đó đoạn mã máy của hàm được bắt đầu (xem trang 131, [1]).

Bảng 8.4 chỉ ra các tham số hình thức của hàm g (xem trang 127, [1]) như `newFrame` sẽ đặt chúng trên ba kiến trúc khác nhau: Pentium, MIPS, và Sparc. Tham số đầu tiên biến mất, do đó nó cần là `InFrame` trên cả ba máy. Các tham số còn lại là `InFrame` trên Pentium, nhưng `InReg` trên các máy khác.

Bảng 8.4: Các tham số hình thức cho $g(x_1, x_2, x_3)$ trong đó x_1 biến mất.

	Pentium	MIPS	Sparc
1	<code>InFrame(8)</code>	<code>InFrame(0)</code>	<code>InFrame(68)</code>
Formals 2	<code>InFrame(12)</code>	<code>InReg(t_{157})</code>	<code>InReg(t_{157})</code>
3	<code>InFrame(16)</code>	<code>InReg(t_{158})</code>	<code>InReg(t_{158})</code>
	$M[sp + 0] \leftarrow fp$ $sp \leftarrow sp - K$		<code>save %sp, -K, %sp</code>

Bảng 8.4: Các tham số hình thức cho $g(x_1, x_2, x_3)$ trong đó x_1 biến mất.

Pentium	MIPS	Sparc
View Shift $fp \leftarrow sp$	$M[sp + K + 0] \leftarrow r_4$	$M[fp + 68] \leftarrow i_0$
$sp \leftarrow sp - K$	$t_{157} \leftarrow r_5$	$t_{157} \leftarrow i_1$
	$t_{158} \leftarrow r_6$	$t_{158} \leftarrow i_2$

Các biến tạm thời mới được tạo ra t_{157} và t_{158} , và các chỉ thị *move*, mà sẽ sao chép r_4 và r_5 vào chúng (hoặc trên Sparc, i_1 và i_2), có vẻ như thừa. Tại sao thân của g không truy cập đến các tham số này trực tiếp từ các thanh ghi nơi mà chúng đến đó? Để thấy tại sao không, chúng ta xét

```
void m(int x, int y) { h(y, y); h(x, x); }
```

Nếu x ở thanh ghi tham số số 1 - "parameter register 1"- đến tận m , và y được truyền đến h trong thanh ghi tham số 1, thì ở đây có vấn đề.

Bộ cấp thanh ghi đương nhiên sẽ chọn thanh ghi máy mà giữ t_{157} . Nếu không có sự va chạm về kiểu được nêu trong hàm m , thì (trên MIPS) bộ cấp sẽ quan tâm đến việc chọn thanh ghi r_4 để giữ t_{157} và r_5 để giữ t_{158} . Khi đó các chỉ thị *move* sẽ không cần thiết và sẽ bị loại bỏ tại thời điểm đó.

(Xem các trang 157 và 251 [1] để bàn thêm về dịch chuyển tầm nhìn).

CÁC BIẾN CỤC BỘ

Một số biến cục bộ được giữ trong khung; một số khác được giữ trên thanh ghi. Để cấp chỗ cho biến cục bộ mới trong khung f , pha thông dịch sẽ gọi

```
f.allocLocal(false)
```

Nó trả về truy cập `InFrame` với offset từ con trỏ khung. Chẳng hạn, để cấp hai biến cục bộ trên Sparc, `allocLocal` sẽ được gọi hai lần, trả về thành công `InFrame(-4)` và `InFrame(-8)`, mà là offsets con trỏ khung chuẩn cho các biến cục bộ.

Biến Bool cho `allocLocal` chỉ rõ biến mới có biến mất và cần đến khung không; nếu nó là sai, thì các biến có thể được cấp trong thanh ghi. Như vậy, `f.allocLocal(false)` có thể tạo `InReg( $t_{481}$ )`.

Đối với MiniJava, ở đây không có biến nào biến mất. Điều này đúng vì trong MiniJava:

- Không có lồng nhau về lớp và các phương thức;
- Không thể lấy địa chỉ của biến;

- Các số nguyên và biến bool được truyền bởi giá trị; và
- Các đối tượng, bao gồm các mảng nguyên, có thể được thể hiện như các con trỏ mà cũng được truyền bởi giá trị.

Các lời gọi đến `allocLocal` không cần đến ngay sau khi khung được tạo. Trong nhiều ngôn ngữ lập trình, ở đây có thể có các khối khai báo biến lồng nhau bên trong thân của hàm. Chẳng hạn,

```
void f()
{int v=6;
  print(v);
  {int v=7;
    print(v);
  }
  print(v);
  {int v=8;
    print(v);
  }
  print(v);
}
```

Trong trường hợp này có ba biến `v` khác nhau. Chương trình sẽ in ra dãy 6 7 6 8 6. Khi việc khai báo mỗi biến bắt gặp trong quá trình xử lý trong chương trình, chúng ta sẽ cấp không gian mới hoặc tạm thời trong khung, liên kết với tên `v`. Mỗi khi `end` (hoặc đóng ngoặc) bắt gặp, liên kết với `v` sẽ được quên đi – nhưng không gian vẫn được giữ lại trong khung. Như vậy, ở đây có các khe khung hoặc tạm thời khác nhau cho mỗi biến khai báo bên trong toàn bộ hàm.

Bộ cấp thanh ghi sẽ sử dụng một số thanh ghi mới có thể để lưu tạm. Trong ví dụ này, các biến `v` thứ hai và thứ ba (được khởi tạo bằng 7 và 8) có thể được giữ trên cùng một thanh ghi. Chương trình dịch thông minh có thể sẽ tối ưu kích thước khung bằng việc cảnh báo khi hai biến thường trú trong khung được cấp định vị trên cùng khe.

TẠM THỜI VÀ NHÃN

Pha thông dịch của chương trình dịch sẽ muốn chọn các thanh ghi cho các tham số và các biến cục bộ, và chọn địa chỉ mã máy cho thân các thủ tục. Nhưng lúc đó là quá sớm để xác định chính xác thanh ghi nào dùng được, hoặc chính xác thân thủ tục sẽ được đặt ở đâu. Chúng ta sử dụng biến *tạm thời temporary* để nói về giá trị mà tạm thời giữ trong thanh ghi, và *nhãn label* để nói về vị trí ngôn ngữ máy mà địa chỉ chính xác của nó còn cần được xác định – giống như nhãn trong ngôn ngữ assembly.

Tạm thời `Temps` là các tên trừu tượng cho các biến cục bộ; `labels` là các tên trừu tượng cho các địa chỉ bộ nhớ tĩnh. Module `Temp` quản lý hai tập tên khác nhau.

```
package Temp;
public class Temp {
  public String toString();
  public Temp();
}
```

```
}  
public class Label {  
    public String toString();  
    public Label();  
    public Label(String s);  
    public Label(Symbol s);  
}  
public class TempList {...}  
public class LabelList {...}
```

`new Temp.Temp()` trả về tạm thời mới từ tập vô hạn các biến tạm thời. `new Temp.Label()` trả về nhãn mới từ tập vô hạn các nhãn. Và `new Temp.Label(string)` trả về nhãn mới mà tên ngôn ngữ assembly của nó là *string*.

Khi xử lý khai báo `m(...)`, nhãn cho địa chỉ mã máy của `m` có thể được tạo ra bởi `new Temp.Label()`. Khi thử gọi `new Temp.Label("m")` - chương trình ngôn ngữ assembly sẽ trở nên dễ hơn cho việc bắt lỗi nếu sử dụng nhãn `m` thay cho `L213` – nhưng không may ở đây có hai phương pháp khác nhau tên `m` trong các lớp khác nhau. Ý tưởng tốt hơn là gọi `new Temp.Label("C"+"$"+"m")`, trong đó `C` là tên của lớp mà ở đó phương thức `m` xuất hiện.

QUẢN LÝ CÁC LIÊN KẾT TĨNH

Module `Frame` cần phải độc lập với ngôn ngữ gốc đặc thù đang được dịch. Nhiều ngôn ngữ gốc, bao gồm MiniJava, không có các khai báo hàm lồng nhau; như vậy, `Frame` không thể biết gì về các liên kết tĩnh. Thay vào đó, đây là trách nhiệm của pha thông dịch. Pha thông dịch có thể biết rằng mỗi khung có chứa liên kết tĩnh. Liên kết tĩnh có thể được truyền cho hàm trong thanh ghi và được lưu vào khung. Vì liên kết tĩnh có hành vi giống như tham số hình thức, chúng ta có thể coi chúng là một (tối đa có thể).

^[1] Trước năm 1960, chúng được truyền không phải trên ngăn xếp mà trên các khối bộ nhớ định vị tĩnh, mà loại trừ việc sử dụng các hàm đệ qui.

^[2] Tuy nhiên, với việc cập các thanh ghi theo các biên của hàm, các biến cục bộ được truy cập bởi các hàm bên trong có thể ghi vào các thanh ghi, dài tùy theo mức mà hàm bên trong biết sẽ khóa ở đâu.

^[3] Tuy nhiên, một số chương trình dịch trải rộng giá trị biên dưới vào trong một số thanh ghi cho hiệu quả.

^[4] Chương trình này có thể sạch hơn nếu `show` gọi `write` ở đây thay vì thao tác output trực tiếp, nhưng nó không thể là chỉ lệnh được.

CÁC KHUNG CHƯƠNG TRÌNH

Nếu bạn dịch cho Sparc, cài đặt gói `Sparc package` chứa `Sparc/SparcFrame.java`. Nếu bạn dịch cho MIPS, cài đặt gói `Mips`, và tiếp tục như vậy.

Nếu bạn làm việc trên máy RISC (như MIPS hoặc Sparc) mà truyền k tham số đầu tiên trên thanh ghi và phần còn lại trên bộ nhớ, giữ mọi vật đơn giản bằng cách chỉ kiểm soát trường hợp ở đó có k hoặc ít hơn các tham số.

Hỗ trợ các files sử dụng trong `$MINIJAVA/chap6` bao gồm:

`Temp/*` là module hỗ trợ tạm thời và các nhãn.

`Util/BoolList.java` là lớp cho danh sách các biến bool.

Tùy chọn: Kiểm soát các hàm có nhiều hơn k tham số hình thức.

8.3 VUN RÁC

- Các hệ thống thời gian thực quản lý heap, bảo trì việc giải phóng danh sách. Cấp phát theo kiểu lấp đầy hoặc lấp tốt nhất (tùy biến với hệ thống).
- Thu thập rác cần có khả năng phân biệt dữ liệu đang sống từ các dữ liệu không chạm tới được. Thu thập rác đánh các dữ liệu không chạm tới được vào danh sách loại bỏ. Thu thập rác có thể cần gói gọn dữ liệu khi dùng để tăng cường tính địa phương của chương trình.

Trong Java, hiện tượng rò rỉ bộ nhớ hầu như không xảy ra do bộ nhớ được quản lý bởi Java Virtual Machine (JVM) bằng cách tự động "dọn dẹp rác". Người lập trình không phải quan tâm đến việc cấp phát và xóa bộ nhớ như C, C++. Tuy nhiên khi sử dụng những tài nguyên mạng, file IO, database (nằm ngoài kiểm soát của JVM) mà người lập trình không đóng (close) các streams thì rò rỉ dữ liệu vẫn có thể xảy ra.

DỊCH RA CÂY

Dịch từ biểu thức cú pháp trừu tượng sang cây trung gian là tương đối trực tiếp; nhưng ở đây có một số trường hợp cần kiểm soát. Chúng ta sẽ bao quát việc dịch các cấu trúc ngôn ngữ khác nhau, bao gồm cả nhiều cấu trúc của MiniJava.

CÁC KIỂU BIỂU THỨC

Văn phạm MiniJava có các biểu thức và các lệnh khác nhau một cách rõ ràng. Tuy nhiên, trong ngôn ngữ như C, sự khác biệt là mập mờ; chẳng hạn, phép gán trong C có thể được sử dụng như biểu thức. Trong khi dịch các ngôn ngữ như vậy, chúng ta sẽ hỏi câu hỏi sau. Thể hiện của biểu thức cú pháp trừu tượng trong dạng ngôn ngữ cây sẽ như thế nào? Thoạt đầu nó rõ ràng cần phải dưới dạng `Tree.Exp`. Tuy nhiên, nó đúng chỉ với một số biểu thức, những cái mà tính giá trị. Những biểu thức mà không trả giá trị thì chúng được biểu diễn một cách tự nhiên hơn dưới dạng `Tree.Stm`. Và biểu thức với giá trị bool, như

$a > b$, có thể thể tốt nhất biểu diễn dạng nhảy có điều kiện – là sự kết hợp của `Tree.Stm` và cặp đích thể hiện bởi `Temp.Labels`.

Tốt hơn sẽ hỏi, "Biểu thức sẽ được sử dụng như thế nào?" Khi đó chúng ta có thể xác định đúng kiểu của phương thức đối với giao diện hướng đối tượng cho biểu thức. Cả đối với MiniJava và ngôn ngữ khác, chúng ta đi đến kết quả là lớp `Translate.Exp`, không phải cùng lớp như `Tree.Exp`, sẽ có ba phương thức:

```
package Translate;
public abstract class Exp {
    abstract Tree.Exp unEx();
    abstract Tree.Stm unNx();
    abstract Tree.Stm unCx(Temp.Label t, Temp.Label f);
}
```

- `Ex` ký hiệu cho biểu thức "expression", được thể hiện như `Tree.Exp`.
- `Nx` ký hiệu cho "no result", được thể hiện như lệnh `Tree`.
- `Cx` ký hiệu cho "conditional", được thể hiện như hàm chuyển cặp nhãn thành lệnh.

Nếu bạn truyền nó như đích đến đúng đích đến sai, nó sẽ tạo nên lệnh mà tính một điều kiện nào đó và sau đó nhảy đến một trong các đích (lệnh này sẽ không bao giờ bị lỗi "fall through").

Chẳng hạn, lệnh MiniJava

```
if (a<b && c<d) {
    // true block
}
else {
    // false block
}
```

sẽ được dịch sang `Translate.Exp` mà phương thức `unCx` của nó trông giống như

```
Tree.Stm unCx(Label t, Label f) {
    Label z = new Label();
    return new SEQ(new CJUMP(CJUMP.LT, a, b, z, f),
                   new SEQ(new LABEL(z),
                           new CJUMP(CJUMP.LT, c, d, t, f)));
}
```

Lớp trừu tượng `Translate.Exp` có thể được khởi tạo bởi một số lớp con: `Ex` cho việc biểu diễn bình thường mà sinh ra giá trị đơn, `Nx` cho biểu thức không sinh ra giá trị nào, và `Cx` cho biểu thức điều kiện mà nhảy đến t hoặc f :

```
class Ex extends Exp {
    Tree.Exp exp;
    Ex(Tree.Exp e) {exp=e;}
    Tree.Exp unEx() {return exp;}
}
```



```

    Tree.Stm unNx() { ... ?... }
    Tree.Stm unCx(Label t, Label f) { ... ?... }
}
class Nx extends Exp {
    Tree.Stm stm;
    Nx(Tree.Stm s) {stm=s;}
    Tree.Exp unEx() { ... ?... }
    Tree.Stm unNx() {return stm;}
    Tree.Stm unCx(Label t, Label f) { ... ?... }
}

```

Nhưng phương thức `unNx` của `Ex` làm gì? Chúng ta có `Tree.Exp` đơn giản tạo ra giá trị, và chúng ta được yêu cầu tạo ra `Tree.Stm` mà không sinh ra giá trị. Có toán tử chuyển đổi `Tree.EXP`, và `unNx` cần áp dụng nó:

```

class Ex extends Exp {
    Tree.Exp exp;
    :
    Tree.Stm unNx() {return new Tree.EXP(exp); }
    :
}

```

Mỗi kiểu của lớp `Translate.Exp` cần có phương thức chuyển đổi tương tự. Chẳng hạn, lệnh `MiniJava`

```
flag = (a<b && c<d);
```

yêu cầu phương thức `unEx` của đối tượng `Cx` sao cho 1 (cho đúng) hoặc 0 (cho sai) có thể được lưu vào `flag`.

Chương trình 8.4 mô tả lớp `Cx`. Phương thức `unEx` cũng có khía cạnh thú vị riêng. Để chuyển điều kiện "conditional" thành biểu thức giá trị "value expression", chúng ta nghĩ ra tạm thời mới `r` và các nhãn mới `t` và `f`. Khi đó chúng ta tạo ra `Tree.Stm` mà chuyển giá trị 1 cho `r`, và nhảy có điều kiện `unCx(t, f)` mà cài đặt điều kiện. Nếu điều kiện là false, thì 0 được truyền cho `r`; nếu nó true, thì việc thực hiện được tiến hành tại `t` và việc chuyển thứ hai được bỏ qua. Kết quả của toàn bộ việc này là biến tạm thời `r` chứa không hoặc một.

CHƯƠNG TRÌNH 8.4: Lớp `Cx`.

```

abstract class Cx extends Exp {
    Tree.Exp unEx() {
        Temp r = new Temp();
        Label t = new Label();
        Label f = new Label();

        return new Tree.ESEQ(
            new Tree.SEQ(new Tree.MOVE(new Tree.TEMP(r),
                                      new Tree.CONST(1)),
                        new Tree.SEQ(unCx(t, f),
                                      new Tree.SEQ(new Tree.LABEL(f),

```

```

        new Tree.SEQ(new Tree.MOVE(new Tree.TEMP(r),
                                   new Tree.CONST(0)),
                    new Tree.LABEL(t))))),
        new Tree.TEMP(r));
    }

    abstract Tree.Stm unCx(Label t, Label f);

    Tree.Stm unNx() { ... }
}

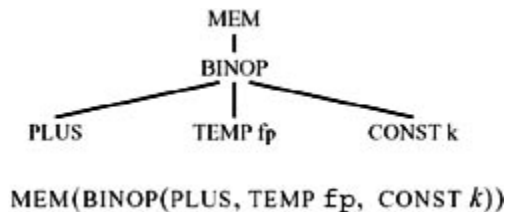
```

Phương thức `unCx` vẫn còn trừu tượng: Chúng ta sẽ bàn thêm sau, với việc dịch các toán tử so sánh. Nhưng các phương thức `unEx` và `unNx` có thể vẫn được cài đặt dựa trên phương thức `unCx`. Chúng ta đã nêu rõ `unEx`; chúng ta để lại `unNx` (vì nó đơn giản) như bài tập.

Phương thức `unCx` của lớp `Ex` cũng để lại như bài tập. Sẽ có ích khi coi `unCx` như các trường hợp của `CONST 0` và `CONST 1`, vì chúng có phần dịch hiệu quả và đặc biệt đơn giản. Các phương thức `unEx` và `unCx` của lớp `Nx` không cần cài đặt, vì các trường hợp này không bao giờ xảy ra trong khi dịch chương trình MiniJava đúng kiểu.

CÁC BIẾN ĐƠN GIẢN

Đối với biến đơn giản `v` được khai báo trong khung ngăn xếp thủ tục hiện thời, chúng ta dịch như sau



trong đó k là offset của `v` bên trong khung và `TEMP  $f_p$` là thanh ghi con trỏ khung. Đối với chương trình dịch MiniJava chúng ta đưa ra giả thiết đơn giản đi là mọi biến có cùng kích thước – kích thước của từ tự nhiên trong máy.

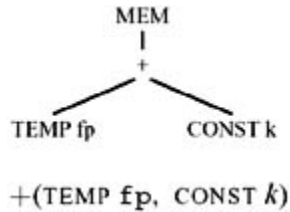
Lớp `Frame` chứa mọi định nghĩa phụ thuộc máy; ở đây chúng ta cộng thêm vào nó thanh ghi con trỏ khung `FP` và hằng số mà giá trị của nó là độ dài của từ tự nhiên trong máy:

```

package Frame;
public class Frame {
    :
    abstract public Temp FP();
    abstract public int wordSize();
}
public abstract class Access {
    public abstract Tree.Exp exp(Tree.Exp framePtr);
}

```

Trong chương này và các chương về sau, chúng ta sẽ viết tắt BINOP(PLUS, e_1 , e_2) như là $+(e_1, e_2)$, như vậy cây ở trên có thể viết dạng



Phương thức `exp` của `Frame.Access` được sử dụng bởi `Translate` để chuyển `Frame.Access` sang biểu thức cây `Tree`. Đối số `Tree.Exp` là địa chỉ của khung ngăn xếp mà `Access` ở đó. Như vậy, đối với truy cập a giống như `InFrame(k)`, ta có

```

a.exp(new TEMP(frame.FP())) =
    MEM(BINOP(PLUS, TEMP(frame.FP()), CONST(k)))
  
```

Nếu a là truy cập thanh ghi giống như `InReg( $t_{832}$ )`, thì đối số địa chỉ khung cho `a.exp()` sẽ được bỏ qua, và kết quả đơn giản sẽ là `TEMP  $t_{832}$` .

Giá trị l -value như vậy như v hoặc $a[i]$ hoặc $p:next$ có thể xuất hiện hoặc trong vế trái hoặc trong vế phải của lệnh gán - l ký hiệu cho *left*, để phân biệt với r -values, mà xuất hiện chỉ trong vế phải của lệnh gán. May mắn, cả hai đỉnh `MEM` và `TEMP` có thể xuất hiện ở vế trái của đỉnh `MOVE`.

CÁC BIẾN MẢNG

Đối với phần còn lại của chương chúng ta sẽ không đặc tả mọi hàm giao diện của `Translate`, như chúng ta đã làm cho `simpleVar`. Nhưng qui tắc chung mới đưa ra áp dụng cho tất cả chúng: Cần có hàm `Translate` kiểm soát các chỉ số của mảng, một cho mỗi trường bản ghi, một cho mỗi kiểu biểu thức.

Các ngôn ngữ lập trình khác nhau làm việc với các biến giá trị mảng khác nhau.

Trong Pascal, biến mảng ký hiệu cho nội dung của mảng – trong trường hợp này có 12 số nguyên. Chương trình Pascal

```

var a,b : array[1..12] of integer
begin
    a:=b
end;
  
```

sẽ sao chép nội dung của mảng a sang mảng b .

Trong C, ở đây không có việc tương tự như biến mảng. Có các biến con trỏ; mảng giống như "hằng con trỏ". Do đó, điều này là không hợp lệ:

```

{int a[12], b[12];
  
```

```
a=b;  
}
```

Nhưng điều này là hợp lệ:

```
{int a[12], *b;  
  b=a;  
}
```

Lệnh `b=a` không sao các phần tử của `a`; thay vào đó, nó có nghĩa là `b` bây giờ trỏ đến bắt đầu của mảng `a`.

Trong MiniJava (như trong Java và ML), các biến mảng cư xử giống như con trỏ. Tuy nhiên MiniJava không đặt tên hằng mảng như trong C. Thay vào đó, giá trị mảng mới được tạo (và khởi tạo) bởi hàm tạo `new int[n]`; trong đó `n` là số phần tử, và 0 là giá trị ban đầu của mỗi phần tử. Trong chương trình

```
int [] a;  
a = new int[12];  
b = new int[12];  
a = b;
```

biến mảng `a` trỏ đến 12 giá trị số không cũng giống như `b`; 12 giá trị số không gốc được cấp dành cho `a` được bỏ qua.

Các đối tượng của MiniJava cũng là các con trỏ. Lệnh gán đối tượng, cũng giống như gán mảng, là phép gán con trỏ và không copy mọi trường (xem dưới đây). Điều này cũng đúng cho các ngôn ngữ lập trình hàm và hướng đối tượng khác, mà tìm cách che mờ sự khác nhau cú pháp giữa con trỏ và đối tượng. Trong C hoặc Pascal, tuy nhiên, giá trị bản ghi là “lớn” và phép gán bản ghi có nghĩa là sao chép mọi trường.

L-GIÁ TRỊ CÓ CẤU TRÚC

l-giá trị là kết quả của biểu thức mà có thể xảy ra ở bên trái của lệnh gán, như là `x` hoặc `p.y` hoặc `a[i+2]`. *r*-giá trị là cái chỉ xuất hiện ở bên phải của phép gán, như `a+3` hoặc `f(x)`. Như vậy, *l*-giá trị ký hiệu vị trí có thể được gán tới, và *r*-giá trị thì không.

Tuy nhiên, *l*-giá trị có thể xuất hiện cả ở bên phải của lệnh gán; trong trường hợp này nội dung của vị trí đó được lấy một cách không tường minh.

Chúng ta nói rằng số nguyên hoặc giá trị con trỏ là vô hướng, vì chúng chỉ có một thành phần. Giá trị như vậy chiếm một từ trong bộ nhớ và có thể vừa khớp vào thanh ghi. Mọi biến và *l*-giá trị trong MiniJava là vô hướng. Ngay cả biến mảng hay biến đối tượng trong MiniJava thực tế cũng là con trỏ (một dạng của vô hướng); Hướng dẫn tham khảo ngôn ngữ *Java* có thể không nói tường minh như vậy, vì nó được lấy từ ngữ nghĩa *Java* chứ không phải cài đặt *Java*.

Trong C hoặc Pascal có các *l*-giá trị có cấu trúc – là cấu trúc trong C, mảng và bản ghi trong Pascal – mà không là các đại lượng vô hướng. Để cài đặt ngôn ngữ với các biến “lớn” như các mảng hay bản ghi trong C hoặc Pascal đòi hỏi thêm một số công việc. Trong chương trình dịch C, kiểu `access` có thể đòi hỏi thông tin về kích cỡ của biến. Và thao tác MEM của ngôn ngữ trung gian TREE có thể cần được mở rộng với định nghĩa về cỡ:

```
package Tree;
abstract class Exp
MEM(Exp exp, int size)
```

Phần dịch của biến cục bộ vào cây IR có dạng sau

MEM(+ (TEMP f_p , CONST k_n), S)

Trong đó S chỉ rõ cỡ của đối tượng được đánh và lưu (phụ thuộc vào cây này có xuất hiện ở vế trái hoặc phải của MOVE không).

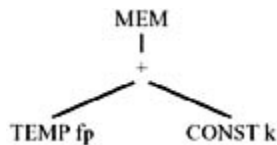
Bỏ qua cỡ các đỉnh MEM làm cho chương trình dịch MiniJava dễ cài đặt hơn, nhưng nó hạn chế tính tổng quát biểu diễn trung gian của nó.

LỰA CHỌN TRƯỜNG VÀ CHỈ SỐ DƯỚI

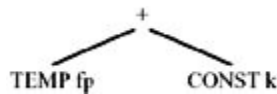
Để đánh chỉ số dưới cho mảng trong Pascal hoặc C (để tính $a[i]$), chỉ cần tính địa chỉ của phần tử thứ i của a : $(i - l) \times s + a$, trong đó l là cận dưới của miền chỉ số, s là cỡ theo bytes của mỗi phần tử mảng, và a là địa chỉ cơ sở của các phần tử mảng. Nếu a là toàn cục, với địa chỉ không đổi trong thời gian dịch, thì hiệu $a - s \times l$ có thể được tính trong thời gian dịch.

Tương tự, để chọn trường f của bản ghi *l*-giá trị a (để tính $a.f$), chỉ cần cộng offset trường không đổi của f vào địa chỉ a .

Biến mảng a là *l*-giá trị; do đó biểu thức mảng $a[i]$ cũng là *l*-giá trị, ngay cả nếu i không là *l*-giá trị. Để tính *l*-giá trị $a[i]$ từ a , chúng ta tính toán địa chỉ của a . Vì vậy, trong chương trình dịch Pascal, phần dịch *l*-giá trị (đặc biệt *l*-giá trị có cấu trúc) sẽ không được giống như



Nhưng thay vào đó là biểu thức cây Tree biểu diễn địa chỉ cơ sở của mảng:

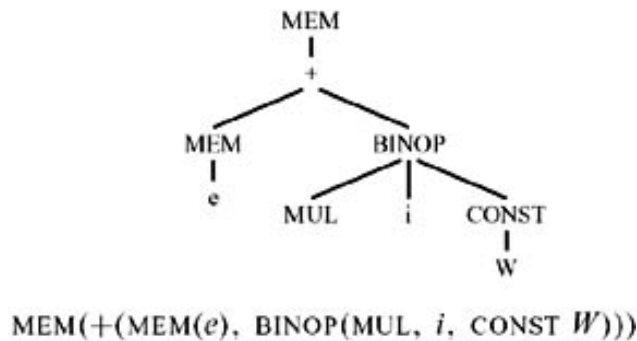


Điều gì xảy ra với l -giá trị này?

- Một phần tử riêng biệt nào đó có thể được đánh chỉ số, tạo ra (smaller) l -giá trị (nhỏ hơn). Đỉnh "+" sẽ cộng cờ phần tử nhân với chỉ số cho l -giá trị của cơ sở mảng đó.
- l -giá trị này (biểu diễn toàn bộ mảng) có thể được sử dụng trong bối cảnh ở đó r -giá trị được yêu cầu (chẳng hạn truyền tham trị, hoặc gán cho biến mảng khác). Khi đó l -giá trị được ép buộc thành r -giá trị bằng việc áp dụng toán tử MEM cho nó.

Trong ngôn ngữ MiniJava, không có l -giá trị có cấu trúc hoặc “lớn”. Vì mọi đối tượng và giá trị mảng thực tế là các con trỏ đến đối tượng và cấu trúc mảng. “Địa chỉ cơ sở” của mảng thực tế là nội dung của biến con trỏ, do đó MEM được yêu cầu để tìm địa chỉ cơ sở này.

Như vậy, nếu a là biến mảng thường trú trong bộ nhớ biểu diễn bởi $\text{MEM}(e)$, thì nội dung của địa chỉ e sẽ là giá trị con trỏ một từ p . Các nội dung của các địa chỉ p , $p + W$, $p + 2W$, ... (trong đó W là cỡ của từ) sẽ là các phần tử của mảng (mọi phần tử đều có độ dài một từ). Vậy, $a[i]$ đơn giản là



các l -giá trị và các đỉnh MEM. Về mặt kỹ thuật, l -giá trị (hoặc biến gán được) cần được thể hiện như địa chỉ (không có đỉnh MEM ở trên cùng như trong sơ đồ trên). Chuyển l -giá trị thành r -giá trị (khi nó được sử dụng trong biểu thức) có nghĩa là tìm được từ địa chỉ này; gán nó cho l -giá trị có nghĩa là lưu vào địa chỉ đó. Chúng ta đính kèm đỉnh MEM vào l -giá trị trước khi biết nó có được tìm thấy là lưu lại không; việc này thực hiện được chỉ vì trong cây biểu diễn trung gian Tree , MEM có nghĩa là cả lưu *store* (khi được sử dụng như con trái của MOVE) và tìm *fetch* (khi được sử dụng ở nơi khác).

TRAO ĐỔI VỀ TÍNH AN TOÀN

Khi chương trình có lỗi, nó cần phải được phát hiện càng sớm càng tốt và tuyên bố về việc đó trước khi lỗi gây ra thiệt hại nào đó.

Một số lỗi khó phát hiện. Nhưng không qua khó để phát hiện chỉ số mảng vượt quá giới hạn: nếu mảng có giới hạn là $L..H$, và chỉ số mảng là i , thì $i < L$ hoặc $i > H$ là lỗi biên của mảng. Hơn nữa, máy tính được trang bị tốt phần cứng có khả năng tính điều kiện $i >$

H. Trong vài thập kỷ gần đây, chúng ta biết rằng chương trình dịch có thể tự động sinh ra đoạn mã để kiểm tra điều kiện này. Không có lời biện hộ nào cho chương trình dịch nếu nó không thể sinh ra đoạn code để kiểm tra các giới hạn của mảng. Chương trình dịch tối ưu có thể loại bỏ một cách an toàn việc kiểm tra trong thời gian dịch.

Một ai đó có thể nói, một cách bao biện, "nhưng ngôn ngữ mà tôi lập trình có kiểu tính toán địa chỉ mà không thể biết được giới hạn của mảng."

Trong một số hoàn cảnh, phần chương trình đòi hỏi tốc độ nhanh, và quỹ thời gian không cho phép kiểm tra giới hạn biên. Trong trường hợp như vậy, tốt nhất là chương trình dịch tối ưu có thể phân tích biểu thức chỉ số và chứng minh rằng chỉ số vẫn luôn ở trong giới hạn, như vậy không cần thiết việc kiểm tra biên tường minh nào cả. Nếu điều đó không thể, có thể chấp nhận được là trong những trường hợp khó xảy ra đó đó cho phép lập trình viên chỉ rõ thao tác chỉ số không được kiểm tra một cách tường minh. Nhưng điều này không miễn cho chương trình dịch khỏi việc kiểm tra mọi biểu thức chỉ số khác trong chương trình đó.

Cũng không cần phải nói là chương trình dịch cần phải kiểm tra các con trỏ `nil` trước khi sử dụng chúng.^[1]

PHÉP TOÁN SỐ HỌC

Số học số nguyên dễ dàng dịch. Mỗi phép toán số học tương ứng với phép toán cây.

Ngôn ngữ cây `Tree` không có phép toán số học một ngôi. Phép đảo dấu một ngôi của số nguyên được cài đặt như phép lấy 0 trừ nó; Phần bù logic một ngôi được cài đặt như XOR với toàn số 1.

Phép đảo dấu số phẩy động không thể cài đặt như phép trừ 0 cho nó, vì nhiều biểu diễn số phẩy động cho phép không âm *negative zero*. Đảo của số không âm *negative zero* là số không dương, và ngược lại. Như vậy, ngôn ngữ cây `Tree` không hỗ trợ đảo một ngôi.

Cũng may là ngôn ngữ MiniJava không hỗ trợ các số phẩy động; nhưng trong chương trình dịch thực tế, phép toán mới có thể được bổ sung cho đảo của số dấu phẩy động.

PHÉP TOÁN CÓ ĐIỀU KIỆN

Kết quả của phép toán so sánh sẽ là biểu thức `Cx`: lệnh `s` sẽ nhảy đến đích đúng hoặc đích sai mà bạn đã đặc tả.

Làm cho biểu thức `Cx` “đơn giản” từ các phép so sánh được thực hiện với phép toán `CJUMP`. Tuy nhiên, toàn bộ điểm của biểu diễn `Cx` là biểu thức có điều kiện mà dễ dàng được kết hợp với phép toán MiniJava.

Khi đó, biểu thức như `x < 5` sẽ được dịch như `Cx(s1)`, trong đó

$$s_1(t, f) = \text{CJUMP}(\text{LT}, x, \text{CONST}(5), t, f)$$

đối với các nhân t và f bất kỳ.

Để làm điều đó, chúng ta mở rộng lớp Cx tạo thành lớp con subclass RelCx mà có trường riêng để giữ các biểu thức trái và phải (trong trường hợp này là x và 5) và phép toán so sánh (trong trường hợp này Tree.CJUMP.LT). Khi đó chúng ta ghi đề phương thức unCx để sinh ra CJUMP từ các dữ liệu này. Không cần thiết phải tạo các phương thức unEx và unNx , vì chúng được kế thừa từ lớp cha Cx .

Việc đơn giản nhất là xử lý với biểu thức if

if e_1 then e_2 else e_3

được coi như e_1 là biểu thức Cx , và e_2, e_3 là biểu thức Ex . Như vậy, sử dụng phương thức unCx của e_1 và unEx của e_2 và e_3 . Dùng hai nhân t và f để điều kiện rẽ nhánh. Cấp biến tạm thời r , và sau nhân t , chuyển e_2 về r ; sau nhân f , chuyển e_3 về r . Cả hai nhánh sẽ kết thúc bằng việc nhảy đến nhân "join" mới được tạo ra.

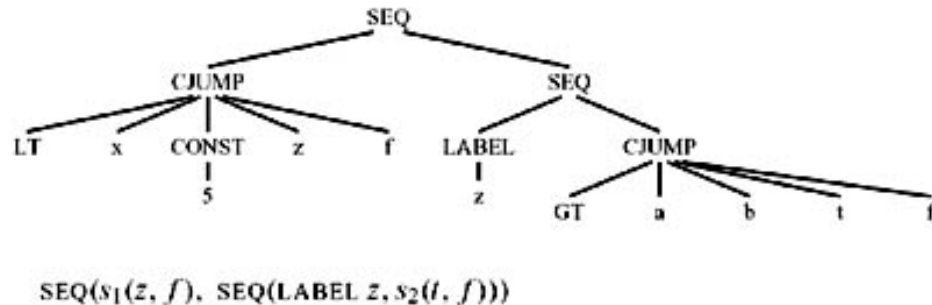
Điều đó sẽ tạo ra kết quả đúng hoàn toàn. Tuy nhiên, mã dịch có thể không hiệu quả một chút nào. Nếu e_2 và e_3 cả hai đều là lệnh "statements" (biểu thức mà không trả về giá trị), thì biểu diễn của chúng giống như Nx , chứ không phải Ex . Áp dụng unEx cho chúng sẽ hoạt động – sự ép buộc sẽ tự động được áp dụng – nhưng có thể sẽ tốt hơn nếu thừa nhận trường hợp này riêng biệt.

Sẽ tồi tệ hơn, nếu e_2 hoặc e_3 là biểu thức Cx , khi đó ép kiểu unEx cho nó sẽ tạo ra tình trạng các bước nhảy và nhân một cách đáng sợ. Tốt hơn nhiều nếu thừa nhận đây là trường hợp riêng biệt.

Ví dụ, xét

if $x < 5$ then $a > b$ else 0

Như đã chỉ ra ở trên, $x < 5$ dịch sang $Cx(s_1)$; tương tự, $a > b$ sẽ được dịch như $Cx(s_2)$ đối với s_2 nào đó. Toàn bộ lệnh if -statement cần tạo ra giống như sau



đối với nhân mới z nào đó.

Như vậy, việc dịch của **if** yêu cầu lớp con mới của biểu thức `Exp`:

```
class IfThenElseExp extends Exp {
    Exp cond, a, b;
    Label t = new Label();
    Label f = new Label();
    Label join = new Label();
    IfThenElseExp(Exp cc, Exp aa, Exp bb) {
        cond=cc; a=aa; b=bb;
    }
    Tree.Stm unCx(Label tt, Label ff) { ... }
    Tree.Exp unEx() { ... }
    Tree.Stm unNx() { ... }
}
```

Các nhãn `t` và `f` chỉ ra bắt đầu của mệnh đề then-clause và mệnh đề elseclause, tương ứng. Các nhãn `tt` và `ff` là hoàn toàn khác: Đó là các vị trí mà đến đó các điều kiện bên trong then-clause (hoặc else-clause) cần nhảy tới, phụ thuộc vào tính đúng đắn của các biểu thức con đó.

CÁC XÂU

Xâu chữ thường được cài đặt như địa chỉ không đổi của đoạn bộ nhớ khởi tạo cho các ký tự thích hợp. Trong ngôn ngữ assembly, nhãn được sử dụng để tham chiếu đến địa chỉ này từ giữa dãy lệnh nào đó. Tại vị trí khác nào đó của chương trình assembly, định nghĩa của nhãn này được xuất hiện, tiếp theo là giả lệnh của assembly để dành và khởi tạo cho khối bộ nhớ cho các ký tự thích hợp.

Đối với mỗi xâu chữ `lit`, bộ dịch cần tạo nhãn mới `lab`, và trả về cây `Tree.NAME(lab)`. Nó cũng cần đặt đoạn viết trên ngôn ngữ assembly `frame.string(lab, lit)` vào trong danh sách tổng thể các đoạn như vậy để chuyển qua bộ phận sinh mã. Các đoạn "Fragments" sẽ được bàn đến sau.

Mọi thao tác xâu được thực hiện dạng các hàm số được cấp bởi hệ thống thời gian thực; Các hàm này cấp không gian đồng heap-allocate cho kết quả của chúng, và trả về các con trỏ. Vì vậy chương trình dịch (hầu như) không cần biết đến biểu diễn dạng nào, mà chỉ biết rằng mỗi con trỏ xâu có độ dài chính xác là một từ. Chúng ta nói "hầu như" vì xâu chữ cần phải được biểu diễn.

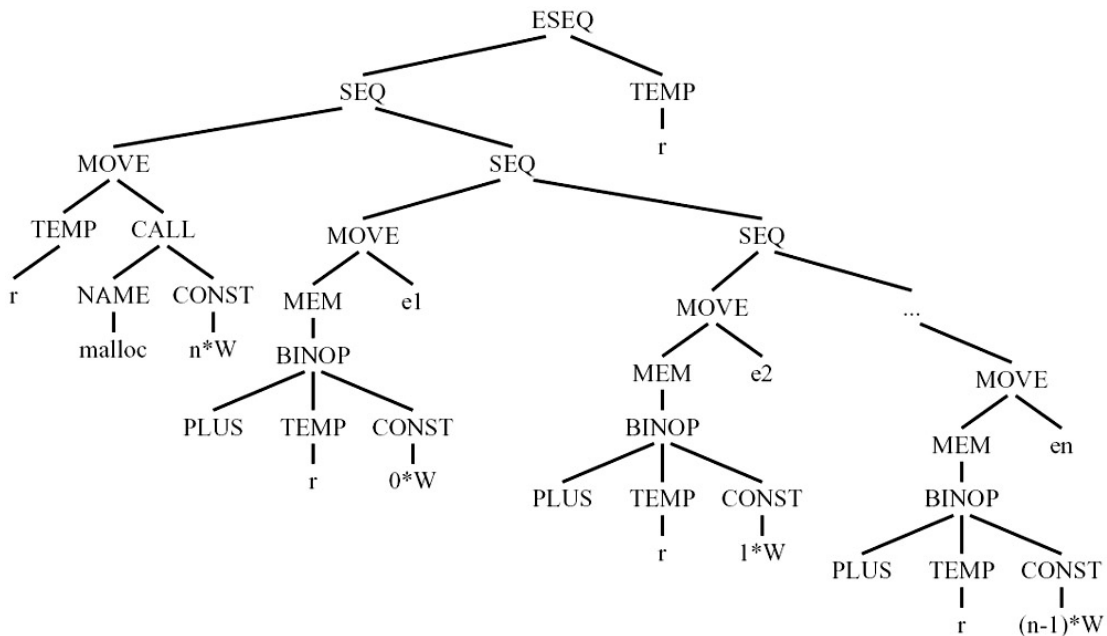
Trong Pascal, các xâu là các mảng ký tự có độ dài cố định; các chữ được bổ xung các khoảng trống thích hợp. Điều này không có ích lắm. Trong C, xâu là các con trỏ đến các dãy kết thúc 0 có độ dài thay đổi. Điều này có ích hơn nhiều, mặc dù xâu chứa byte 0 không thể được biểu diễn.

TẠO MẢNG VÀ BẢN GHI

Tương tượng cấu trúc ngôn ngữ $\{e_1, e_2, \dots, e_n\}$ mà tạo ra bản ghi n -phần tử được khởi tạo bởi các giá trị của biểu thức e_i . Điều này giống như cấu trúc đối tượng mà khởi tạo

mọi biến ban đầu của đối tượng. Bản ghi như vậy có thể sống vượt qua cả bản ghi kích hoạt thủ tục mà tạo ra nó, như vậy nó không thể được đặt trên ngăn xếp. Thay vào đó, nó cần được đặt trên đồng *heap*. Nếu không có kế hoạch để giải phóng các bản ghi (hoặc các xâu) các hệ thống có sức mạnh công nghiệp cần có bộ thu dọn rác *garbage collector* để giải phóng các bản ghi mà không dùng tới nữa.

Cách đơn giản nhất để tạo bản ghi là gọi hàm cấp bộ nhớ ngoài mà trả về con trỏ đến vùng n -từ vào trong biến tạm thời mới r . Sau đó dãy các cây MOVE có thể khởi tạo offsets $0, 1W, 2W, \dots, (n-1)W$ từ r với việc dịch các biểu thức e_i . Cuối cùng, kết quả của toàn bộ biểu thức là $\text{TEMP}(r)$, như đã nêu trên Hình 8.5.



Hình 8.5: Khởi tạo đối tượng.

Trong chương trình dịch công nghiệp, gọi `malloc` (hoặc tương đương với nó) cho mỗi lần tạo bản ghi có thể là quá chậm; xem 13.7 [1].

Việc tạo mảng rất giống với tạo bản ghi, trừ việc mọi trường được khởi tạo với cùng một giá trị. Hàm ngoài `initArray` có thể lấy độ dài mảng và giá trị ban đầu như đối số.

Trong MiniJava chúng ta có thể tạo ra mảng các số nguyên bởi cấu trúc

```
new int [exp]
```

trong đó `exp` là biểu thức tính toán trên số nguyên. Nó sẽ tạo ra mảng số nguyên có độ dài được xác định bởi giá trị của `exp` và với mỗi giá trị khởi tạo bằng 0.

Để dịch việc tạo mảng, chương trình dịch cần thực hiện các bước sau:

1. Xác định cần bao nhiêu không gian cho mảng. Nó có thể được tính như sau:

$$((\text{Length of array}) + 1) \times (\text{size of Integer, chẳng hạn } 4)$$

Lý do chúng ta bổ sung thêm 1 vào độ dài của mảng là chúng ta muốn lưu độ dài của mảng cùng với mảng. Điều này cần thiết để kiểm tra biên và xác định độ dài trong thời gian chạy.

2. Gọi hàm ngoài để nhận được không gian đồng. Lời gọi sẽ trả về con trỏ đến đầu của mảng.
3. Sinh mã để lưu độ dài của mảng tại offset 0.
4. Sinh mã khởi tạo mỗi giá trị trong mảng bằng 0 bắt đầu tại offset 4.

Gọi hàm hệ thống thời gian chạy. Để gọi hàm ngoài tên `init-Array` với đối số `a, b`, đơn giản sinh *CALL* như sau

```
static Label initArray = new Label("initArray");
new CALL(new NAME(initArray),
         new Tree.ExpList(a, new Tree.ExpList(b, null)));
```

Nó tham chiếu đến hàm ngoài `initArray` mà được viết trên ngôn ngữ C như hoặc assembly – nó không thể được viết trên MiniJava vì MiniJava không có cơ chế thao tác bộ nhớ thô.

Nhưng trên hệ điều hành nào đó, chương trình dịch C sẽ bổ sung gạch dưới vào đầu mỗi nhãn; và qui ước gọi hàm cho C có thể khác so với MiniJava; và các hàm C không hy vọng sẽ nhận được liên kết tĩnh, Mọi chi tiết mang đặc thù của máy đích cần được đóng gói thành hàm cung cấp bởi cấu trúc `Frame`:

```
public abstract class Frame {
    :
    abstract public Tree.Exp externalCall(String func,
                                         Tree.ExpList args);
}
```

trong đó `externalCall` lấy tên của thủ tục ngoài và đối số được truyền.

Cài đặt `externalCall` phụ thuộc vào quan hệ giữa qui ước lời gọi thủ tục của MiniJava và hàm bên ngoài. Cài đặt có thể đơn giản nhất trông giống như

```
Tree.Exp externalCall(String s, Tree.ExpList args) {
    return new Tree.CALL(new Tree.NAME(new Temp.Label(s)),
                        args);
}
```

nhưng có thể được điều chỉnh các liên kết tĩnh, hoặc các gạch dưới trong nhãn, ... Cũng như vậy, lời gọi `new Label(s)` được lặp lại với cùng một tên `s` tạo ra một số đối tượng nhãn mà tất cả chúng đều ý nghĩa như nhau; nó có thể làm nhầm lẫn cho các phần của

chương trình dịch, như vậy nó có thể có ích nhằm duy trì bảng xâu tới nhãn để tránh lặp trùng.

VÒNG LẶP WHILE

Giao diện tổng quan của vòng lặp **while** có dạng sau

```
test:
        if not(condition) goto done body
        goto test
done:
```

Nếu lệnh **break** xảy ra bên trong thân vòng lặp *body* (và không lồng bên trong các lệnh **while** bên trong), phần dịch đơn giản sẽ nhảy JUMP đến *done*.

Phần dịch lệnh **break** cần phải cần phải có tham số hình thức mới *break* mà là nhãn *done* của vòng lặp đóng gần nhất. Trong khi dịch vòng lặp **while**, bộ dịch sẽ được gọi đệ qui thân vòng lặp với nhãn *done* được truyền như tham số *break*. Khi bộ dịch gọi đệ qui chính nó trong bối cảnh không vòng lặp, nó đơn giản truyền cùng tham số *break* mà vừa được truyền cho nó.

VÒNG LẶP FOR

Lệnh **for** có thể được biểu diễn sử dụng các kiểu lệnh khác:

```
for (i=lo; i<=hi; i++;) {
    // body
}

i=lo;
limit=hi;
while (i<=limit) {
    // body
    i++;
}
```

Tiếp cận trực tiếp cho phần dịch lệnh **for** là viết lại cú pháp trừu tượng dưới dạng cú pháp trừu tượng của lệnh **while** được nêu trên, và sau đó dịch kết quả đó.

Điều này gần đúng, nhưng xét trường hợp ở đó $limit = maxint$. Khi đó $i + 1$ sẽ bị tràn; hoặc lỗi phần cứng sẽ đưa ra, hoặc $i \leq limit$ sẽ luôn luôn đúng! Giải pháp là đặt việc kiểm tra ở đáy *bottom* của vòng lặp, ở đó $i < limit$ sẽ được kiểm tra trước khi tăng. Sau đó việc kiểm tra bổ sung là cần thiết trước khi vào vòng lặp để kiểm tra $lo \leq hi$.

LỜI GỌI HÀM

Dịch lời gọi hàm $f(a_1, \dots, a_n)$ đơn giản là:

```
CALL(NAME  $f$ , [ $e_1, e_2, \dots, e_n$ ])
```

Trong đó l_f là nhãn của hàm f . Trong ngôn ngữ hướng đối tượng, biến ẩn `this` cần được làm thành đối số tường minh của lời gọi. Vậy $p.m(a_1, \dots, a_n)$ được dịch là

CALL(NAME $l_{c\$m}$, [$p, e_1, e_2, \dots, e_n$])

trong đó p thuộc lớp c , và $c\$m$ là phương thức m của lớp c . Đối với các phương thức tĩnh, việc tính toán địa chỉ $l_{c\$m}$ có thể được thực hiện trong thời gian dịch – đây là nhãn đơn giản, như trong MiniJava. Đối với các phương thức động, việc tính toán này phức tạp hơn (như giải thích trong Chương 14 [1]).

LIÊN KẾT TĨNH

Một số ngôn ngữ lập trình (như Pascal, Scheme, và ML) hỗ trợ lồng hàm sao cho các hàm bên trong có thể tham chiếu đến các biến được khai báo trong các hàm bên ngoài. Khi xây dựng chương trình dịch cho ngôn ngữ lập trình như vậy, biểu diễn khung và truy cập biến phức tạp hơn một chút.

Khi biến x được khai báo ở mức ngoài tầm nhìn tĩnh, liên kết tĩnh cần được sử dụng. Dạng tổng quát là

**MEM(+ (CONST k_n , MEM(+ (CONST $k_{n-1}, \dots$
MEM(+ (CONST k_1 , TEMP FP)) ...)))**

trong đó $k_1, \dots, k_{n-1}$ là các offsets liên kết tĩnh khác nhau trong các hàm lồng nhau, và k_n là offset của x trong khung riêng của nó.

Trong khi tạo các biến tạm thời `TEMP`, các biến tạm mà biến mất (tức là đã được gọi từ trong các hàm bên trong) cần được cất trên khung ngăn xếp, không phải trong thanh ghi. Khi truy cập các biến tạm đó từ cùng một hàm hoặc từ hàm bên trong, chúng ta cần phải truyền liên kết tĩnh tương ứng. Phương thức `exp` của `Frame.Access` cần phải tính toán dãy tham chiếu thích hợp.

Phản dịch lời gọi hàm $f(a_1, \dots, a_n)$ sử dụng các liên kết tĩnh đòi hỏi một liên kết tĩnh cần được bổ sung như đối số ẩn được thêm vào:

CALL(NAME l_f , [$sl, e_1, e_2, \dots, e_n$])

Ở đây l_f là nhãn cho f , và sl là liên kết tĩnh, tính toán như mô tả trong chương 6. Để thực hiện tính toán đó, cả mức (`level`) của f và mức của hàm gọi f đều được yêu cầu. Dãy các offsets (có thể không có) tìm thấy ở các mô tả mức liên tiếp sẽ nhận được, bắt đầu từ con trỏ khung `TEMP(FP)` xác định bởi module `Frame`.

[1] Cách khác để kiểm tra `nil` là không sắp đặt trang 0 trong bảng trang bộ nhớ ảo, sao cho việc tìm cách lấy/lưu các trường của bản ghi `nil` cho lại kết quả lỗi trang.

KHAI BÁO

Khi khai báo mỗi biến bên trong thân hàm, không gian bổ sung được giành riêng ở trên khung. Cũng như vậy, mỗi khai báo hàm, một đoạn "fragment" mới của mã cây Tree sẽ được giành cho thân của hàm này.

ĐỊNH NGHĨA BIẾN

Phần dịch khai báo biến cần trả về môi trường kiểu đối số mà được sử dụng trong việc xử lý phần còn lại của thân hàm.

Tuy nhiên, phần khởi đầu biến dịch qua biểu thức cây cần phải được đặt trước thân của hàm. Vì vậy, phần dịch cần trả về `Translate.Exp` chứa các biểu thức gán mà thực hiện các khởi tạo này.

Nếu phần dịch được áp dụng cho hàm và khai báo kiểu, kết quả sẽ là biểu thức "no-op" như `Ex(CONST(0))`.

ĐỊNH NGHĨA HÀM

Hàm được dịch sang đoạn ngôn ngữ assembly với mở đầu, thân và kết luận. Phần thân hàm là biểu thức, và thân phần dịch đơn giản là dịch biểu thức.

Phần mở đầu, mà đứng trước thân trong phiên bản ngôn ngữ assembly của hàm chứa

1. Các chỉ dẫn giả mã, như cần thiết trong ngôn ngữ assembly cụ thể để tuyên bố bắt đầu hàm;
2. Định nghĩa nhãn cho tên của hàm;
3. Chỉ dẫn để điều chỉnh con trỏ ngăn xếp (để cấp khung mới);
4. Chỉ dẫn để lưu các đối số biến mất vào khung, và để chuyển các đối số không biến mất vào thanh ghi tạm thời mới;
5. Giữ các chỉ dẫn để lưu thanh ghi được lưu nơi gọi – bao gồm thanh ghi địa chỉ trả về - được sử dụng trong hàm. Và sau đó đến
6. Thân hàm.
Phần kết đến sau thân hàm và bao gồm
7. Chỉ dẫn chuyển và trả về giá trị (kết quả của hàm) cho thanh ghi lưu giữ cho mục đích này;
8. Tái các chỉ dẫn để khôi phục thanh ghi lưu nơi gọi;
9. Chỉ dẫn để tái khởi tạo con trỏ ngăn xếp (để cấp lại khung);
10. Chỉ lệnh trả về *return* (nhảy JUMP đến địa chỉ trả về);
11. Các chỉ lệnh giả mã, mà cần thiết, để tuyên bố kết thúc hàm.

Một số mục này (1, 3, 9 và 11) phụ thuộc vào hiểu biết chính xác kích thước khung, mà không được biết cho đến sau khi bộ cấp thanh ghi xác định có bao nhiêu biến cần phải giữ trong khung vì chúng không thể chứa hết trong thanh ghi. Vì các chỉ dẫn này có thể

được sinh ra rất muộn, trong hàm `FRAME` gọi `procEntryExit3`. Mục 2 (và 10), lồng nhau giữa 1 và 3 (và 9 và 11, tương ứng) cũng được kiểm soát tại cùng một thời điểm.

Để cài đặt 7, giai đoạn dịch `Translate` cần phải sinh ra chỉ thị dịch chuyển `move`

```
MOVE(RV, body)
```

mà đặt kết quả tính toán thân `body` vào vị trí giá trị trả về (`RV`) được chỉ ra bởi cấu trúc khung `frame` mang đặc thù máy:

```
package Frame;
public abstract class Frame {
    :
    abstract public Temp RV();
}
```

Mục 4 (chuyển các tham số hình thức đến), và 5 và 8 (lưu và khôi phục các thanh ghi lưu nơi gọi), là phần của dịch chuyển tầm nhìn *view shift* được mô tả ở phần sau. Chúng có thể được thực hiện bởi hàm trong module `Frame`:

```
package Frame;
public abstract class Frame {
    :
    abstract public Tree.Stm procEntryExit1(Tree.Stm body);
}
```

Việc cài đặt hàm này sẽ được bàn đến trong tài liệu tham khảo. Phần dịch `Translate` cần được áp dụng cho mỗi thân thủ tục (mục 5-7) như nó được dịch.

CÁC PHÂN ĐOẠN - FRAGMENTS

Cho trước định nghĩa hàm gồm biểu thức `body` đã được dịch, giai đoạn dịch `Translate` sẽ tạo ra mô tả hàm chứa các thông tin cần thiết:

- **frame:** mô tả khung chứa thông tin đặc thù máy về các biến và tham số cục bộ;
- **body:** Kết quả trả về từ `procEntryExit1`.

Gọi cặp này là phân đoạn *fragment* cần được dịch ra ngôn ngữ assembly. Đây là kiểu đoạn thứ hai mà chúng ta đã thấy; kiểu đoạn khác là dãy giả chỉ lệnh ngôn ngữ assembly cho xâu chữ. Vì vậy, sẽ là hữu ích để định nghĩa (trong giao diện `Translate`) kiểu dữ liệu `frag`:

```
package Translate;
public abstract class Frag { public Frag next; }
public ProcFrag(Tree.Stm body, Frame.Frame frame);
public DataFrag(String data);
```

CHƯƠNG TRÌNH 8.6: A MiniJava program.

```
class Vehicle {
    int position;
    int gas;
    int move (int x) {
        position = position + x;
        return position;
    }
    int fill (int y) {
        gas = gas + y;
        return gas;
    }
}

public class Translate {
    :
    private Frag frags; // linked list of accumulated fragments
    public void procEntryExit(Exp body);
    public Frag getResult();
}
```

Giai đoạn phân tích ngữ nghĩa gọi `new Translate.Level(...)` trong khi xử lý phần đầu hàm. Sau đó nó gọi các phương thức khác của `Translate` để dịch thân của hàm. Cuối cùng bộ phân tích ngữ nghĩa gọi `procEntryExit`, mà có tác động phụ của việc nhớ `ProcFrag`.

Mọi phân đoạn được nhớ đưa vào danh sách phân đoạn riêng bên trong `Translate`; khi đó `getResult` có thể được sử dụng để lấy ra danh sách phân đoạn.

CÁC LỚP VÀ CÁC ĐỐI TƯỢNG

Hình 8.6 nêu lớp MiniJava `Vehicle` mà có hai biến khởi tạo `position` và `gas`, và hai phương thức `move` và `fill`. Chúng ta có thể tạo ra nhiều đối tượng `Vehicle`. Mỗi đối tượng `Vehicle` sẽ có các biến `position` và `gas` riêng của mình. Hai đối tượng `Vehicle` có thể có giá trị các biến khác nhau, và trong MiniJava, chỉ các phương thức của đối tượng truy cập đến các biến của đối tượng đó. Phần dịch `new Vehicle()` rất giống với phần dịch việc tạo bản ghi và có thể thực hiện qua hai bước:

1. Sinh mã để cấp không gian đống *heap* cho các biến khởi tạo; trong trường hợp này chúng ta cần cấp 8 bytes (2 số nguyên, mỗi số cỡ 4).
2. Lấp qua các vị trí bộ nhớ cho các biến này và khởi tạo chúng – trong trường hợp này cả hai chúng được khởi tạo bằng 0.

Các phương thức và con trỏ *this*. Lời gọi phương thức trong MiniJava tương tự như lời gọi hàm; nhưng trước hết, chúng ta cần xác định lớp mà trong đó phương thức được khai báo và tìm kiếm phương thức trong lớp đó. Thứ hai, chúng ta gặp phải câu hỏi sau. Giả sử chúng ta có nhiều đối tượng `Vehicle` và chúng ta muốn gọi một phương thức của một

trong các đối tượng đó; chúng ta tin tưởng như thế nào rằng cài đặt biết đối tượng nào chúng ta gọi phương thức? Giải pháp là truyền đối tượng như đối số bổ sung cho phương thức đó; đối số này là con trỏ *this*. Đối với lời gọi phương thức

```
Vehicle v;  
...  
v.move();
```

đối tượng `Vehicle` trong biến `v` sẽ là con trỏ *this* khi gọi phương thức `move`.

Phần dịch khai báo phương thức rất giống phần dịch hàm, nhưng chúng ta cần tránh xung đột tên trong các phương thức với cùng một tên nhưng khai báo trong các phương thức khác nhau. Chúng ta thực hiện việc này bằng cách chọn sơ đồ tên sao cho tên của phương thức dịch là sự nối tiếp giữa tên lớp và tên phương thức. Chẳng hạn, phần dịch `move` có thể được cho bởi tên `Vehicle move`.

Truy cập các biến trong MiniJava, các biến có thể được truy cập từ các phương thức trong cùng một lớp. Các biến được truy cập qua con trỏ *this*; vì vậy, phần dịch tham chiếu biến giống như chọn trường cho bản ghi. Vị trí của biến trong đối tượng có thể được tìm trong bảng ký hiệu *Symbol table* đối với lớp này.

DỊCH CHƯƠNG TRÌNH SANG CÂY

Thiết kế tập các visitors mà dịch chương trình MiniJava sang cây biểu diễn trung gian.

Các files hỗ trợ trong `$MINIJAVA/chap7` bao gồm:

```
Tree/* Data types for the Tree language.  
Tree/Print.java Functions to display trees for debugging.
```

Bộ dịch đơn giản. Để đơn giản bớt việc cài đặt bộ dịch, bạn có thể thực hiện không cần các cấu trúc `Ex`, `Nx`, `Cx`. Toàn bộ bộ dịch cần được thực hiện với các biểu thức giá trị thông thường. Điều đó có nghĩa là ở đây chỉ có một lớp `Exp` (không có các lớp con); lớp này chứa một trường kiểu `Tree.Exp` và chỉ có phương thức `unExp()`. Thay vì `Nx(s)`, sử dụng `Ex(ESEQ(s, CONST 0))`. Với lệnh điều kiện thay vì `Cx`, sử dụng biểu thức mà tính giá trị 1 hoặc 0.

Các cây biểu diễn trung gian tạo ra từ kiểu dịch ngây thơ này sẽ là công kênh hơn và chậm hơn bộ dịch “đẹp”. Nhưng chúng sẽ làm việc đúng, và về nguyên tắc bộ tối ưu phần sau tốt có thể được sử dụng để xóa bỏ mọi chỗ làm vụng về. Trong trường hợp bất kỳ, sự vụng về nhưng dịch đúng vẫn tốt hơn bộ dịch “đẹp” nhưng không làm việc.

BAO ĐÓNG (CLOSURES)

Trong các ngôn ngữ (như C) mà không có các hàm lồng nhau, biểu diễn thời gian chạy của giá trị hàm cần phải là địa chỉ mã máy cho hàm đó. Địa chỉ này cần truyền như đối số, được lưu trong biến; khi đến lúc gọi hàm, địa chỉ sẽ được tải đến thanh ghi máy, và “gọi đến chỉ lệnh “địa chỉ chứa trong thanh ghi” được sử dụng.

Trong biểu diễn trung gian Cây, điều này rất dễ biểu diễn. Giả sử hàm bắt đầu tại nhãn L_{123} ; chúng ta gán địa chỉ cho biến t_{57} sử dụng

```
MOVE (TEMP ( $t_{57}$ ) NAME ( $L_{123}$ ))
```

Và sau đó gọi đến hàm với một điều gì đó như

```
CALL (TEMP ( $t_{57}$ ), ... parameters ...).
```

Nhưng điều này không làm việc đối với các hàm lồng nhau; nếu chúng ta biểu diễn hàm h bằng địa chỉ, ở khung ngoài nào có thể truy cập đến biến n ? Tương tự, hàm g truy cập đến biến f như thế nào?

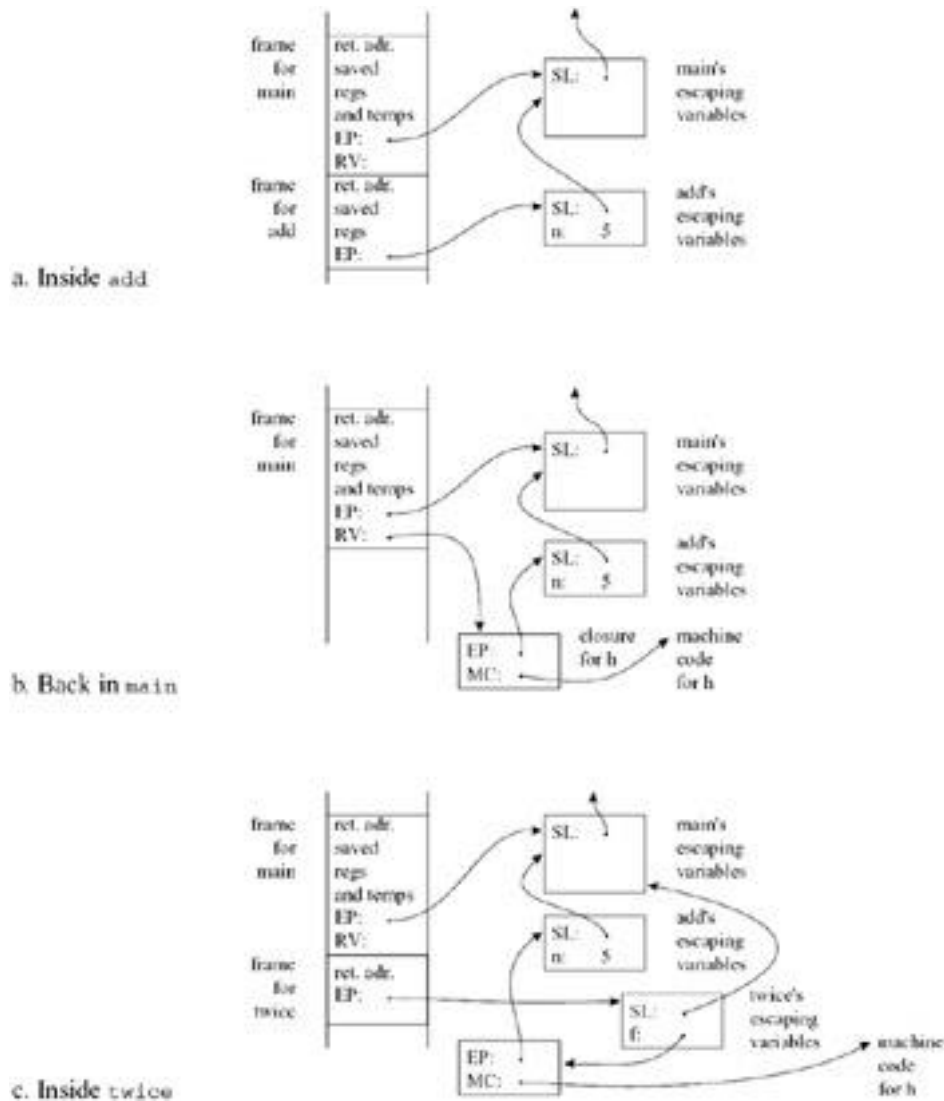
Giải pháp là biểu diễn biến hàm như bao đóng *closure*: bản ghi mà chứa con trỏ mã máy và cách truy cập các biến không cục bộ cần thiết. Điều này rất giống với đối tượng có phương thức duy nhất (con trỏ mã máy) và một số biến bản tạo. Một phần bao đóng cho phép truy cập đến giá trị của các biến thường được gọi là môi trường (*environment*).

Các bao đóng cần không được dựa trên các đối tượng; bất cứ cấu trúc dữ liệu nào khác mà cho phép truy cập đến các biến không cục bộ sẽ được dùng. Tuy nhiên, trong chương này chúng ta sẽ sử dụng các đối tượng cho đơn giản.

BẢN GHI KÍCH HOẠT ĐƯỢC CẤP TRÊN HEAP

Các biến cục bộ cho `add` cần không được hủy khi `add` trả về, vì n còn cần đến để thực thi h . Để giải quyết vấn đề này, chúng ta cần tạo đối tượng được cấp trên heap để giữ các biến cục bộ của mỗi hàm; sau đó chúng ta dựa trên bộ thu dọn rác để giải phóng đối tượng khi mọi tham chiếu đã biến mất (kể cả các giá trị hàm lồng bên trong).

Làm mịn kỹ thuật này là lưu trên heap chỉ những biến mất đi (những cái đã sử dụng bởi các hàm lồng bên trong). Khung ngăn xếp sẽ giữ các thanh ghi, địa chỉ trả về và cả con trỏ đến *bản ghi biến mất đi*. Bản ghi biến mất đi đó giữ (1) biến cục bộ bất kỳ mà các thủ tục lồng bên trong có thể cần và (2) con trỏ đến môi trường (bản ghi biến mất đi) được cung cấp bởi hàm bao bọc. Con trỏ này từ một bao đóng đến bao đóng của hàm bao bọc tĩnh được gọi là liên kết tĩnh (*static link*); xem Hình 8.7.



Hình 8.7: Các bao đóng cho việc thực thi `twice(add(5))`. SL=static link; RV=return value; EP=escaping-variables-pointer hoặc environment-pointer

CHUYỂN ĐỔI BAO ĐÓNG

Hàm được truyền như đối số được biểu diễn như bao đóng (*closure*): sự kết hợp con trỏ mã máy và phương tiện truy cập đến các biến không cục bộ (còn được gọi là *biến tự do*).

Ví dụ của biến không cục bộ trong ngôn ngữ lập trình hướng đối tượng là biến khởi tạo của một lớp. Một phương thức tham chiếu đến một biến khởi tạo truy cập đến nó qua **this**,

mà là tham số hình thức ẩn của phương thức đó. Một cách dịch việc truy cập biến tự do cho cho các hàm lồng nhau là biểu diễn các bao đóng như các đối tượng.

Pha chuyển đổi bao đóng của chương trình dịch của ngôn ngữ hàm sẽ biến đổi chương trình sao cho không hàm nào có vẻ như truy cập đến các biến tự do (không cục bộ). Điều này được thực hiện bằng cách biến việc truy cập mỗi biến tự do thành truy cập biến khởi tạo.

Một số biến cục bộ được khai báo trong một hàm f cũng được truy cập bằng các hàm lồng nhau bên trong f ; chúng ta nói các biến này *trốn mất*. Chẳng hạn, trong Chương trình 7.11, trong hàm `putInt`, các biến `dig` và `c` trốn mất (vì chúng được sử dụng trong hàm lồng bên trong `doDigit`), nhưng biến `rest` không trốn mất.

Cho hàm $f(a_1, \dots, a_n)$ ở độ sâu lồng d với các biến cục bộ trốn mất (và các tham số hình thức) $x_1, x_2, \dots, x_n$ và các biến không trốn $y_1, \dots, y_n$, chúng ta có thể viết lại thành

```
f(this, a1, ..., an) {
    c272 r = new c272(this, x1, x2, ..., xn);
    return B'
}
```

Tham số mới `this` là con trỏ bao đóng, bây giờ được biến thành đối số tường minh. Biến r là đối tượng chứa tất cả các biến trốn mất và con trỏ bao đóng bao ngoài. Biến r này trở thành đối số con trỏ bao đóng khi gọi hàm ở độ sâu $d + 1$. Lớp (trong trường hợp này là `c272`) cần phải được tạo riêng cho mỗi hàm, vì danh sách các biến trốn (và kiểu của chúng) là khác nhau đối với mỗi hàm.

Bất cứ việc dùng biến cục bộ nào (mà ở độ sâu lồng nhau $< d$) bên trong B cần phải được chuyển thành việc truy cập đến lẽ nào đó bên trong bản ghi `this` (trong thân hàm viết lại B').

Các giá trị hàm. Chúng ta cần biểu diễn một giá trị hàm như một đối tượng với phương thức duy nhất (mà chúng ta sẽ gọi là `exec`) và không có hoặc nhiều hơn các biến khởi tạo (để giữ các biến cục bộ). Chúng ta sẽ biểu diễn kiểu $t_1 \rightarrow t_2$ như lớp

```
abstract class c_t1_t2 { abstract public t2 exec(t1 x); }
```

và bất cứ giá trị hàm thực tế nào thuộc về kiểu này sẽ là mở rộng của lớp đó, bổ sung các biến khởi tạo và phương thức `exec`.

Chương trình 8.8 là kết quả của việc chuyển đổi bao đóng Chương trình 8.7. Chúng ta có thể thấy rằng mỗi kiểu hàm là một lớp trừu tượng, và mỗi hàm là một lớp con khác nhau của lớp trừu tượng đó. Các biến cục bộ trốn thoát được đặt vào trong các đối tượng của các hàm lồng bên trong. Hơn nữa, khi các hàm lồng sâu, thông thường bao đóng của hàm lồng bên trong có liên kết `link` đến bao đóng của hàm bao ngoài để thuận tiện cho việc truy cập đến các biến của các hàm ở ngoài nữa.

CHƯƠNG TRÌNH 8.8: `printTable` sau khi chuyển đổi bao đóng (các hàm tạo lớp được bỏ qua).

```
abstract class cont {
    abstract public answer exec(); }

abstract class c_list_cont_answer {
    abstract public answer exec(list l, cont c); }

class printTable extends c_list_cont_answer {
    public answer exec(list l, cont c) {
        doListX r1 = new doListX(this, c);
        return r1.exec(l);
    } }

abstract class c_list_answer {
    abstract public answer exec(list l); }

class doListX extends c_list_answer {
    printTable link;
    cont c;
    public answer exec (list l) {
        return if (l==null) c.exec()
            else {doRest r2 = new doRest(this,l);
                int i = l.head;
                again r3 = new again(i,doRest);
                return putInt.exec(i,again);
            }
    }

    }

abstract class c_void_answer { abstract public answer exec(); }

class doRest extends c_void_answer {
    doListX link;
    list l;
    public answer exec() {
        return doListX.exec(l);
    } }

class again extends c_void_answer {
    int i;
    doRest d;
    public answer exec() {
        return putInt(i+i, d);
    } }
```

ĐỆ QUI ĐUÔI HIỆU QUẢ

Các chương trình hàm thể hiện các vòng lặp và luồng điều khiển khác qua các lời gọi hàm. Ở đây Chương trình 7.13b có vòng lặp **while**, Chương trình 8.8 có lời gọi hàm đến `doListX`. Hàm `putInt` của Chương trình 7.13b đơn giản trả về cho hai điểm gọi bên trong `printTable`, Chương trình 8.7 có các hàm tiếp diễn. Chương trình dịch FunJava

cần phải dịch các lời gọi đến `doListX`, `doRest`, và hiệu quả như chương trình dịch MiniJava lại dịch các vòng lặp và trả về hàm.

Nhiều lời gọi hàm trong Chương trình 8.7 là ở vị trí đuôi. Một lời gọi hàm $f(x)$ bên trong thân của hàm khác $g(y)$ là ở vị trí đuôi nếu "gọi f là cái cuối cùng mà g sẽ làm trước khi trả về. Một cách hình thức hơn, trong mỗi biểu thức sau, B_i là ở ngữ cảnh đuôi, nhưng C_i thì không:

1. `{int x =  $C_1$ ; return  $B_1$ ;`
2. `$C_1$ ( $C_2$ )`
3. `if  $C_1$   $B_1$  else  $B_2$`
4. `$C_1$  +  $C_2$`

Chẳng hạn, C_2 trong biểu thức 4 là không ở ngữ cảnh đuôi, mặc dù có vẻ nó là “cuối cùng”, vì vậy sau khi C_2 hoàn thành ở đây vẫn cần phải có chỉ lệnh **add**. Nhưng B_1 trong biểu thức 3 là ở ngữ cảnh đuôi, mặc dù nó không phải “cuối cùng” về mặt cú pháp.

Nếu lời gọi hàm $f(x)$ là ở ngữ cảnh đuôi đối với biểu thức bao ngoài của nó, và biểu thức này là ở ngữ cảnh đuôi, và tiếp tục như vậy cho đến thân của định nghĩa hàm bao ngoài `int  $g(y)$   $B$` , thì $f(x)$ là lời gọi đuôi.

Các lời gọi đuôi có thể được cài đặt hiệu quả hơn các lời gọi thông thường. Cho

```
int g(int y) {int x = h(y); return f(x)}
```

thì $h(y)$ không phải lời gọi đuôi, nhưng $f(x)$ thì đúng. Khi $f(x)$ trả về kết quả z nào đó, thì z sẽ là kết quả trả về từ g . Thay vì đẩy địa chỉ trả về mới cho f để trả về, g chỉ cần cho f trả về địa chỉ đã cho g , và buộc f trả về trực tiếp nó.

Như vậy, một lời gọi đệ qui đuôi có thể được cài đặt giống như một bước nhảy hơn là một lời gọi. Các bước cho lời gọi đuôi là

1. Chuyển các tham số thực tế vào các thanh ghi đối số.
2. Khôi phục các thanh ghi callee-save.
3. Dỡ khung ngăn xếp của hàm gọi, nếu nó có.
4. Nhảy đến nơi được gọi callee.

Trong nhiều trường hợp, mục 1 (chuyển các tham số) được loại bỏ bằng pha copy chuẩn bị của chương trình dịch. Thông thường, các mục 2 và 3 được loại bỏ vì hàm gọi không có khung ngăn xếp – hàm bất kỳ mà có thể làm mọi tính toán trong các thanh ghi caller-save không cần khung. Vì vậy, lời gọi đuôi có thể rẻ hơn chỉ lệnh bước nhảy.

Trong Chương trình 8.8, *mỗi* lời gọi là lời gọi đuôi! Cũng vậy, không hàm nào trong chương trình này cần khung ngăn xếp. Điều này không cần phải là đúng; chẳng hạn, lời gọi đến `double` trong Chương trình 8.8 là không ở vị trí đuôi, và lời gọi không đuôi này chỉ biến mất vì mở rộng trên hàng thực hiện với nó.

Các lời gọi đuôi được cài đặt như bước nhảy. Chương trình dịch của chương trình 8.8 và 7.13b là để giảng dạy. Hình vẽ 8.9 chỉ ra rằng chương trình hàm thuần túy và chương trình chỉ lệnh là thực hiện hầu hết chính xác cùng các chỉ lệnh! Hình vẽ này không chỉ ra việc định chương trình hàm từ các bản ghi liên kết tĩnh; và nó không chỉ ra việc lưu chương trình chỉ lệnh và khôi phục các thanh ghi callee-save.

printTable:	allocate object	printTable:	allocate stack
r1		frame	
	jump to doListX		jump to whileL
doListX:	allocate record	whileL:	
r2			if l=nil goto
	if l=nil goto	doneL	
doneL			i := l.head
	i = l.head		call putInt
	allocate object		add i+i
r3			call putInt
	jump to putInt		jump to whileL
again:	add	doneL:	return
this.i+this.i			
	jump to putInt		
doRest:	jump to doListX		
doneL :	jump to this.c		
(a) Chương trình hàm		(b) Chương trình chỉ lệnh	

Hình 8.9: `printTable` được dịch.

Tính không hiệu quả còn lại trong chương trình hàm là nó tạo ba đối tượng được cấp trên đồng, `r1`, `r2`, `r3`, trong khi chương trình mệnh lệnh tạo chỉ một khung ngăn xếp. Tuy nhiên, các thuật toán chuyển đổi bao đóng nâng cao có thể thành công khi tạo chỉ một bản ghi (ở đầu của `printTable`). Do đó sự khác biệt giữa hai chương trình có thể bé hơn việc tạo bản ghi đồng đối với tạo khung ngăn xếp.

Cấp đối tượng trên đồng dọn rác có thể đắt hơn là đẩy và lấy ra trên khung ngăn xếp. Tối ưu chương trình dịch cho các ngôn ngữ hàm giải quyết vấn đề này theo các cách khác nhau:

- Phân tích *chạy trốn* thời gian dịch có thể xác định bao đóng nào không sống lâu hơn hàm mà tạo nó. Các đối tượng này có thể được cấp ngăn xếp. Trong trường hợp của `printTable`, điều này có thể làm cho mã “hàm” gần giống với mã “mệnh lệnh”.
- Hoặc cấp đóng và dọn rác có thể được làm rất rẻ. Khi đó tạo (và thu gom rác) một đối tượng cấp đóng gồm chỉ bốn hoặc năm chỉ lệnh, tạo `printTable` hàm hầu như nhanh bằng phiên bản mệnh lệnh.

ƯỚC LƯỢNG SAU

Suy luận đẳng thức hỗ trợ hiểu các chương trình hàm. Một nguyên lý quan trọng của suy luận đẳng thức là phép thế beta (β -substitution): nếu $f(x) = B$ với thân hàm nào đó B , thì áp dụng bất kỳ $f(E)$ cho một biểu thức E là tương đương với B với mỗi xuất hiện của x được thay thế bằng E :

$$f(x) = B \text{ suy ra } f(E) \equiv B[x \mapsto E]$$

Nhưng hãy xét một đoạn chương trình PureFunJava,

<pre>{int loop (int z) { return if (z>0) z else loop(z); } int f (int x) { return if (y>8) x else -y; } return f(loop(y)); }</pre>	<pre>{int loop (int z) { return if (z>0) z else loop(z); } int f (int x) { return if (y>8) x else -y; } return if (y>8) loop(y) else -y; }</pre>
--	---

Nếu biểu thức B là $\text{if } (y>8) \ x \ \text{else } -y$, và biểu thức E là $\text{loop}(y)$, thì rõ ràng chương trình ở bên trái chứa $f(E)$ và chương trình ở bên phải chứa $B[x \mapsto E]$. Như vậy hai chương trình này tương đương, sử dụng suy luận đẳng thức.

Tuy nhiên, *hai chương trình không thực hiện như nhau!* Nếu $y = 0$, thì chương trình ở bên phải sẽ trả về 0, còn chương trình ở bên trái trước hết sẽ chọc vào $\text{loop}(0)$, mà là vòng lặp vô hạn.

Rõ ràng, nếu chúng ta muốn khẳng định hai chương trình là tương đương, thì chúng cần cho kết quả như nhau. Trong PureFunJava, nếu chúng ta nhận được chương trình A bằng cách thế trên chương trình B , thì A và B sẽ không bao giờ cho cho các kết quả khác nhau nếu cả hai đều dừng; nhưng A hoặc B có thể không dừng trên cùng tập đầu vào.

Để khắc phục lỗi (một phần) này của suy luận đẳng thức, chúng ta đưa ra khái niệm *ước lượng sau* cho ngôn ngữ lập trình. Haskell là ngôn ngữ ước lượng sau được sử dụng rộng rãi nhất. Một chương trình được dịch với ước lượng sau (gọi là *lười*) sẽ không ước lượng bất cứ biểu thức nào trừ khi giá trị của nó được yêu cầu bởi một phần tính toán khác. Ngược lại, các ngôn ngữ chặt chẽ như MiniJava, PureFunJava, ML, C, và Java tính toán mỗi biểu thức khi luồng điều khiển của chương trình đến nó.

Để triển khai việc dịch cho các ngôn ngữ hàm lười, chúng ta sẽ sử dụng ngôn ngữ LazyJava. Cú pháp của nó giống như PureFunJava, và ngữ nghĩa của nó gần như vậy, ngoại trừ ước lượng sau được sử dụng trong khi dịch nó.

ƯỚC LƯỢNG GỌI BẰNG TÊN

Đa số các ngôn ngữ lập trình (Pascal, C, ML, Java, MiniJava, PureFunJava) sử dụng lời gọi bằng giá trị truyền cho đối số hàm: để tính $f(g(x))$, trước hết $g(x)$ được tính, và giá trị này được truyền cho f . Nhưng nếu f không thực sự cần sử dụng đối số của nó, thì tính toán $g(x)$ sẽ là không cần thiết.

Để tránh tính biểu thức trước khi kết quả của nó là cần thiết, chúng ta có thể sử dụng ước lượng *gọi bằng tên*. Về bản chất, mỗi biến không chỉ là giá trị đơn, nhưng được nghĩ là: hàm số chỉ tính giá trị khi được yêu cầu. Chương trình dịch sẽ thay mỗi biểu thức kiểu `int` bằng giá trị hàm kiểu `() -> int`, và tương tự cho mọi kiểu khác.

Tại mỗi chỗ ở nơi biến được tạo, chương trình dịch tạo giá trị hàm; và mỗi chỗ nơi biến được sử dụng, chương trình dịch đưa ra áp dụng hàm.

Do đó chương trình LazyJava

```
{int a = 5+7; return a + 10; }
```

được tự động biến thành

```
{int a() {return 5+7;} return a() + 10; }
```

Ở đâu các biến được tạo? Tại nơi khai báo biến và nơi gắn kết các tham số hàm. Như vậy, mỗi biến thành một hàm, và tại mỗi lời gọi hàm, chúng ta cần khai báo nhỏ về hàm cho mỗi biểu thức tham số thực tế.

Chương trình 8.10 thể hiện việc chuyển đổi này được áp dụng cho hàm `look` của Chương trình 7.9a

CHƯƠNG TRÌNH 8.10: Chuyển đổi gọi bằng tên được áp dụng cho Chương trình 7.9a.

```
type c_void_int = () -> int;
type c_void_tree = () -> tree;

class tree {
    c_void_String key;
    c_void_int binding;
    c_void_tree left;
    c_void_tree right;
}

public c_void_int look(c_void_tree t, c_void_String k) {
    c_void_int c = t().key().compareTo(k);
    if (c() < 0) return look(t().left, k);
    else if (c() > 0) return look(t().right, k);
    else return t().binding;
}
```

Vấn đề với việc gọi bằng tên là mỗi một thunk có thể được thực hiện nhiều lần, mỗi lần (dư thừa) cho cùng một giá trị. Chẳng hạn, giả sử một cây được biểu diễn bằng một thunk

t1. Mỗi lần `look(t1,k)` được gọi, `t1()` được tính toán, mà tái xây dựng cùng một cây mỗi lần!

GỌI THEO YÊU CẦU

Ước lượng lười, mà còn được gọi là *gọi theo yêu cầu (call-by-need)*, là cải tiến của gọi bằng tên mà không bao giờ ước lượng cùng một thunk hai lần. Mỗi thunk được cấp một rãnh nhớ (*memo slot*) để lưu giá trị đó. Khi một thunk được tạo lần đầu, rãnh nhớ là rỗng. Mỗi ước lượng thunk đó sẽ kiểm tra rãnh nhớ: Nếu đầy, nó trả về giá trị được nhớ; nếu rỗng, nó gọi hàm thunk.

Để tạo luồng cho quá trình này, chúng ta sẽ thể hiện một thunk lười như một đối tượng với *một hàm thunk*, *rãnh nhớ*, và (như với đối tượng bao đóng bất kỳ) các biến khởi tạo để biểu diễn các biến tự do là cần thiết đối với việc sử dụng bởi hàm thunk. Một thunk *chưa ước lượng* có rãnh nhớ rỗng, và *and* hàm thunk, khi được gọi, tính toán giá trị và ặt nó và đặt nó vào trong rãnh nhớ. Một thunk đã *được ước lượng* có giá trị đã được tính trước trong rãnh nhớ của nó, và hàm thunk của nó chỉ trả về giá trị trong rãnh nhớ.

Chẳng hạn, khai báo LazyJava `int twenty = addFive(15)` (trong Chương trình 2.4) được dịch trong ngữ cảnh ở đó con trỏ môi trường EP sẽ trỏ đến bản ghi chứa hàm `addFive`. Thể hiện của `addFive(15)` không phải là lời gọi hàm mà sẽ thực thi và tính câu trả lời ngay bây giờ, nhưng một thunk sẽ nhớ tính theo yêu cầu như thế nào sau đó. Chúng ta có thể dịch đoạn chương trình LazyJava thành chương trình FunJava như sau:

```
/* this luôn trỏ đến bản ghi chứa addFive */
c_void_int twenty = new intThunk(this);
```

mà được hỗ trợ bởi các khai báo bổ sung

```
class intThunk {public int eval(); int memo; boolean done; }

class c_int_int {public int exec(int x);}
class intFuncThunk {public c_int_int eval();
                    c_int_int memo; boolean done; }

class twentyThunk extends intThunk {
    intFuncThunk addFive;
    public int exec() {
        if (!done) {
            memo = addFive.eval().exec(15);
            done = true;
        }
        return memo;
    }
    twentyThunk(addFive) {this.addFive=addFive;}
}

twentyThunk twenty = new twentyThunk(...);
```

Để tạo một thunk như `twenty`, cần phải gán giá trị cho các biến tự do của nó (trong trường hợp này là `addFive`) sao cho khi nó ước lượng sau này, nó sẽ có mọi thông tin cần thiết; điều này cũng giống như chuyển đổi bao đóng. Để dùng một thunk lười `t`, chúng ta chỉ tính `t.eval()`. Lần đầu `t.eval()` được tính, sẽ thấy `done` là `false`, và nó sẽ tính kết quả và đặt nó trong `memo`. Mỗi thời điểm `t` sau đó được xét đến, `t.eval()` sẽ đơn giản trả về trường `memo` đó.

ƯỚC LƯỢNG CHƯƠNG TRÌNH LƯỜI

Sau đây là chương trình mà sử dụng hàm `enter` của Chương trình 7.9b để xây dựng một cây mà ánh xạ 3 thành $3!$, $-1 \mapsto (-1)!$:

```
{int fact(int i) {
    return if (i==0) 1 else i * fact(i-1)
}
tree t0 = new tree("",0,null,null);
tree t1 = t0.enter("-one", if i=0 then 1 else i * fact(i-1));
tree t2 = t1.enter("three", fact(3));
return putInt(t2.look("three", exit));
}
```

Một việc kỳ lạ về chương trình này là `fact(-1)` là không được xác định. Như vậy, nếu chương trình này được dịch bằng chương trình dịch của PureFunJava (chặt), nó sẽ lặp vô hạn (hoặc sẽ làm tràn phép tính toán mã máy như duy trì phép trừ 1 từ các số âm).

Nhưng nếu dịch bằng chương trình dịch LazyJava, chương trình sẽ thành công, in ra 3 giai thừa! Trước hết, biến `t1` được xác định; nhưng nó thực tế không gọi `enter` – mà chỉ tạo một thunk mà sẽ làm như vậy theo yêu cầu. Sau đó, `t2` được xác định, mà cũng không làm gì ngoài việc tạo một thunk. Do đó một thunk được tạo cho `look(t2, "three")` (nhưng `look` thực tế không được gọi).

Cuối cùng, một thunk cho biểu thức `putInt(..., exit)` được tạo. Đây là kết quả của chương trình. Nhưng hệ thống thời gian chạy sau đó “yêu cầu” câu trả lời `answer` từ chương trình này, mà có thể được tính chỉ bằng cách gọi thunk ở vòng ngoài nhất. Vậy thân của `putInt` thực thi, mà lập tức yêu cầu giá trị nguyên của đối số thứ nhất; nó buộc thunk `look(t2, "three")` phải ước lượng.

Thân của `look` cần so sánh `k` với `t.key`. Vì `k` và `t` mỗi cái đều là thunk, chúng ta cần tính giá trị nguyên bằng việc ước lượng `k.eval()` và một cây bằng việc ước lượng `t.eval()`. Từ cây này chúng ta có thể trích rút ra trường `key`; nhưng mỗi trường là một thunk, vậy chúng ta thực sự cần phải tính `(t.eval().key)()` để nhận được giá trị nguyên.

Giá trị `t.key` sẽ trở thành -1 , vậy `look(t().right, k)` được gọi. *Chương trình này không bao giờ ước lượng thunk binding tại định `-one`, vậy `fact(-1)` không bao giờ cho cơ hội dẫn đến vòng lặp vô hạn.*

ĐỌC THÊM

Church [1941] phát triển λ -calculus, là "ngôn ngữ lập trình" của các hàm lồng nhau mà được truyền như đối số và trả về như kết quả. Ông ta bị cản trở bởi việc không có máy nào để dịch.

Bao đóng. Landin [1964] chỉ ra thông dịch λ -calculus như thế nào trên máy trừu tượng, sử dụng các bao đóng được cấp trên đóng. Steele [1978] đã sử dụng biểu diễn bao đóng chuyên dụng cho các mẫu khác nhau của việc dùng hàm, sao cho trong nhiều trường hợp các biến không cục bộ được truyền như các đối số bổ sung cho hàm bên trong để tránh cấp đóng cho bản ghi. Cousineau [1985] chỉ ra cách chuyển bao đóng được thể hiện như biến đổi ngược lại ngôn ngữ nguồn, sao cho phân tích bao đóng có thể được tách hoàn toàn khỏi các pha sinh code khác.

Liên kết tĩnh trên thực tế không là cơ sở tốt nhất để chuyển bao đóng; vì nhiều lý do sẽ là tốt hơn nếu xét các biến không cục bộ riêng ra, thay vì luôn nhóm chúng tất cả với nhau tại mức lồng như nhau. Kranz [1986] thực hiện *phân tích trốn* để xác định bao đóng nào được cấp ngăn xếp vì chúng không tạo cho các hàm sống lâu hơn và cũng như phân tích bao đóng tích hợp với việc cấp thanh ghi để tạo chương trình dịch tối ưu hiệu năng cao. Shao và Appel [1994] tích hợp bao đóng với việc sử dụng thanh ghi được gọi lưu để tối thiểu lưu lượng tải/lưu được sinh ra bởi việc truy cập các biến cục bộ và không cục bộ. Appel [1992] có tổng quan tốt về chuyển bao đóng.

Tiếp diễn. Các lời gọi đuôi là đặc biệt hiệu quả và dễ dàng phân tích. Strachey và Wadsworth [1974] chỉ ra rằng luồng điều khiển của chương trình bất kỳ (ngay cả chương trình chỉ lệnh) có thể được biểu diễn như các lời gọi hàm, sử dụng khái niệm tiếp diễn (*continuations*). Steele [1978] biến đổi sớm chương trình thành *kiểu truyền tiếp diễn* trong khi dịch, chuyển mọi lời gọi hàm thành lời gọi đuôi, để đơn giản mọi phân tích và các pha tối ưu của chương trình dịch. Kranz [1986] xây dựng tối ưu chương trình dịch cho Lược đồ sử dụng kiểu truyền tiếp diễn; Appel [1992] mô tả tối ưu dựa trên tiếp diễn chương trình dịch cho ML.

Mở rộng dòng. Cocke và Schwartz [1970] mô tả mở rộng dòng của thân hàm; Scheifler [1977] chỉ ra rằng nó đặc biệt hữu ích cho các ngôn ngữ hỗ trợ trừu tượng dữ liệu, ở đó có xu hướng có nhiều hàm nhỏ cài đặt các thao tác trên kiểu dữ liệu. Appel [1992] mô tả các trực quan thực tế để kiểm soát việc bùng nổ mã.

Vào ra dựa trên tiếp diễn. Wadler [1995] mô tả sử dụng monads để sinh ra khái niệm tương tác dựa trên tiếp diễn.

Ước lượng lười. Algol-60 [Naur, 1963] đã sử dụng ước lượng gọi theo tên cho các đối số hàm, được cài đặt sử dụng thunks – nhưng cũng cho phép các tác động phụ, do đó các lập trình viên cần phải họ đang làm gì! Đa số những người nổi nghiệp sử dụng gọi theo giá trị. Henderson và Morris [1976], Friedman và Wise [1976] độc lập phát minh ra ước lượng lười (gọi theo nhu cầu). Hughes [1989] tranh luận rằng các ngôn ngữ hàm lười cho phép lập trình modular tốt hơn các ngôn ngữ chỉ lệnh.

Một số ngôn ngữ hàm lười đã được phát triển trong những năm 1980; cộng đồng các nhà nghiên cứu trong lĩnh vực này thiết kế và cho ra ngôn ngữ Haskell [Hudak và những người khác, 1992] như một chuẩn. Peyton Jones [1987; 1992] mô tả nhiều kỹ thuật cài đặt và tối ưu cho các ngôn ngữ hàm lười;

BÀI TẬP

8.1 Giả sử một chương trình dịch nào đó dịch tất cả các lệnh và các biểu thức sang các cây $Tree.Exp$, và không sử dụng các cấu trúc Nx và Cx để biểu diễn các biểu thức theo các cách khác nhau. Vẽ bức tranh cây IR mà là kết quả của một trong các lệnh và các biểu thức sau.

- a. $a+5$
- b. $b[i+1]$
- c. $a < b$, mà cần được cài đặt bởi việc thực hiện ESEQ mà vế trái của nó chuyển 1 hoặc 0 thành biến tạm thời mới được định nghĩa, và vế phải của nó là biến tạm thời.
- d. $a = x+y$; mà cần được dịch với đỉnh EXP ở trên cùng.
- e. $if (a < b) c=a; else c=b$; được dịch sử dụng cây $a < b$ từ phần (c) ở trên; Toàn bộ lệnh khi đó sẽ rắc rối và không hiệu quả.
- f. $if (a < b) c=a; else c=b$; được dịch theo cách ít rắc rối hơn.

8.2 Dịch các lệnh và các biểu thức MiniJava sau đây sang cây IR , nhưng sử dụng các cấu trúc Ex , Nx , và Cx thích hợp. Trong mỗi trường hợp, chỉ cần vẽ nên bức tranh của cây; cây Ex sẽ là cây Exp , cây Nx sẽ là cây Stm , và cây Cx sẽ là cây Stm với các hồ được đánh nhãn true và false mà các nhãn về sau sẽ được đặt vào đó.

- g. $a+5$;
- h. $b[i+1]=0$;
- i. $while (a < 0) a=a+1$;
- j. $a < b$ chuyển 1 hoặc 0 vào biến tạm thời mới được định nghĩa, và vế phải là biến tạm thời.
- k. $a = x+y$;
- l. $if (a < b) c=a; else c=b$;

8.3 Sử dụng chương trình dịch C tùy theo sự lựa chọn của bạn (hoặc chương trình dịch cho ngôn ngữ khác), dịch một số hàm sang ngôn ngữ assembly. Trên Unix điều này được thực hiện với tùy chọn `-s` cho chương trình dịch C.

Sau đó xác định mọi thành phần của dãy gọi (các mục 1-11), và giải thích từng dòng của ngôn ngữ assembly làm gì (đặc biệt các giả lệnh bao gồm các mục 1 và 11). Thử một hàm nhỏ mà trả về không tính toán nhiều (hàm lá), và một hàm mà gọi hàm khác trước khi trả về ngay.

8.4 Ngôn ngữ trung gian cây không có thao tác nào trên các biến dấu phẩy động. Hãy chỉ ra ngôn ngữ sẽ thế nào với thao tác nhị phân mới cho số học dấu phẩy động và quan hệ nhị phân mới cho việc so sánh dấu phẩy động. Bạn có thể tìm thấy lợi ích khi đưa vào một biến thể của đỉnh MEM để mô tả việc tìm thấy và lưu trữ các giá trị dấu phẩy động.

***8.5** Ngôn ngữ trung gian cây *Tree* không quan tâm đến các giá trị dữ liệu mà không có độ dài chính xác một từ. Ngôn ngữ C có các số nguyên có dấu và không dấu một số kích thước, với các phép toán qui ước trên các cỡ khác nhau đó. Trang bị cho ngôn ngữ trung gian để có thêm một số cỡ số nguyên, với các phép toán chuyển đổi giữa chúng.

Gợi ý: Không phân biệt các giá trị có dấu và không có dấu trên cây trung gian, nhưng phải phân biệt các phép toán có dấu và không có dấu. Xem thêm Fraser và Hanson [1995], Phần 5.5 và 9.1 trong sách tham khảo với bảng các chỉ lệnh số học và bộ nhớ sau:

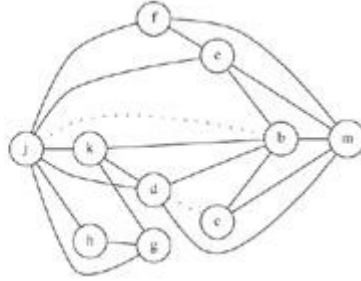
Name	Effect	Trees
—	r_i	TEMP
ADD	$r_i \leftarrow r_j + r_k$	
MUL	$r_i \leftarrow r_j \times r_k$	
SUB	$r_i \leftarrow r_j - r_k$	
DIV	$r_i \leftarrow r_j / r_k$	
ADDI	$r_i \leftarrow r_j + c$	
SUBI	$r_i \leftarrow r_j - c$	
LOAD	$r_i \leftarrow M[r_j + c]$	
STORE	$M[r_j + c] \leftarrow r_i$	
MOVEM	$M[r_j] \leftarrow M[r_i]$	

8.7 Vẽ hình ảnh của cấu trúc dữ liệu bao đóng biểu diễn `add24` và `a` trong Chương trình 2.4 tại điểm ở đó `add24(a)` gần như được gọi. Đánh nhãn mọi thành phần.

***8.8** Hình 8.9 tóm tắt các chỉ dẫn cần thiết để cài đặt `printTable` trong kiểu hàm và chỉ lệnh. Nhưng nó để lại các chỉ lệnh `MOVE` mà truyền tham số cho các lời gọi. Hoàn thiện các phương án hàm và chỉ lệnh với mọi chỉ lệnh được bỏ qua, viết ngôn ngữ tựa assembly trong phong cách của chương trình đi kèm với Đồ thị sau. Chỉ ra rằng các chỉ lệnh `MOVE` mà bạn muốn xóa bằng cách lan truyền bản sao.

ĐỒ THỊ: Đồ thị xen vào với chương trình. Các đường đứt nét là Dotted lines là các cung xen vào nhưng chỉ rõ các chỉ lệnh move.

```
live-in: k j
g := mem[j+12]
h := k - 1
f := g * h
e := mem[j+8]
m := mem[j+16]
b := mem[f]
c := e + 8
d := c
k := m + 4
j := b
live-out: d k j
```



***8.9** Giải thích tại sao ở đây không có vòng lặp trong đồ thị bao đóng và bản ghi của chương trình PureFunJava. Chú giải về việc áp dụng thu dọn rác dựa trên bộ đếm tham chiếu cho một chương trình như vậy. **Gợi ý:** Dưới các hoàn cảnh như thế nào các bản ghi hoặc các bao đóng được cập nhật sau khi chúng được khởi tạo?

8.10

- Thực hiện Thuật toán 7.15 (biến đổi tiền đề vòng lặp) về hàm `look` của Chương trình 7.9a.
- Thực hiện Thuật toán 7.16 (nhắc nhở bất biến vòng lặp) về kết quả đó.
- Thực hiện Thuật toán 7.14 (mở rộng trên dòng) về lời gọi sau `look` (giả thiết rằng hai phép biến đổi trước đã được áp dụng):

```
look(mytree, a+1)
```

PHỤ LỤC

A. NGÔN NGỮ MINIJAVA

MiniJava là tập con của Java. Ý nghĩa của chương trình MiniJava được cho bởi ý nghĩa của nó như chương trình Java. Tải chồng không cho phép trong MiniJava. Lệnh MiniJava `System.out.println( ...);` chỉ có thể in số nguyên. Biểu thức MiniJava `e.length` chỉ áp dụng cho biểu thức kiểu `int[]`.

A1. TỪ VỰNG

Định danh - Identifiers: *identifier* là dãy các chữ, số và gạch chân, bắt đầu từ chữ. Các chữ hoa là khác với các chữ thường. Trong phụ lục này ký hiệu *id* viết tắt cho identifier.

Hằng nguyên - Integer literals: Một dãy các số thập phân là một hằng nguyên *integer constant* mà ký hiệu giá trị số nguyên tương ứng. Trong Phụ lục này ký hiệu *INTEGER_LITERAL* được viết cho hằng nguyên.

Các phép toán nhị phân - Binary operators: Phép toán nhị phân *binary operator* là một trong các phép toán sau

`&& < + - *`

Trong Phụ lục này ký hiệu *op* viết tắt cho phép toán nhị phân.

Chú giải - Comments: Một chú giải có thể xuất hiện giữa hai từ tổ bất kỳ. Ở đây có hai dạng chú giải: Một bắt đầu từ `/*` và kết thúc với `*/`, và chúng có thể lồng nhau; cái khác bắt đầu với `//` và kéo dài đến hết dòng.

A2. VĂN PHẠM

Trong văn phạm MiniJava, chúng ta sử dụng ký hiệu N^* , trong đó N là một chưa tận, có nghĩa là 0, 1, hoặc nhiều hơn phép lặp của N .

GRAMMAR A.2

```
Program → MainClass ClassDecl*
MainClass → class id { public static void main ( String [] id )
              { Statement }}
ClassDecl → class id { VarDecl* MethodDecl* }
           → class id extends id { VarDecl* MethodDecl* }
VarDecl → Type id ;
MethodDecl → public Type id ( FormalList )
```



```

        { VarDecl* Statement* return Exp ;}
FormalList → Type id FormalRest*
        →
FormalRest → , Type id
        Type → int []
        → boolean
        → int
        → id
Statement → { Statement* }
        → if ( Exp ) Statement else Statement
        → while ( Exp ) Statement
        → System.out.println ( Exp ) ;
        → id = Exp ;
        → id [ Exp ] = Exp ;
Exp → Exp op Exp
        → Exp [ Exp ]
        → Exp . length
        → Exp . id ( ExpList )
        → INTEGER LITERAL
        → true
        → false
        → id
        → this
        → new int [ Exp ]
        → new id ()
        → ! Exp
        → ( Exp )
ExpList → Exp ExpRest*
        →
ExpRest → , Exp

```

A3. CHƯƠNG TRÌNH MẪU

```

class Factorial {
    public static void main(String[] a) {
        System.out.println(new Fac().ComputeFac(10));
    }
}
class Fac {
    public int ComputeFac(int num) {
        int num_aux;
        if (num < 1)
            num_aux = 1;
        else
            num_aux = num * (this.ComputeFac(num-1));
        return num_aux;
    }
}

```

B. BÀI TẬP BỔ SUNG

Bài tập bổ sung chương 3:

1. Mô tả bằng biểu thức chính qui tập các dãy bit chứa số chẵn các bit 1, còn lại là các bit 0.

Trả lời: $0^*(10^*1)^*0^*$

2. Xây dựng văn phạm biểu diễn số nguyên lẻ

GRAMMAR:

```

<odd> ::= <tail> | <head><tail> | <head><mid><tail>
<head> ::= <pdigit>
<mid> ::= <pdigit> | <pdigit><mid> | 0 | 0 <mid>
<tail> ::= 1 | 3 | 5 | 7 | 9
<pdigit> ::= 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9
    
```

3. Hằng số nguyên và tên biến đơn giản (bắt đầu chữ và được nối tiếp bởi chữ hoặc số) được sinh bởi biểu thức chính qui sau:

```

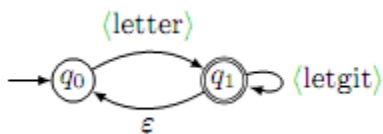
<digit> ≡ 0|1|2|3|4|5|6|7|8|9
<letter> ≡ a|b|c|d|e|f|g|h|i|j|k|l|m|n|o|p|q|r|s|t|u|v|w|x|y|z
           | A|B|C|D|E|F|G|H|I|J|K|L|M|N|O|P|Q|R|S|T|U|V|W|X|Y|Z
<letgit> ≡ <letter> | <digit>
<integer> ≡ <digit>+
<identifier> = <letter> <letgit>*
    
```

4. Định nghĩa tên biến phức tạp hơn (bắt đầu chữ và được nối tiếp bởi chữ hoặc số hoặc gạch chân, nhưng không được kết thúc bởi gạch chân):

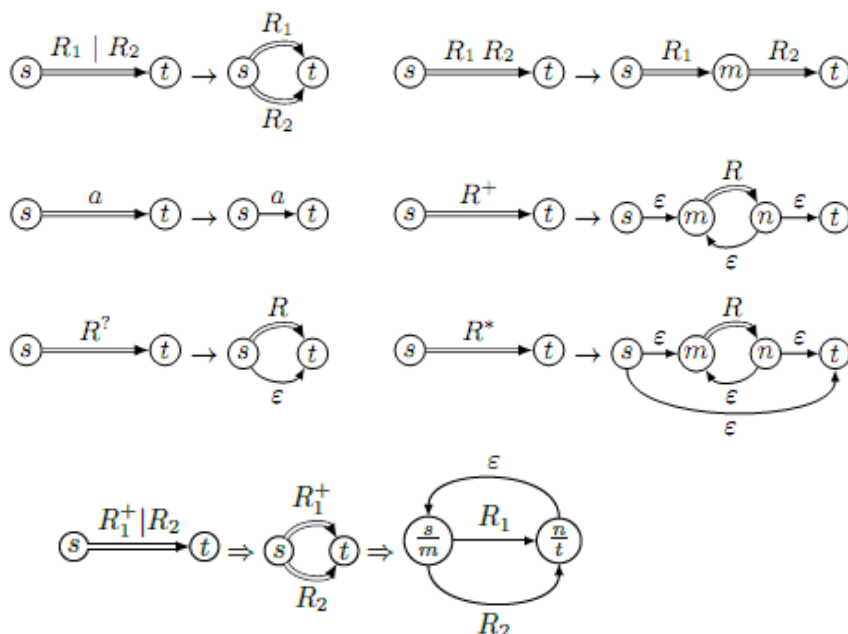
```

<identifier> = <letter> ((-)? (<letter> | <digit>)) *
<letter> = a | ... z | A | ... Z
<digit> = 0 | ... 9
    
```

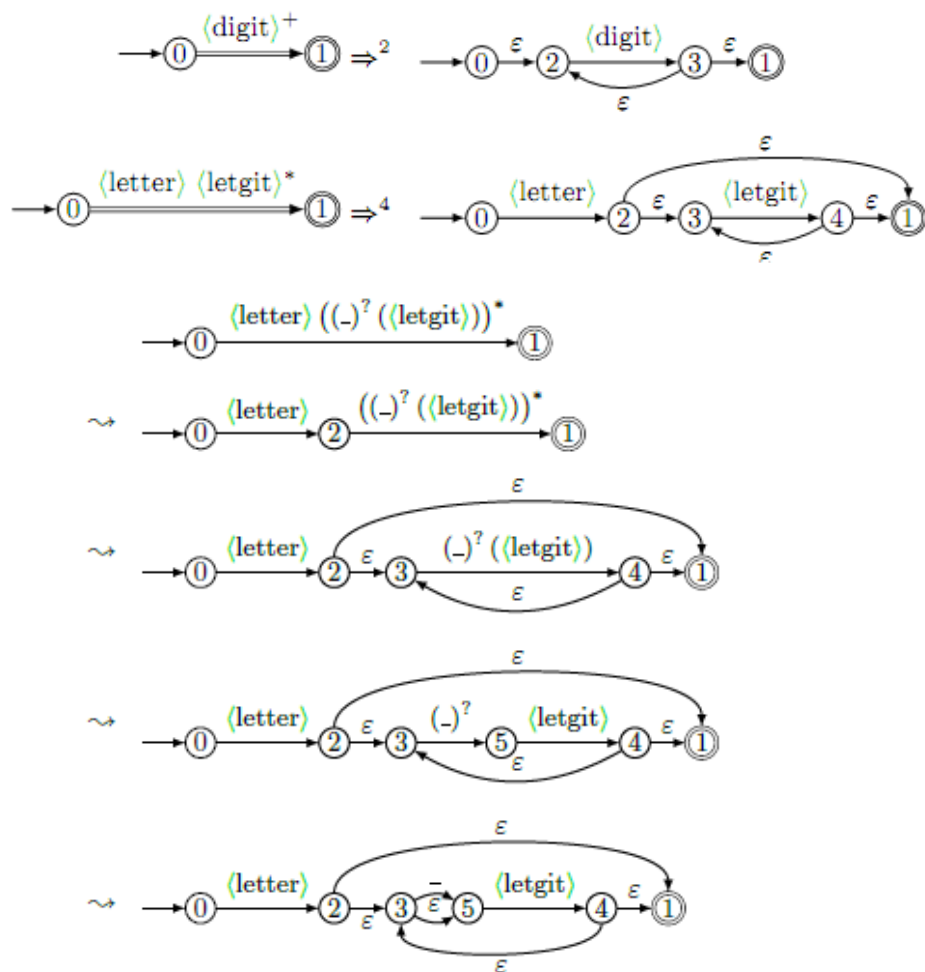
5. Otomat đoán nhận biểu thức chính qui biểu diễn tên biến đơn giản:

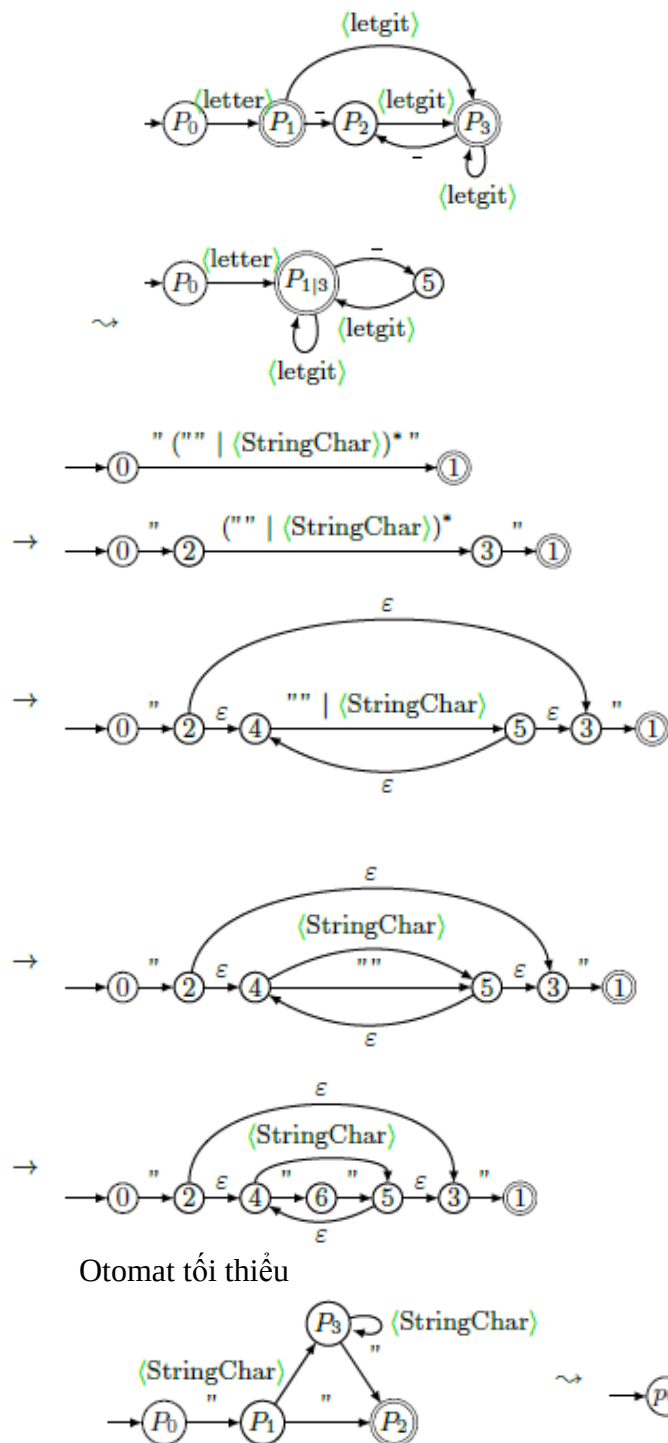


6. Qui tắc chuyển từ biểu thức chính qui sang NFA



Ví dụ. Xây dựng NFA đoán nhận số nguyên và tên biến (đơn giản và phức tạp):

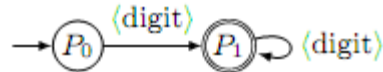




7. Chuyển đổi từ NFA sang DFA

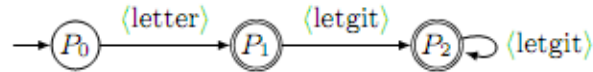
Trong ví dụ trước ta nhóm các trạng thái của NFA để tạo nên DFA

1. $P_0 = E(0) = \{0, 2\}$.
2. $P_1 = F(P_0, \langle \text{digit} \rangle) = \{1, 2, 3\}$.
3. $P_2 = F(P_1, \langle \text{digit} \rangle) = \{1, 2, 3\} = P_1$.



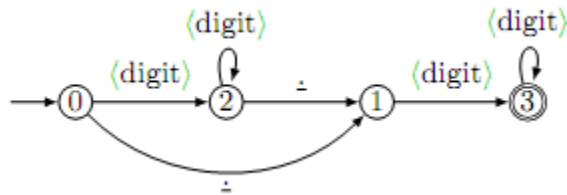
DFA cho hằng số nguyên

1. $P_0 = E(0) = \{0\}$.
2. $P_1 = F(P_0, \langle \text{letter} \rangle) = \{1, 2, 3\}$.
3. $P_2 = F(P_1, \langle \text{letgit} \rangle) = \{1, 3, 4\}$.
4. $F(P_2, \langle \text{letgit} \rangle) = \{1, 3, 4\} = P_2$.

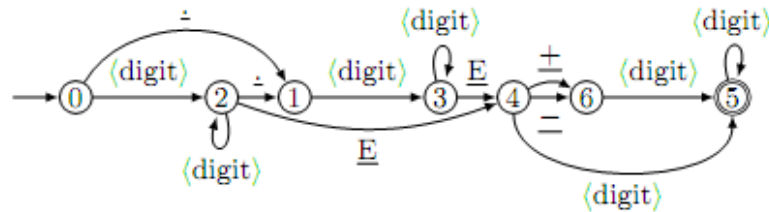


DFA cho tên biến

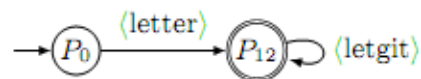
8. Xây dựng DFA cho số thập phân



9. Xây dựng DFA cho số float



10. Xây dựng Otomat tối thiểu cho otomat trong ví dụ 8.

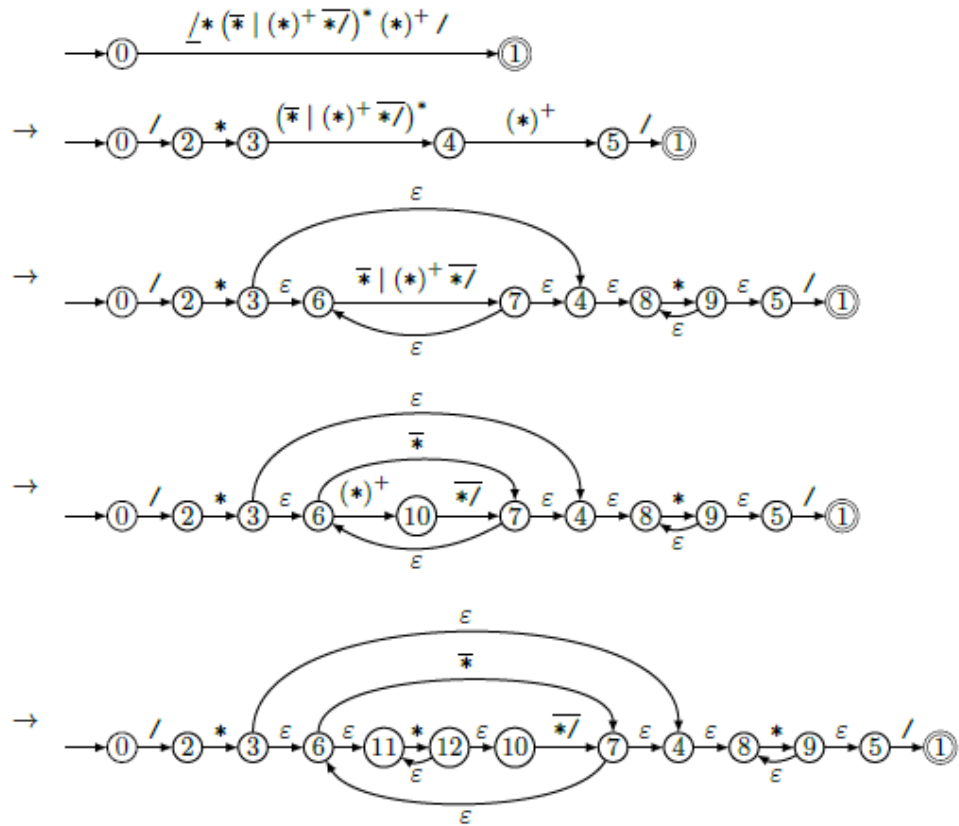


11. Xây dựng Văn phạm cho chú giải

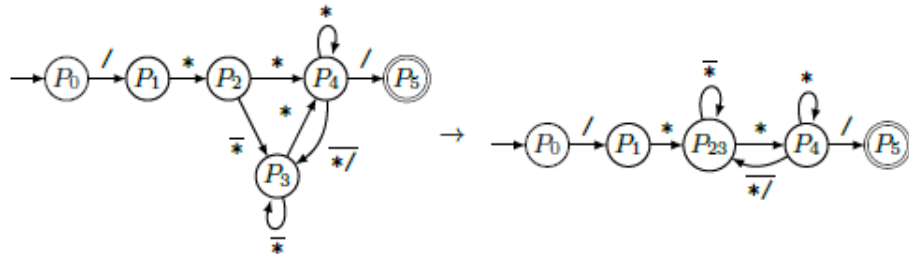
$$\langle \text{comment} \rangle = _ * \left(\bar{*} \mid (*)^+ \bar{*} / \right)^* (*)^+ /$$

Ở đây $\bar{i}$ là biểu thức cho xâu ở đó không chứa i

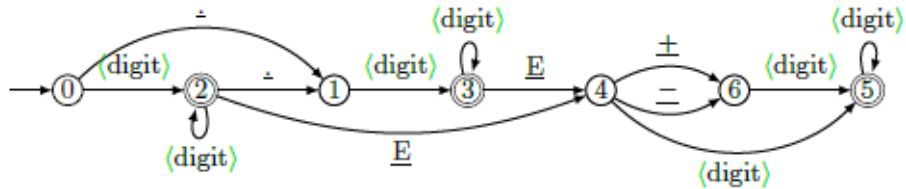
NFA cho chú giải:



DFA cho chú giải:



12. Xây dựng DFA kết hợp cho số nguyên, số thập phân, số dấu phẩy động



Bài tập bổ sung chương 4:

1. Cho văn phạm sau:

$S \rightarrow A \mid B \mid C$
 $A \rightarrow B \mid \epsilon$
 $B \rightarrow x \mid C y$
 $C \rightarrow B C S$

Trong đó S là ký tự ban đầu.

- Nêu các chưa tận mà đệ qui trái, đệ qui phải, có thể Null hoặc không có ích
- Ngôn ngữ sinh ra bởi văn phạm là gì?
- Văn phạm có nhập nhằng không?

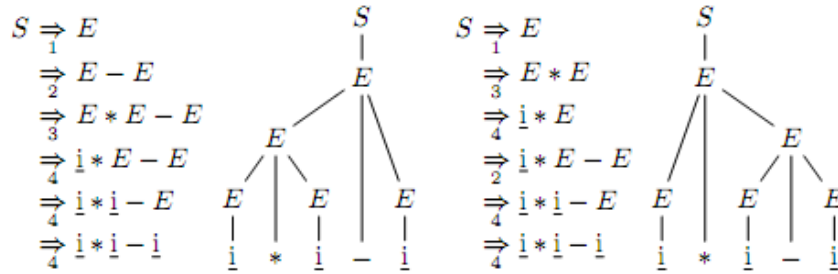
Trả lời: (a). đệ qui trái: B, C; đệ qui phải S, C; có thể Null: S, A; Không có ích C.

(b). Tập $\{x, \varepsilon\}$. (c) có: x có hai phép suy diễn: $S \rightarrow A \rightarrow B \rightarrow x$ và $S \rightarrow B \rightarrow x$.

- Văn phạm chính qui cho biểu thức đơn giản. Giả sử từ điển

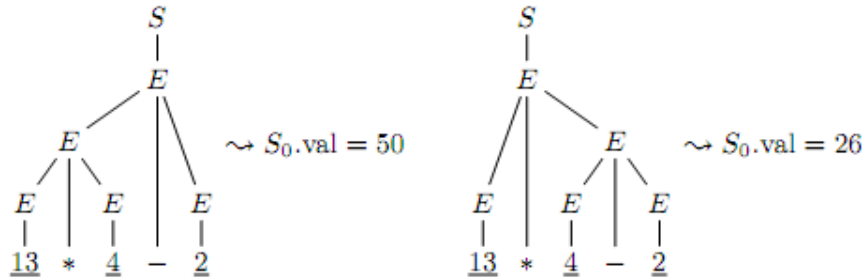
$V = N \cup T$ được cho với các chưa tận $N = \{S, E\}$ và các tận cùng $T = \{-, *, i, (,)\}$ (i biểu diễn hằng nguyên). Tập các qui tắc được cho như sau:

$$R = \left\{ \begin{array}{ll} S \rightarrow E & (1) \\ E \rightarrow E - E & (2) \\ E \rightarrow E * E & (3) \\ E \rightarrow i & (4) \\ E \rightarrow (E) & (5) \end{array} \right\}.$$



Hai suy diễn trái nhất trong G_0 , và các cây suy diễn tương ứng của chúng

- Văn phạm trong câu 1 là nhập nhằng, vì có hai suy diễn cho biểu thức $13*4 - 2$:



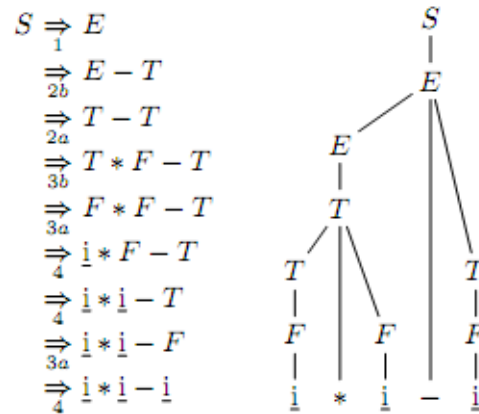
- Văn phạm không nhập nhằng. Văn phạm phi ngữ cảnh G_2 được cho với bộ luật sau:

$$R_2 = \left\{ \begin{array}{ll} S \rightarrow E & (1) \\ E \rightarrow T & (2a) \\ E \rightarrow E - T & (2b) \\ T \rightarrow F & (3a) \\ T \rightarrow T * F & (3b) \\ F \rightarrow i & (4) \\ F \rightarrow (E) & (5) \end{array} \right\}.$$

G_2 là không nhập nhằng. Điều này đạt được bằng cách phân biệt ba kiểu biểu thức: các đóng mở ngoặc bên ngoài, các nhân tử F không chứa cả $-$ và $*$ và các

số hạng T không chứa - . Như vậy các biểu thức chứa – không bao giờ xuất hiện trong các thành phần của phép nhân.

Các qui tắc đệ qui (2b) và (3b) cho E và T là đệ qui trái chứ không phải đệ qui phải, các suy diễn kề với các phép toán – và * luôn được nhóm tuân theo kết hợp trái. Xem hình vẽ sau làm ví dụ:



Suy diễn (và cây) của văn phạm G_2

5. Cho Văn phạm sinh địa chỉ các trang Web.

url	=	httpaddress ftpaddress telnetaddress mailtoaddress.
httpaddress	=	"http://" hostport ["/" path] ["?" search].
ftpaddress	=	"ftp://" login "/" path.
telnetaddress	=	"telnet://" login.
mailtoaddress	=	"mailto:" xalphas "@" hostname.
login	=	[user ":" password] "@"] hostport.
hostport	=	hostname [":" port].
hostname	=	xalphas { "." xalphas }.
port	=	digits.
path	=	segment { "/" segment }.
search	=	xalphas { "+" xalphas }.
user	=	xalphas.
password	=	xalphas.
segment	=	{ xalpha }.
xalphas	=	xalpha { xalpha }.
xalpha	=	alpha digit.
digits	=	digit { digit }.
alpha	=	"A" .. "Z" "a" .. "z".
digit	=	"0" .. "9".

- Xây dựng cây cú pháp cho địa chỉ <http://utc.edu.vn>
- Xây dựng cây cú pháp trừu tượng cho địa chỉ ở mục a.

6. Các qui tắc chuyển đổi BNF thành EBNF và ngược lại:

Conversion from EBNF to BNF and Vice Versa

• BNF to EBNF:

(i) Look for recursion in grammar:

$$A ::= a A \mid B$$

$$\Rightarrow A ::= a \{ a \} B$$

(ii) Look for common string that can be factored out with grouping and options.

$$A ::= a B \mid a$$

$$\Rightarrow A ::= a [B]$$

• EBNF to BNF:

(i) Options: $[]$

$$A ::= a [B] C$$

$$\Rightarrow A' ::= a N C \quad N ::= B \mid \epsilon$$

(ii) Repetition: $\{ \}$

$$A ::= a \{ B_1 B_2 \dots B_n \} C$$

$$\Rightarrow A' ::= a N C \quad N ::= B_1 B_2 \dots B_n N \mid \epsilon$$

Bài tập bổ sung chương 5:

1. Xây dựng Bảng FIRST cho văn phạm:

$$G: \quad E \rightarrow TE'$$

$$E' \rightarrow +TE' / \epsilon$$

$$T \rightarrow FT'$$

$$T' \rightarrow *FT' / \epsilon$$

$$F \rightarrow (E) / i$$

Ta nhận được bảng FIRST như sau:

	I	+	*	(	)	\$
E	$E \rightarrow TE'$			$E \rightarrow E'$		
E'		$E' \rightarrow TE'$			$E' \rightarrow \epsilon$	$E' \rightarrow \epsilon$
T	$T \rightarrow FT'$			$T \rightarrow FT'$		
T'		$T' \rightarrow \epsilon$	$T \rightarrow *FT'$		$T' \rightarrow \epsilon$	$T' \rightarrow \epsilon$
F	$F \rightarrow i$			$F \rightarrow (E)$		

Xây dựng Phân tích trên xuống cho biểu thức: $i + i * i$

<u>Stack</u>	<u>Input</u>	<u>Output</u>
\$E	i + i * i \$	
\$E'T	i + i * i \$	$E \rightarrow TE'$
\$E'T'F	i + i * i \$	$T \rightarrow FT'$
\$E'T'i	i + i * i \$	$F \rightarrow i$
\$E'T'	+ i * i \$	pop i
\$E'	+ i * i \$	$T \rightarrow \varepsilon$
\$E'T+	+ i * i \$	$E' \rightarrow +TE'$
\$E'T	i * i \$	pop +
\$E'T'F	i * i \$	$T \rightarrow FT'$
\$E'T'i	i * i \$	$F \rightarrow i$
\$E'T'	* i \$	Pop i
\$E'T'F*	* i \$	$T \rightarrow *FT'$
\$E'T'F	i \$	pop *
\$E'T'i	i \$	$F \rightarrow i$
\$E'T'	\$	pop i
\$E'	\$	$T' \rightarrow \varepsilon$
\$	\$	$E' \rightarrow \varepsilon$

2. Cho Văn phạm LL(1) với lựa chọn Null

```

input → expression EoF
expression → term rest_expression
term → IDENTIFIER | parenthesized_expression
parenthesized_expression → '(' expression ')'
rest_expression → '+' expression | ε

```

Xây dựng chương trình Phân tích trên – xuống Đoán nhận đệ qui cho văn phạm trên:

```
#include "tokennumbers.h"

/* PARSER */
int input(void) {
    return expression() && require(token(EoF));
}

int expression(void) {
    return term() && require(rest_expression());
}

int term(void) {
    return token(IDENTIFIER) || parenthesized_expression();
}

int parenthesized_expression(void) {
    return token('(') && require(expression()) && require(token(')'));
}

int rest_expression(void) {
    return token('+') && require(expression()) || 1;
}

int token(int tk) {
    if (tk != Token.class) return 0;
    get_next_token(); return 1;
}

int require(int found) {
    if (!found) error ();
    return 1;
}
```

Xây dựng Bảng FIRST và FOLLOW cho Văn phạm trên:

Rule	FIRST set	FOLLOW set
input	{ IDENTIFIER '(' }	{ }
expression	{ IDENTIFIER '(' }	{ EoF ')' }
term	{ IDENTIFIER '(' }	{ '+' EoF ')' }
parenthesized_expression	{ '(' }	{ '+' EoF ')' }
rest_expression	{ '+' ϵ }	{ EoF ')' }

3. Cho văn phạm LL(1)

$$S \rightarrow A 'a' 'b'$$

$$A \rightarrow 'a' | \epsilon$$

Xây dựng Bảng FIRST/FOLLOW (có xung đột):

Rule	FIRST set	FOLLOW set
$S \rightarrow A 'a' 'b'$	$\{ 'a' \}$	$\{ \}$
$A \rightarrow 'a' \epsilon$	$\{ 'a' \epsilon \}$	$\{ 'a' \}$

Xây dựng Phân tích cú pháp dự báo cho văn phạm trên:

```

void S(void) {
    switch (Token.class) {
        case 'a': A(); token('a'); token('b'); break;
        default: error();
    }
}

void A(void) {
    switch (Token.class) {
        case 'a': token('a'); break;
        case 'a': break;
        default: error();
    }
}

```

Tạo văn phạm LL(1)

4. Loại thừa số chung trái

$$\text{term} \rightarrow \text{IDENTIFIER} | \text{IDENTIFIER} '[' \text{expression} ']' | \dots$$

$$\text{term} \rightarrow \text{IDENTIFIER after_identifier} | \dots$$

$$\text{after_identifier} \rightarrow '[' \text{expression} ']' | \epsilon$$

5. Khử đệ qui trái

$$\text{expression} \rightarrow \text{expression} '-' \text{term} | \text{term}$$

$$N = \text{expression}$$

$$\alpha = '-' \text{term}$$

$$\beta = \text{term}$$

$$\text{expression} \rightarrow \text{term expression_tail_option}$$

$$\text{expression_tail_option} \rightarrow '-' \text{term expression_tail_option} | \epsilon$$

6. Chuyển cả ngữ nghĩa

```

expression(int *e) →
    expression(int *e) '-' term(int *t) { *e -= *t; }
| term(int *t) { *e = *t; }

```

```

N = expression(int *e)
α = '-' term(int *t) { *e -= *t; }
β = term(int *t) { *e = *t; }

```

```

expression(int *e) →
    term(int *t) *e = *t; expression_tail_option(int *e)
expression_tail_option(int *e) →
    '-' term(int *t) *e -= *t; expression_tail_option(int *e) |

```

7. Otomat đẩy xuống cho văn phạm câu 6:

Top of stack/state:	Look-ahead token		
	IDENTIFIER	+	() EoF
input	expression EoF		expression EoF
expression	term rest_expression		term rest_expression
term	IDENTIFIER		parenthesized_ expression
parenthesized_ expression			(expression)
rest_expression		+ expression	ε ε

8. LR(0) phân tích dưới lên

```

input → expression EoF
expression → term | expression '+' term
term → IDENTIFIER | '(' expression ')'

```

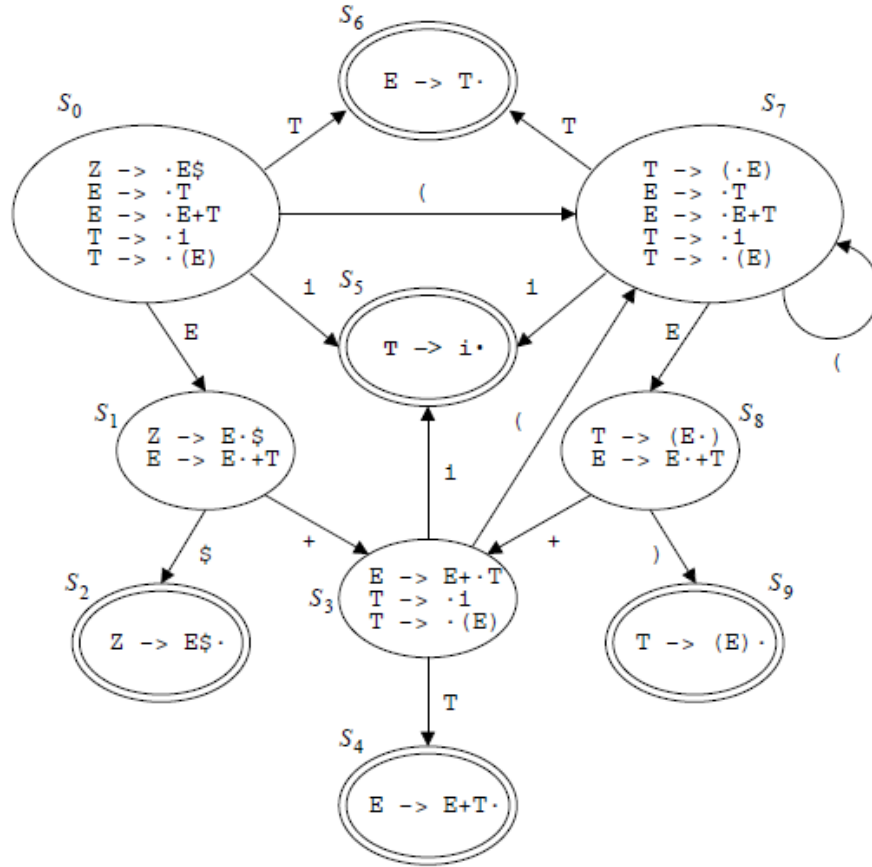
Văn phạm đơn giản cho phân tích dưới lên

```

Z → E $
E → T | E '+' T
T → 'I' | '(' E ')'

```

Dạng rút gọn của văn phạm trên



Lược đồ truyền ứng cho LR(0) của văn phạm đã cho

← GOTO table → ACTION table					
state	symbol				
	i	$+$	$($	$)$	$\$$
0	5	7	1	6	shift
1		3	2		shift
2					$Z \rightarrow E\$$
3	5	7		4	shift
4					$E \rightarrow E+T$
5					$T \rightarrow i$
6					$E \rightarrow T$
7	5	7	8	6	shift
8		3	9		shift
9					$T \rightarrow (E)$

Bảng GOTO và ACTION của LR(0) otomat trên

Stack	Input	Action
S_0	$i + i \$$	shift
$S_0 i S_5$	$+ i \$$	reduce by $T \rightarrow i$
$S_0 T S_6$	$+ i \$$	reduce by $E \rightarrow T$
$S_0 E S_1$	$+ i \$$	shift
$S_0 E S_1 + S_3$	$i \$$	shift
$S_0 E S_1 + S_3 i S_5$	$\$$	reduce by $T \rightarrow i$
$S_0 E S_1 + S_3 T S_4$	$\$$	reduce by $E \rightarrow E + T$
$S_0 E S_1$	$\$$	shift
$S_0 E S_1 \$ S_2$		reduce by $Z \rightarrow E \$$
$S_0 Z$		stop

Phân tích LR(0) cho đầu vào $i + i$

9. Phân tích SLR(1) cho văn phạm trên

$\text{FOLLOW}(Z) = \{ \$ \}$
 $\text{FOLLOW}(E) = \{) + \$ \}$
 $\text{FOLLOW}(T) = \{) + \$ \}$

state	look-ahead token				
	i	+	(	)	\$
0	shift		shift		
1		shift			shift
2					$Z \rightarrow E \$$
3	shift		shift		
4		$E \rightarrow E + T$		$E \rightarrow E + T$	$E \rightarrow E + T$
5		$T \rightarrow i$		$T \rightarrow i$	$T \rightarrow i$
6		$E \rightarrow T$		$E \rightarrow T$	$E \rightarrow T$
7	shift		shift		
8		shift		shift	
9		$T \rightarrow (E)$		$T \rightarrow (E)$	$T \rightarrow (E)$

ACTION Table cho văn phạm SLR(1) ở câu trên

1: $Z \rightarrow E \$$
 2: $E \rightarrow T$
 3: $E \rightarrow E + T$
 4: $T \rightarrow i$
 5: $T \rightarrow (E)$

Đánh số thứ tự các luật suy diễn

state	stack symbol/look-ahead token				
	i	+	(	)	\$ E T
0	s5		s7		s1 s6
1		s3			s2
2					r1
3	s5		s7		s4
4		r3		r3	r3
5		r4		r4	r4
6		r2		r2	r2
7	s5		s7		s8 s6
8		s3		s9	
9		r5		r5	r5

Bảng ACTION/GOTO cho văn phạm SLR(1) đang xét

10. Phân tích LR(1)

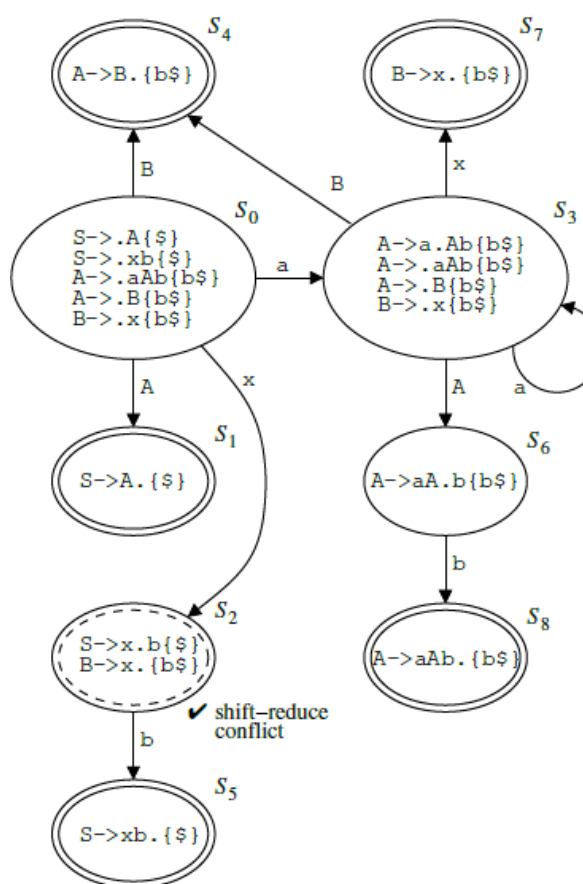
$S \rightarrow A \mid 'x' 'b'$
 $A \rightarrow 'a' A 'b' \mid B$
 $B \rightarrow 'x'$

Văn phạm dùng để mô tả kỹ thuật LR(1)

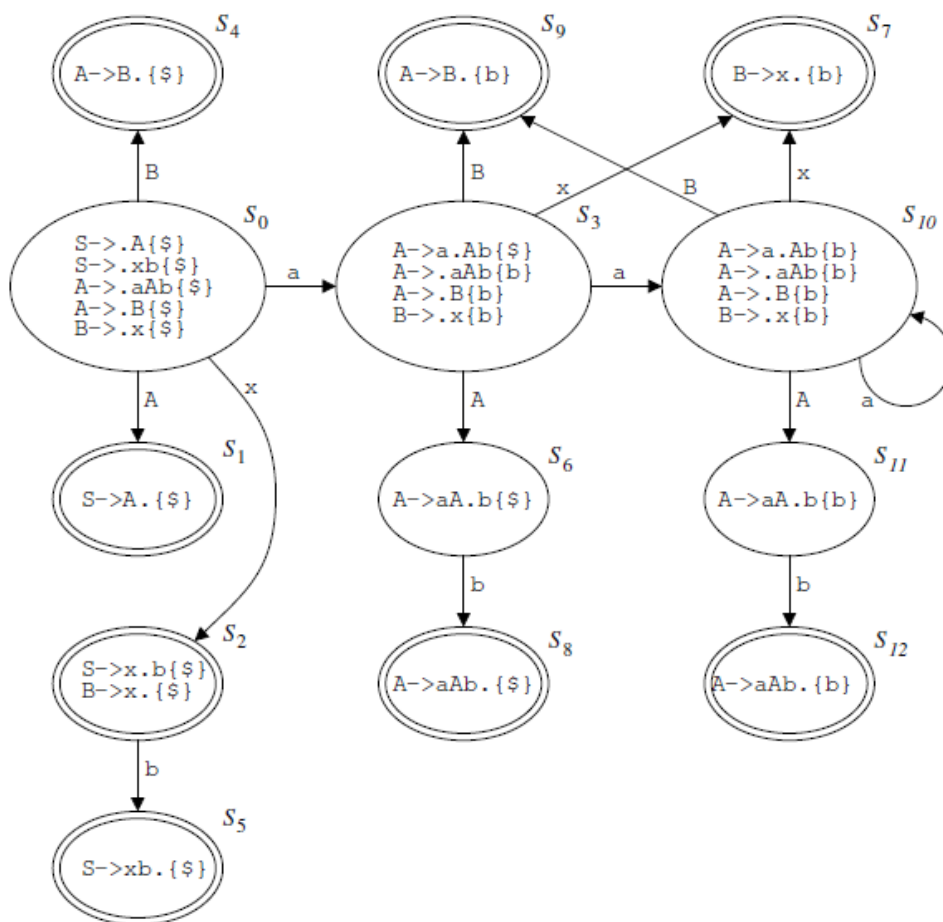
Ngôn ngữ sinh bởi văn phạm:

$\{ xb, a^nxb^n \mid n \geq 0 \}$

Nó không là LL(1) vì mâu thuẫn FIRST/FIRST tại x.

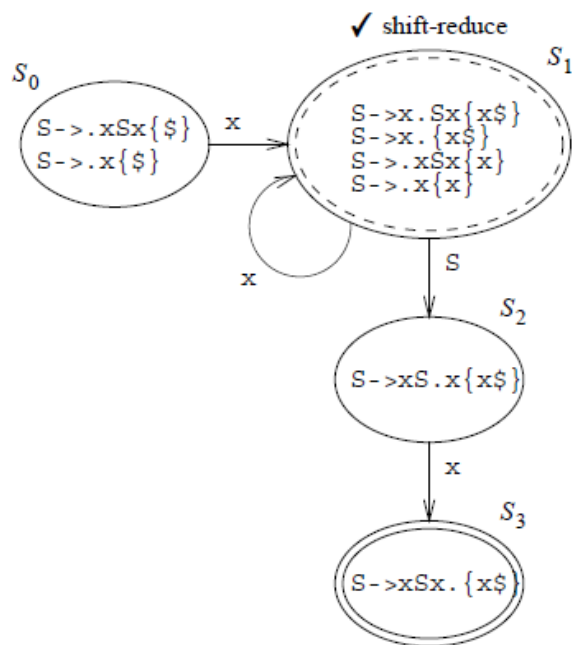


Otomat SLR(1) cho văn phạm trên

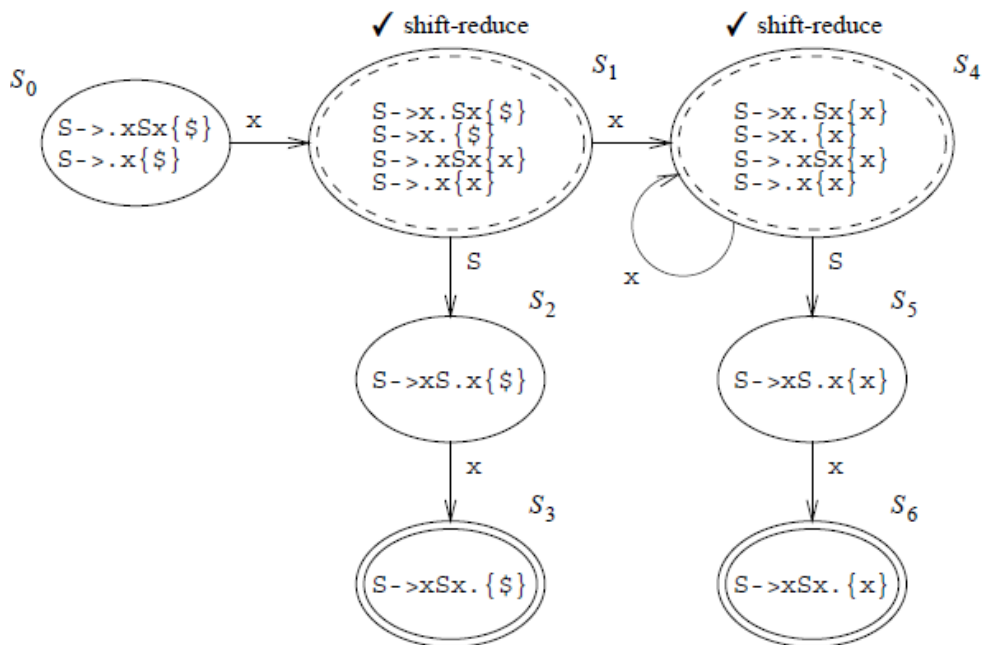


Otomat LR(1) cho văn phạm đã cho

11. Otomat SLR(1) cho văn phạm $S \rightarrow xSx \mid x$



12. Otomat LR(1) cho văn phạm $S \rightarrow xSx \mid x$



13. Cho văn phạm

$E' \rightarrow E$
 $E \rightarrow E + T$
 $E \rightarrow T$
 $T \rightarrow T * F$
 $T \rightarrow F$
 $F \rightarrow (E)$
 $F \rightarrow i$

Xây dựng bảng SLR(1)

State	A C T I O N						G O T O		
	← →						← →		
	i	+	*	(	)	S	E	T	F
0	S5			S4			1	2	3
1		S6				Accept			
2		r2	S7		r2	r2			
3		r4	r4		r4	r4			
4	S5			S4			8	2	3
5		r6	r6		r6	r6			
6	S5			S4				9	3
7	S5			S4					10
8		S6			S11				
9		r1	S7		r1	r1			
10		r3	r3		r3	r3			
11		r5	r5		r5	r5			

Phân tích $i * i + i$

Line	Stack	Input
	0	$i * i + i \$$
	0i5	$* i + i \$$
	0F3	$* ij + i \$$
	0T2	$i + i \$$
	0T2*7	$+ i \$$
	0T2*7i5	$+ i \$$
	0T2*7F10	$+ i \$$
	0T2	$+ i \$$
	0E1	$+ i \$$
	0E1+6	$i \$$
	0EH6i5	$\$$
	0E1+6F3	$\$$
	0E1+6T9	$\$$
	0E1	$\$$
	Accept	

Bài tập bổ sung chương 6:

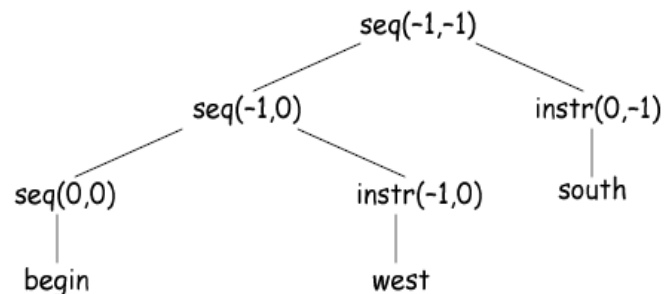
1. Văn phạm thuộc tính để tính tọa độ dịch chuyển theo các hướng khác nhau:

Attributed Grammar for Robot Moves

- Production

$seq(x_0, y_0) \rightarrow$	begin	$x_0 := 0; y_0 := 0$
$ seq(x_1, y_1) instr(dx, dy)$	$x_0 := x_1 + dx; y_0 := y_1 + dy$	
$instr(dx, dy) \rightarrow$	east	$dx := 1; dy := 0$
$ north$	$dx := 0; dy := 1$	
$ west$	$dx := -1; dy := 0$	
$ south$	$dx := 0; dy := -1$	

- Parse tree for begin west south:



Xây dựng cây cú pháp chú giải cho: begin east north west south

2. Dịch trung tố sang hậu tố và khử đệ qui trái

```

expr -> expr + term  { print ('+') }
expr -> expr - term  { print ('-') }
expr -> term
term -> 0             { print ('0') }
term -> 1             { print ('1') }
...
term -> 9             { print ('9') }
expr -> term rest
rest -> + term { print('+') } rest |
        - term { print('-') } rest | e
term -> 0 { print('0') }
term -> 1 { print('1') }
...
term -> 9 { print('9') }

```

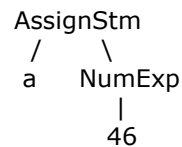
Xây dựng cây cú pháp chú giải cho biểu thức: 7-3+4

C. HƯỚNG DẪN VÀ TRẢ LỜI BÀI TẬP CÁC CHƯƠNG

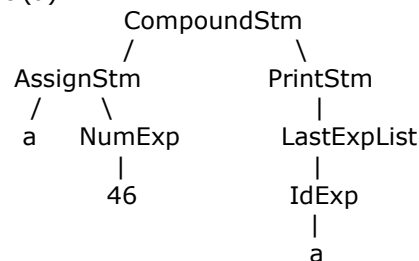
CHƯƠNG 1:

1. Vẽ cây cú pháp trừu tượng AST cho chương trình dòng trực tiếp sau:

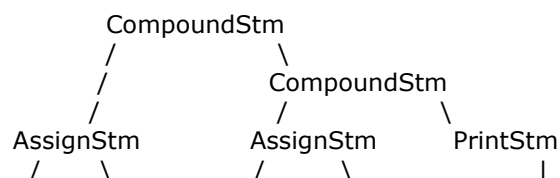
a. a := 46

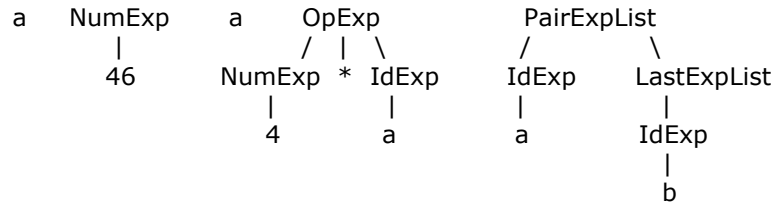


b. a :=46; print (a)



c. a :=46; b := 4 * a; print (a, b)





2. Viết chương trình Java cho chương trình dòng trực tiếp sau:

d. `a := 46;`

```

    Stm program = new AssignStm( "a",
                                new NumExp(46)
                                );
  
```

e. `a :=46; print (a)`

```

    Stm program = new CompoundStm(
        new AssignStm( "a",
            new NumExp(46)
        ),
        new PrintStm(
            new LastExpList(
                new IdExp("a")
            )
        )
    );
  
```

f. `a :=46; b := 4 * a; print (a, b)`

```

    Stm program =
        new CompoundStm(
            new AssignStm( "a",
                new NumExp(46)
            ),
            new CompoundStm(
                new AssignStm( "b",
                    new OpExp(
                        new NumExp(4),
                        Times,
                        new IdExp("a")
                    )
                ),
                new PrintStm(
                    new PairExp(
                        new IdExp("a"),
                        new LastExpList(
                            new IdExp("b")
                        )
                    )
                )
            )
        )
  
```

);

CHƯƠNG 3:

1. Viết các biểu thức chính qui sau:

- a. Các xâu trên bảng chữ cái $\{a,b,c\}$ ở đó chữ a đứng trước chữ b đầu tiên.

$c^*a[ac]^*b[abc]^*$ giả thiết rằng có ít nhất một a và một b

- b. Các xâu trên bảng chữ $\{a,b,c\}$ với số chẵn số chữ a .

$((b|c)^*a(b|c)^*a)^*$ vì 0 cũng là số chẵn

- c. Các số nhị phân mà là bội của 4.

Tức là 0, 000000, 0100, 1000, 00001100, 000000010000, 00000000010100, ...

$0^*[01]^*00 \mid 0^+$

- d. Các xâu trên bảng chữ $\{a,b,c\}$ mà không chứa xâu con liền nhau baa .

Từ chối: ccccaababcbacabaaccca

Chấp nhận: ccccaababcbacabacccab

Chấp nhận: ccccaaacacaaccca

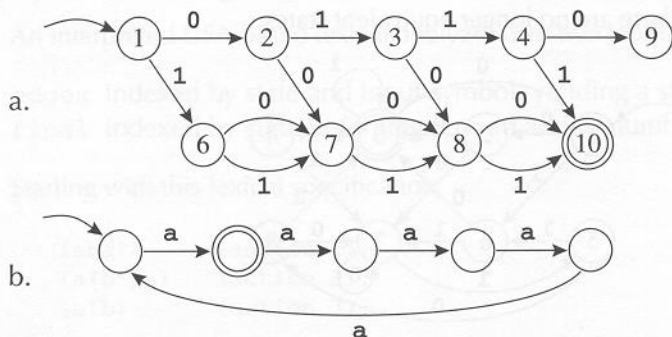
- e. Ngôn ngữ với các hằng số nguyên không âm trong C , mà ở đó các số bắt đầu bởi 0 là hằng số hệ 8 và các số khác là hằng số thập phân.

$0[0-7]^+ \mid [0-9]^+$

2. Tại sao bạn không ngạc nhiên là không có biểu thức chính qui xác định các xâu của a và b mà là từ palindromes (tức là đọc tới cũng như đọc lùi). Gợi ý: Xét số trạng thái cần thiết của otomat hữu hạn.

Yêu cầu vô hạn các trạng thái: $a, aa, aba, abba, \dots$ vì hai đầu về điểm giữa cần phải bằng nhau. Chỉ có thể đặc tả bằng cách liệt kê từng trường hợp.

3. Giải thích trong tiếng Anh thông thường cái gì mỗi otomat hữu hạn sau đây đoán nhận.



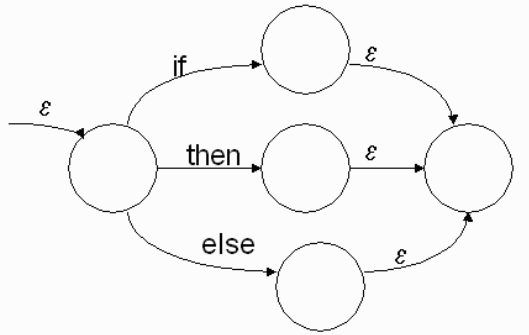
- a. Trạng thái 10 là kết, do đó đoán nhận các xâu: $1xxx, 00xx, 010x$ hoặc

0111 trong đó $x = 0|1$

b. a hoặc $5n+1$ chữ a với $n=1$ đến vô cùng

4. Biến đổi các biểu thức chính qui sau thành NFA.

a. $(if|then|else)$



5. Biến đổi các NFA sau thành DFA.

- a.
- $1 = \{1,2,3,4\} = \text{state}[1]$
 - $\{1,2,3,4\}_x = \{5,6,7\} = \text{state}[2]$
 - $\{1,2,3,4\}_y = \{6,7\} = \text{state}[3]$
 - $\{5,6,7\}_z = \{1,2,3,4\} = \text{state}[1]$

CHƯƠNG 4:

1. Chuyển mỗi biểu thức chính qui sau thành văn phạm phi ngữ cảnh. Sử dụng $|$ cho mỗi phép lựa chọn.

- a. $((xy*x)|(yx*y))?$
 $auxy \rightarrow y auxy \mid \epsilon$
 $auxx \rightarrow x auxx \mid \epsilon$
 $opxy \rightarrow x \mid y \mid \epsilon$
 $answer \rightarrow (x auxy x \mid y auxx y) opxy$
- b. $((0|1)^+.(0|1)^*)|((0|1)^*.(0|1)^+)$
 $1ormore = (0|1) 1ormore \mid (0|1)$
 $0ormore = (0|1) 0ormore \mid \epsilon$
 $1ormore "." 0ormore \mid 0ormore "." 1ormore$

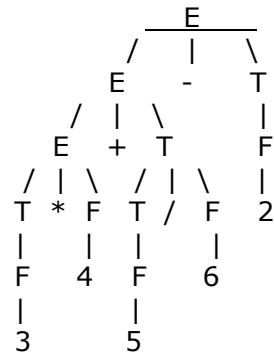
2. Viết văn phạm phi ngữ cảnh mà định nghĩa các dãy lệnh sau:

- a. Phân cách bởi $;$ chẳng hạn: $s; s; s; s$
 $stmtlist \rightarrow stmt ; stmtlist \mid stmt$
 $stmt \rightarrow s$
- b. Kết thúc bởi $;$ chẳng hạn: $s; s; s; s;$
 $stmtlist \rightarrow stmt ; stmtlist \mid \epsilon$
 $stmt \rightarrow s$

3. Giả sử cho các luật sinh định nghĩa *class*, *public*, *static*, *void*, *main*, *string*, *id*, mở vòng trái $[$, vòng phải $]$, ngoặc trái $($, ngoặc phải $)$, móc trái $\{$, móc phải $\}$, các từ tổ và lệnh, *vardecl* và *methoddecl*. Viết văn phạm phi ngữ cảnh trong SableCC cho MiniJava (xem Phụ lục) mà định nghĩa:

- a. *MainClass* - $\text{main_class} = \text{class id lbrace public static void main lparen string lbracket rbracket id r_parenthese lbrace statement rbrace rbrace;$

- b. *ClassDecl* - class_decl = class identifier lbrace vardecl* method_decl* rbrace;
- c. Sử dụng Văn phạm 4.8 xây dựng cây cú pháp cho biểu thức:
 $3*4+5/6-2$



CHƯƠNG 5:

1. Tìm các tập FIRST và FOLLOW
 - a. Tìm tập FIRST cho Văn phạm:

$E \rightarrow TQ$ $Q \rightarrow +TQ \mid -TQ \mid \epsilon$ $T \rightarrow FR$ $R \rightarrow *FR \mid /FR \mid \epsilon$ $F \rightarrow (E) \mid i$	$FIRST(E) = FIRST(T) = FIRST(F) = \{ (, i \}$ $FIRST(Q) = \{ +, -, \epsilon \}$ $FIRST(R) = \{ *, /, \epsilon \}$ $FIRST(+TQ) = \{ + \}$ $FIRST(-TQ) = \{ - \}$ $FIRST(*FR) = \{ * \}$ $FIRST(/FR) = \{ / \}$ $FIRST((E) = \{ (\}$ $FIRST(i) = \{ i \}$
--	--

- b. Tìm tập FOLLOW cho văn phạm trên.
 $\text{FOLLOW}(E) = \{\$, \epsilon\}$ vì E là bắt đầu và trên vế phải E xuất hiện trong $F \rightarrow (E)$

$FOLLOW(Q) = \{ \$, \} = FOLLOW(E)$ Q chỉ xuất hiện ở cuối ($E \rightarrow TQ$) do đó bao gồm $FOLLOW(E)$. Các luật khác chỉ có Q ở trái hoặc phải.

$$\text{FOLLOW}(T) = \{+, -,), \$\}$$

- Chứa $\text{FIRST}(Q) - \{\epsilon\} = \{+, -\}$.
- Q là null được ($\text{FIRST}(Q)$ chứa ϵ) chứa $\text{FOLLOW}(E)$. Để thấy tại sao $\text{FOLLOW}(E)$ được chứa, xét $F \rightarrow (E)$. Thế $E \rightarrow TQ$ cho ta $F \rightarrow (TQ)$. Nếu $Q = \epsilon$ thì ta có $F \rightarrow (T)$ vậy suy ra $)$ sau T .
- Các luật Q không bổ sung gì vì $+TQ$ và $-TQ$.

$$\text{FOLLOW}(R) = \text{FOLLOW}(T) = \{+, -,), \$\}$$

$FOLLOW(F) = \{+, -, *, /,), \$\}$ chứa $FIRST(R) - \{\epsilon\} = \{*, /\}$. R là null được chứa $FOLLOW(T)$.

c. Viết bảng phân tích dự báo cho Văn phạm trong Câu hỏi 5.

$E \rightarrow TQ$	$FIRST(E) = FIRST(T) = FIRST(F) = \{ (, i \}$	$FOLLOW(E) = \{ \$,) \}$
$Q \rightarrow +TQ \mid -TQ \mid \epsilon$	$FIRST(Q) = \{ +, -, \epsilon \}$	$FOLLOW(Q) = \{ \$,) \}$
$T \rightarrow FR$	$FIRST(R) = \{ *, /, \epsilon \}$	$FOLLOW(T) = \{ +, -,), \$ \}$
$R \rightarrow *FR \mid /FR \mid \epsilon$	$FIRST(+TQ) = \{ + \}$	$FOLLOW(R) = \{ +, -,), \$ \}$
$F \rightarrow (E)$	$FIRST(-TQ) = \{ - \}$	$FOLLOW(F) = \{ +, -, *, /,), \$ \}$
$\mid i$	$FIRST(*FR) = \{ * \}$	
	$FIRST(/FR) = \{ / \}$	
	$FIRST((E)) = \{ (\}$	
	$FIRST(i) = \{ i \}$	

2. $E \rightarrow TQ$ $FIRST(T) = \{ (, i \}$ vậy $[E, (]$ và $[E, i] = E \rightarrow TQ$
 $Q \rightarrow +TQ$ $FIRST(+TQ) = \{ + \}$ vậy $[Q, +] = Q \rightarrow +TQ$
 $Q \rightarrow -TQ$ $FIRST(-TQ) = \{ - \}$ vậy $[Q, -] = Q \rightarrow -TQ$
 $T \rightarrow FR$ $FIRST(F) = \{ (, i \}$ vậy $[T, (]$ và $[T, i] = T \rightarrow FR$
 $R \rightarrow *FR$ $FIRST(*FR) = \{ * \}$ vậy $[R, *] = R \rightarrow *FR$
 $R \rightarrow /FR$ $FIRST(/FR) = \{ / \}$ vậy $[R, /] = R \rightarrow /FR$
 $F \rightarrow (E)$ $FIRST((E)) = \{ (\}$ vậy $[F, (] = F \rightarrow (E)$
 $F \rightarrow i$ $FIRST(i) = \{ i \}$ vậy $[F, i] = F \rightarrow i$

3. Tìm các tập FIRST và FOLLOW cho Văn phạm:

Cho chưa tận A, tập **FOLLOW(A)** bao gồm các tận cùng và có thể \$ được định nghĩa như sau:

- a. Nếu A là ký hiệu bắt đầu thì \$ nằm trong Follow(A).
b. Nếu luật $B \rightarrow \alpha A \gamma$, thì $FIRST(\gamma) - \{\epsilon\}$ nằm trong Follow(A)
c. Nếu luật $B \rightarrow \alpha A \gamma$, sao cho ϵ ở trong $FIRST(\gamma)$, thì Follow(A) chứa Follow(B); vì cái gì sau B có thể sau A khi $\gamma = \epsilon$
d. Nếu luật $B \rightarrow \alpha A$, thì Follow(A) chứa Follow(B); vì cái gì sau B có thể sau A

$S \rightarrow ABC$	$FIRST(S) = \{ a, c, d, e, f \}$	$FOLLOW(S) = \{ \$ \}$ Luật 1.
$A \rightarrow a \mid Cb \mid \epsilon$	$FIRST(A) = \{ a, e, f, \epsilon \}$	$FOLLOW(A) = FIRST(B) \cup FIRST(C) \cup FOLLOW(B) = \{ c, d, e, f \}$
$B \rightarrow c \mid dA \mid \epsilon$	$FIRST(B) = \{ c, d, \epsilon \}$	Luật 2, 3 và 4.
$C \rightarrow e \mid f$	$FIRST(C) = \{ e, f \}$	$FOLLOW(B) = FIRST(C) = \{ e, f \}$ Luật 2 $FOLLOW(C) = FIRST(b) = \{ b \} \cup FOLLOW(S) = \{ b, \$ \}$ Luật 2 và 4

5. Xây dựng bảng phân tích cú pháp cho Câu hỏi 1.

Bản nhập rỗng là các điều kiện lỗi. Các luật cho phép sinh tiêu biểu $X \rightarrow \beta$ là:

- a. Đối với mọi tận cùng a trong $FIRST(\beta)$ ngoại trừ ϵ , $[X, a] = X \rightarrow \beta$
b. Nếu $\beta = \epsilon$ hoặc nếu ϵ ở trong $FIRST(\beta)$ thì: đối với mọi a trong

FOLLOW(X), [X,a]=X →□ε						
	A	b c	D	e	F	\$
S	S → ABC	S → ABC	S → ABC	S → ABC	S → ABC	
A	A → a	A → ε	A → ε	A → Cb	A → Cb	
B		B → c	B → dA	B → ε	B → ε	
C				C → e	C → f	

CHƯƠNG 6:

1. Kiểm tra gói SymbolTable chứa các lớp SymbolTable, Class, Method và Variable. Vẽ lược đồ tổ chức các class, fields, methods, parameters, và local variables của chương trình sau.

```

class ex5 {
    public static void main(String [] args) {
        System.out.println((new B()).bm(10, 20, 30));
    }
}

class A {
    int aglobal1;
    int aglobal2;
    public int am1(int am1parm1, int am1parm2, int am1parm3) {
        int am1local1;
        int am1local2;
        return aglobal1;
    }
    public int am2(int am2parm1, int am2parm2, int am2parm3) {
        int am2local1;
        int am2local2;
        return aglobal1;
    }
}

class B extends A {
    int bglobal;
    public int bm(int bparm1, int bparm2, int bparm3) {
        int bmlocal1;
        int bmlocal2;
        bmlocal2 = 40;
        return bmlocal2;
    }
}

```

Classes

B

Fields

bglobal class minijava.node.AIntType

Methods

bm class minijava.node.AIntType

Parameters

bparm1 class minijava.node.AIntType

bparm2 class minijava.node.AIntType

ex5			bparm3	class minijava.node.AIntType
			Variables	
			bmlocal1	class minijava.node.AIntType
			bmlocal2	class minijava.node.AIntType
	Fields			
	Methods			
	A			
	Fields			
		aglobal1	class minijava.node.AIntType	
		aglobal2	class minijava.node.AIntType	
	Methods			
		am2	class minijava.node.AIntType	
		Parameters		
			am2parm1	class minijava.node.AIntType
			am2parm2	class minijava.node.AIntType
			am2parm3	class minijava.node.AIntType
		Variables		
			am2local2	class minijava.node.AIntType
			am2local1	class minijava.node.AIntType
		am1	class minijava.node.AIntType	
		Parameters		
			am1parm1	class minijava.node.AIntType
			am1parm2	class minijava.node.AIntType
			am1parm3	class minijava.node.AIntType
		Variables		
			am1local2	class minijava.node.AIntType
			am1local1	class minijava.node.AIntType

2. Đối với cú pháp trừu tượng MiniJava, các phương thức nào cần được cài đặt để xây dựng Bảng ký hiệu?

Về cơ bản bất cứ khai báo nào cần phải đặt trong Bảng ký hiệu với kiểu đã được khai báo của nó:

Abstract Syntax Tree

```

program = main_class class_decl*;
main_class = [classname]:id [arg]:id statement;
class_decl =
    id var_decl* method_decl* |
    {extends} [classname]:id
    [extend]:id var_decl* method_decl*;
var_decl = type id;
method_decl = type id formal_list var_decl* statement* exp;
formal_list = type id formal_rest* | {empty};
formal_rest = type id;

type= {int_array} |
      {boolean} |
      {int} |
      {id} id;

```

```

public void caseAProgram(AProgram node)
public void caseAMainClass(AMainClass node)
public void caseAClassDecl(AClassDecl node)
public void caseAExtendsClassDecl(AExtendsClassDecl node)
public void caseAVarDecl(AVarDecl node)

```

```

public void caseAMethodDecl(AMethodDecl node)
public void caseAFormalList(AFormalList node)

public void caseAFormalRest(AFormalRest node)

public void caseAIntType(AIntType node)
public void caseABooleanType(ABooleanType node)
public void caseAIntArrayType(AIntArrayType node)
public void caseAIdType(AIdType node)

```

- a. variable **type id**
- b. statement **nothing**
- c. method_decl **type id formal_list var_decl***

CHƯƠNG 7:

1. Đối với cú pháp trừu tượng MiniJava trên, các phương thức nào cần được cài đặt để kiểm tra kiểu biểu thức sau?
 - a. 5+4
 - b. M(3,2);
 - c. true && false
 - d. if true then x=5 else x=3;

Mọi việc sử dụng một ký hiệu (mọi thứ cần được khai báo) hoặc literal trong biểu thức hoặc lệnh, các khai báo cần được cài đặt theo thứ tự *apply* kiểm tra kiểu cho khai báo phương thức mà bao gồm cả lệnh và biểu thức. Mỗi luật trong văn phạm cần được cài đặt trong kiểm tra kiểu.

Abstract Syntax Tree

```

program = main_class class_decl*;
main_class = [classname]:id [arg]:id statement;
class_decl =
    id var_decl* method_decl* |
    {extends} [classname]:id
    [extend]:id var_decl* method_decl*;
var_decl = type id;
method_decl = type id formal_list var_decl* statement* exp;
statement =
    {statementlist} statement* |
    {if} exp [true]:statement [false]:statement |
    {while} exp statement |
    {println} exp |
    {assign} id exp |
    {array_assign} id [index]:exp [r]:exp;
formal_list = type id formal_rest* | {empty};
formal_rest = type id;

type= {int_array} |
    {boolean} |
    {int} |
    {id} id;
exp = {and} [l]:exp [r]:exp |
    {lt} [l]:exp [r]:exp |
    {plus} [l]:exp [r]:exp |
    {minus} [l]:exp [r]:exp |

```

```

{times} [l]:exp [r]:exp |
{length} exp |
{arrayindex} [array]:exp [index]:exp |
{methodcall} exp id exp_list |
{integerliteral} integer_literal |
{trueliteral} true |
{falseliteral} false |
{id} id |
{this} this |
{new} id |
{newarray} exp |
{not} exp ;
exp_list = exp*;

```

- d. 4 * a **AIntegerliteralExp ATimesExp AIdExp**
- e. while(a<b) a = a + 1; **AIntegerliteralExp APlusExp AIdExp AltExp AWhileStm AAssignStm**

- 2. Viết phương thức để kiểm tra kiểu
 - a. falseliteral

```

public void caseAFalseliteralExp(AFalseliteralExp node) {
    setType(node, new ABooleanType());
}

```

- b. It

```

public void caseAltExp(AltExp node) {
    node.getL().apply(this);
    if (!((Node)getType(node.getL())) instanceof AIntType) {
        System.out.println("Left side of < must be of type integer");
        System.exit(-1);
    }
    node.getR().apply(this);
    if (!((Node)getType(node.getR())) instanceof AIntType) {
        System.out.println("Right side of < must be of type integer");
        System.exit(-1);
    }
    setType(node, new ABooleanType());
}

```

CHƯƠNG 8:

<pre> 1. public class Q1 { 2. public static void main(String [] args) { 3. Q2 qa = new Q2(); 4. qa.f1(5); 5. Q2 qb = new Q2(); 6. qa.f2(qb, 4); 7. } 8. } </pre>	<pre> 7. class Q2 { 8. int a; 9. public int f1(int n) { 10. int i; 11. int j; 12. i = n; 13. j = i + 1; 14. this.a = n; 15. return this.a; 16. } 17. } </pre>	<pre> 18. public void f2(Q2 q, int m) { 19. int i; 20. i = q.f1(m); 21. } 22. } </pre>
--	--	---

1. Vẽ nội dung khung JVM (chỉ ra các tham số hình thức và thực tế) cho lời gọi phương thức: **qa.f1(5);**

qa.f1(5)	
0	this=qa
1	n=5
2	i
3	j

2. Vẽ ngăn xếp (nêu TOS) cho lời gọi phương thức: **qa.f1(5);**

qa.f1(5)	
0	qa
1 TOS	5

3. Vẽ nội dung khung JVM (chỉ ra các tham số hình thức và thực tế) cho hai lời gọi phương thức bắt đầu bởi: **qa.f2(qb, 4);**

qa.f2(qb, 4)		q.f1(m)	
0	this=qa	0	this=qb
1	q=qb	1	n=4
2	m=4	2	i
3	i	3	j

4. Vẽ ngăn xếp (nêu TOS) cho lời gọi phương thức tại dòng 20: **qa.f2(qb, 4);**

qa.f2(qb, 4)	
0	qa
1	qb
2 TOS	4

5. Vẽ ngăn xếp (nêu TOS) cho lời gọi phương thức: **q.f1(m);**

q.f1(m)	
0	qb
1 TOS	4

6. Trong phương thức visitor nào offsets khung (đối số và biến cục bộ) cần được tính toán?

AMethodDecl

7. Tính toán offset khung xảy ra sau khi xây dựng bảng ký hiệu. Nêu ba lớp và tên phương thức cần để gán offsets mà lấy từ gói SymbolTable.
 1. Method sizeParams
 2. Method sizeVars
 3. Variable setOffset

Viết biểu diễn trung gian cho biểu thức sau:

8. $3 * 5 - 4$

```
BINOP(  
  BINOP.SUB,  
  BINOP(  
    BINOP.MUL,  
    CONST(3),  
    CONST(5)  
  ),  
  CONST(4)  
)
```

9. $a = 3$, giả sử a ở trong khung tại offset 5 và $TEMP(fp)$ là giá trị con trỏ khung.

```
MOVE(  
  MEM(  
    BINOP(  
      BINOP.PLUS,  
      TEMP(fp),  
      CONST(5)  
    )  
  ),  
  CONST(3)  
)
```

10. $a = b$, giả sử a và b ở trong khung tại offset 5 và 7 tương ứng và $TEMP(fp)$ là giá trị con trỏ khung.

```
MOVE(  
  MEM(          ;a  
    BINOP(  
      BINOP.PLUS,  
      TEMP(fp),  
      CONST(5)  
    )  
  ),  
  MEM(          ;b  
    BINOP(  
      BINOP.PLUS,  
      TEMP(fp),  
      CONST(7)  
    )  
  ),  
)
```

11. $4 < 5$

```
ESEQ(  
  SEQ(  
    SEQ(  
      SEQ(  
        MOVE(TEMP(t),CONST(0)),  
        CJUMP(Tree.CJUMP.LT, CONST(4), CONST(5), T, F)  
      )  
    )  
  )  
)
```

```
    ),  
    SEQ(  
        LABEL(T),  
        MOVE(TEMP(t),CONST(1))  
    )  
    ),  
    LABEL(F)  
    ),  
    TEMP(t)  
)
```

12. while (a < b)
 a = b

```
ESEQ(  
    SEQ(  
        SEQ(  
            LABEL(test),  
            CJUMP(Tree.CJUMP.EQ,  
                condition  
                CONST(1),T,F)  
        ),  
        SEQ(  
            SEQ(  
                LABEL(T),  
                body  
            ),  
            Tree.JUMP( test )  
        )  
    ),  
    LABEL(F)  
)
```

condition từ Câu hỏi 4

body từ Câu hỏi 3

TÀI LIỆU THAM KHẢO

1. Alfred V. Aho, Ravi Sethi, and Jeffrey D. Ullman. *Compilers: Principles, Techniques, and Tools*. Second edition. Wesley: Pearson Addison, 2007
2. Andrew W. Appel và Jens Palsberg. *Modern Compiler Implementation in Java*. Second Princeton University, 2002.
3. Dick Grune. *Modern Compiler Design*. Second Edition. Springer, 2012.

MỤC LỤC

LỜI NÓI ĐẦU	2
CHƯƠNG 1 MỞ ĐẦU VỀ CHƯƠNG TRÌNH DỊCH	4
1.1. Tổng quan về chương trình dịch	4
1.2. Các bước chính của chương trình dịch	5
1.3. Thông dịch và biên dịch	7
Ví dụ: Chương trình dòng lệnh trực tiếp	
Bài tập	
CHƯƠNG 2 THIẾT KẾ NGÔN NGỮ LẬP TRÌNH	17
2.1. Lịch sử ngôn ngữ lập trình	17
2.2. Thiết kế ngôn ngữ lập trình	22
2.3. Dịch sang mã trung gian	32
2.4. Máy ảo Java	36
Đọc thêm	
Bài tập	
CHƯƠNG 3 PHÂN TÍCH TỪ VỰNG	38
3.1. Biểu thức chính qui	38
3.2. Otomat hữu hạn	42
3.3. Văn phạm chính qui	45
3.4. Bộ phân tích từ vựng	52
Chương trình phân tích từ vựng	
Bài tập	
CHƯƠNG 4 PHÂN TÍCH CÚ PHÁP	59
4.1. Văn phạm phi ngữ cảnh	61
4.2. Phân tích văn phạm phi ngữ cảnh	62
4.3. Phân tích dạng chuẩn tắc Chomsky.....	67
4.4. Bộ phân tích BNF và EBNF.....	70
Chương trình Phân tích cú pháp	
Bài tập	
CHƯƠNG 5 PHÂN TÍCH CÚ PHÁP DỰA BẢNG	77
5.1. Phân tích trên xuống văn phạm LL(1)	77

5.2.	Phân tích dưới lên văn phạm LR	85
Chương trình cú pháp trừu tượng		
Bài tập		
CHƯƠNG 6 PHÂN TÍCH NGŨ NGHĨA		114
6.1	Lược đồ dịch	114
6.2	Văn phạm thuộc tính tổng hợp	121
6.3	Văn phạm thuộc tính kế thừa	129
Chương trình kiểm tra kiểu		
Bài tập		
CHƯƠNG 7 KIỂM TRA KIỂU		140
7.1	Kiểm tra kiểu	140
7.2	Xác định phép toán.....	144
7.3	Giải quyết kiểm nghiệm	157
Chương trình kiểm tra kiểu		
Bài tập		
CHƯƠNG 8 MÔI TRƯỜNG TÍNH TOÁN.....		175
8.1	Tổ chức bộ nhớ	175
8.2	Lưu trữ thông tin	181
8.3	Vun rác	190
Chương trình dịch sang cây trung gian		
Bài tập		
PHỤ LỤC		
A.VĂN PHẠM MINIJAVA		223
B. BÀI TẬP BỔ SUNG.....		225
C. HƯỚNG DẪN VÀ TRẢ LỜI		243